

Installs

```
In [28]: !pip install scipy matplotlib
!pip install ipywidgets
```

```
Requirement already satisfied: scipy in ./venv/lib/python3.13/site-packages (1.16.0)
Requirement already satisfied: matplotlib in ./venv/lib/python3.13/site-packages (3.10.3)
Requirement already satisfied: numpy<2.6,>=1.25.2 in ./venv/lib/python3.13/site-packages (from scipy) (2.3.1)
Requirement already satisfied: contourpy>=1.0.1 in ./venv/lib/python3.13/site-packages (from matplotlib) (1.3.2)
Requirement already satisfied: cyclor>=0.10 in ./venv/lib/python3.13/site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in ./venv/lib/python3.13/site-packages (from matplotlib) (4.58.4)
Requirement already satisfied: kiwisolver>=1.3.1 in ./venv/lib/python3.13/site-packages (from matplotlib) (1.4.8)
Requirement already satisfied: packaging>=20.0 in ./venv/lib/python3.13/site-packages (from matplotlib) (25.0)
Requirement already satisfied: pillow>=8 in ./venv/lib/python3.13/site-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in ./venv/lib/python3.13/site-packages (from matplotlib) (3.2.3)
Requirement already satisfied: python-dateutil>=2.7 in ./venv/lib/python3.13/site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in ./venv/lib/python3.13/site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: ipywidgets in ./venv/lib/python3.13/site-packages (8.1.7)
Requirement already satisfied: comm>=0.1.3 in ./venv/lib/python3.13/site-packages (from ipywidgets) (0.2.2)
Requirement already satisfied: ipython>=6.1.0 in ./venv/lib/python3.13/site-packages (from ipywidgets) (9.4.0)
Requirement already satisfied: traitlets>=4.3.1 in ./venv/lib/python3.13/site-packages (from ipywidgets) (5.14.3)
Requirement already satisfied: widgetsnbextension~=4.0.14 in ./venv/lib/python3.13/site-packages (from ipywidgets) (4.0.14)
Requirement already satisfied: jupyterlab_widgets~=3.0.15 in ./venv/lib/python3.13/site-packages (from ipywidgets) (3.0.15)
Requirement already satisfied: decorator in ./venv/lib/python3.13/site-packages (from ipython>=6.1.0->ipywidgets) (5.2.1)
Requirement already satisfied: ipython-pygments-lexers in ./venv/lib/python3.13/site-packages (from ipython>=6.1.0->ipywidgets) (1.1.1)
Requirement already satisfied: jedi>=0.16 in ./venv/lib/python3.13/site-packages (from ipython>=6.1.0->ipywidgets) (0.19.2)
Requirement already satisfied: matplotlib-inline in ./venv/lib/python3.13/site-packages (from ipython>=6.1.0->ipywidgets) (0.1.7)
Requirement already satisfied: pexpect>4.3 in ./venv/lib/python3.13/site-packages (from ipython>=6.1.0->ipywidgets) (4.9.0)
Requirement already satisfied: prompt_toolkit<3.1.0,>=3.0.41 in ./venv/lib/python3.13/site-packages (from ipython>=6.1.0->ipywidgets) (3.0.41)
```

3.0.51)

Requirement already satisfied: pygments>=2.4.0 in ./venv/lib/python3.13/site-packages (from ipython>=6.1.0->ipywidgets) (2.19.2)

Requirement already satisfied: stack_data in ./venv/lib/python3.13/site-packages (from ipython>=6.1.0->ipywidgets) (0.6.3)

Requirement already satisfied: wcwidth in ./venv/lib/python3.13/site-packages (from prompt_toolkit<3.1.0,>=3.0.41->ipython>=6.1.0->ipywidgets) (0.2.13)

Requirement already satisfied: parso<0.9.0,>=0.8.4 in ./venv/lib/python3.13/site-packages (from jedi>=0.16->ipython>=6.1.0->ipywidgets) (0.8.4)

Requirement already satisfied: ptyprocess>=0.5 in ./venv/lib/python3.13/site-packages (from pexpect>4.3->ipython>=6.1.0->ipywidgets) (0.7.0)

Requirement already satisfied: executing>=1.2.0 in ./venv/lib/python3.13/site-packages (from stack_data->ipython>=6.1.0->ipywidgets) (2.2.0)

Requirement already satisfied: asttokens>=2.1.0 in ./venv/lib/python3.13/site-packages (from stack_data->ipython>=6.1.0->ipywidgets) (3.0.0)

Requirement already satisfied: pure-eval in ./venv/lib/python3.13/site-packages (from stack_data->ipython>=6.1.0->ipywidgets) (0.2.3)

Kapitel 1: Riemann-Integrale mit Ober- und Untersummen, Trapezregel

Grundidee

Die Idee hinter den Riemann-Integralen ist es den Flächeninhalt in einem Intervall $[a, b]$ einer Kurve/eines Graphen näherungsweise zu berechnen. Dafür wird das Intervall in n -Teilintervalle geteilt.

$$\Delta x = \frac{b-a}{n}$$

Die einzelnen Teilintervalle x_i werden mit der folgenden Formel erstellt:

$$x_i = a + i \cdot \Delta x$$

Ein Teilintervall wäre dann $[x_i, x_{i+1}]$ für $i \in \mathbb{N}$

Untersumme

Bei der Untersumme werden Rechtecke innerhalb der Teilintervalle erstellt. Der kleinste Funktionswert innerhalb des Teilintervalls dient bei den Rechtecken als Höhe, das Δx als Breite. Um den Flächeninhalt der Funktion anschließend zu berechnen, nutzen wir die Formel der Rechtecke:

$$A = a \cdot b$$

Dabei ist unser a unsere Höhe, was der kleinste Funktionswert bei der Untersumme ist und b unsere Breite, was unser Δx ist:

$$U = \sum_{i=0}^{n-1} \min_{x \in [x_i, x_{i+1}]} f(x) \cdot \Delta x$$

Obersumme

Bei der Obersumme werden Rechtecke innerhalb der Teilintervalle erstellt. Der größte Funktionswert innerhalb des Teilintervalls dient bei den Rechtecken als Höhe, das Δx als Breite. Um den Flächeninhalt der Funktion anschließend zu berechnen, nutzen wir die Formel der Rechtecke:

$$A = a \cdot b$$

Dabei ist unser a unsere Höhe, was der größte Funktionswert bei der Obersumme ist und b unsere Breite, was unser Δx ist, dass benutzt wird um die Teilintervalle zu berechnen:

$$O = \sum_{i=0}^{n-1} \max_{x \in [x_i, x_{i+1}]} f(x) \cdot \Delta x$$

Trapezregel

Bei der Trapezregel wird der Funktionswert an $f(x_i)$ mit einer Geraden zu $f(x_{i+1})$ verbunden. Dadurch entstehen Trapeze unterhalb des Graphen in den jeweiligen Teilintervallen. Die beiden Funktionswerte an x_i und x_{i+1} sind dabei unsere Höhen und Δx unsere Breite. Um den Flächeninhalt der Funktion anschließend zu berechnen, nutzen wir die Formel der Trapeze:

$$A = \frac{1}{2} (h_1 + h_2) \cdot b$$

Dabei ist unsere Höhe h_1 und h_2 unsere Funktionswerte $f(x_i)$ und $f(x_{i+1})$ bei der Trapezregel ist unsere Breite b auch unser Δx . Wir können unsere Formel außerdem noch etwas vereinfachen, wir summieren alle Funktionswerte zusammen und da wir jedes x_i und x_{i+1} immer doppelt benutzen, können wir den Bruch $\frac{1}{2}$ ignorieren und müssen diesen nur bei unserem x_0 und x_n verwenden, da wir diese nur einmal haben beim erstellen unserer Trapeze. Deswegen subtrahieren wir am Ende noch mit $\frac{1}{2} \cdot (f(x_0) + f(x_n))$. Damit lautet die Formel:

$$T = \left(\sum_{i=0}^{n-1} f(x_i) \right) - \frac{1}{2} \cdot (f(x_0) + f(x_n)) \cdot \Delta x$$

Erklärung: `find_extrema(func, a, b)`

Diese Funktion dient der **Bestimmung des Minimums und Maximums** einer Funktion $f(x)$ auf einem **endlichen Intervall** $[a, b]$.

Funktionssignatur

```
def find_extrema(func: Callable[[float], float], a: float, b: float) -> Tuple[float, float]
```

Parameter:

Parameter	Typ	Bedeutung
<code>func</code>	Funktion	Die zu untersuchende Funktion ($f(x)$)
<code>a</code> , <code>b</code>	float	Die Intervallgrenzen ($a < b$)

Rückgabewert:

Ein Tupel (`f_min`, `f_max`) mit:

- `f_min` : Funktionsminimum im Intervall
- `f_max` : Funktionsmaximum im Intervall

Idee

Für die numerische Bestimmung von Extremwerten wird `scipy.optimize.minimize_scalar` verwendet.

Ablauf

1. Überprüfung des Intervalls:

- `a` muss kleiner als `b` sein, sonst wird ein Fehler geworfen.

2. Minimum berechnen:

- `minimize_scalar(func, bounds=(a, b), method='bounded')`
- Rückgabe: `result_min.fun`

3. Maximum berechnen:

- Trick: Das **Maximum** einer Funktion ist das **Minimum der negativen Funktion**.
- Deshalb: `minimize_scalar(lambda x: -func(x), ...)`
- Danach wird das Ergebnis negiert: `-result_max.fun`

4. Fehlerbehandlung:

- Wenn die Optimierung fehlschlägt (`.success == False`), wird ein `ValueError` ausgelöst.

Hinweis

Diese Methode ist **numerisch**. Sie garantiert kein exaktes Ergebnis, sondern eine Näherung. Für glatte und stetige Funktionen ist sie jedoch in der Praxis sehr zuverlässig.

```
In [29]: from typing import Callable, Tuple
from scipy.optimize import minimize_scalar

def find_extrema(func: Callable[[float], float], a: float, b: float)
    """
    Findet das Minimum und Maximum der Funktion func im Intervall [a, b].

    Rückgabe:
        Ein Tupel (f_min, f_max), wobei:
        - f_min der kleinste Funktionswert im Intervall ist
        - f_max der größte Funktionswert im Intervall ist
    """

    # Überprüfung: Das Intervall muss gültig sein (a < b)
    if a >= b:
        raise ValueError("Ungültiges Intervall: a muss kleiner als b sein")

    # ----- Minimum berechnen -----
    # Verwende eine numerische Methode ("bounded"), um das Minimum zu finden
    result_min = minimize_scalar(func, bounds=(a, b), method='bounded')

    # Überprüfung, ob die Optimierung erfolgreich war
    if not result_min.success:
        raise ValueError("Minimum konnte nicht gefunden werden.")

    # Speichere den gefundenen minimalen Funktionswert
    f_min = result_min.fun

    # ----- Maximum berechnen -----
    # Trick: Statt das Maximum direkt zu berechnen, minimieren wir -func(x)
    # Dadurch erhalten wir das Maximum von func, weil: max(func(x)) = -min(-func(x))
    result_max = minimize_scalar(lambda x: -func(x), bounds=(a, b), method='bounded')

    # Überprüfung, ob die Optimierung erfolgreich war
    if not result_max.success:
        raise ValueError("Maximum konnte nicht gefunden werden.")

    # Da wir -f(x) minimiert haben, negieren wir das Ergebnis wieder
    f_max = -result_max.fun

    # Rückgabe des gefundenen Minimums und Maximums als Tupel
    return f_min, f_max
```

Erklärung: Allgemeine numerische Integration mit Fehlerbehandlung und Visualisierung

Dieser Code implementiert eine **robuste und flexible Methode zur numerischen Integration**. Es werden drei klassische Verfahren unterstützt: **Untersumme**, **Obersumme** und **Trapezregel**.

sicher_auswerten(f, x)

Diese Funktion wertet die Funktion `f(x)` sicher aus. Sie fängt typische Fehler wie Division durch Null, nicht-definierte Werte (z. B. `\log(0)`) oder `NaN` / `Inf` ab und gibt eine verständliche Fehlermeldung aus.

vektorisiere(f)

Da NumPy-Funktionen mit Arrays arbeiten, wird `f` hier vektorisiert. Intern nutzt die Methode `np.vectorize()` zusammen mit `sicher_auswerten`, um `f` auf NumPy-Arrays anzuwenden.

numerische_integration(...)

Hauptfunktion zur numerischen Berechnung eines bestimmten Integrals mit optionaler grafischer Ausgabe.

Signatur:

```
numerische_integration(  
    f: Callable[[float], float],  
    a: float,  
    b: float,  
    n: int,  
    methode: str,  
    zeige_plot: bool = True  
) -> float
```

Parameter:

Parameter	Bedeutung
<code>f</code>	Funktion $f(x)$
<code>a</code> , <code>b</code>	Intervallgrenzen
<code>n</code>	Anzahl der Teilintervalle
<code>methode</code>	"Untersumme", "Obersumme" oder "Trapezregel"

`zeige_plot` Ob ein grafischer Plot erzeugt wird

Vorgehen

1. Fehlertests:

- `n > 0`, `a < b`, `f` muss aufrufbar sein.

2. Intervallaufteilung:

- `deltaX = (b - a) / n`
- Erzeuge `x_i` mit `n+1` Knotenpunkten.

3. Integration je nach Methode:

- `"Untersumme"` : Minima in jedem Teilintervall bestimmen.
- `"Obersumme"` : Maxima in jedem Teilintervall bestimmen.
- `"Trapezregel"` : Durchschnitt der Funktionswerte am Rand.

4. Fehlerbehandlung:

- Jeder Rechenschritt ist abgesichert durch `try-except`.

5. Grafische Darstellung:

- Zeige `f(x)` und:
 - Rechtecke für Unter-/Obersumme,
 - Trapeze für Trapezregel.
- Plot mit Legende, Raster und Achsentiteln.

```
In [30]: from typing import Callable, Tuple
import numpy as np
import matplotlib.pyplot as plt
import math
from scipy.optimize import minimize_scalar

# -----
# Hilfsfunktion: Sichere Auswertung von Funktionen
# -----
def sicher_auswerten(f: Callable[[float], float], x: float) -> float:
    """
    Führt eine sichere Auswertung der Funktion f an der Stelle x durch.
    Fängt typische Fehler wie Division durch Null oder unendliche Werte ab.
    """
    try:
        y = f(x)
        if not math.isfinite(y):
            raise ValueError(f"f({x}) = {y} ist kein endlicher Wert")
        return y
    except ZeroDivisionError:
        raise ValueError(f"f({x}): Division durch Null.")
    except ValueError as ve:
        raise ValueError(f"f({x}): Ungültiger mathematischer Ausdruck")
```

```

except Exception as e:
    raise ValueError(f"f({x}): Allgemeiner Fehler bei der Funkti

# -----
# Hilfsfunktion: Funktion vektorisieren für NumPy
# -----
def vektorisiere(f: Callable[[float], float]) -> Callable[[np.ndarray], float]:
    """
    Wandelt eine skalare Funktion f(x) in eine NumPy-kompatible Funktion um,
    sodass ganze Arrays an x-Werten verarbeitet werden können.
    """
    return np.vectorize(lambda x: sicher_auswerten(f, x))

# -----
# Hauptfunktion: Numerische Integration mit drei Verfahren
# -----
def numerische_integrations(
    f: Callable[[float], float], # Die zu integrierende Funktion
    a: float, # Untere Intervallgrenze
    b: float, # Obere Intervallgrenze
    n: int, # Anzahl der Teilintervalle
    methode: str, # "Untersumme", "Obersumme", "Trapezregel"
    zeige_plot: bool = True # Optional: grafische Darstellung
) -> float:

    # ----- Vorbedingungen prüfen -----
    if n <= 0:
        raise ValueError("Die Anzahl der Unterteilungen (n) muss größer als 0 sein.")
    if a >= b:
        raise ValueError("Ungültiges Intervall: a muss kleiner als b sein.")
    if not callable(f):
        raise ValueError("Ungültiges Argument: f muss eine aufrufbare Funktion sein.")

    deltaX = (b - a) / n # Breite eines Teilintervalls
    x_i = np.linspace(a, b, n + 1) # Teilintervallgrenzen
    f_vec = vektorisiere(f) # Funktion für Array-Argumente
    summe = 0.0 # Akkumulator für das Ergebnis

    try:
        # ----- Auswahl des Integrationsverfahrens -----
        if methode == "Untersumme":
            for i in range(n):
                f_min, _ = find_extrema(f, x_i[i], x_i[i+1])
                summe += f_min * deltaX

        elif methode == "Obersumme":
            for i in range(n):
                _, f_max = find_extrema(f, x_i[i], x_i[i+1])
                summe += f_max * deltaX

        elif methode == "Trapezregel":
            y_i = f_vec(x_i) # Funktionswerte an allen x_i
            # Trapezregel-Formel: Mittelwert aus benachbarten y-Werten
            summe = deltaX * (np.sum(y_i) - 0.5 * (y_i[0] + y_i[-1]))

        else:
            raise ValueError(f"Unbekanntes Integrationsverfahren: {methode}")
    except Exception as e:
        raise ValueError(f"Fehler bei der Integration: {e}")

```



```

        raise ValueError('Ungültige Methode: Erlaubt sind "Unter- und Obersumme"')

except Exception as fehler:
    raise fehler

# ----- Optionale Visualisierung -----
if zeige_plot:
    try:
        xs = np.linspace(a, b, 1000)      # Feinere x-Werte zum
        ys = f_vec(xs)                    # Funktionswerte zum
        fig, ax = plt.subplots(figsize=(8, 5))
        ax.plot(xs, ys, label="f(x)", color="black", linewidth=2)
        ax.axhline(0, color="gray", linestyle="--")

        label_gesetzt = False # Für Legende nur ein Label setzen

        if methode == "Untersumme":
            for i in range(n):
                f_min, _ = find_extrema(f, x_i[i], x_i[i+1])
                ax.bar(x_i[i] + deltaX / 2, f_min, width=deltaX,
                       edgecolor='blue', color='blue', alpha=0.4,
                       label='Untersumme' if not label_gesetzt else None)
                label_gesetzt = True

        elif methode == "Obersumme":
            for i in range(n):
                _, f_max = find_extrema(f, x_i[i], x_i[i+1])
                ax.bar(x_i[i] + deltaX / 2, f_max, width=deltaX,
                       edgecolor='red', color='red', alpha=0.4,
                       label='Obersumme' if not label_gesetzt else None)
                label_gesetzt = True

        elif methode == "Trapezregel":
            y_i = f_vec(x_i)
            for i in range(n):
                ax.fill([x_i[i], x_i[i], x_i[i+1], x_i[i+1]],
                        [0, y_i[i], y_i[i+1], 0],
                        color='green', alpha=0.3,
                        label='Trapezregel' if not label_gesetzt else None)
                label_gesetzt = True

        ax.set_title(f"Numerische Integration mittels {methode}"
                     f"Ergebnis bei n={n} |  $\approx$  {round(summe, 5)}")
        ax.set_xlabel("x")
        ax.set_ylabel("y")
        ax.legend()
        ax.grid(True)
        plt.tight_layout()
        plt.show()

    except Exception as err:
        print(f"Warnung beim Zeichnen der Funktion: {err}")

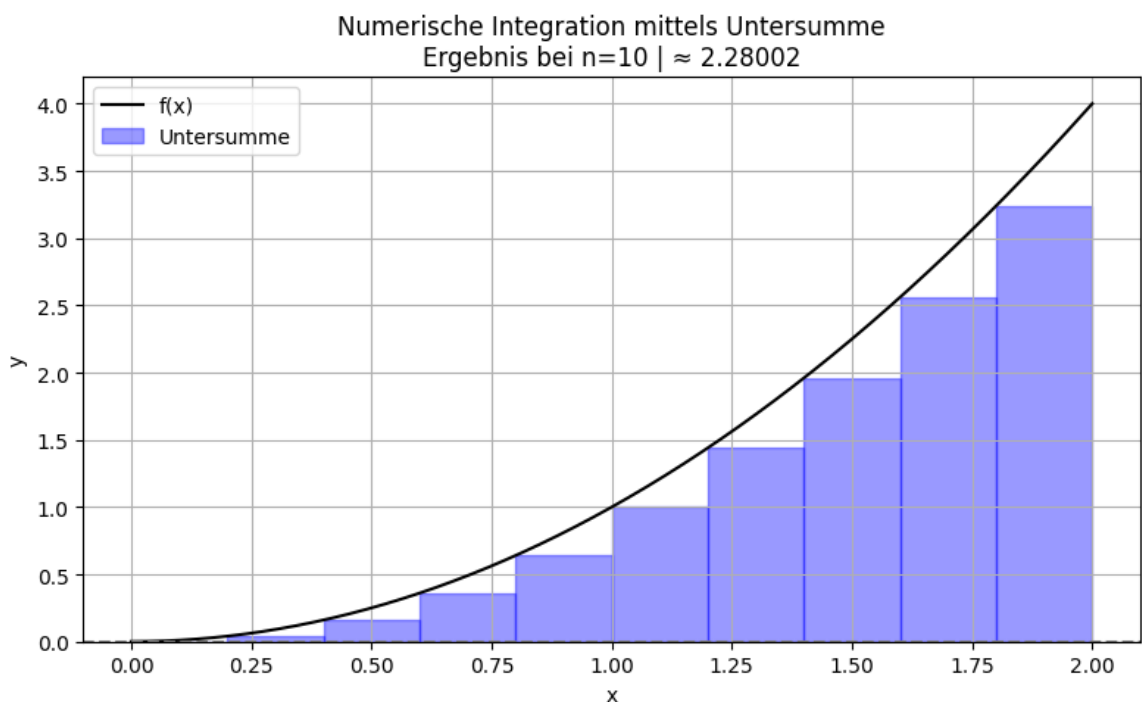
# Rückgabe des gerundeten Integralergebnisses
return round(summe, 3)

```

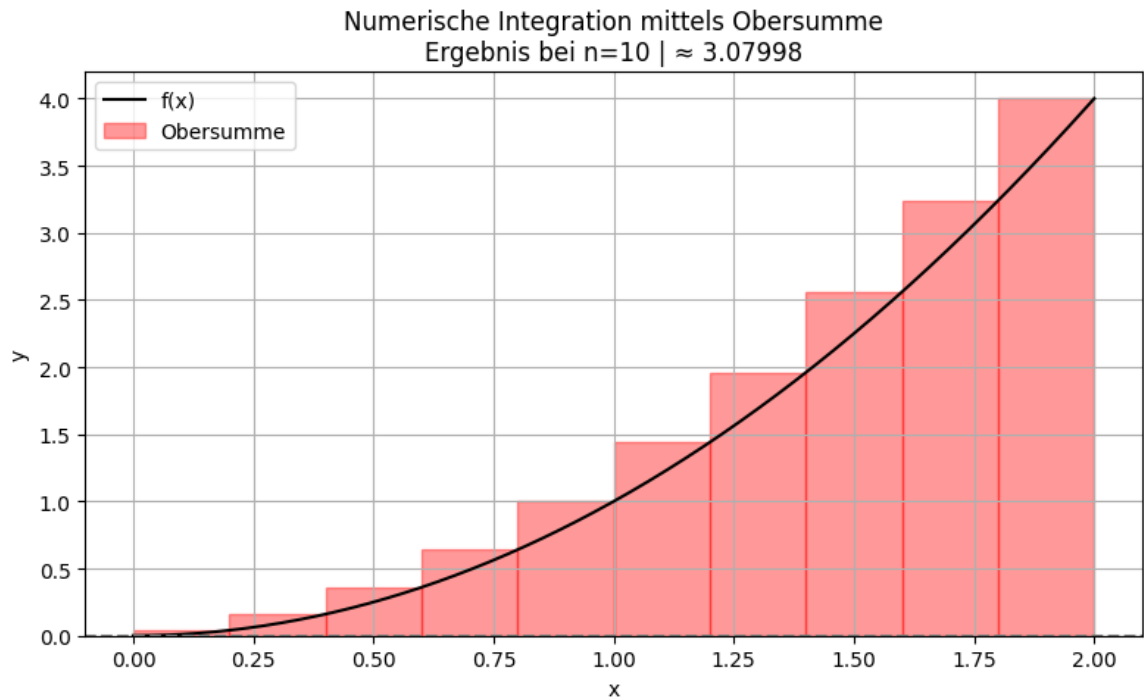
```
In [31]: # Beispiel 1: Untersumme für  $f(x) = x^2$  im Intervall  $[0, 2]$  mit 10 Teilintervallen
# Erwartung: Das Ergebnis liegt unter dem exakten Wert (da Untersumme)
ergebnis_unter = numerische_integrations(
    lambda x: x**2,      # Funktion  $f(x) = x^2$ 
    a=0,                 # Untere Grenze des Intervalls
    b=2,                 # Obere Grenze des Intervalls
    n=10,                # Anzahl der Teilintervalle
    methode="Untersumme" # Integrationsmethode: Untersumme
)
print("Untersumme Ergebnis:", ergebnis_unter)

# Beispiel 2: Obersumme für  $f(x) = x^2$  im Intervall  $[0, 2]$  mit 10 Teilintervallen
# Erwartung: Das Ergebnis liegt über dem exakten Wert (da Obersumme)
ergebnis_ober = numerische_integrations(
    lambda x: x**2,      # Funktion  $f(x) = x^2$ 
    a=0,                 # Untere Grenze des Intervalls
    b=2,                 # Obere Grenze des Intervalls
    n=10,                # Anzahl der Teilintervalle
    methode="Obersumme" # Integrationsmethode: Obersumme
)
print("Obersumme Ergebnis:", ergebnis_ober)

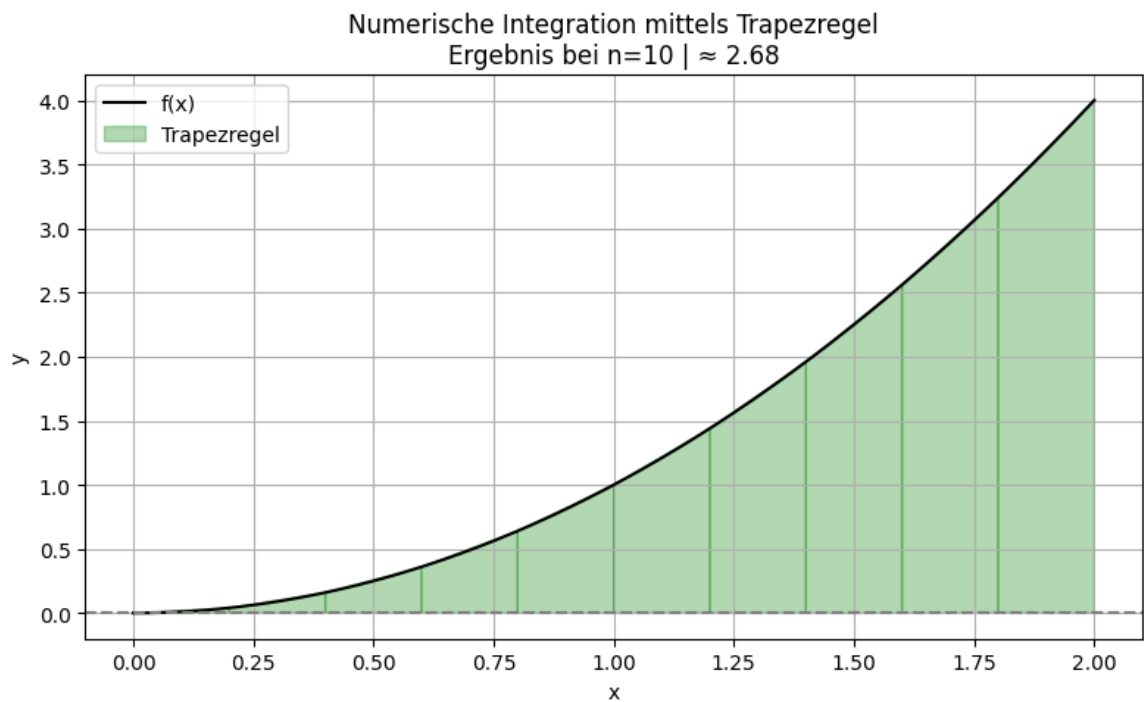
# Beispiel 3: Trapezregel für  $f(x) = x^2$  im Intervall  $[0, 2]$  mit 10 Teilintervallen
# Erwartung: Sehr gute Approximation des exakten Wertes ( $\int_0^2 x^2 dx = \frac{8}{3} \approx 2.6667$ )
ergebnis_trapez = numerische_integrations(
    lambda x: x**2,      # Funktion  $f(x) = x^2$ 
    a=0,                 # Untere Grenze des Intervalls
    b=2,                 # Obere Grenze des Intervalls
    n=10,                # Anzahl der Teilintervalle
    methode="Trapezregel" # Integrationsmethode: Trapezregel
)
print("Trapezregel Ergebnis:", ergebnis_trapez)
```



Untersumme Ergebnis: 2.28



Obersumme Ergebnis: 3.08



Trapezregel Ergebnis: 2.68

Vergleich numerischer Integrationsmethoden am Beispiel von $f(x) = x^2$

Zur numerischen Berechnung des Integrals

$$\int_0^2 x^2 \, dx = \frac{8}{3} \approx 2.6667$$

wurden drei klassische Verfahren verglichen: Untersumme, Obersumme und

Trapezregel.

Die Berechnungen wurden mit `n = 10` gleich breiten Intervallen auf dem Intervall $[0, 2]$ durchgeführt.

Untersumme

```
ergebnis_unter = numerische_integrations(lambda x: x**2, a=0,
b=2, n=10, methode="Untersumme")
```

- Die **Untersumme** verwendet den kleinsten Funktionswert in jedem Teilintervall.
 - Dadurch entsteht eine **systematische Unterschätzung** der Fläche, da x^2 streng monoton wächst.
 - Ergebnis: ≈ 2.280
-

Obersumme

```
ergebnis_ober = numerische_integrations(lambda x: x**2, a=0,
b=2, n=10, methode="Obersumme")
```

- Die **Obersumme** verwendet den größten Funktionswert in jedem Teilintervall.
 - Dadurch wird die Fläche **systematisch überschätzt**.
 - Ergebnis: ≈ 3.080
-

Trapezregel

```
ergebnis_trapez = numerische_integrations(lambda x: x**2,
a=0, b=2, n=10, methode="Trapezregel")
```

- Die **Trapezregel** verwendet den Mittelwert der linken und rechten Randpunkte pro Intervall.
 - Sie liefert in vielen Fällen eine **bessere Näherung** als Ober- oder Untersumme.
 - Ergebnis: ≈ 2.68
-

Fazit

Methode	Ergebnis	Tendenz
Untersumme	≈ 2.28	Unterschätzung

Obersumme	≈ 3.08	Überschätzung
Trapezregel	≈ 2.68	Gute Annäherung

Je feiner die Zerlegung (größeres n), desto genauer wird das Ergebnis bei allen Methoden – aber besonders die Trapezregel konvergiert schneller zum exakten Wert.

Kapitel 2: Monte-Carlo-Integration

Die Monte-Carlo-Integration ist ein numerisches Verfahren zur Berechnung von Integralen, das auf der Verwendung von Zufallszahlen basiert. Sie wird besonders dann eingesetzt, wenn klassische Methoden (wie Trapezregel oder Simpsonregel) aufgrund hoher Dimensionalität oder komplexer Integrationsbereiche nicht praktikabel sind.

Für ein eindimensionales Integral nutzt man n Zufallszahlen aus dem Intervall $[a, b]$.

Für eine explorative Methode benötigt man zusätzlich eine Schätzung der minimalen und maximalen Funktionswerte.

Erklärung der Funktion `monte_carlo_integration`

Diese Funktion dient zur numerischen Approximation eines bestimmten Integrals mittels der **Monte-Carlo-Methode**.

Funktionssignatur

```
monte_carlo_integration(func, a, b, n_punkte=10000,  
zeige_plot=True,  
                        explorativ=True, y_unter=None,  
y_ober=None)
```

Parameter:

Parameter	Typ	Beschreibung
<code>func</code>	Funktion	Die zu integrierende Funktion ($f(x)$).
<code>a</code> , <code>b</code>	float	Untere und obere Grenze des Integrals.
<code>n_punkte</code>	int	Anzahl der Zufallspunkte.
<code>zeige_plot</code>	bool	Ob ein Plot generiert werden soll.

<code>explorativ</code>	<code>bool</code>	Ob die y-Grenzen automatisch bestimmt werden.
<code>y_unter , y_oben</code>	<code>float</code>	Falls <code>explorativ=False</code> , müssen diese explizit gesetzt werden.

Grundidee

Die Fläche unter der Kurve wird geschätzt, indem zufällige Punkte innerhalb eines Rechtecks im Bereich

$[a, b] \times [y_{\text{unter}}, y_{\text{ober}}]$ erzeugt werden.

- **Treffer:** Punkte, die im „richtigen“ Bereich zwischen x-Achse und Funktionsgraph liegen:
 - bei positiver Funktion: zwischen 0 und $f(x)$,
 - bei negativer Funktion: zwischen $f(x)$ und 0.
- **Nettofläche:** Die Differenz zwischen positiven und negativen Treffern, gewichtet mit der Rechtecksfläche.

Ablauf

1. Grenzen validieren:

Prüft, ob $(a < b)$ und (falls notwendig) ob `y_unter` und `y_oben` sinnvoll gesetzt sind.

2. y-Grenzen setzen:

- **Explorativ** (`explorativ=True`):
Die Funktion `find_extrema` berechnet y_{min} und y_{max} auf dem Intervall.
 - Dann wird das Rechteck so gewählt, dass es die Funktion vollständig abdeckt.
- **Manuell** (`explorativ=False`):
Die Grenzen müssen explizit angegeben werden.

3. Zufallspunkte generieren:

- Punkte werden zufällig im Rechteck gezogen.
- Jeder Punkt wird geprüft:
 - Liegt er zwischen Funktionsgraph und x-Achse? → zähle als (positiven oder negativen) Treffer.
 - Sonst: Verfehlt.

4. Integral berechnen: $\int = \text{Fläche Rechteck} \cdot \left(\frac{\text{Netto-Treffer}}{\text{Anzahl Punkte}} \right)$

5. Visualisierung (optional):

- Funktionsverlauf
- Grüne Punkte: Treffer
- Rote Punkte: Verfehlt

find_extrema – Hilfsfunktion

Diese Funktion dient der explorativen Bestimmung der y-Grenzen, indem sie die Funktion auf dem Intervall [a,b] abtastet.

```
def find_extrema(func, a, b, n=1000):
    xs = np.linspace(a, b, n)
    ys = func(xs)
    return np.nanmin(ys), np.nanmax(ys)
```

Theoretisch-mathematischer Hintergrund

Ziel

Approximieren eines bestimmten Integrals:

$$I = \int_a^b f(x) \, dx$$

Grundprinzip der Monte-Carlo-Integration

- Ziehe n Zufallszahlen $x_i \sim \text{Uniform}(a,b)$
- Berechne:

$$\bar{f} = \frac{1}{n} \sum_{i=1}^n f(x_i) \quad \Rightarrow \quad \int_a^b f(x) \, dx \approx (b-a) \cdot \bar{f}$$

Diese Methode liefert einen **erwartungstreuen Schätzer**, konvergiert aber nur langsam: $\text{Fehler} \sim \frac{1}{\sqrt{n}}$

Varianten von Monte-Carlo-Methoden

Methode	Beschreibung
Einfache Monte-Carlo	Gleichmäßiges Zufallssampling im Intervall.
Monte-Carlo mit Rechteck	Zufällige Punkte in 2D-Rechteck $[a,b] \times [y_{\text{unter}}, y_{\text{ober}}]$. Punkte unter der Kurve = Fläche.

Importance Sampling	Punkte werden nach einer Wahrscheinlichkeitsverteilung gewählt, die zur Funktion passt → geringere Varianz.
Markov Chain Monte Carlo	Nutzt Markov-Ketten zur Stichprobenerzeugung aus komplexen Verteilungen. Häufig in der Statistik.
Stratifiziertes Sampling	Der Raum wird in Teilbereiche unterteilt, um gleichmäßigere Abdeckung zu gewährleisten.
Quasi-Monte-Carlo	Verwendet deterministische Low-Discrepancy-Folgen (z. B. Sobol, Halton), um bessere Konvergenz zu erreichen.

Umsetzung unserer Methode

- Wir kombinieren die **2D-Rechteck-Methode** mit einer **explorativen Bestimmung** der y-Grenzen.
- Dadurch können **auch Funktionen mit negativen Werten** korrekt integriert werden.
- Das Sampling erfolgt vollständig zufällig innerhalb eines Rechtecks, das die Kurve nach oben und unten einschließt.

Beispiel

```
import numpy as np

def f(x):
    return np.sin(x)

wert = monte_carlo_integration(f, a=0, b=2*np.pi,
n_punkte=10000, explorativ=True)
print("Monte-Carlo-Ergebnis:", wert)
```

Vorteile

- Sehr **robust**, auch bei unstetigen oder unglatten Funktionen.
- **Intuitiv** und einfach visuell nachvollziehbar.
- Kann auch mit **negativen Funktionswerten** umgehen.

Nachteile

- **Langsame Konvergenz**: Fehler sinkt nur mit $\frac{1}{\sqrt{n}}$
- Genaue Schätzung abhängig von **Rechteckwahl** – schlechtes Rechteck → schlechte Effizienz.
- Wird bei uns durch explorative Methode gelöst

- Für hohe Genauigkeit große Punktzahl nötig.

```
In [32]: import numpy as np
import matplotlib.pyplot as plt

# -----
# Hauptfunktion: Numerische Integration mittels Monte-Carlo-Verfahren
# -----
def monte_carlo_integration(func, a, b, n_punkte=10000, zeige_plot=False,
                             explorativ=True, y_unter=None, y_oben=None):
    """
    Schätzt das Integral der Funktion `func` im Intervall [a, b] mit
    Monte-Carlo-Verfahren.

    Parameter:
        func      - die zu integrierende Funktion
        a, b      - untere und obere Intervallgrenze
        n_punkte  - Anzahl zufälliger Punkte für die Schätzung
        zeige_plot - ob ein Plot erzeugt wird
        explorativ - automatische Bestimmung von y_min und y_max (True)
        y_unter   - untere y-Grenze (nur bei explorativ=False)
        y_oben    - obere y-Grenze (nur bei explorativ=False)

    Rückgabe:
        Geschätzter Integralwert
    """

    # Sicherstellen, dass das x-Intervall korrekt definiert ist
    if a >= b:
        raise ValueError("Untere Grenze a muss kleiner als obere Grenze b sein.")

    # -----
    # y-Grenzen bestimmen (explorativ oder manuell)
    # -----
    if explorativ:
        # Bestimme automatisch min und max der Funktion im Intervall
        f_min, f_max = find_extrema(func, a, b)

        # Überprüfe auf gültige Funktionswerte
        if not np.isfinite(f_min) or not np.isfinite(f_max):
            raise ValueError("Funktion liefert ungültige Extremwerte.")

        # Begrenze auf die Fläche zwischen Funktion und x-Achse
        y_unter = min(f_min, 0)
        y_oben = max(f_max, 0)
    else:
        # Manuelle Festlegung der y-Grenzen erforderlich
        if y_unter is None or y_oben is None:
            raise ValueError("Bei explorativ=False müssen y_unter und y_oben angegeben sein.")
        if y_unter >= y_oben:
            raise ValueError("y_unter muss kleiner als y_oben sein.")

    # Wenn keine Fläche vorhanden ist, gibt es nichts zu integrieren
    if y_unter == y_oben:
        return 0.0
```

```

# -----
# Vorbereitung für Visualisierung
# -----
x_treffer, y_treffer = [], [] # Punkte unter der Kurve
x_verfehlt, y_verfehlt = [], [] # Punkte außerhalb der Kurve

netto_treffer = 0 # Netto-Treffer: positive minus negative Fläche

# -----
# Monte-Carlo-Simulation
# -----
for _ in range(n_punkte):
    # Zufällige Punktwahl im Rechteck [a, b] × [y_unter, y_ober]
    x_rand = np.random.uniform(a, b)
    y_rand = np.random.uniform(y_unter, y_ober)

    try:
        y_func = func(x_rand)
        if not np.isfinite(y_func):
            continue # Ignoriere ungültige Funktionswerte
    except:
        continue # Ignoriere Fehlerhafte Auswertung

    # Fall 1: Punkt liegt im positiven Bereich unterhalb der Funktion
    if y_rand >= 0 and 0 <= y_rand <= y_func:
        netto_treffer += 1
        x_treffer.append(x_rand)
        y_treffer.append(y_rand)

    # Fall 2: Punkt liegt im negativen Bereich oberhalb der Funktion
    elif y_rand < 0 and y_func <= y_rand <= 0:
        netto_treffer -= 1
        x_treffer.append(x_rand)
        y_treffer.append(y_rand)

    # Fall 3: Punkt liegt außerhalb der Funktion → zählt nicht
    else:
        x_verfehlt.append(x_rand)
        y_verfehlt.append(y_rand)

# -----
# Integral berechnen
# -----
rechteck_flaeche = (b - a) * (y_ober - y_unter)
integral = rechteck_flaeche * (netto_treffer / n_punkte)

# -----
# Ergebnis-Visualisierung
# -----
if zeige_plot:
    plt.figure(figsize=(8, 5))
    xs = np.linspace(a, b, 500)
    ys = np.array([func(x) for x in xs])

    plt.plot(xs, ys, label='f(x)', color='black')

```

```

plt.axhline(0, color='gray', linestyle='--') # x-Achse
plt.scatter(x_treffer, y_treffer, s=1, color='green', label='Treffer')
plt.scatter(x_verfehlt, y_verfehlt, s=1, color='red', label='Verfehlt')

plt.title(f"Numerische Integration mittels Monte-Carlo\n"
          f"Ergebnis bei n={n_punkte} |  $\approx$  {integral:.5f}")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid(True)
plt.show()

return integral

# -----
# Hilfsfunktion zur Extremwertbestimmung im Intervall
# -----
def find_extrema(func, a, b, n=1000):
    """
    Näherungsweise Bestimmung von Minimum und Maximum der Funktion

    Rückgabe: (f_min, f_max)
    """
    xs = np.linspace(a, b, n)
    try:
        ys = func(xs) # Versuche direkte Array-Auswertung
        ys = np.where(np.isfinite(ys), ys, np.nan)
    except:
        # Fallback: Einzelne Punkte berechnen
        ys = np.array([func(x) if np.isfinite(func(x)) else np.nan
                       for x in xs])

    return np.nanmin(ys), np.nanmax(ys)

```

```

In [33]: import numpy as np

# Zu integrierende Funktion: f(x) = sin(x)
def f(x):
    return np.sin(x)

# -----
# Variante 1: Manuelle y-Grenzen (explorativ=False)
# -----
# Hier wird das Rechteck manuell auf y ∈ [-0.1, 0.5] gesetzt.
# Achtung: Diese Grenzen decken nicht den gesamten Verlauf von sin(x)
# denn sin(x) ∈ [-1, 1] auf [0, 2π].
# Dadurch werden viele Punkte außerhalb des wahren Funktionsbereichs
# was die Genauigkeit erheblich verschlechtert.
wert = monte_carlo_integration(
    f,                                # Funktion: sin(x)
    a=0,                             # Untere Grenze des Intervalls
    b=2*np.pi,                      # Obere Grenze des Intervalls (≈ 2π)
    n_punkte=10000,                 # Anzahl zufälliger Punkte
    zeige_plot=True,                # Visualisierung aktivieren
    explorativ=False,               # Manuelle y-Grenzen verwenden
    y_unter=-0.1,                   # Untere y-Grenze (zu klein!)
    y_ober=0.5                      # Obere y-Grenze (zu klein!)
)

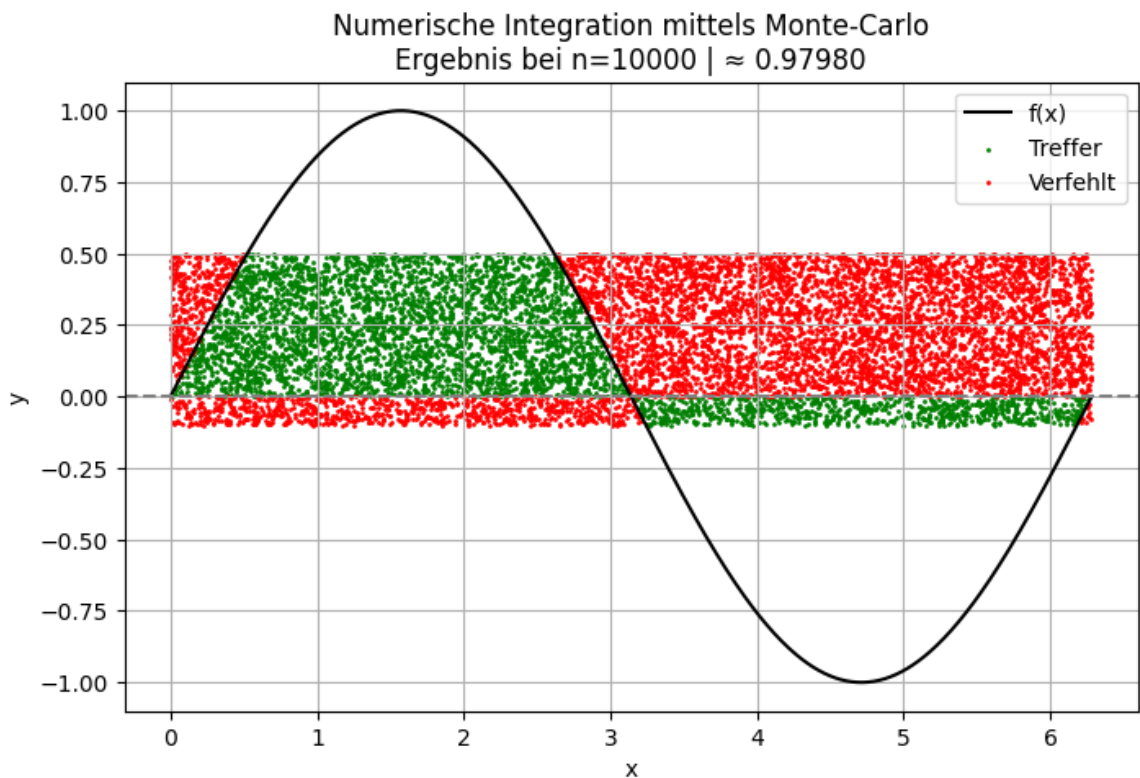
```

```

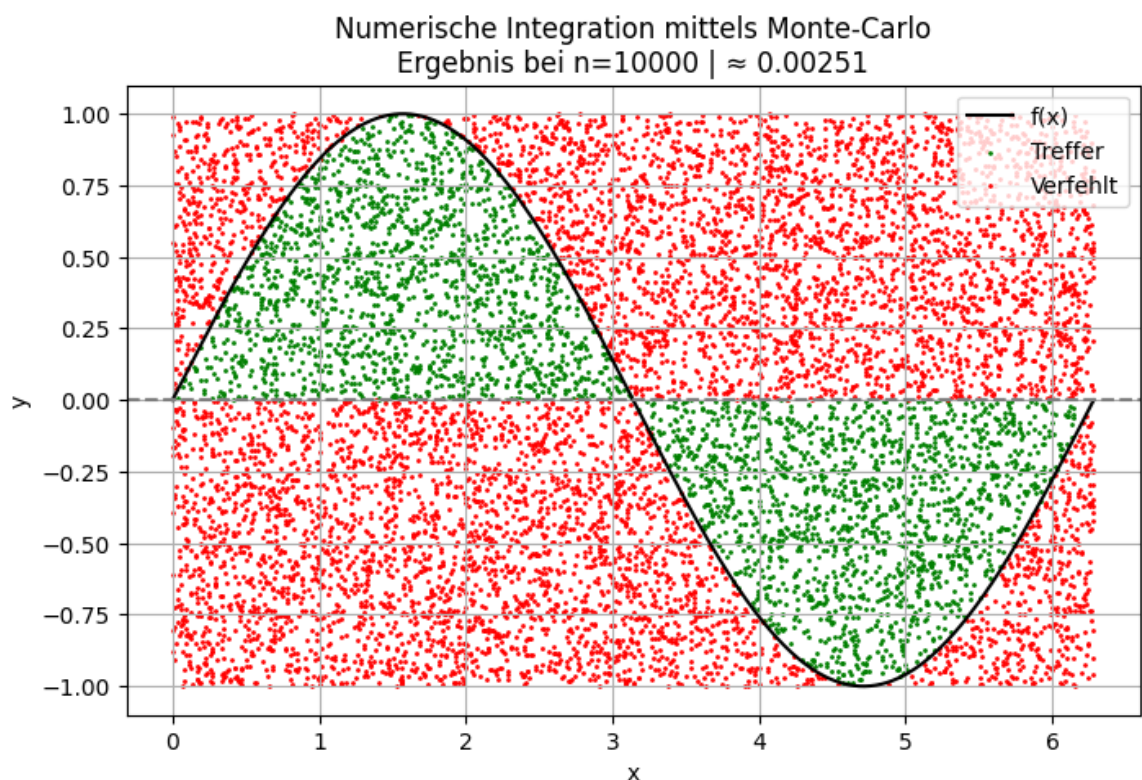
)
print("Monte-Carlo-Ergebnis (manuelle y-Grenzen):", wert)

# -----
# Variante 2: Automatische Grenzen (explorativ=True)
# -----
# Hier werden die y-Grenzen automatisch anhand der Funktionswerte b
# Das Rechteck wird exakt so gewählt, dass es die Fläche zwischen F
# Diese Variante ist **empfohlen**, da sie robust und korrekt ist –
wert = monte_carlo_integration(
    f,                                # Funktion: sin(x)
    a=0,                             # Untere Grenze des Intervalls
    b=2*np.pi,                      # Obere Grenze des Intervalls
    n_punkte=10000,                 # Anzahl der Zufallspunkte
    zeige_plot=True,                # Plot anzeigen
    explorativ=True                  # y-Grenzen automatisch bestimmen
)
print("Monte-Carlo-Ergebnis (automatisch):", wert)

```



Monte-Carlo-Ergebnis (manuelle y-Grenzen): 0.9797999168015847



Monte-Carlo-Ergebnis (automatisch): 0.002513271016034237

Numerische Integration mittels Monte-Carlo

Die Monte-Carlo-Integration ist ein probabilistisches Verfahren zur Approximation von Integralen. Sie funktioniert besonders gut bei komplizierten oder schwer analytisch integrierbaren Funktionen.

Im Folgenden vergleichen wir zwei Varianten der Monte-Carlo-Integration:

Variante 1: Feste y-Grenzen

Ergebnis:

Im oberen Plot (Bild 1) sieht man:

- Es wurden **feste y-Grenzen** vorgegeben: $y_{\text{unter}} = -0.1$, $y_{\text{ober}} = 0.5$.
 - Das ergibt ein inkorrektes Rechteck für die Fläche, die von der Funktion ($f(x) = \sin(x)$) und der x-Achse eingeschlossen wird.
 - Das berechnete Integral entspricht **nicht** dem tatsächlichen Flächeninhalt unter der Funktion, da der negative Anteil ignoriert wird.
-

Variante 2: Automatische y-Grenzen

Ergebnis:

Im unteren Plot (Bild 2) sieht man:

- Die y-Grenzen wurden **automatisch bestimmt**, indem man das **Minimum und Maximum der Funktion im gegebenen Intervall** berechnet hat.
- Dabei wird folgendes beachtet:
 - Wenn $\max(f(x)) < 0$, dann wird $y_{\text{ober}} = 0$ gesetzt.
 - Wenn $\min(f(x)) > 0$, dann wird $y_{\text{unter}} = 0$ gesetzt.
- So wird stets die Fläche **zwischen Funktion und x-Achse** korrekt integriert – **positive und negative Flächenanteile** eingeschlossen.

Das berechnete Ergebnis entspricht hier wesentlich genauer dem exakten Wert des bestimmten Integrals von ($\int_0^{2\pi} \sin(x) \, dx = 0$).

Fazit

- Feste y-Grenzen führen zu falschen Integralergebnissen, wenn sie nicht exakt zur Funktion passen.
 - Die explorative Methode liefert korrekte Ergebnisse, indem sie die **Integrationsfläche dynamisch an die Funktion anpasst**.
 - Für allgemeine Anwendungen und unbekannte Funktionsverläufe sollte immer **explorativ=True** verwendet werden.
-

Kapitel 3: Testfälle und Evaluation

Auswertung numerischer Integrationsverfahren – Erklärung

Diese Datei beschreibt die Funktionsweise und den Ablauf des Python-Skripts `auswertung_durchfuehren`, das zur Analyse verschiedener numerischer Integrationsverfahren dient.

Ziel der Funktion

Die Funktion `auswertung_durchfuehren(...)` dient dem systematischen Vergleich von vier numerischen Verfahren zur Integralapproximation:

- **Untersumme**
- **Obersumme**
- **Trapezregel**
- **Monte-Carlo-Integration**

Diese Methoden werden auf verschiedene Funktionen angewendet. Optional kann zusätzlich der exakte Wert (über eine bekannte Stammfunktion) zur Referenz berechnet werden.

Funktionsparameter

```
def auswertung_durchfuehren(  
    nutze_feste_grenzen: bool,  
    feste_a: float,  
    feste_b: float,  
    moFaktor: int,  
    max_n: int,  
    testfunktionen: List[Tuple[str, Callable[[float],  
float], float, float, Callable[[float], float]]],  
)
```

Parameter	Beschreibung
<code>nutze_feste_grenzen</code>	Wenn <code>True</code> , werden <code>feste_a</code> und <code>feste_b</code> für alle Funktionen erzwungen.
<code>feste_a</code> , <code>feste_b</code>	Intervallgrenzen, falls <code>nutze_feste_grenzen=True</code> .
<code>moFaktor</code>	Faktor für Anzahl der Monte-Carlo-Stichproben: <code>n_punkte = n * moFaktor</code> .
<code>max_n</code>	Maximale Anzahl Teilintervalle <code>n</code> (von 1 bis <code>max_n</code>).
<code>testfunktionen</code>	Liste von Funktionen und ihrer Metadaten: Name, Funktion, a, b, Stammfunktion

Ablauf

1. **Initialisierung:** Farben für Plot-Stile und Fehlerliste werden gesetzt.
 2. **Iteration über alle Testfunktionen:**
 - Intervall anpassen (feste Grenzen oder eigene).
 - Exakter Wert berechnen (sofern Stammfunktion bekannt).
 - Schleife über `n = 1` bis `max_n` :
 - Jede Methode wird separat ausgeführt.
 - Fehlerbehandlung ist integriert, um trotz Singularitäten o.Ä. fortzusetzen.
 - Ergebnisse werden geplottet (nur bei `n == 20` erfolgt ggf. Detailplot).
 - Gesamter Vergleichsplot wird erzeugt, inkl. exaktem Wert als Referenz.
 3. **Fehlerausgabe:** Falls alle Verfahren für eine Funktion fehlschlagen, wird dies am Ende gemeldet.
-

Plot

Die Plots enthalten:

- Kurven für jede Methode mit untersuchten `n`.
 - Horizontaler Strich für exakten Wert (falls vorhanden).
 - Automatisch gewählter y-Achsenbereich.
 - Fehlerhafte oder nicht berechenbare Werte werden durch `NaN` ersetzt.
-

Fehlerbehandlung

Die Funktion bricht nicht vollständig ab, sondern merkt sich:

- Welche Methode bei welchem `n` fehlschlug.
- Welche Funktion komplett unbrauchbar war (z. B. Division durch 0 im gesamten Intervall).

Diese Informationen werden am Ende ausgegeben.

Besonderheiten

- Die Monte-Carlo-Methode arbeitet mit `explorativ=True`, was bedeutet, dass die y-Grenzen aus der Funktion abgeleitet werden.
- Plots werden bei `n=20` zusätzlich zur Visualisierung erzeugt, sofern

gewünscht.

- Farblich einheitliche Darstellung für bessere Vergleichbarkeit.

Ergebnis

Am Ende erhält man:

- Vergleichsplots für jede Funktion
- Automatische Fehlerdokumentation
- Aussagekraft über Konvergenzverhalten der numerischen Methoden

```
In [34]: from typing import Callable, List, Tuple
import math
import matplotlib.pyplot as plt
import numpy as np

# -----
# Hauptfunktion zur Auswertung numerischer Integrationsverfahren
# -----
def auswertung_durchfuehren(
    nutze_feste_grenzen: bool, # True = für alle Funktionen dieselbe
    feste_a: float,           # Feste untere Intervallgrenze
    feste_b: float,           # Feste obere Intervallgrenze
    moFaktor: int,             # Multiplikator für die Anzahl Monte
    max_n: int,                # Maximale Anzahl von Teilintervallen
    testfunktionen: List[Tuple[str, Callable[[float], float], float],
):
    # Farbdefinitionen für die Diagramm-Linien
    farben = {
        "Untersumme": "#f94144",
        "Obersumme": "#f3722c",
        "Trapezregel": "#90be6d",
        "Exakt": "#577590",
        "Montecarlo": "#277da1"
    }

    fehlgeschlagene_funktionen = [] # Sammelnde Liste für Funktionen, die

    # -----
    # Iteration über alle Testfunktionen
    # -----
    for name, f, a, b, F in testfunktionen:
        # Wähle je nach Einstellung feste oder funktionspezifische
        a_aktuell, b_aktuell = (feste_a, feste_b) if nutze_feste_grenzen else (a, b)

        Ns = list(range(1, max_n+1)) # Liste der zu testenden Teilintervallen
        undersummen, obersummen, trapezwerte, montecarlo = [], [], [], []
        funktion_ist_gefaillt = False
        fehler_begrueendung = ""

        # Versuche exakten Wert über Stammfunktion (falls vorhanden)
```

```

try:
    exaktwert = None if F is None else F(b_aktuell) - F(a_aktuell)
except Exception as e:
    exaktwert = None
    funktion_ist_gefailt = True
    fehler_begrueundung = f"Fehler bei exakter Berechnung: {e}"

# -----
# Berechnung für verschiedene n
# -----
for n in Ns:
    zeige_plot = (n == 20) # Optional: Detaillierter Einze

    # Untersumme
    try:
        us = numerische_integrations(f, a_aktuell, b_aktuell, n)
    except Exception as e:
        us = None
        funktion_ist_gefailt = True
        if not fehler_begrueundung:
            fehler_begrueundung = f"Fehler bei Untersumme (n={n}): {e}"

    # Obersumme
    try:
        os = numerische_integrations(f, a_aktuell, b_aktuell, n)
    except Exception as e:
        os = None
        funktion_ist_gefailt = True
        if not fehler_begrueundung:
            fehler_begrueundung = f"Fehler bei Obersumme (n={n}): {e}"

    # Trapezregel
    try:
        tr = numerische_integrations(f, a_aktuell, b_aktuell, n)
    except Exception as e:
        tr = None
        funktion_ist_gefailt = True
        if not fehler_begrueundung:
            fehler_begrueundung = f"Fehler bei Trapezregel (n={n}): {e}"

    # Monte-Carlo-Integration
    try:
        mo = monte_carlo_integrations(f, a_aktuell, b_aktuell, n)
    except Exception as e:
        mo = None
        funktion_ist_gefailt = True
        if not fehler_begrueundung:
            fehler_begrueundung = f"Fehler bei Monte-Carlo (n={n}): {e}"

    # Ergebnisse speichern
    untersummen.append(us)
    obersummen.append(os)
    trapezwerte.append(tr)
    montecarlo.append(mo)

# -----

```

```

# Prüfung: Hat mindestens eine Methode funktioniert?
# -----
mindestens_ein_wert = any(v is not None for v in untersummen)
if not mindestens_ein_wert:
    fehlgeschlagene_funktionen.append((name, fehler_begruehung))
    continue

# -----
# Visualisierung des Vergleichs
# -----
plt.figure(figsize=(9, 5))

# Hilfsfunktion: Ersetzt None durch NaN, damit matplotlib kl
def safe_plot(x, y, label, color):
    y_valid = [v if v is not None else np.nan for v in y]
    plt.plot(x, y_valid, label=label, color=color)

# Plote alle vier Methoden
safe_plot(Ns, untersummen, "Untersumme", farben["Untersumme"])
safe_plot(Ns, obersummen, "Obersumme", farben["Obersumme"])
safe_plot(Ns, trapezwerte, "Trapezregel", farben["Trapezregel"])
safe_plot(Ns, montecarlo, f"Monte-Carlo [n*{moFaktor}]", farben["Monte-Carlo"])

# Plote exakten Wert, falls vorhanden
if exaktwert is not None:
    plt.hlines(exaktwert, min(Ns), max(Ns), label="Exakter Wert", color="black")

# Beschriftung und Design
plt.title(f"{name} auf [{a_aktuell}, {b_aktuell}]")
plt.xlabel("Anzahl Teilintervalle (n)")
plt.ylabel("Flächeninhalt")
plt.legend()
plt.grid(True, linestyle="--", alpha=0.5)

# Dynamische y-Achsen-Skalierung basierend auf gültigen Werten
y_werte = [v for v in untersummen + obersummen + trapezwerte + montecarlo]
if y_werte:
    y_min, y_max = min(y_werte), max(y_werte)
    plt.ylim(y_min - 0.1 * abs(y_max), y_max + 0.1 * abs(y_max))

# Exakten Wert als Text einblenden
if exaktwert is not None:
    plt.text(max(Ns), exaktwert, f"{exaktwert:.5f}", color="black",
             fontsize=10, ha="right", va="bottom",
             bbox=dict(facecolor='white', alpha=0.6, edgecolor='black'))

plt.tight_layout()
plt.show()

# -----
# Zusammenfassung fehlerhafter Funktionen
# -----
if fehlgeschlagene_funktionen:
    print("\nFehlgeschlagene Funktionen:")
    for name, grund in fehlgeschlagene_funktionen:
        print(f"- {name}: {grund}")

```

```
else:
    print("\nAlle Funktionen erfolgreich geplottet.")
```

Testfunktionen

Nun definieren wir die Testfunktionen, sowie das zu integrierende Intervall im Modus ohne feste Grenzen und die Stammfunktion, zur exakten Bestimmung des Integrals

```
In [35]: # Testfunktionen mit analytischer Stammfunktion (wenn bekannt)
testfunktionen = [
    # -----
    # Spezielle und interessante Fälle zur numerischen Integration
    # -----

    # Parabel, symmetrisch, negativ → einfache Stammfunktion
    ("f(x) = -x2", lambda x: -x**2, -2, 2, lambda x: -(1/3) * x**3)

    # Definitionslücke bei x = 1 → nicht integrierbar in [0.5, 1.5]
    ("f(x) = 1/(x - 1)", lambda x: 1 / (x - 1), 0.5, 1.5, None),

    # Logarithmische Stammfunktion → Standardbeispiel
    ("f(x) = 1/x", lambda x: 1/x, 1, 2, lambda x: math.log(x)),

    # Wurzelfunktion mit Problemstelle bei 0 → beginnt knapp darüber
    ("f(x) = 1/√x", lambda x: 1 / math.sqrt(x), 0.0001, 1, lambda x:

    # Lineare Funktion → einfache analytische Lösung
    ("f(x) = 2x + 1", lambda x: 2 * x + 1, 0, 1, lambda x: x**2 + x

    # Konstante Funktion → Rechteckfläche
    ("f(x) = 5", lambda x: 5, 2, 6, lambda x: 5 * x),

    # Kosinusfunktion → glatter Verlauf, Stammfunktion: Sinus
    ("f(x) = cos(x)", math.cos, 0, math.pi, lambda x: math.sin(x)),

    # Kosinus2 mit trigonometrischer Stammfunktion
    ("f(x) = cos2(x)", lambda x: math.cos(x)**2, 0, math.pi,
     lambda x: (x / 2) + (math.sin(2 * x) / 4)),

    # ex: exponentielles Wachstum
    ("f(x) = ex", math.exp, 0, 1, lambda x: math.exp(x)),

    # ex auf großem Intervall → testet Genauigkeit bei starkem Wac
    ("f(x) = ex (erweitert)", math.exp, 0, 10, lambda x: math.exp(x

    # Gaußsche Fehlerfunktion → nicht elementar integrierbar
    ("f(x) = erf(x)", math.erf, -2, 2, None),

    # ln(x): unendlicher Anstieg bei x → ∞, stabile Stammfunktion
    ("f(x) = ln(x)", math.log, 1, 2, lambda x: x * math.log(x) - x)

    # Vershobener Logarithmus → Definitionsgrenze bei x = 2
    ("f(x) = ln(x - 2)", lambda x: math.log(x - 2), 3, 4,
```

```

    lambda x: (x - 2) * math.log(x - 2) - (x - 2)),

# ln(x) auf großem Intervall → testet langsames Wachstum
("f(x) = ln(x) auf großem Intervall", math.log, 1, 1000,
 lambda x: x * math.log(x) - x),

# Sinus auf [0, π] → positive Fläche, glatt
("f(x) = sin(x)", math.sin, 0, math.pi, lambda x: -math.cos(x))

# Sinus auf voller Periode → Ergebnis = 0, wichtiges Testbeispiel
("f(x) = sin(x) (vollständig)", math.sin, 0, 2 * math.pi, lambda x: 0),

# Sinus² auf [0, π] → wie cos², mit periodischer Stammfunktion
("f(x) = sin²(x)", lambda x: math.sin(x)**2, 0, math.pi,
 lambda x: (x / 2) - (math.sin(2 * x) / 4)),

# Hochfrequent: 16 Perioden auf [0, 1] → testet feine Methoden
("f(x) = sin((x + 1) * 16π)", lambda x: math.sin((x + 1) * 16 * math.pi),
 lambda x: -1 / (16 * math.pi) * math.cos((x + 1) * 16 * math.pi)),

# Sehr hochfrequent → numerisch anspruchsvoll
("f(x) = sin(100x)", lambda x: math.sin(100 * x), 0, math.pi,
 lambda x: -math.cos(100 * x) / 100),

# Parabel → Standard-Integrationsbeispiel
("f(x) = x²", lambda x: x**2, 0, 3, lambda x: (1/3) * x**3),

# Allgemeines Polynom 2. Grades → schöne Stammfunktion
("f(x) = x² - 4x + 2", lambda x: x**2 - 4*x + 2, -3, 3,
 lambda x: (1/3)*x**3 - 2*x**2 + 2*x),

# Ungerade Funktion (x³ - x) auf symmetrischem Intervall → Ergebnis 0
("f(x) = x³ - x", lambda x: x**3 - x, -1, 1,
 lambda x: (1/4)*x**4 - (1/2)*x**2),

# Produkt aus x und sin(x) → Kombination aus Wachstum und Welle
("f(x) = x·sin(x)", lambda x: x * math.sin(x), 0, 2 * math.pi,
 lambda x: -x * math.cos(x) + math.sin(x)),

# Wurzelfunktion → Flächen unter konvexer Kurve
("f(x) = √x", math.sqrt, 0, 4, lambda x: (2/3) * x**(3/2)),

# Betrag: abs(x), nicht überall differenzierbar → Sonderfall
("f(x) = |x|", abs, -1, 1,
 lambda x: 0.5 * x**2 if x >= 0 else -0.5 * x**2),

# Normalverteilungskurve → Stammfunktion ist Fehlerfunktion
("f(x) = (2/√π)·e^(-x²)",
 lambda x: (2 / math.sqrt(math.pi)) * math.exp(-x**2),
 -2, 2,
 lambda x: math.erf(x)),

# Sprungfunktion: 1 links, 2 rechts → diskontinuierlich
("Sprung: 1 (<0), 2 (≥0)", lambda x: 1 if x < 0 else 2, -1, 1, 1)

```

]

Aufruf

Im Folgenden Code-Abschnitt rufen wir die bereits vorher definierte Funktion **auswertung_durchfuehren()** auf.

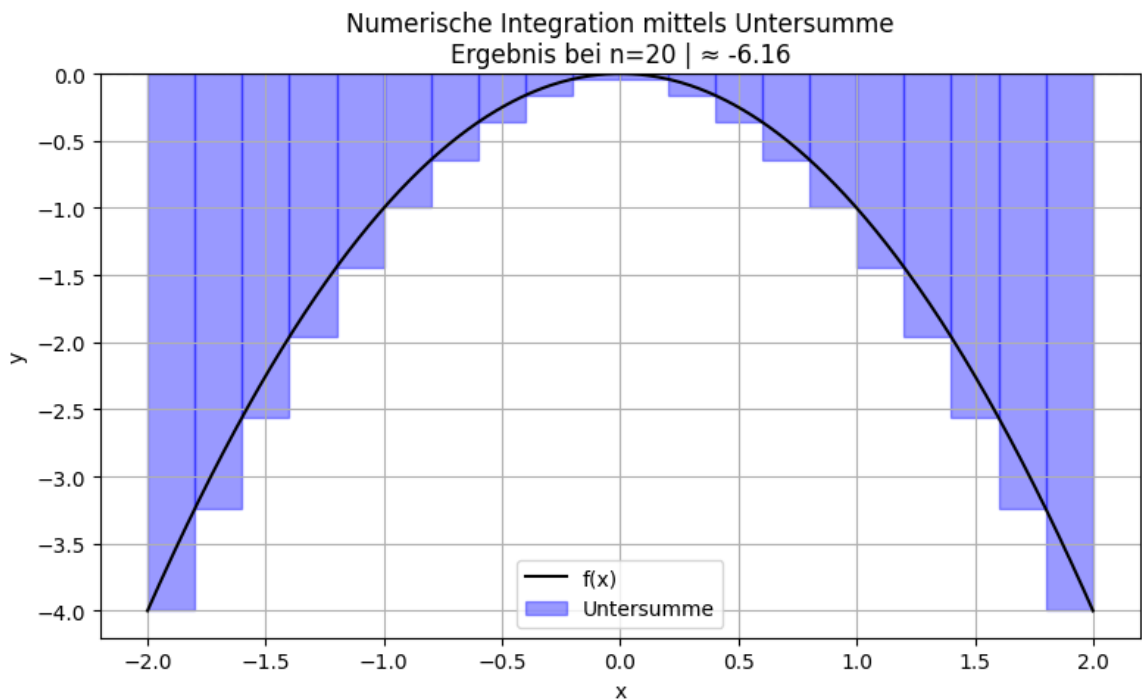
Wie nutzen hierbei die Definierten Grenzen unseres test-arrays indem wir `nutze_feste_grenzen` auf `False` setzen.

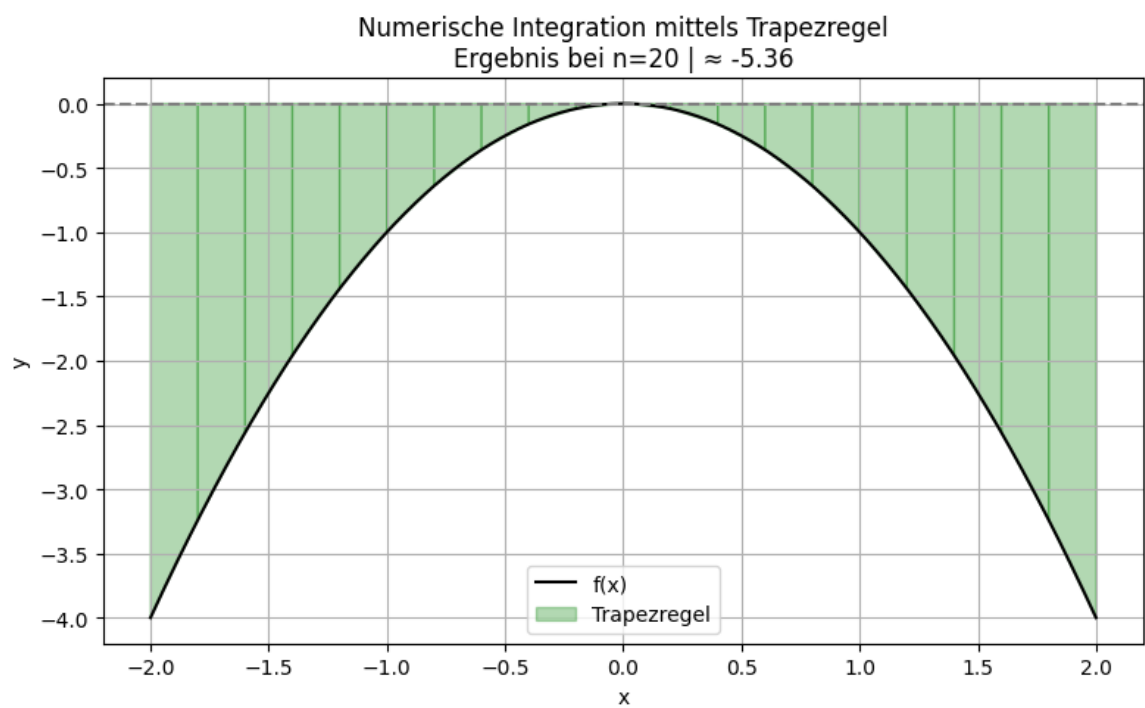
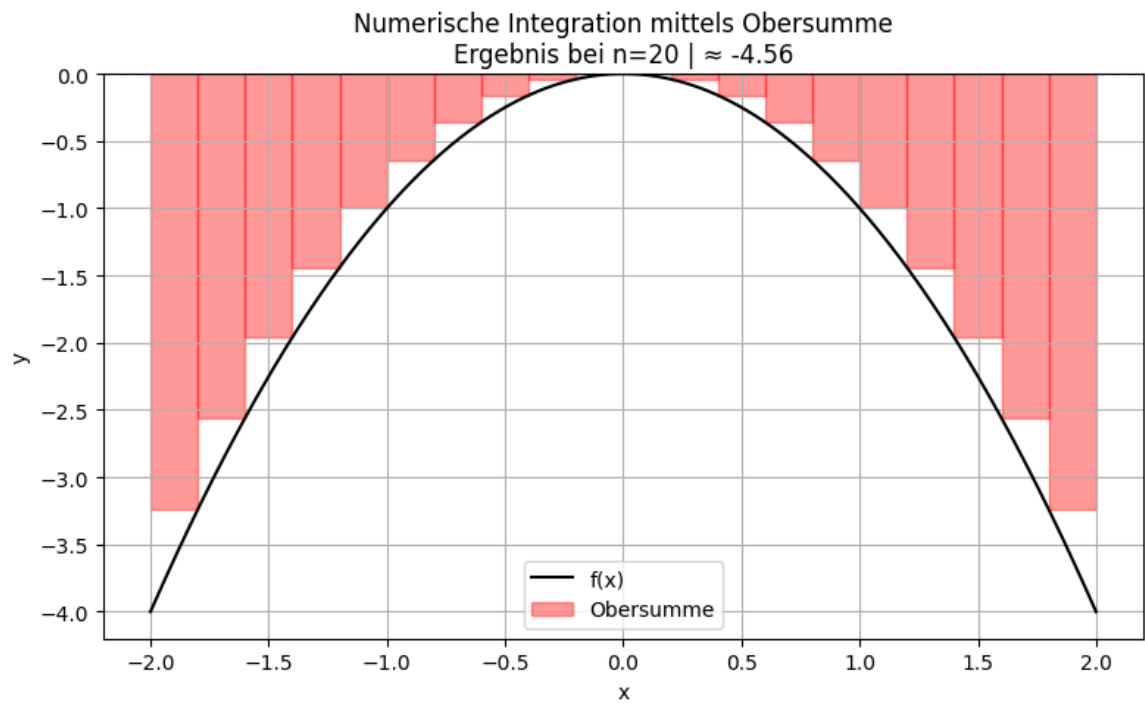
Trotzdem geben wir noch zusätzlich feste Grenzen als Fallback an.

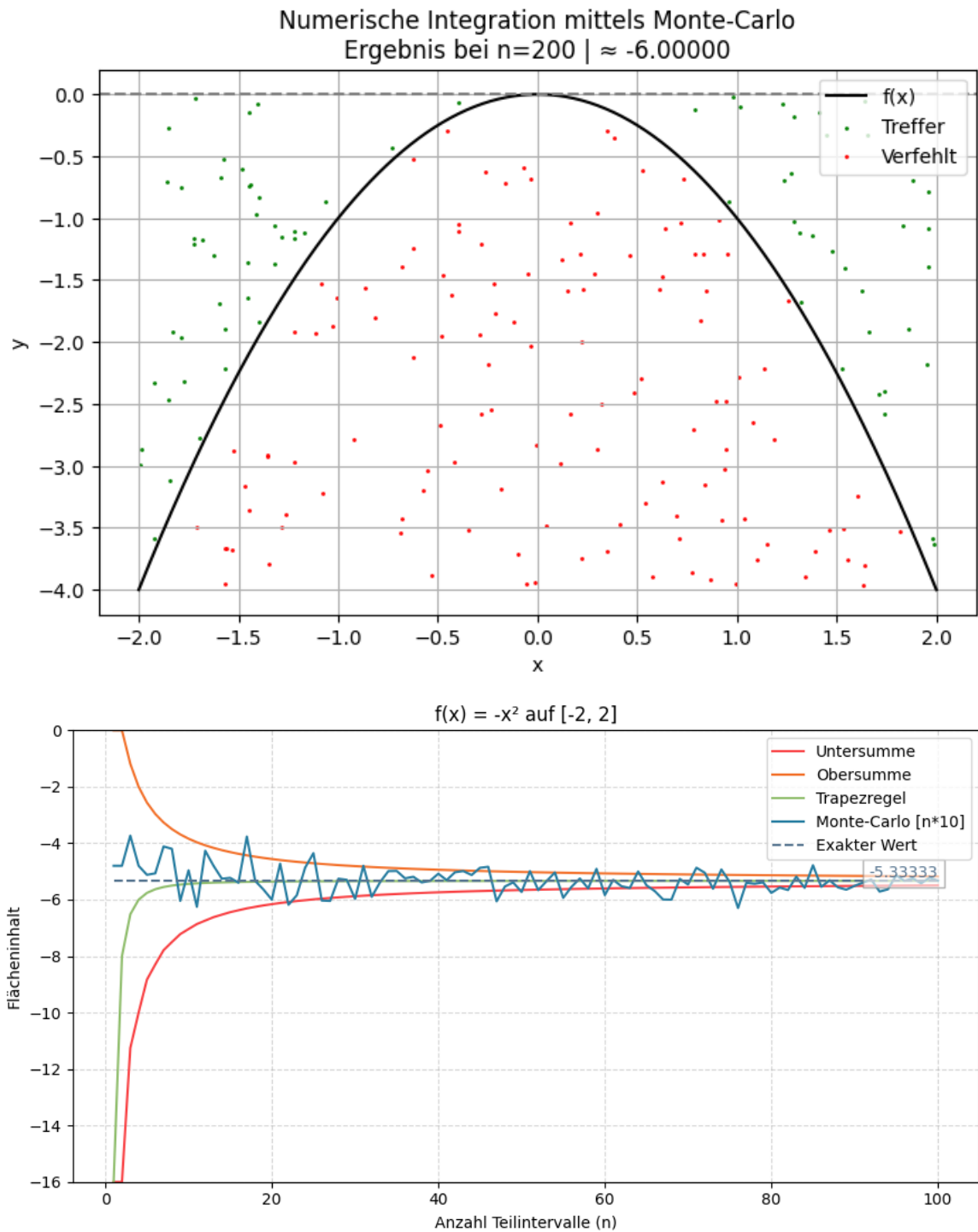
Als `moFaktor` nehmen wir 10, heißt das `n` von der Monte-Carlo Integration läuft nicht von 1 bis 100 sondern von 1 bis 1000 in 10er Schritten.

Als obere Grenze geben wir `max_n = 100` an.

```
In [36]: # Aufruf der Hauptauswertungsfunktion:
# Führt für alle angegebenen Testfunktionen eine vergleichende nume
# mit Untersumme, Obersumme, Trapezregel und Monte-Carlo-Verfahren.
auswertung_durchfuehren(
    nutze_feste_grenzen=False, # False= Jede Funktion verwendet i
    feste_a=-5,                # Wird ignoriert, da nutze_feste_gr
    feste_b=5,                 # Wird ignoriert, da nutze_feste_gr
    moFaktor=10,               # Monte-Carlo verwendet n * 10 Zufa
    max_n=100,                 # Die Anzahl der Teilintervalle n r
    testfunktionen=testfunktionen # Die zuvor definierte Liste an
)
```



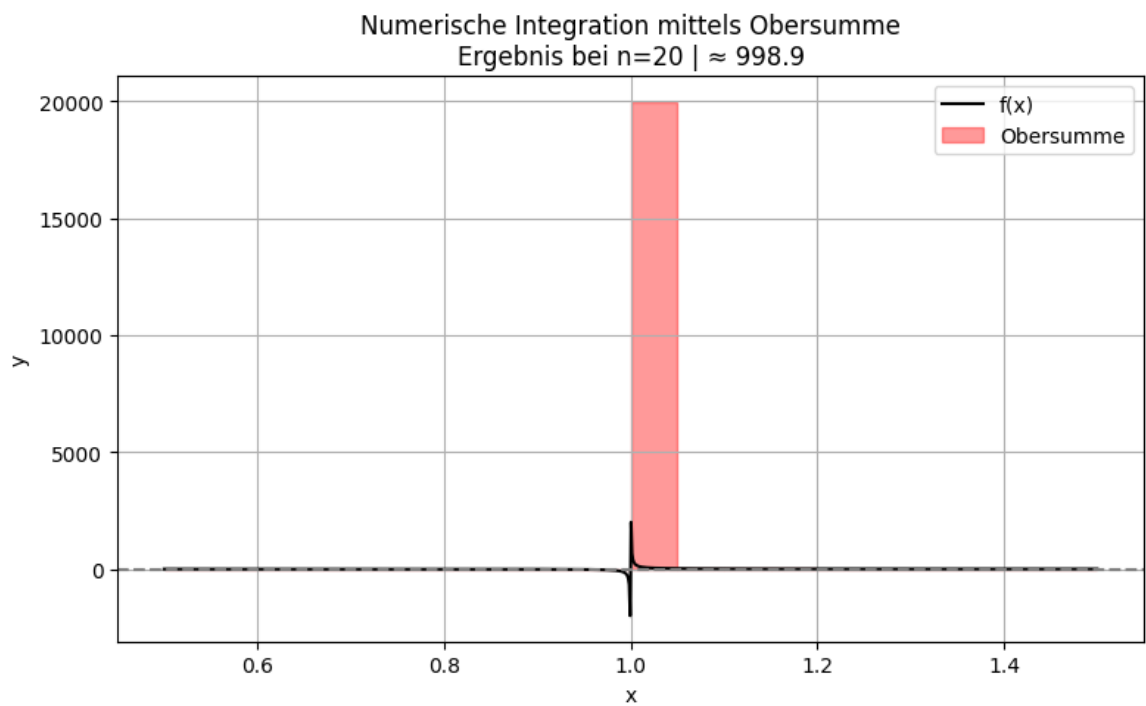
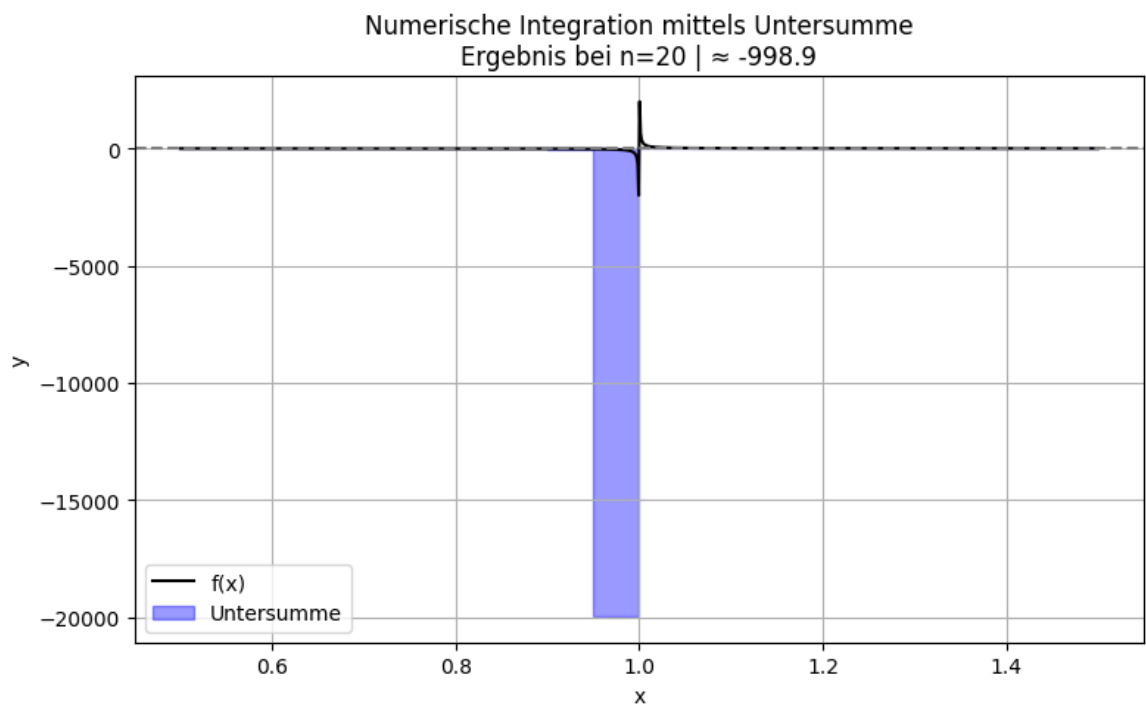


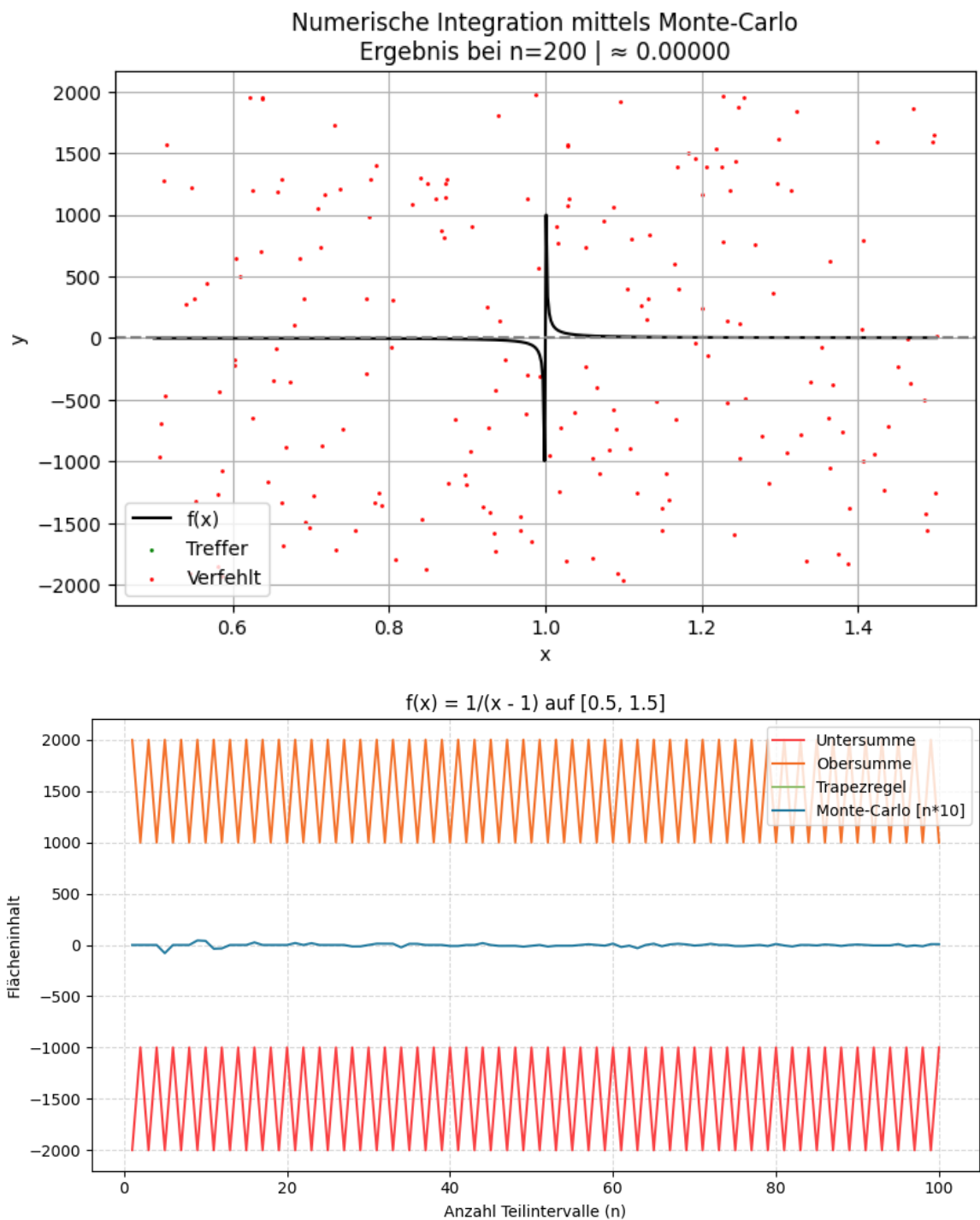


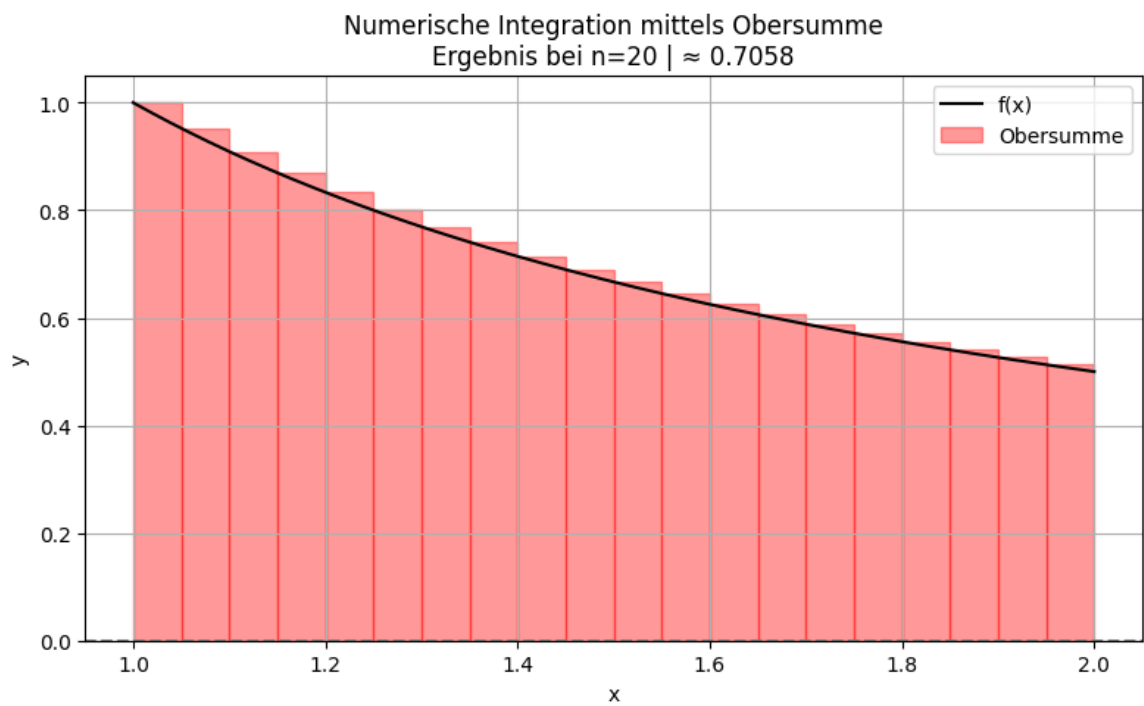
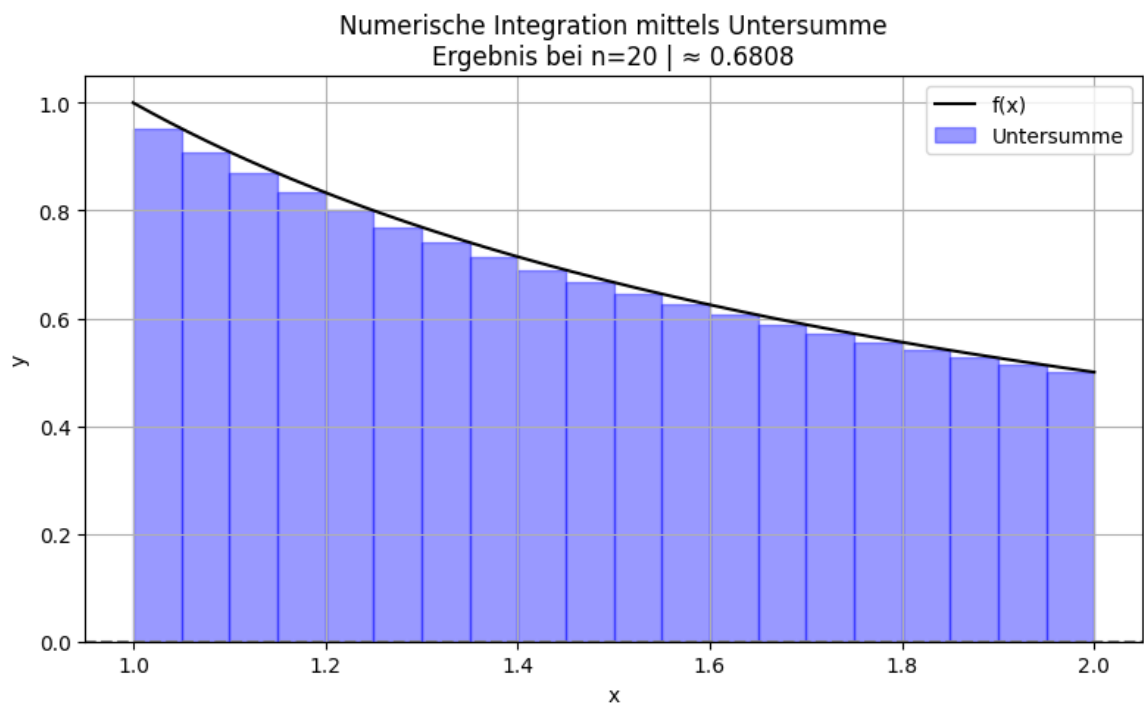
```

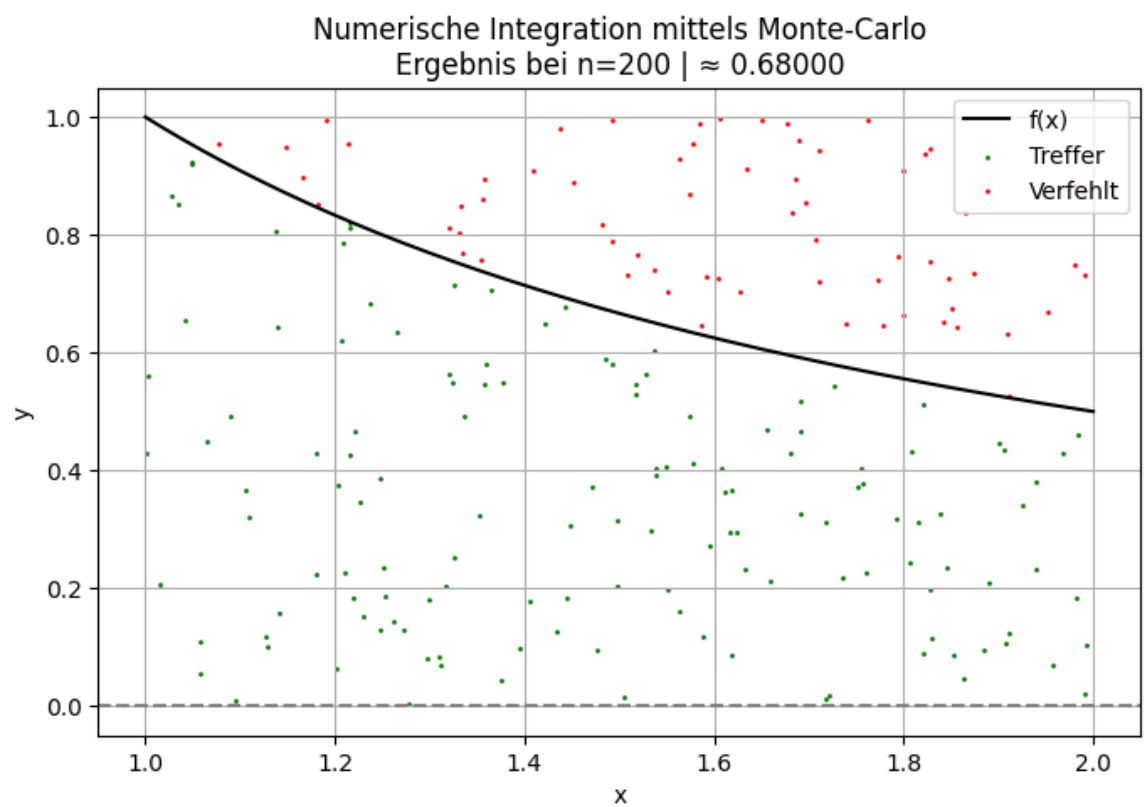
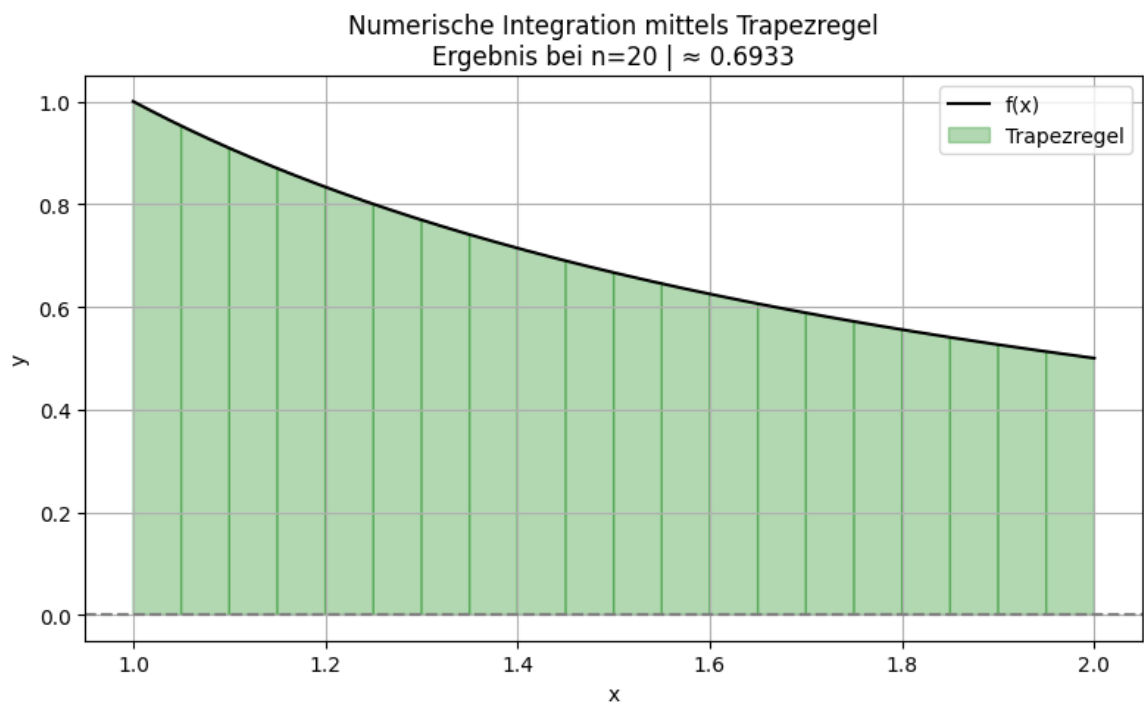
/var/folders/zg/yj0lh8b54cj5ggc2qwkh19300000gn/T/ipykernel_92356/342
7022216.py:11: RuntimeWarning: divide by zero encountered in divide
("f(x) = 1/(x - 1)", lambda x: 1 / (x - 1), 0.5, 1.5, None),

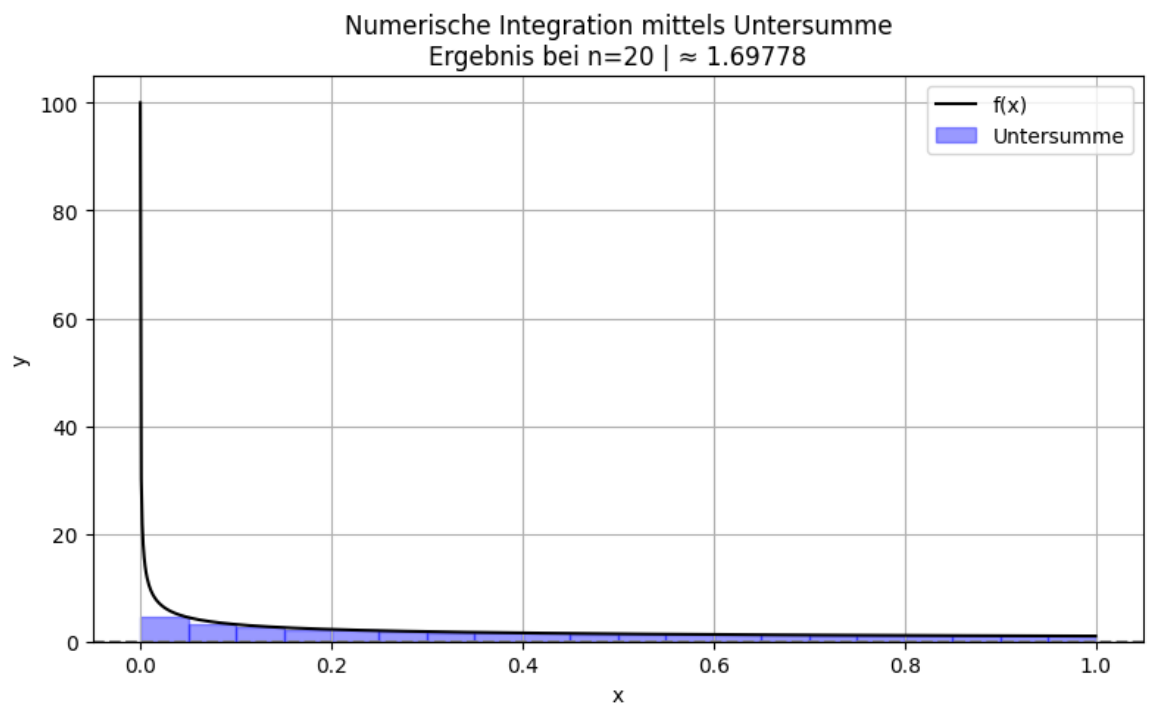
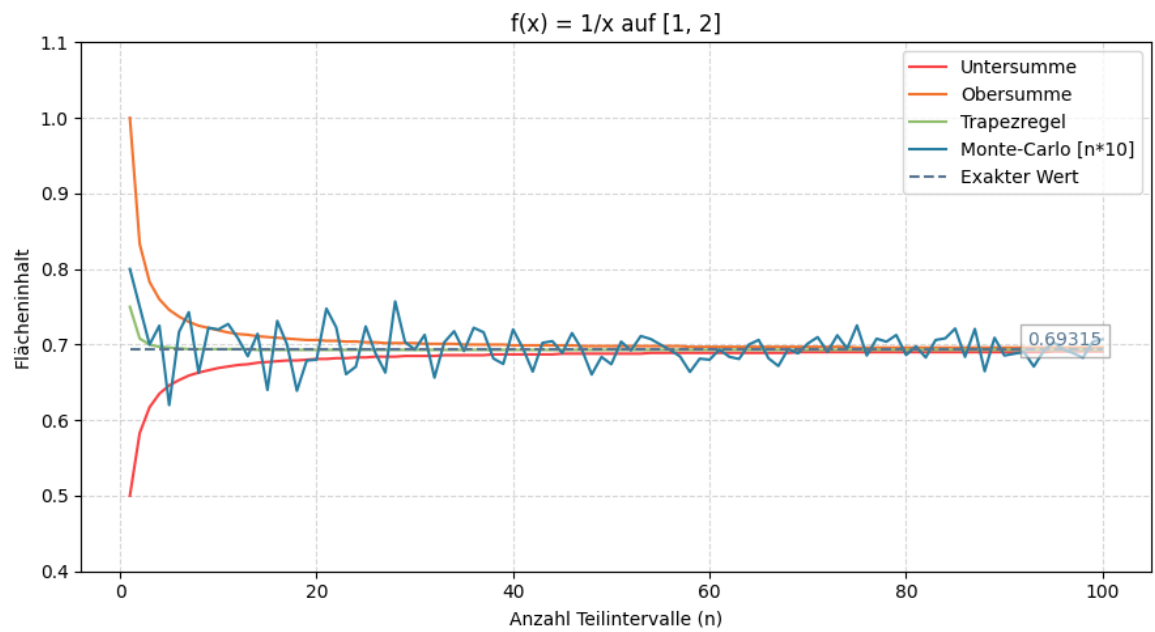
```

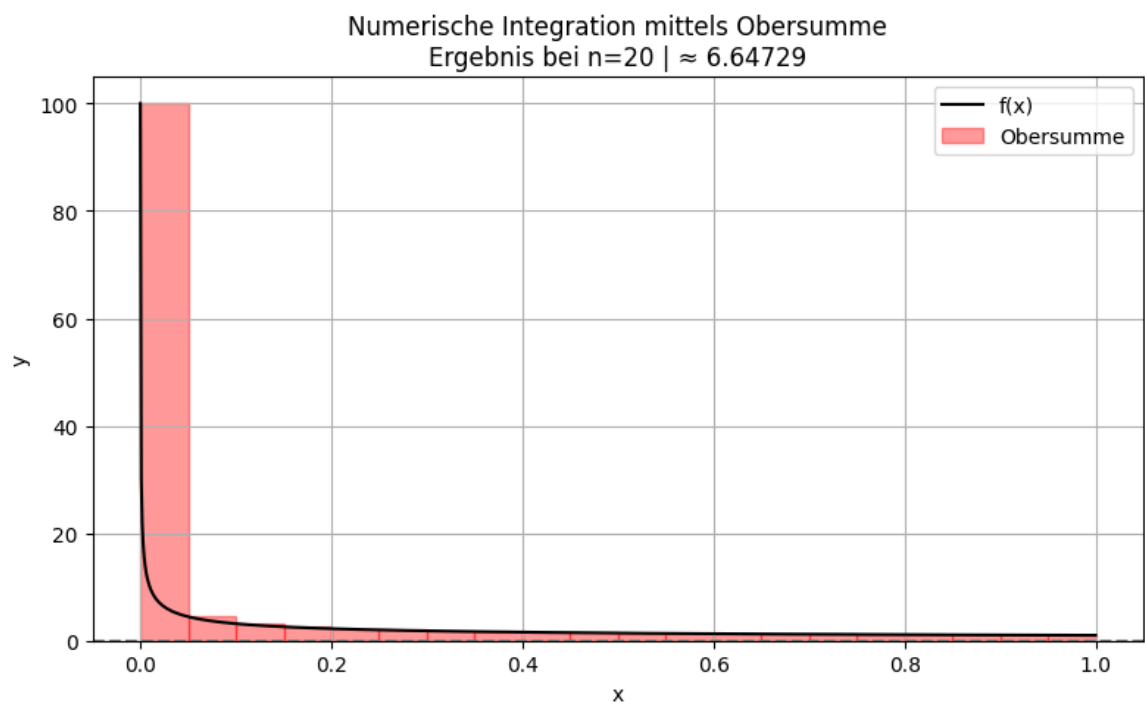



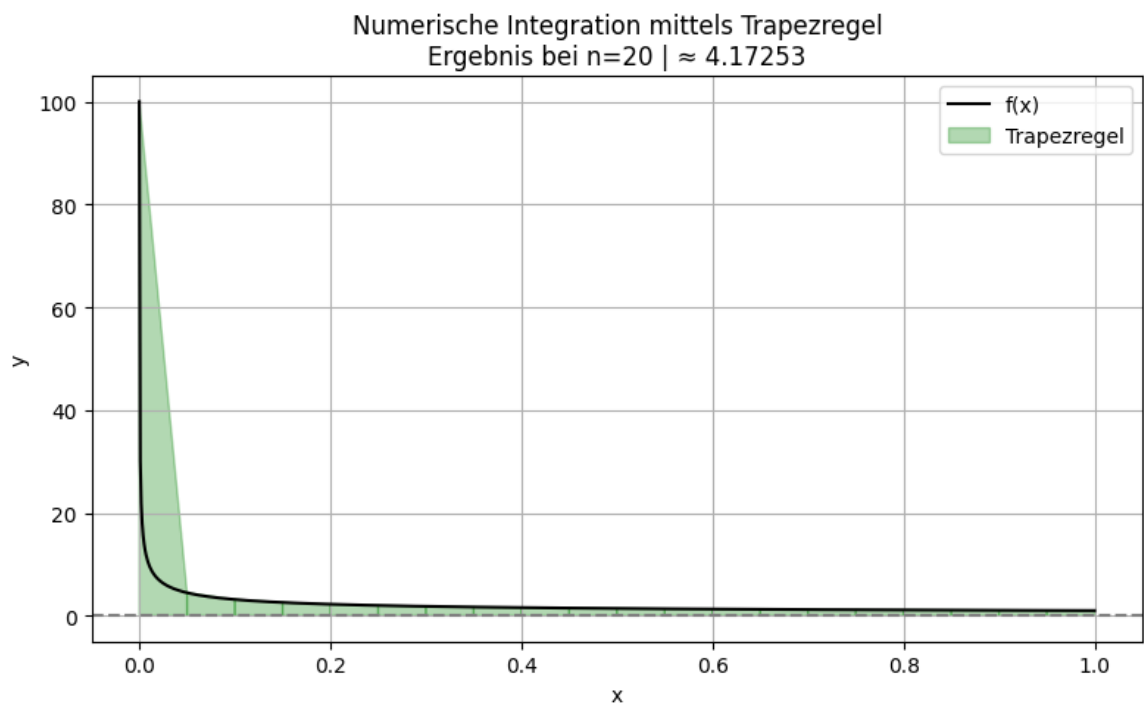


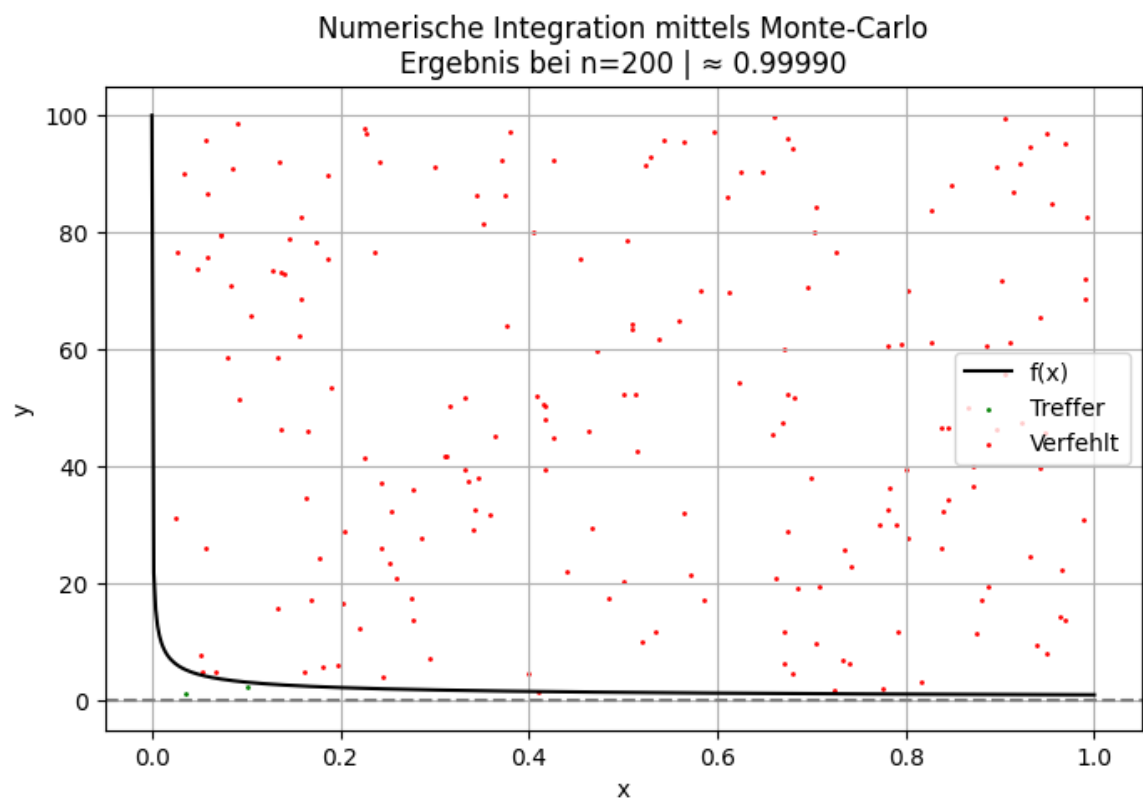


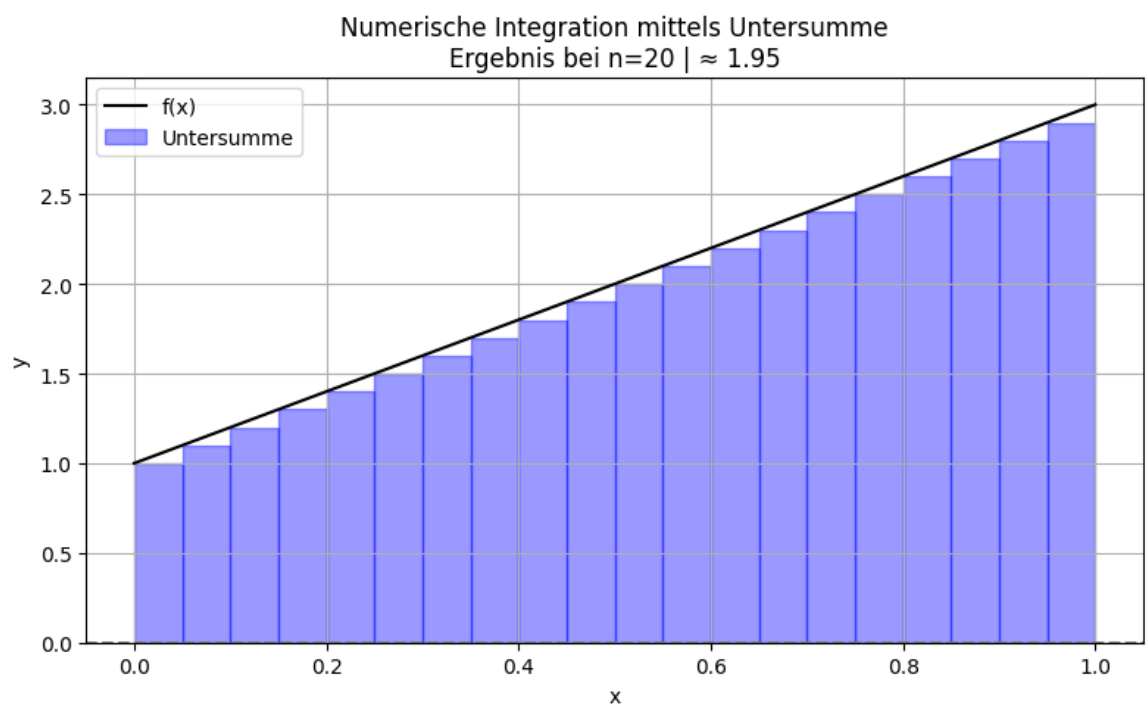
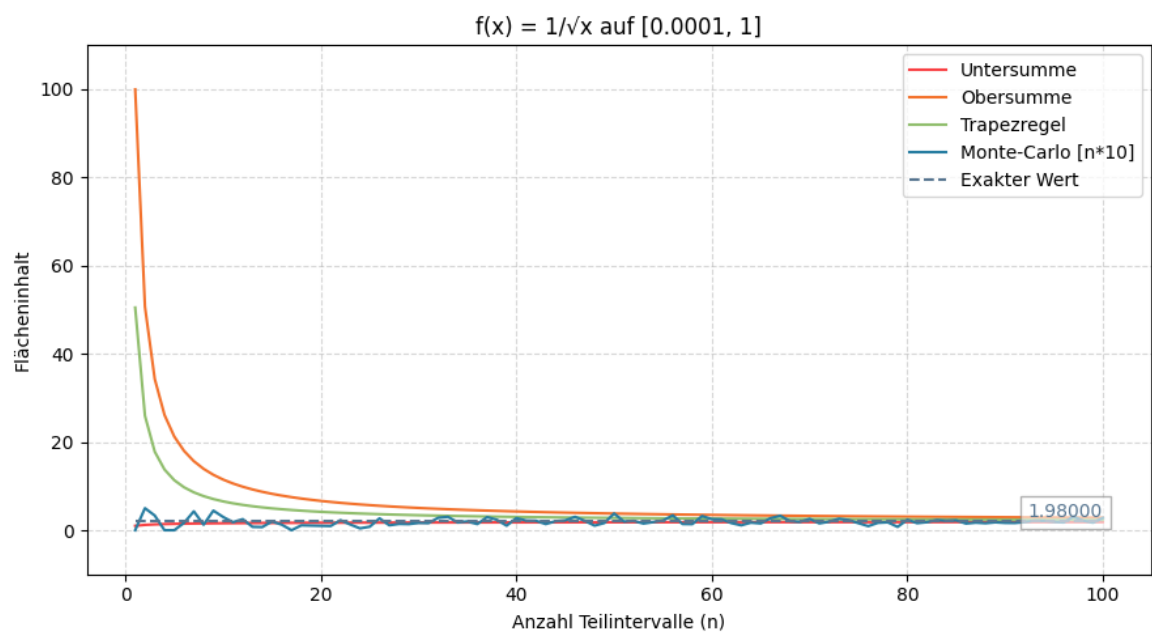




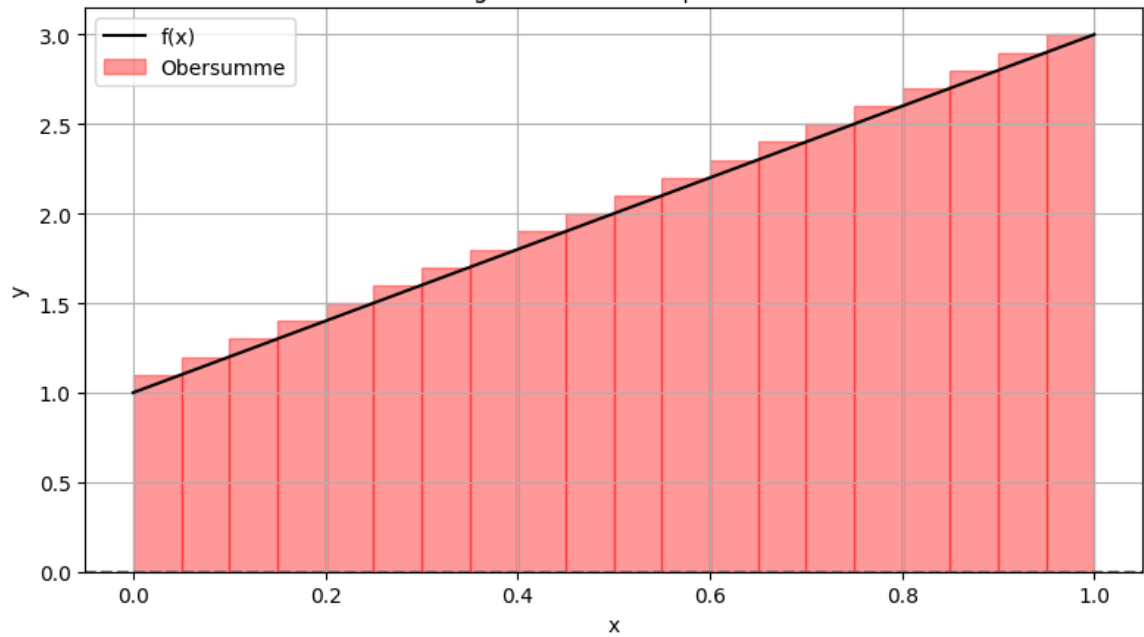




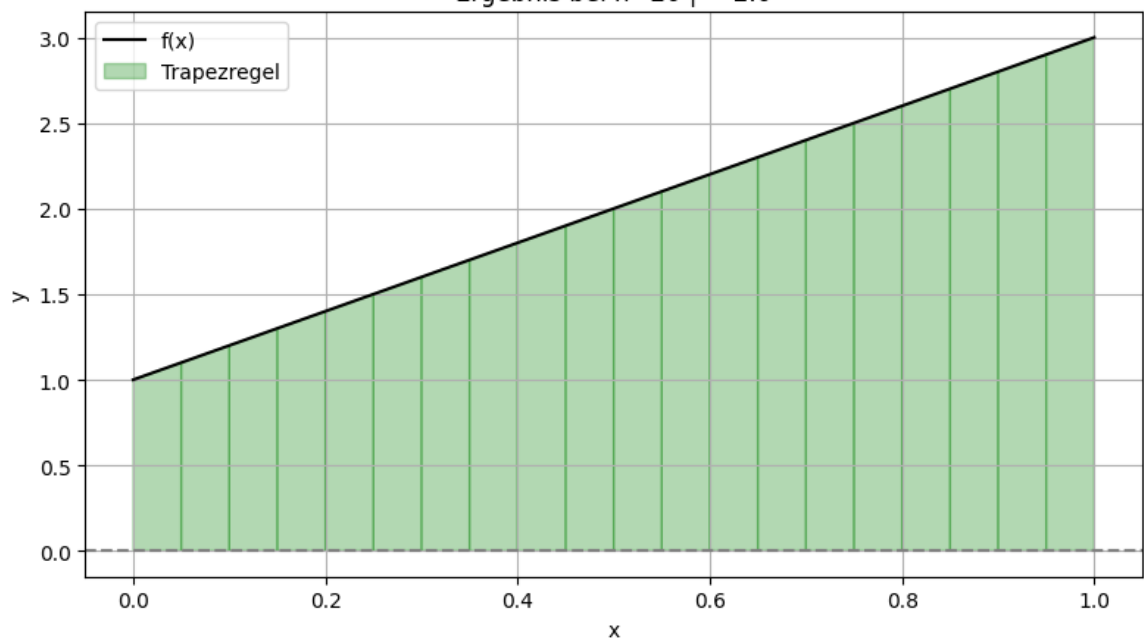


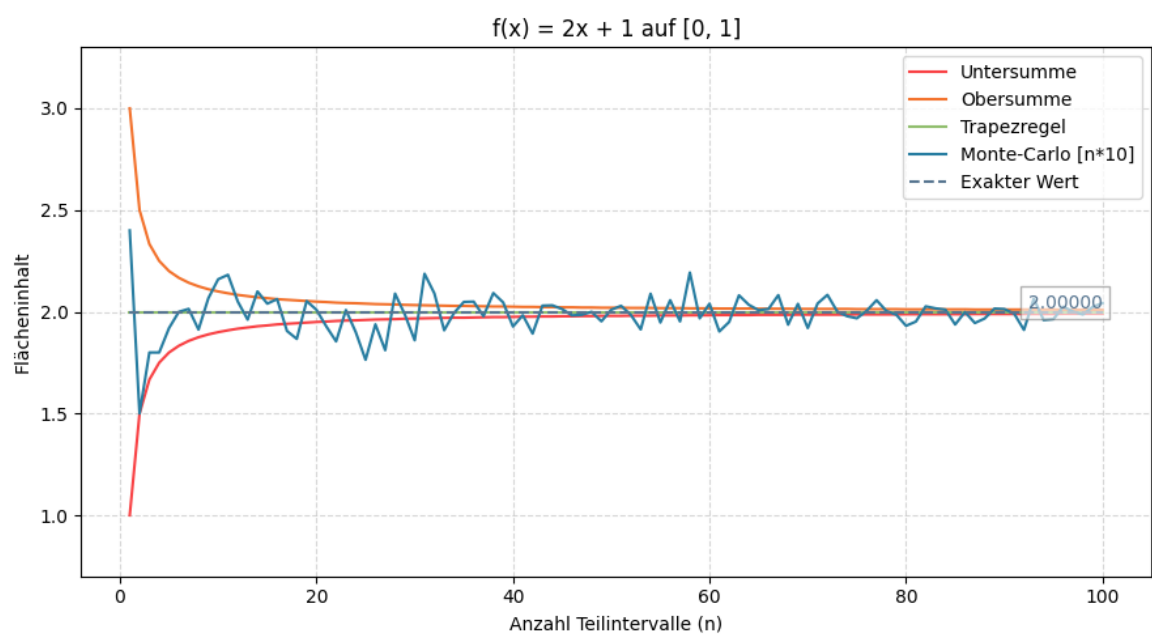
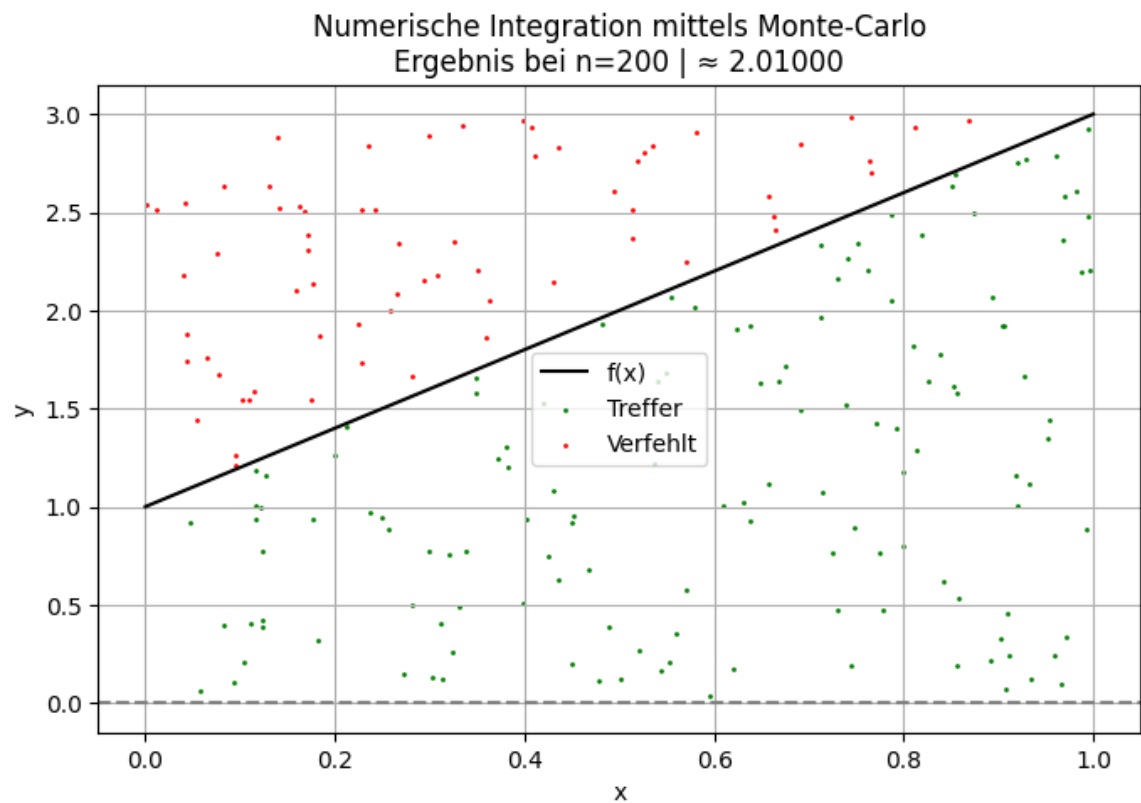


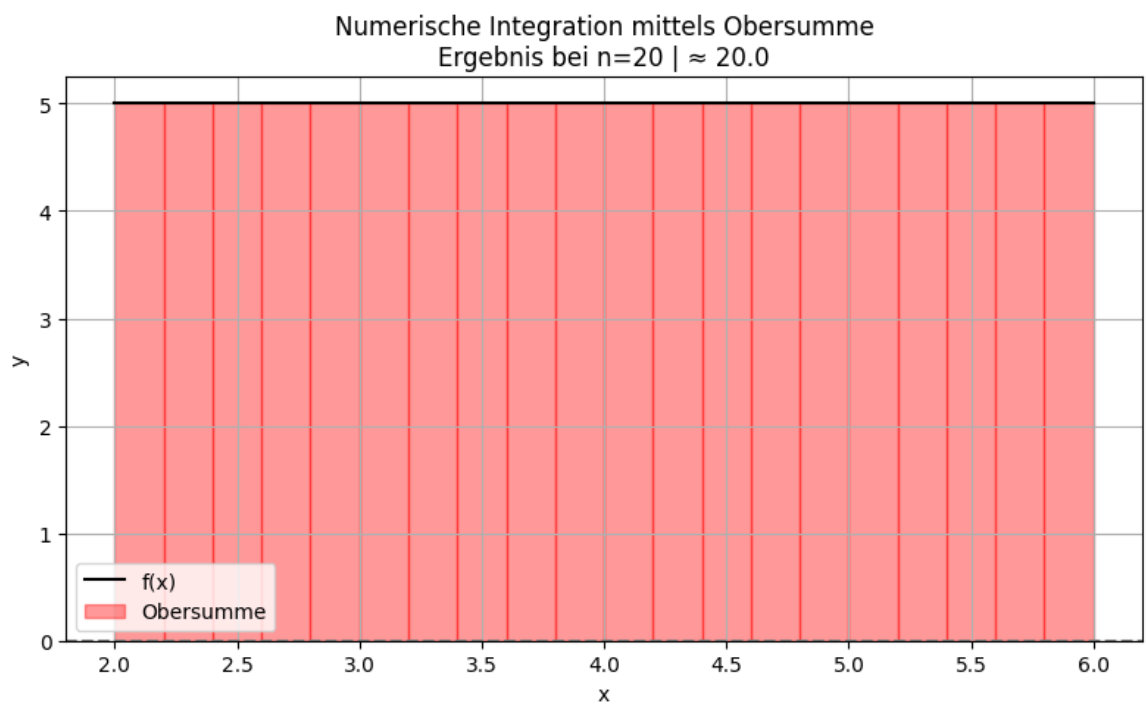
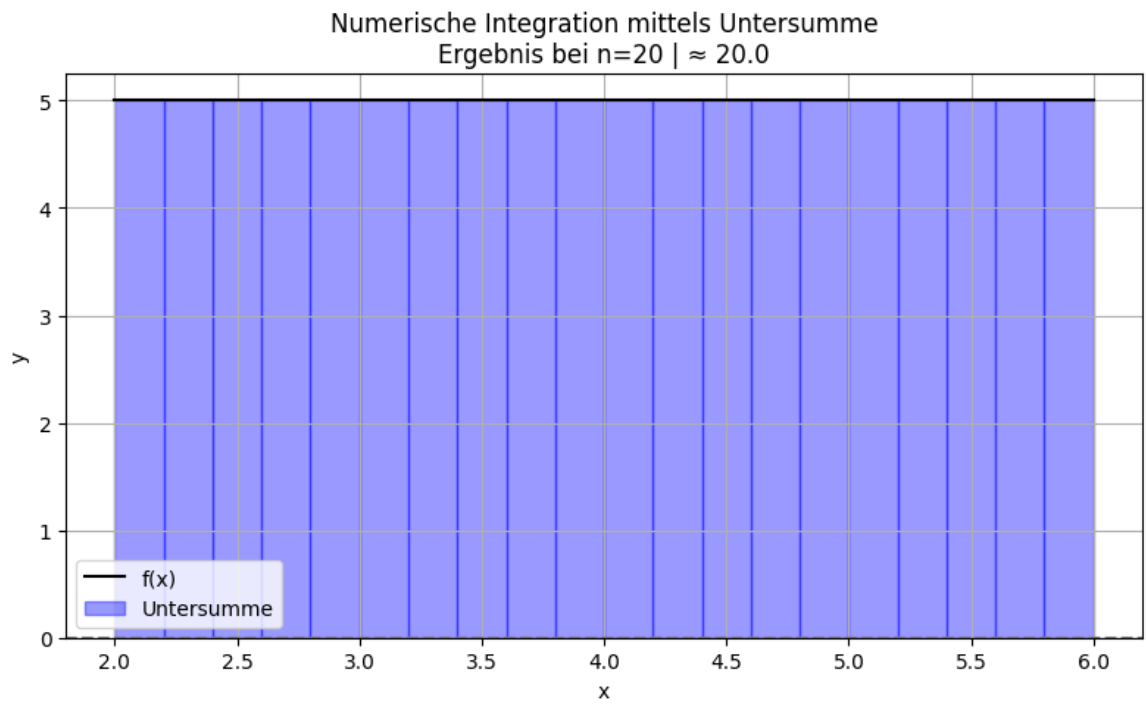
Numerische Integration mittels Obersumme
Ergebnis bei $n=20$ | ≈ 2.05

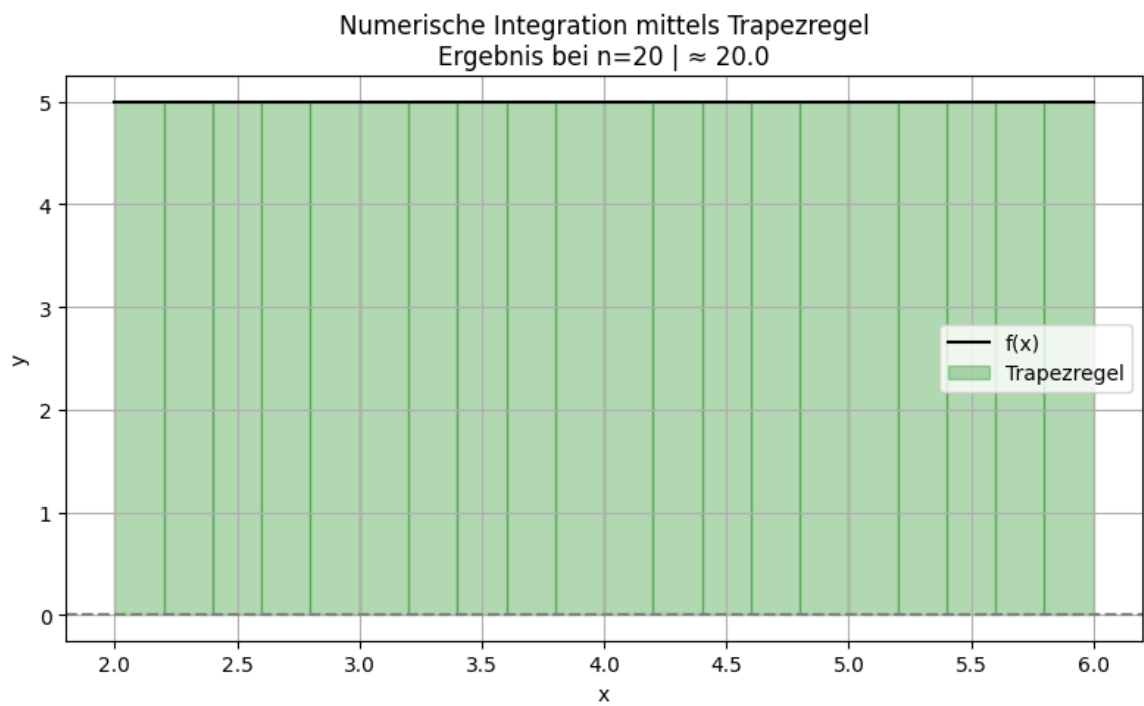


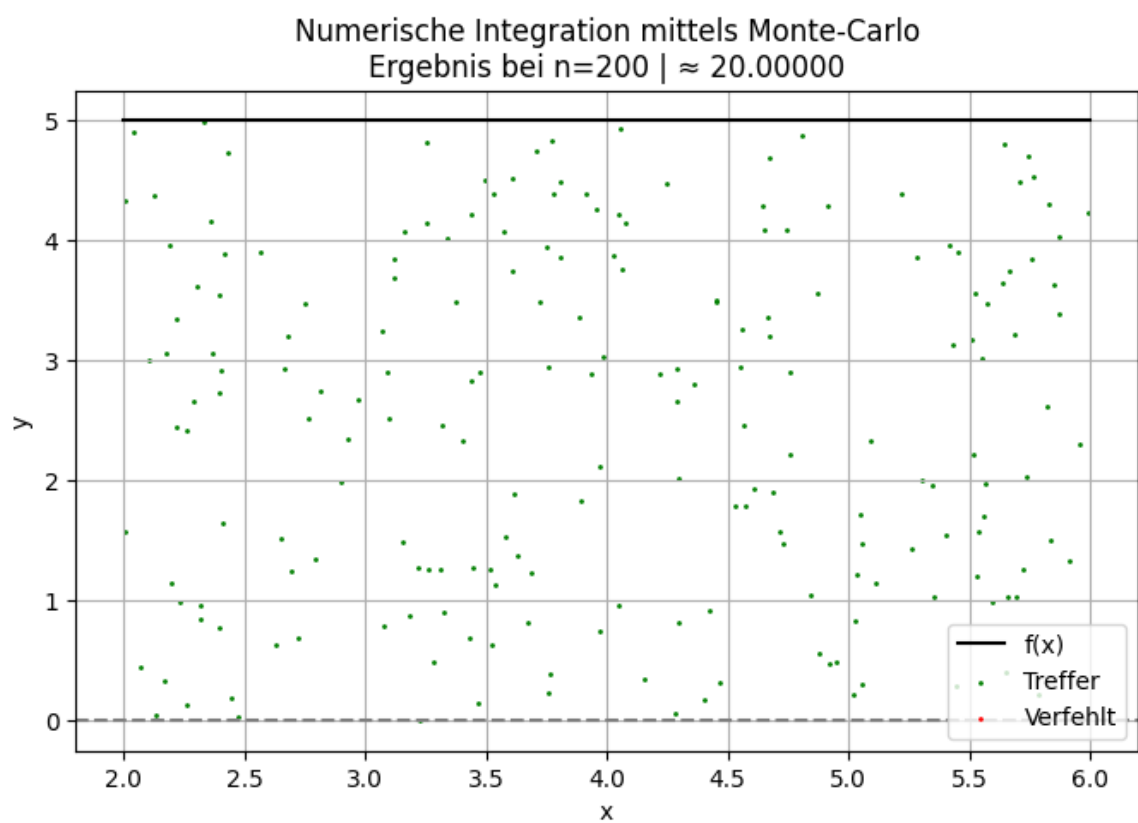
Numerische Integration mittels Trapezregel
Ergebnis bei $n=20$ | ≈ 2.0

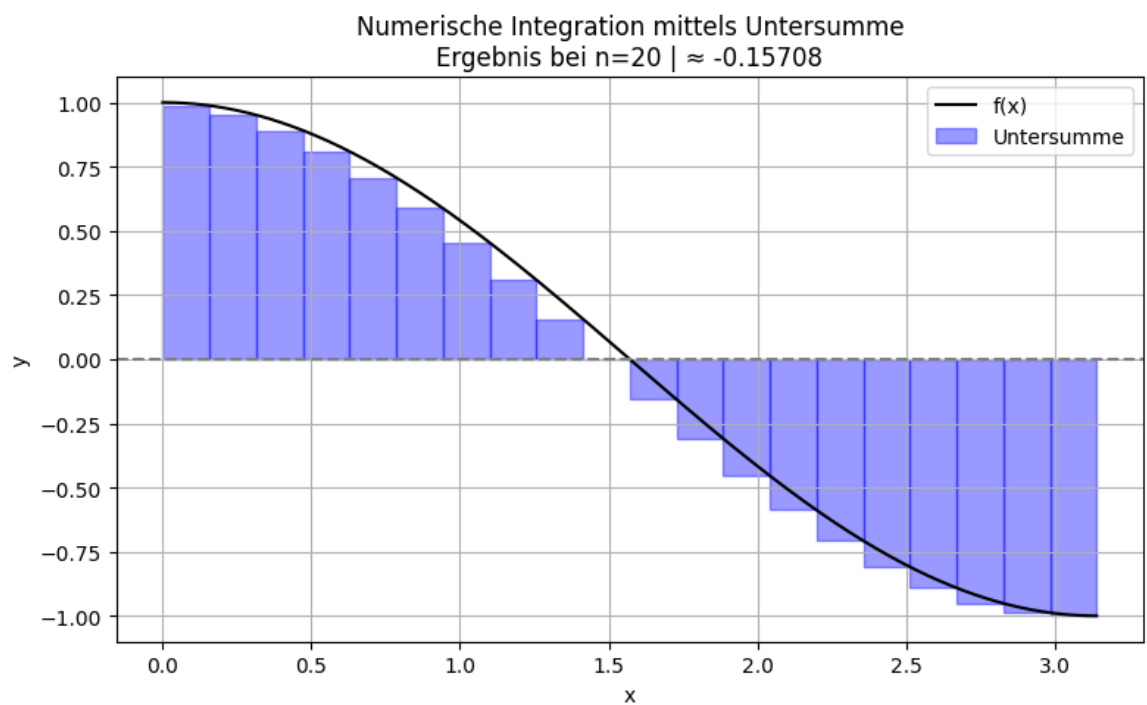
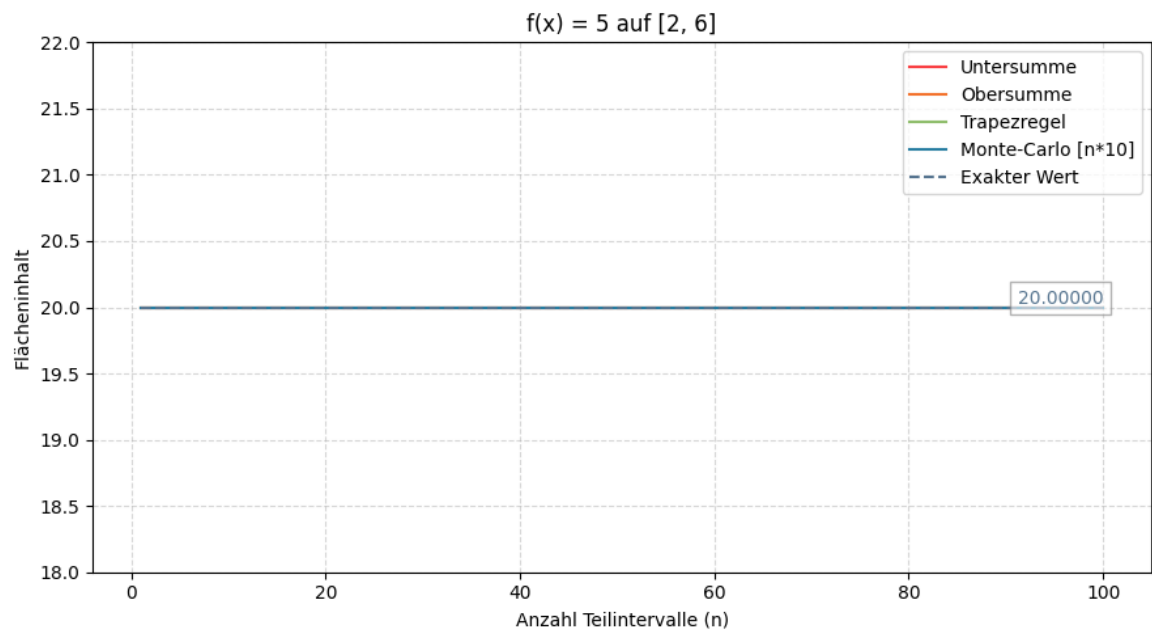


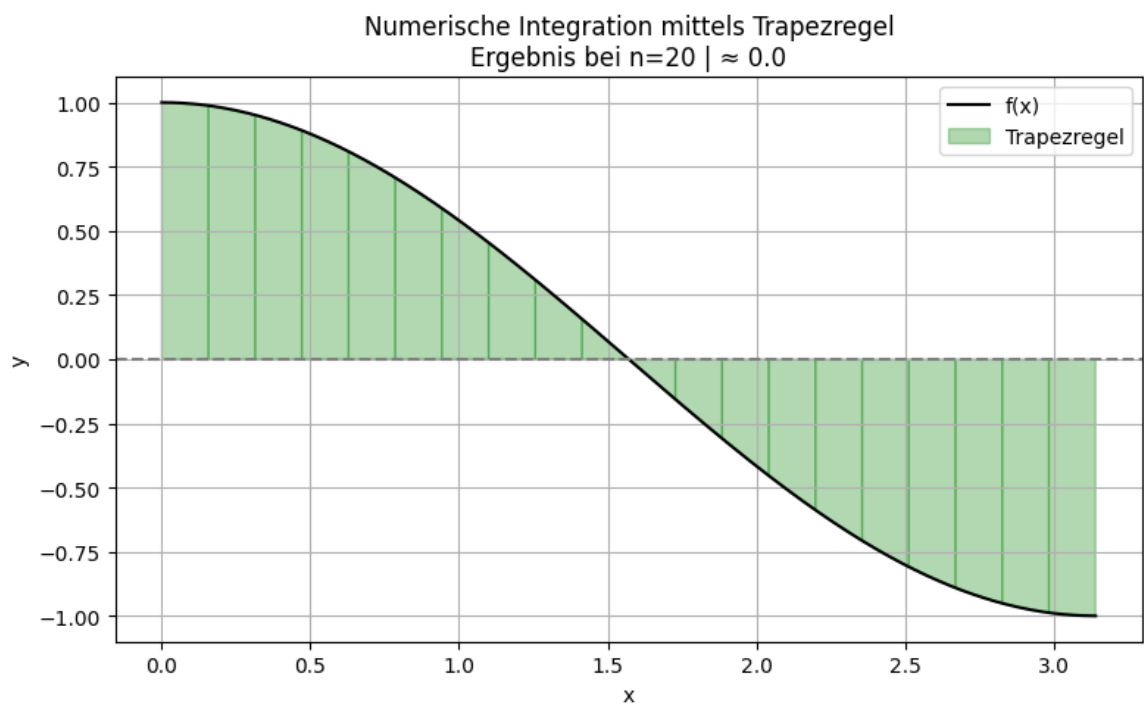
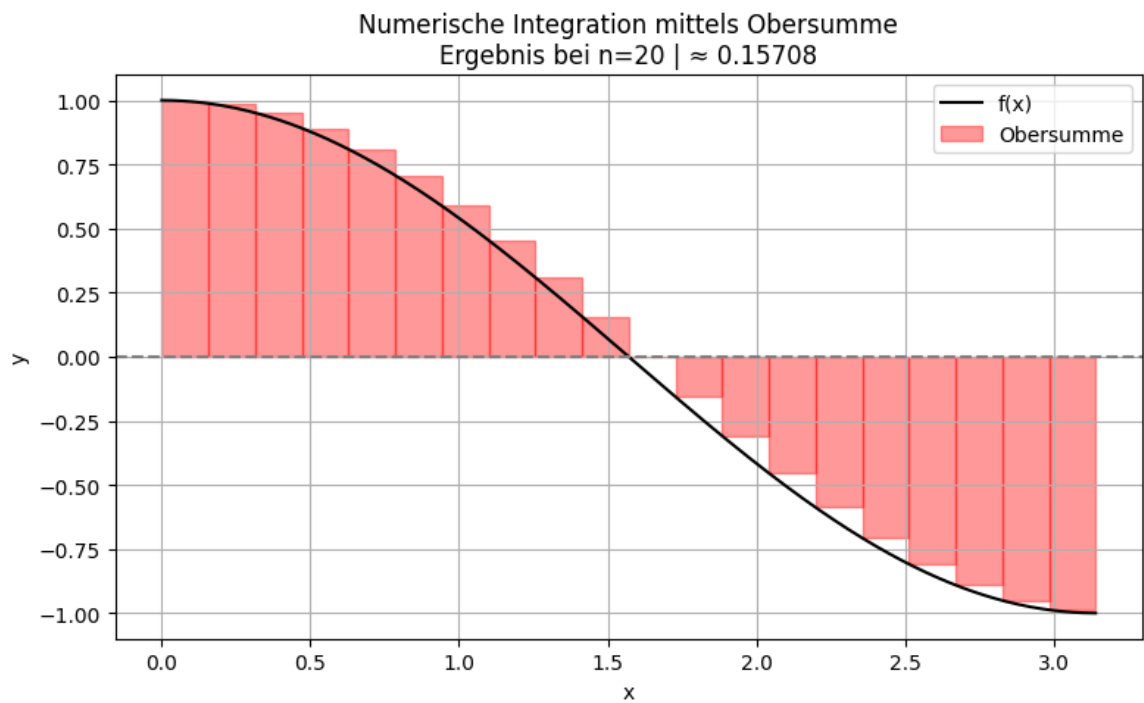


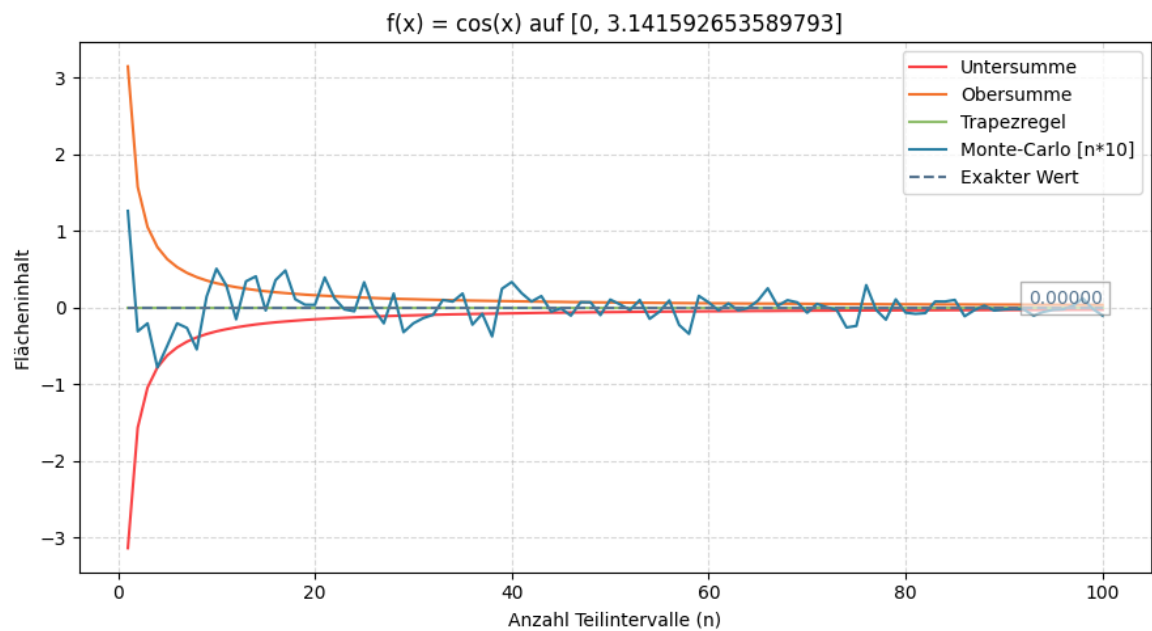
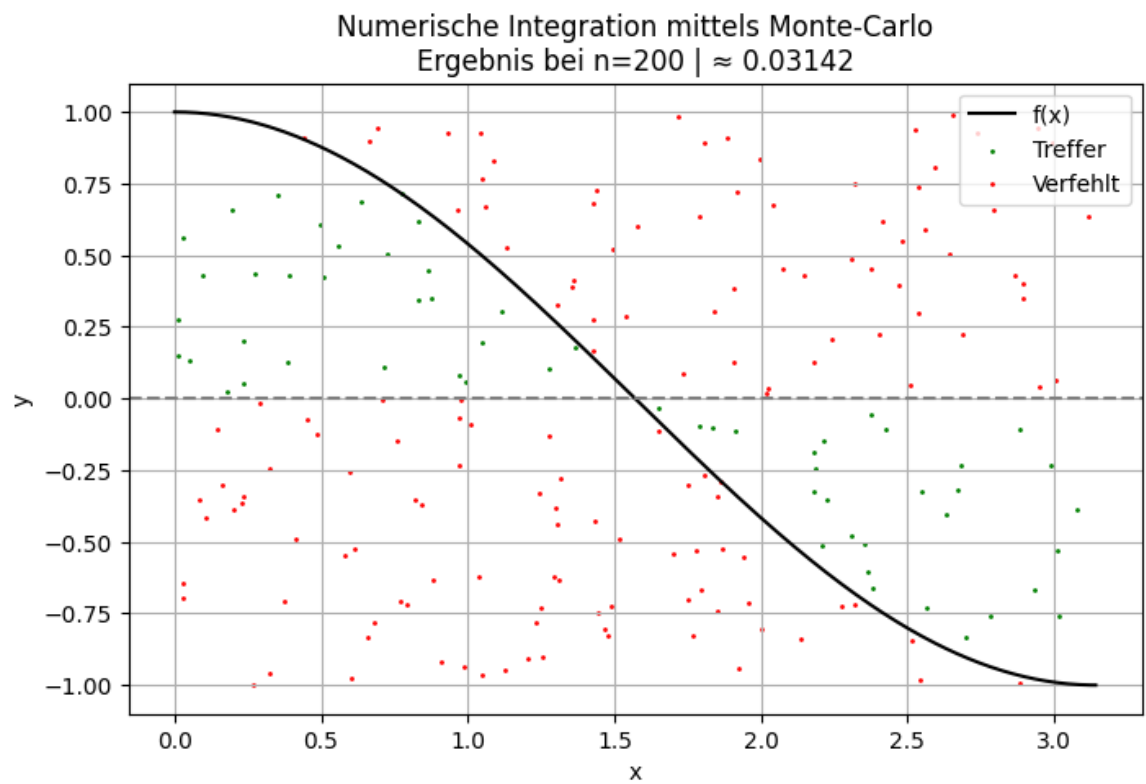




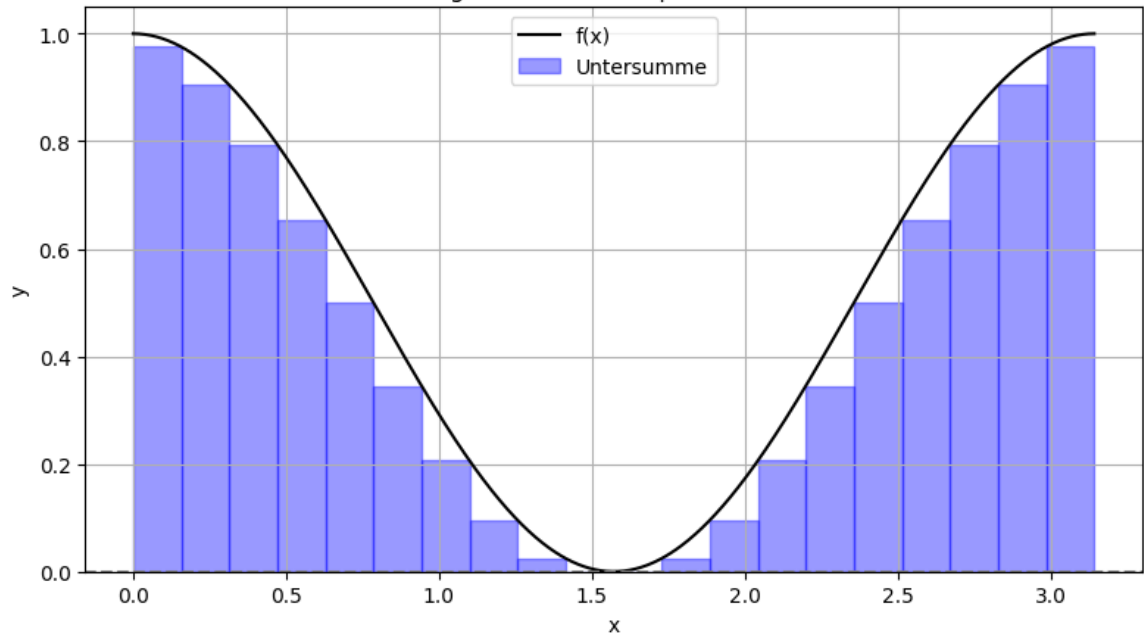




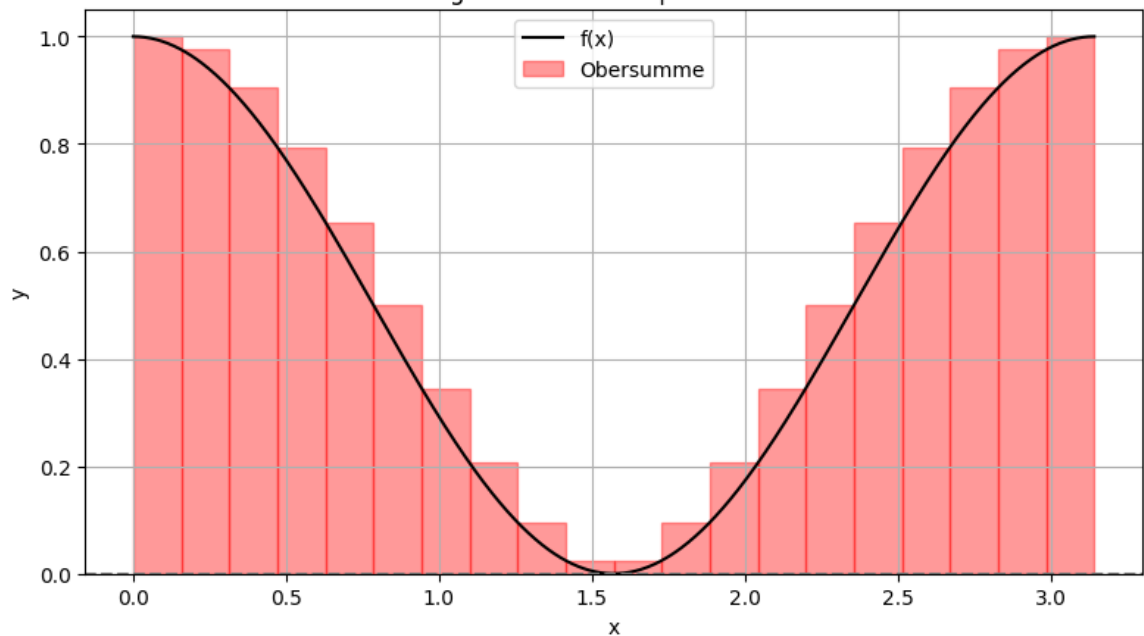


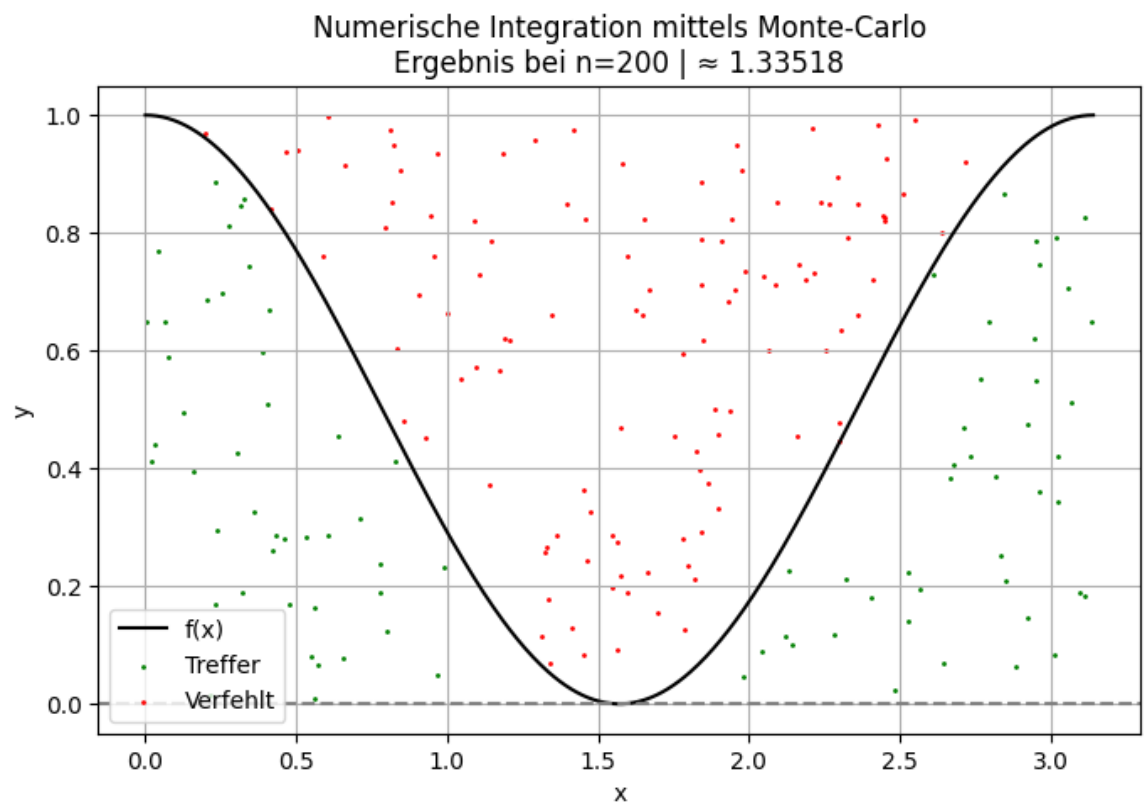
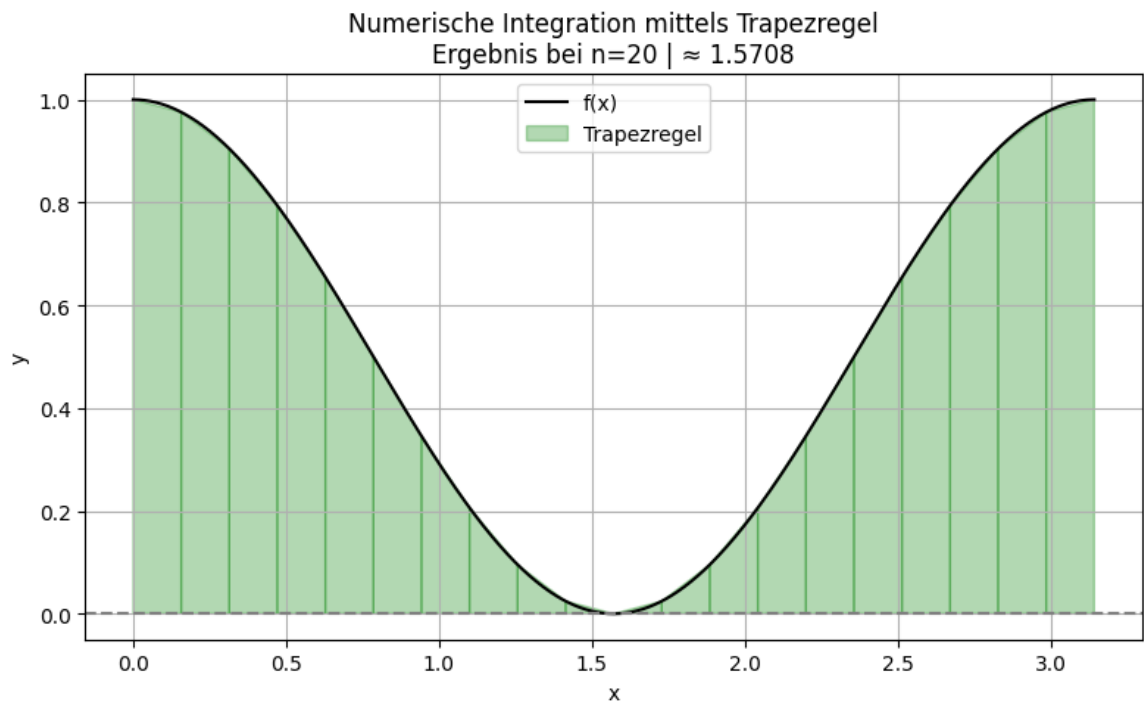


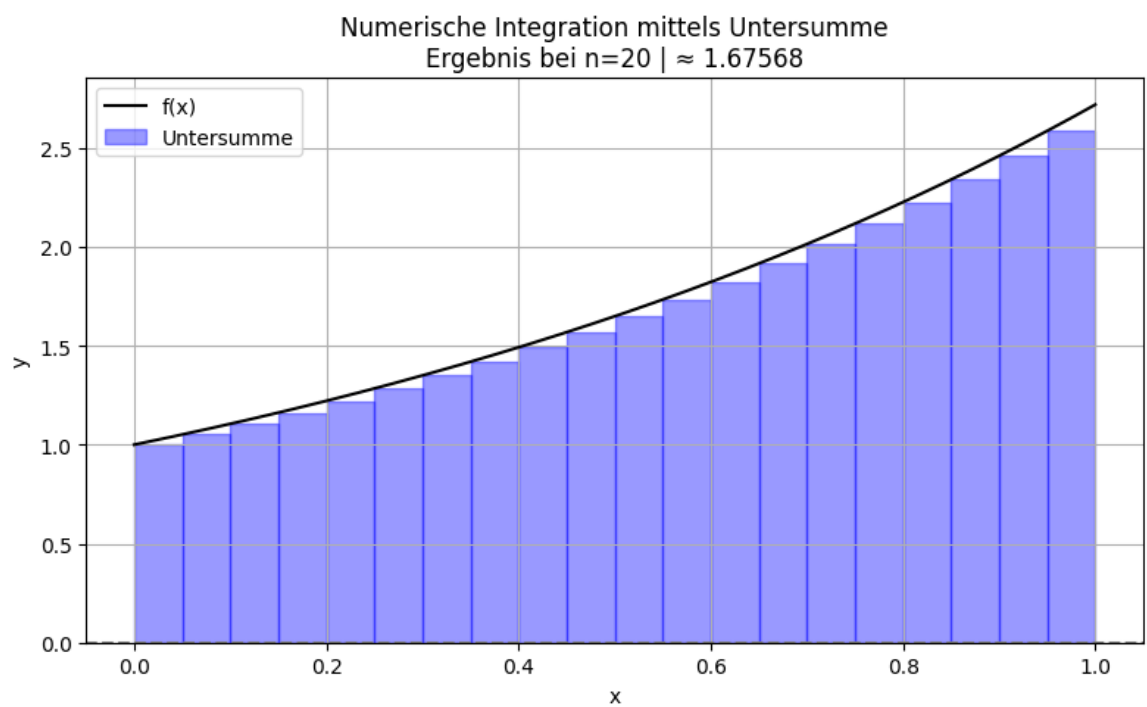
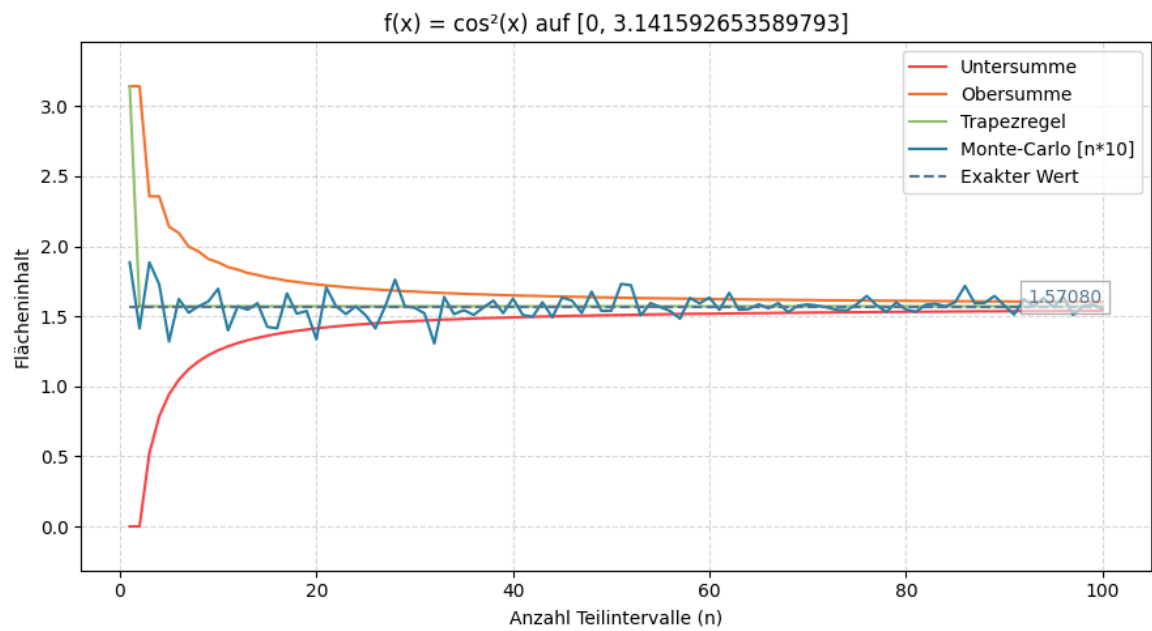
Numerische Integration mittels Untersumme
Ergebnis bei $n=20$ | ≈ 1.41372



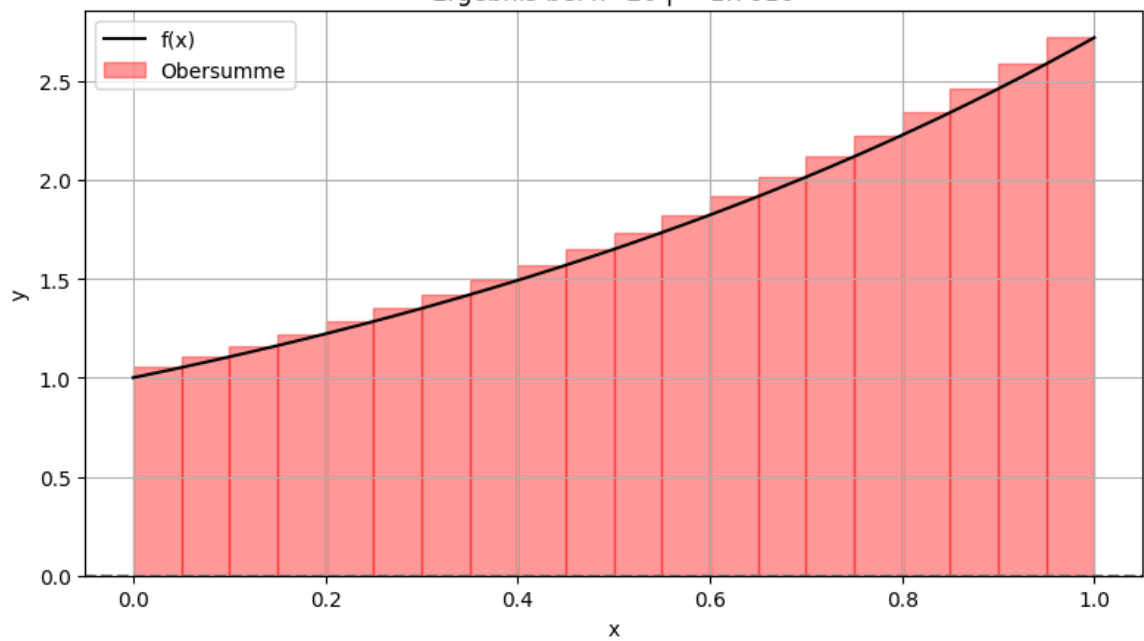
Numerische Integration mittels Obersumme
Ergebnis bei $n=20$ | ≈ 1.72788



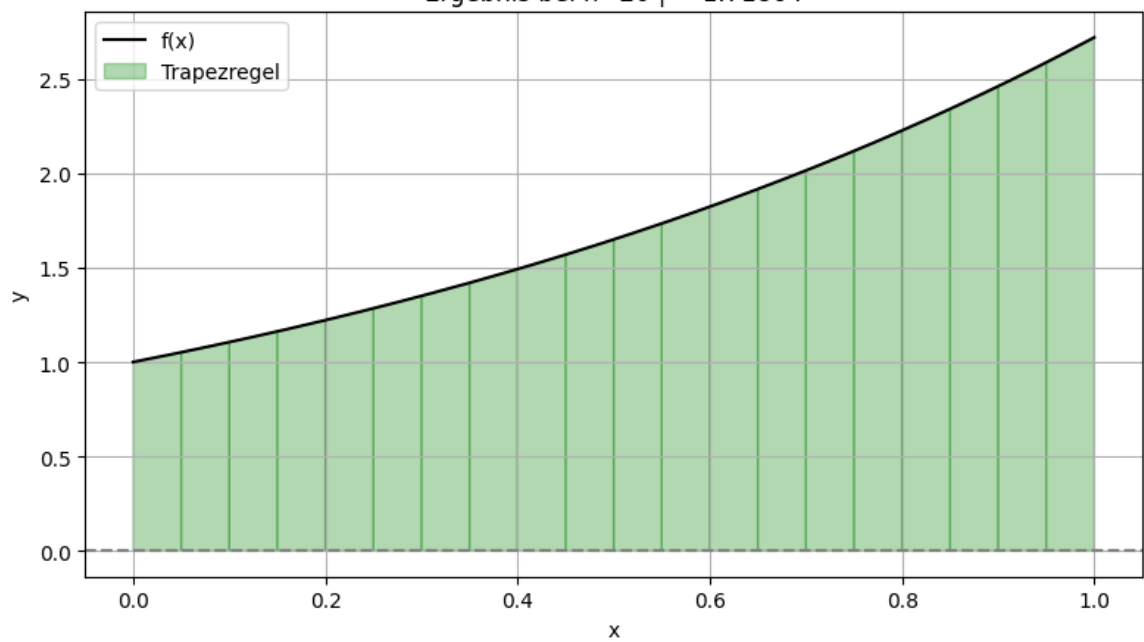




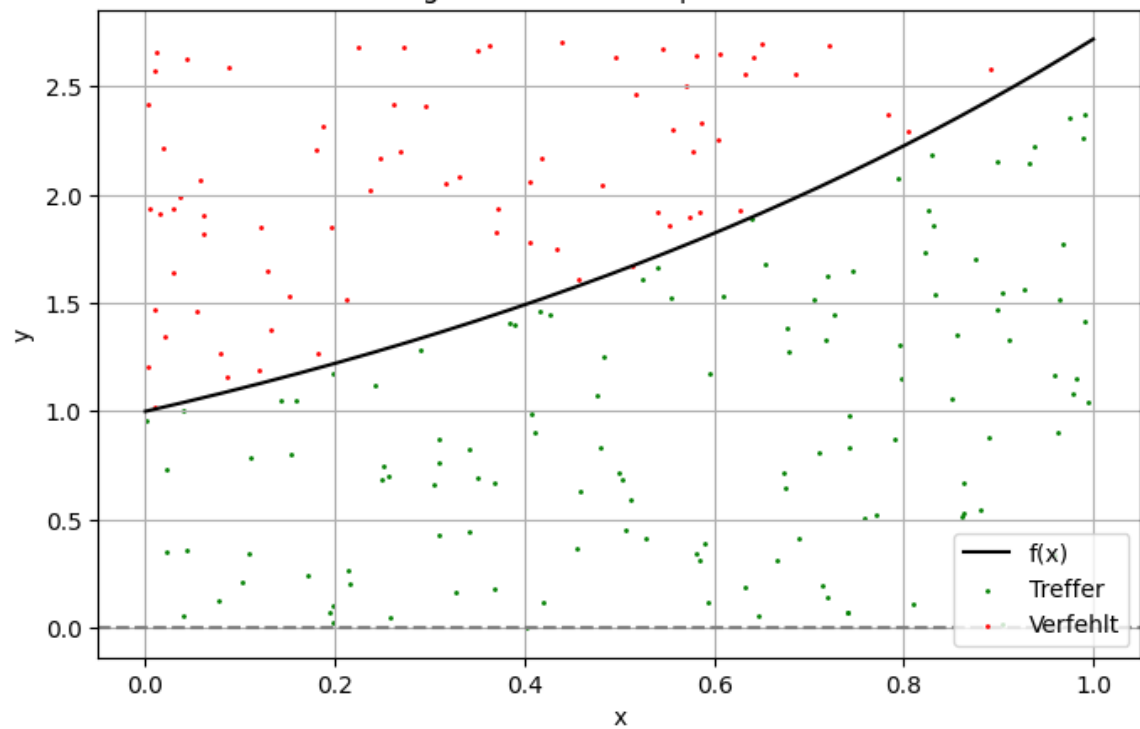
Numerische Integration mittels Obersumme
Ergebnis bei $n=20$ | ≈ 1.7616



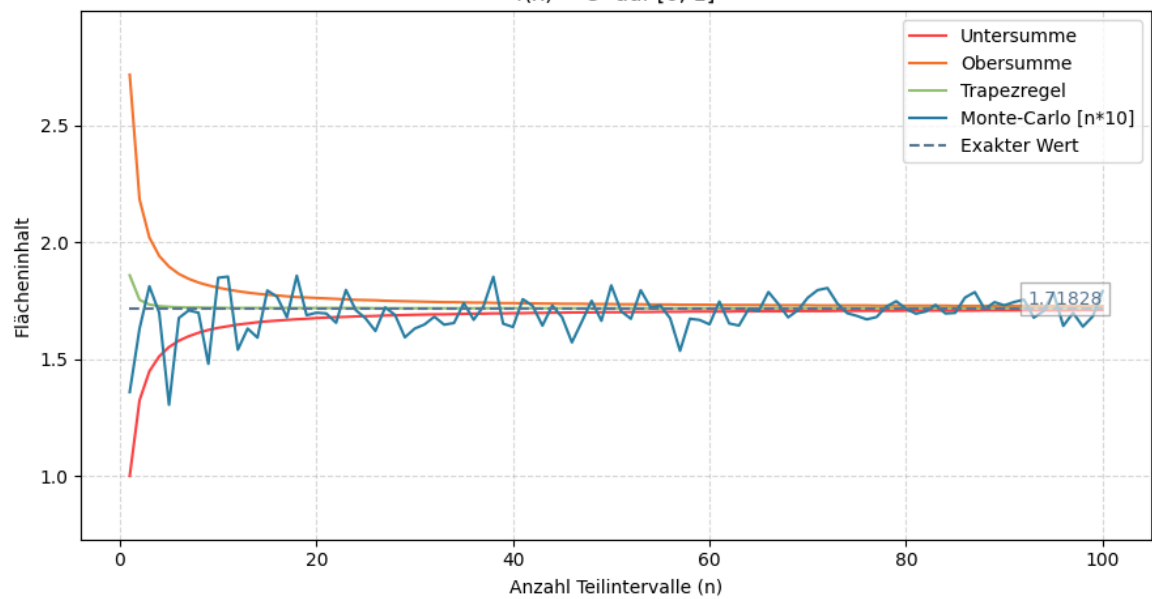
Numerische Integration mittels Trapezregel
Ergebnis bei $n=20$ | ≈ 1.71864

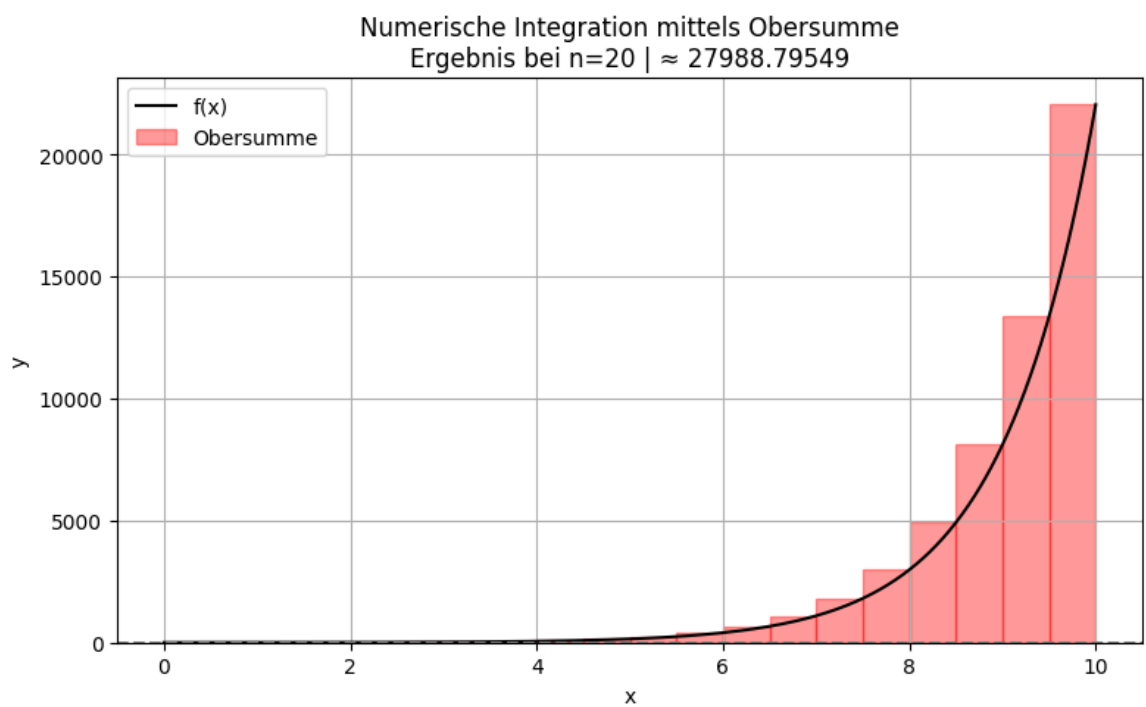
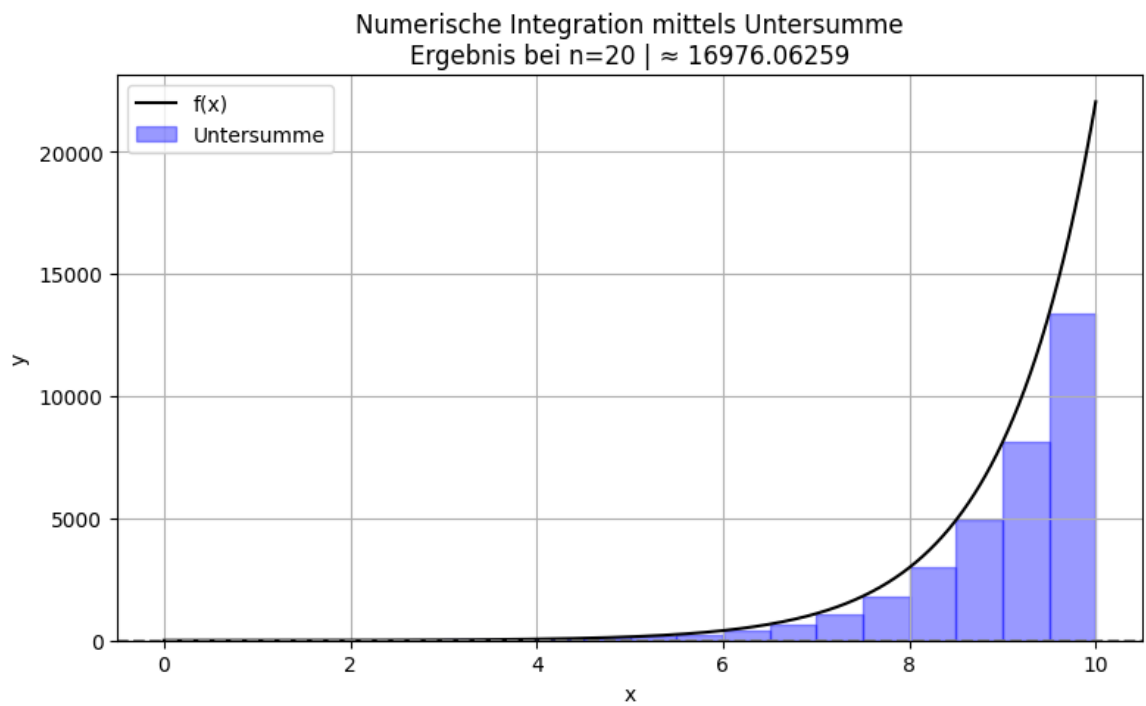


Numerische Integration mittels Monte-Carlo Ergebnis bei $n=200 \mid \approx 1.69893$

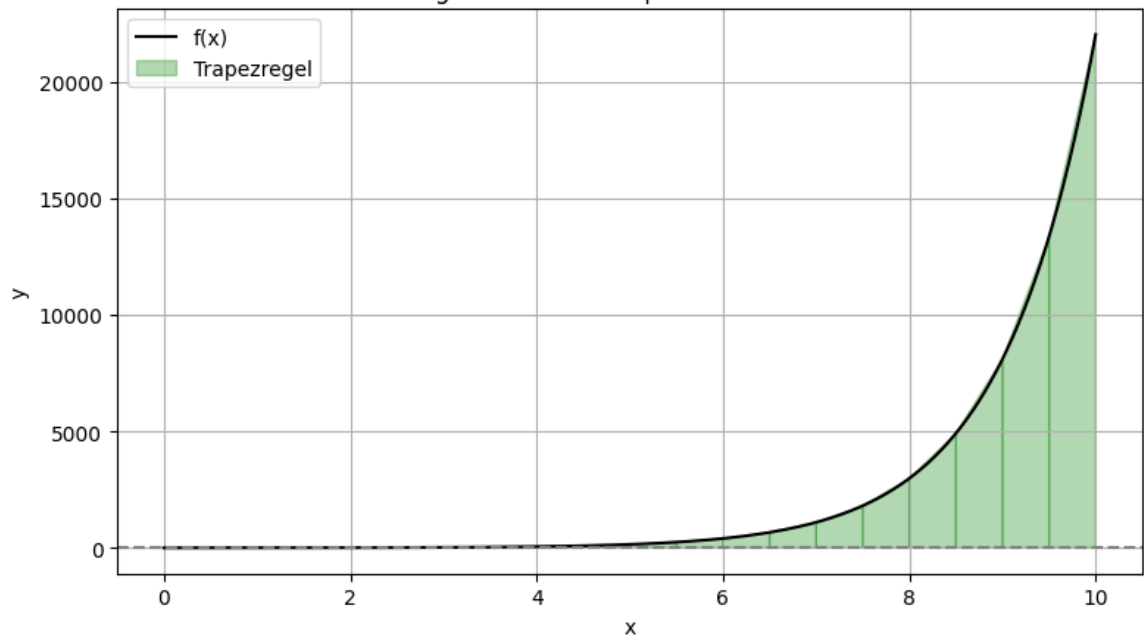


$f(x) = e^x$ auf $[0, 1]$

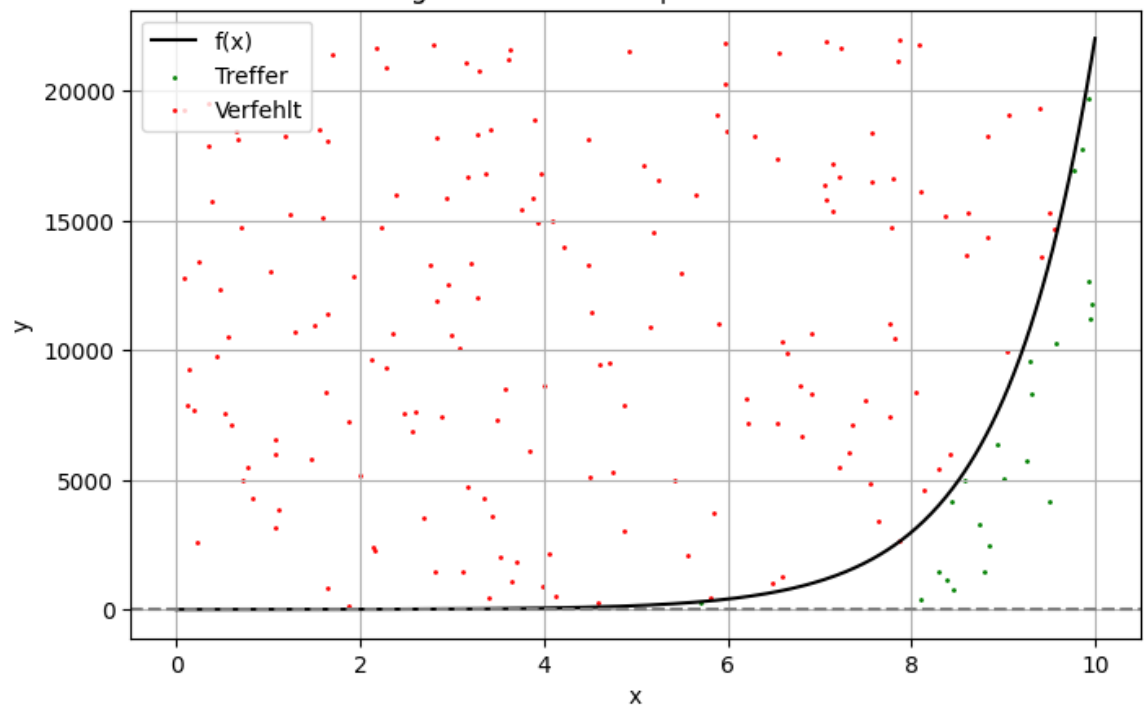


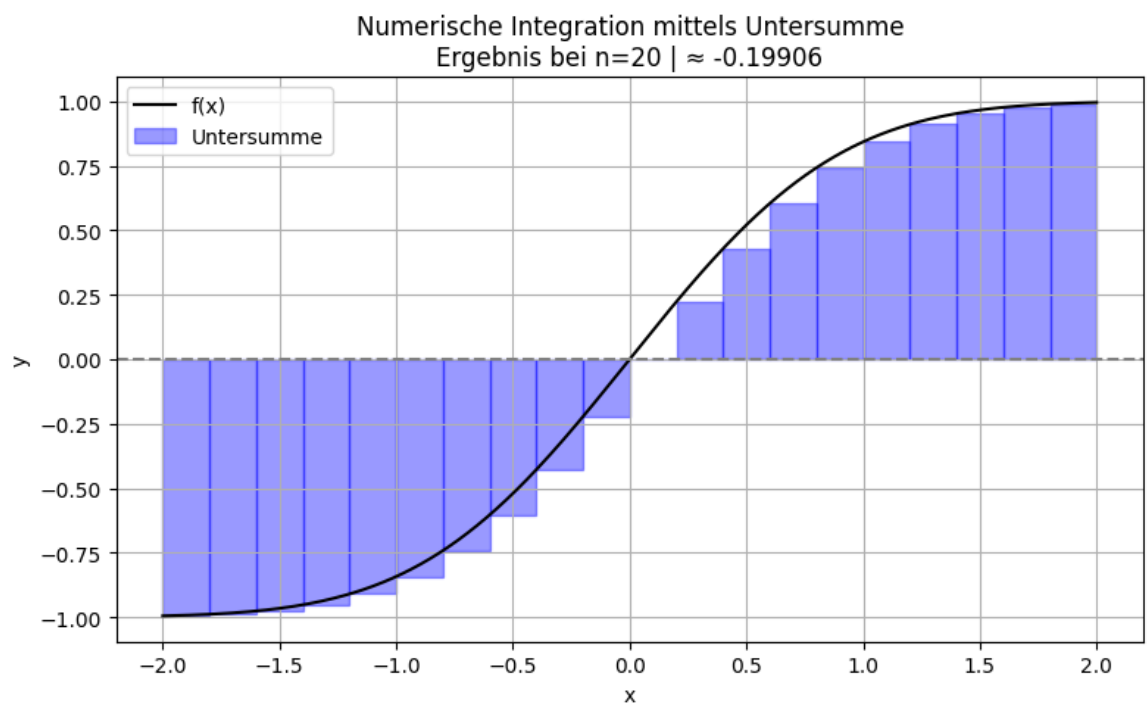
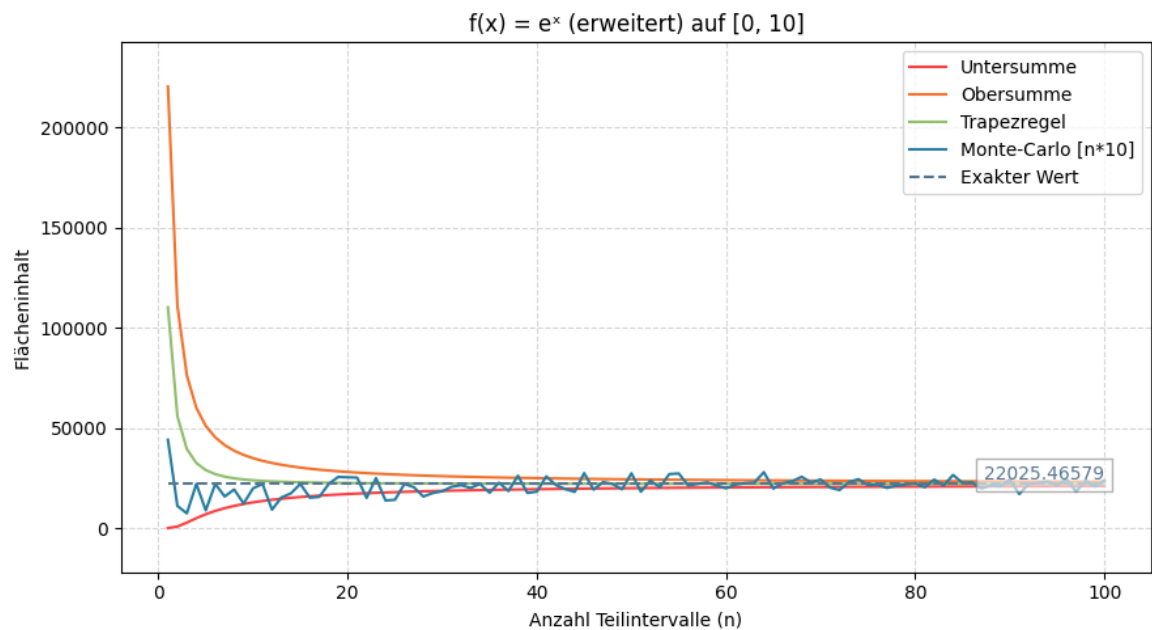


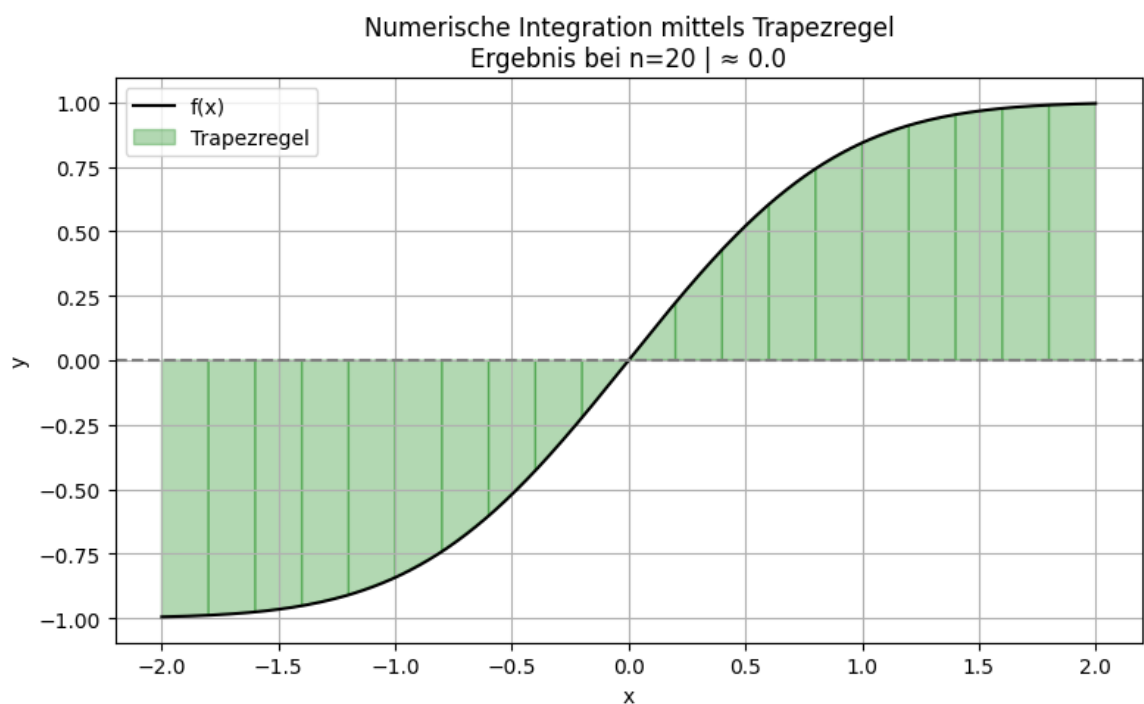
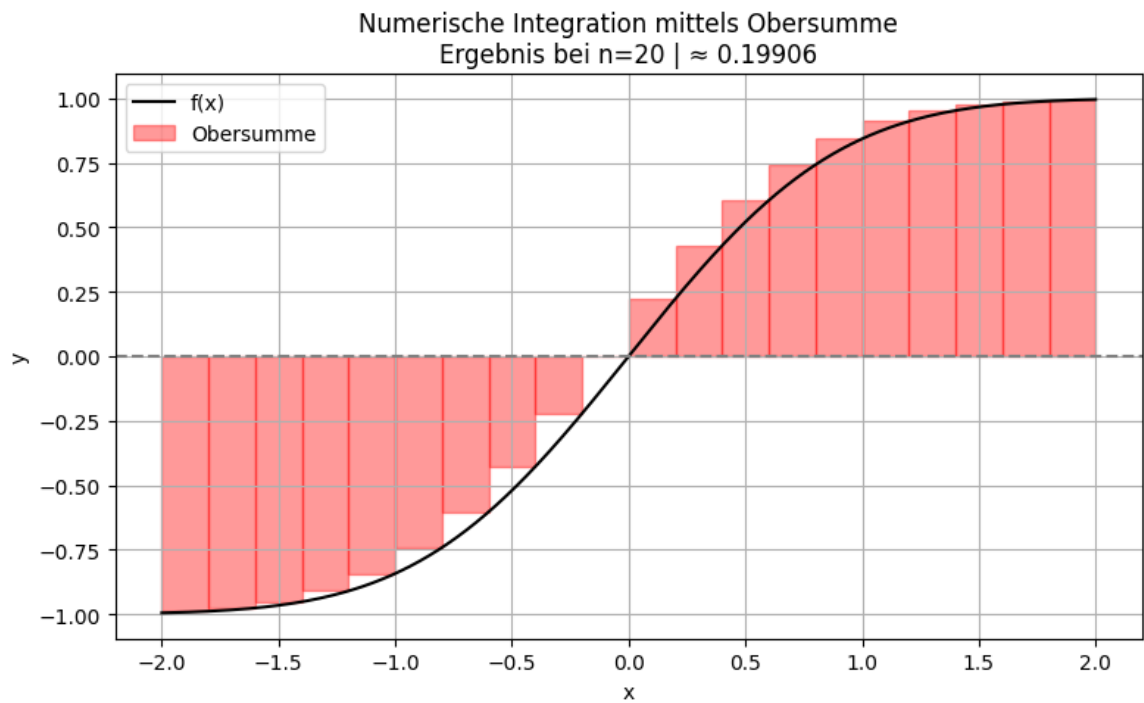
Numerische Integration mittels Trapezregel
Ergebnis bei $n=20$ | ≈ 22482.42904

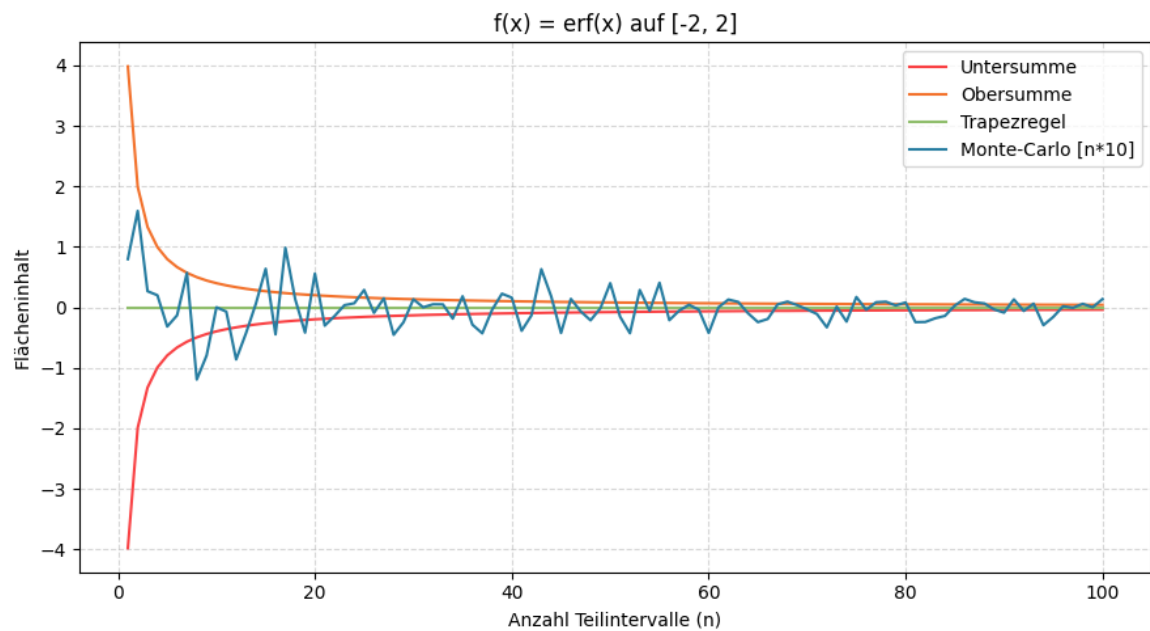
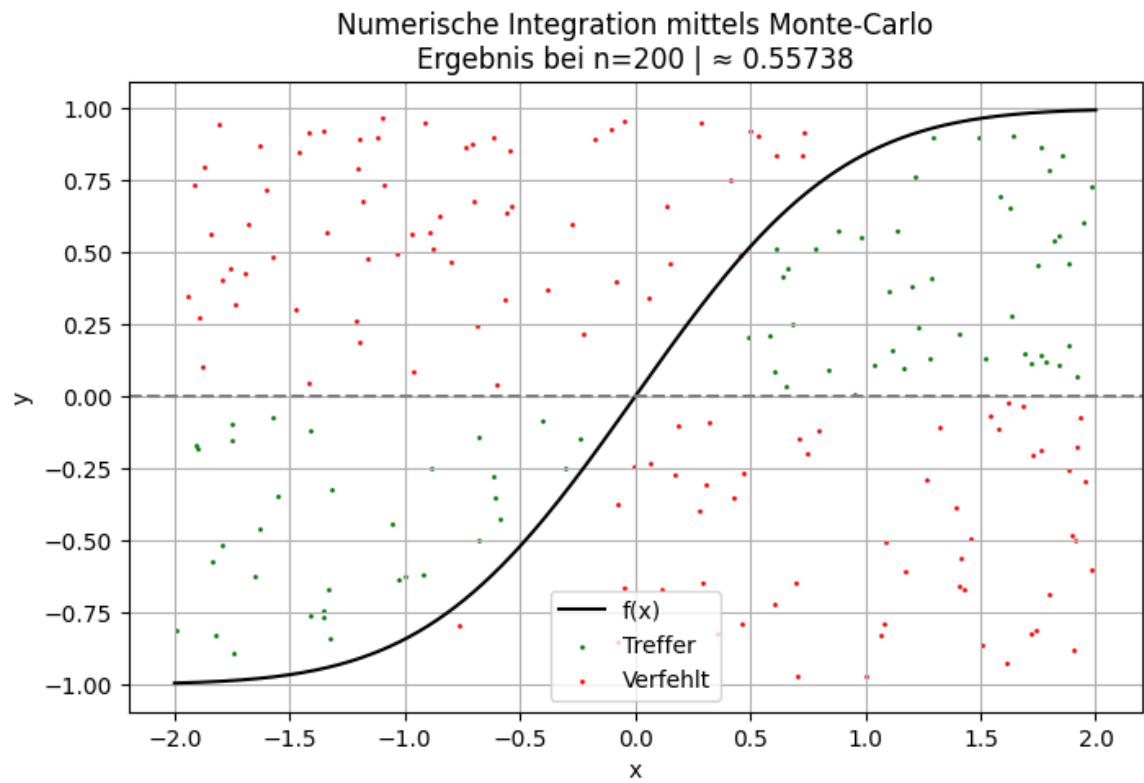


Numerische Integration mittels Monte-Carlo
Ergebnis bei $n=200$ | ≈ 25330.43566

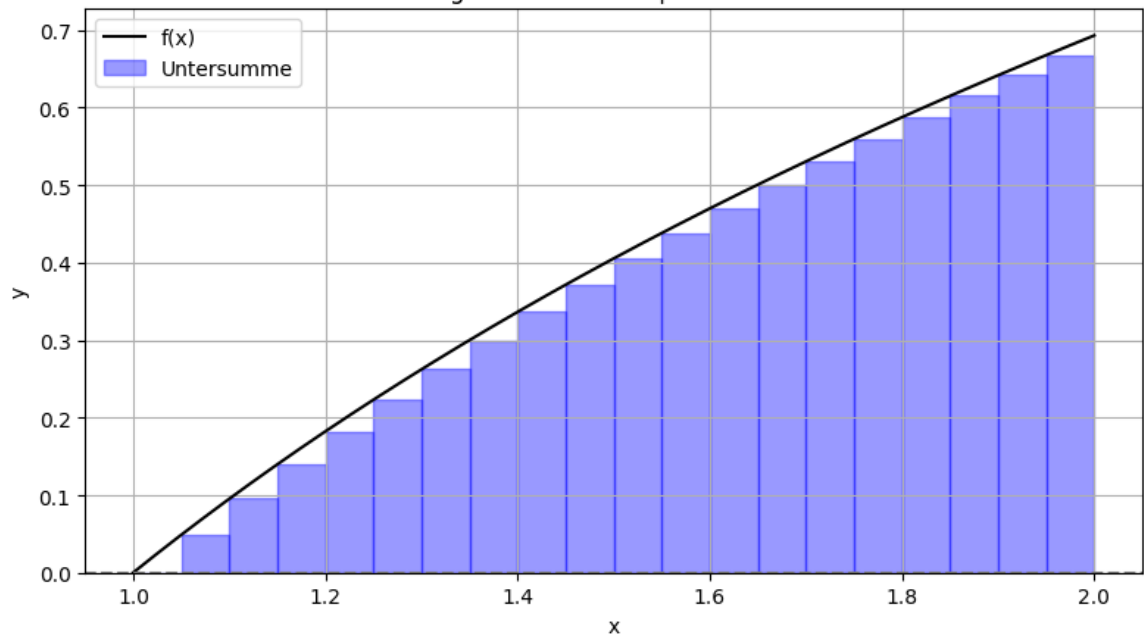




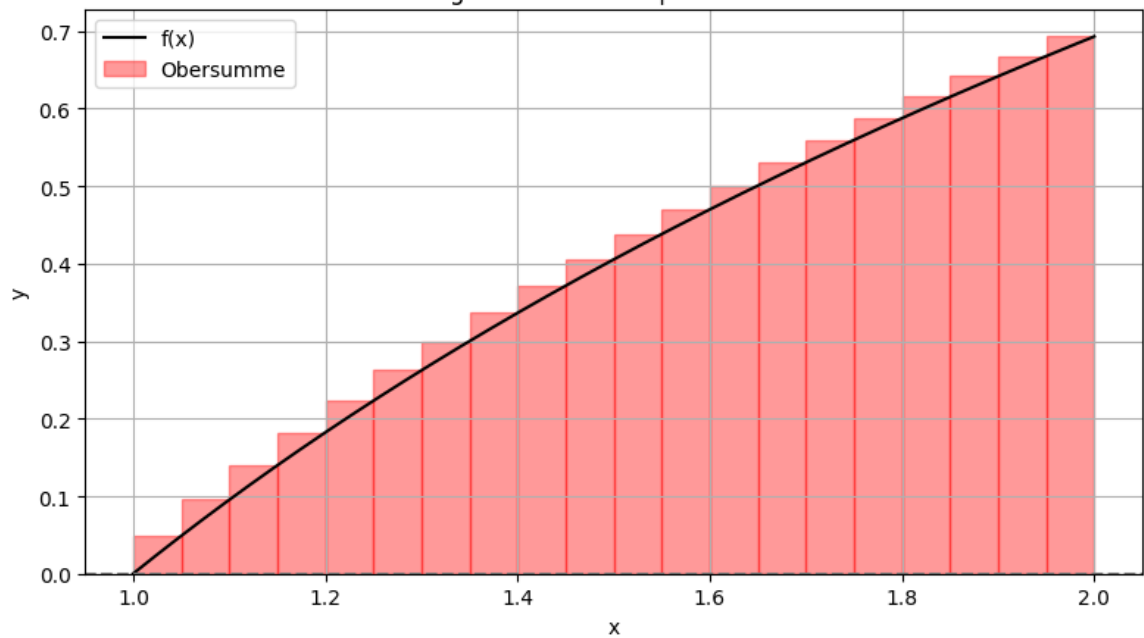




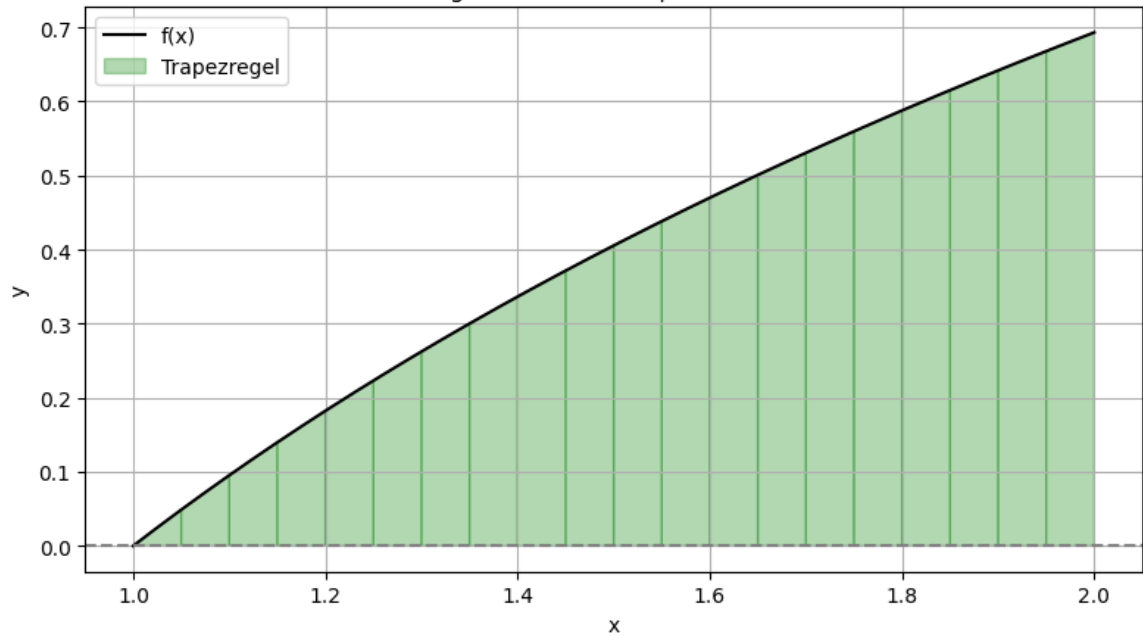
Numerische Integration mittels Untersumme

Ergebnis bei $n=20$ | ≈ 0.36886 

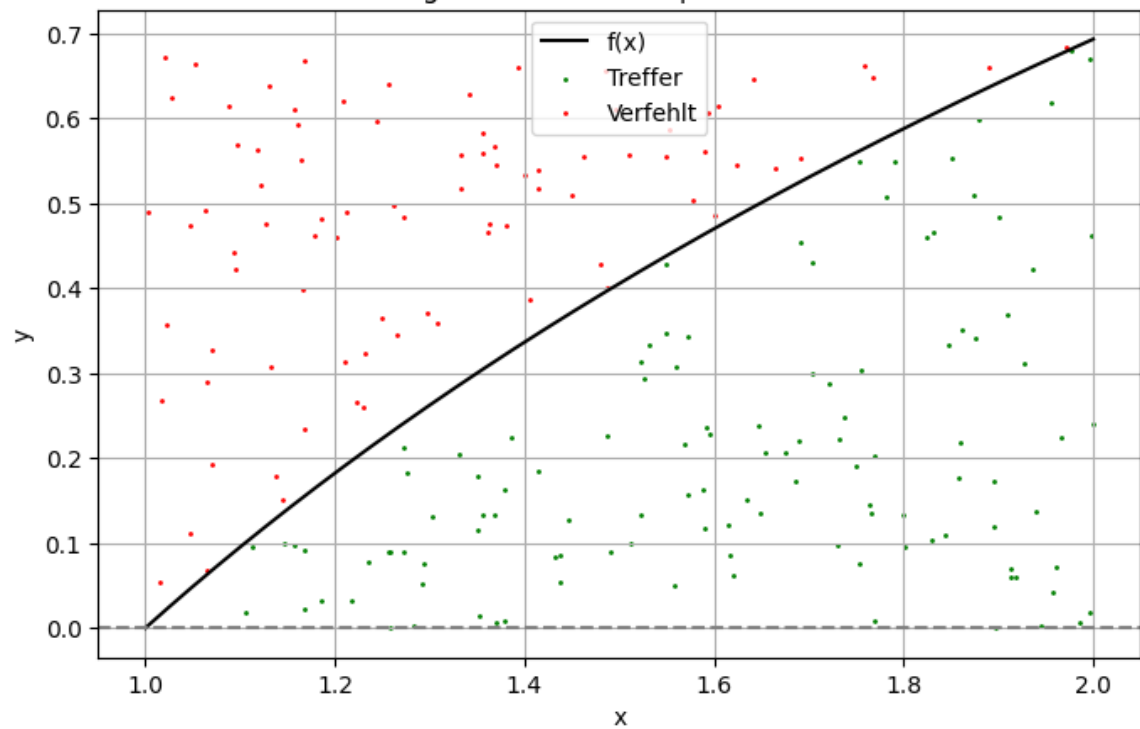
Numerische Integration mittels Obersumme

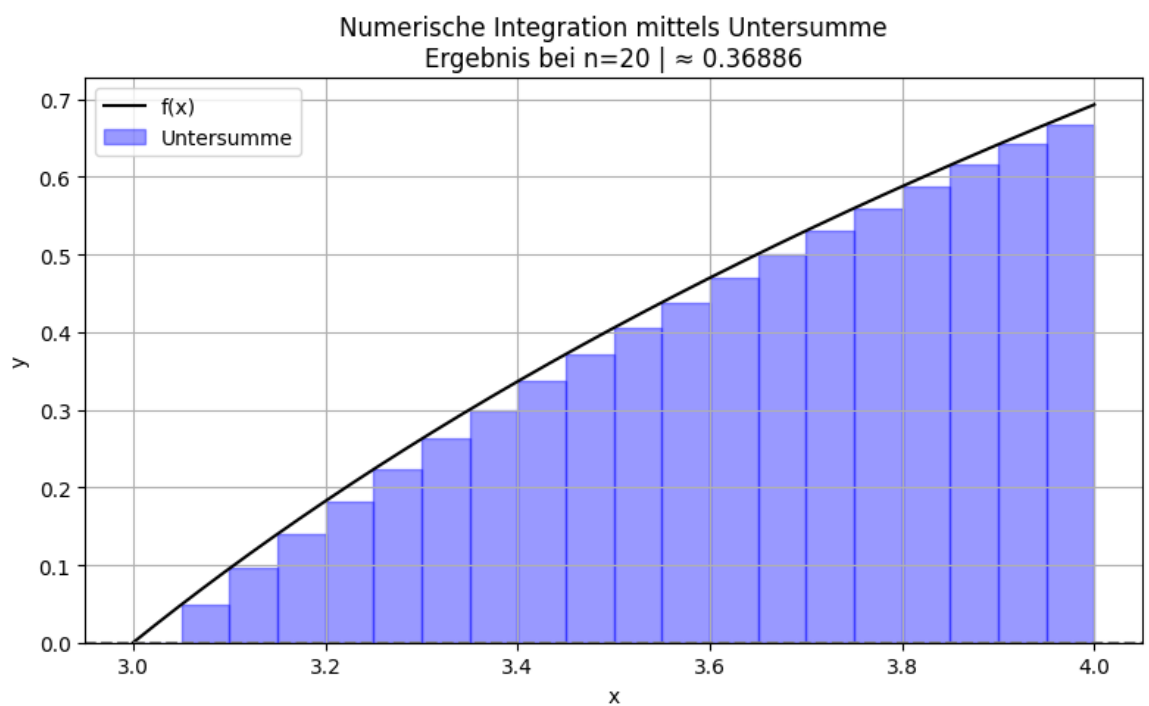
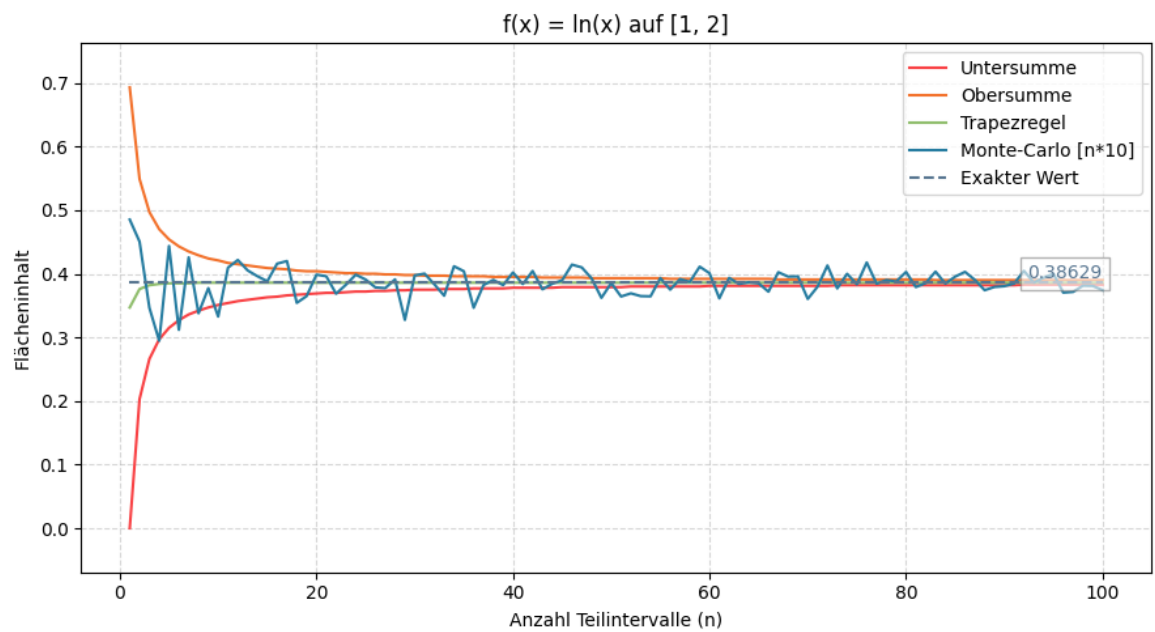
Ergebnis bei $n=20$ | ≈ 0.40352 

Numerische Integration mittels Trapezregel
Ergebnis bei $n=20$ | ≈ 0.38619

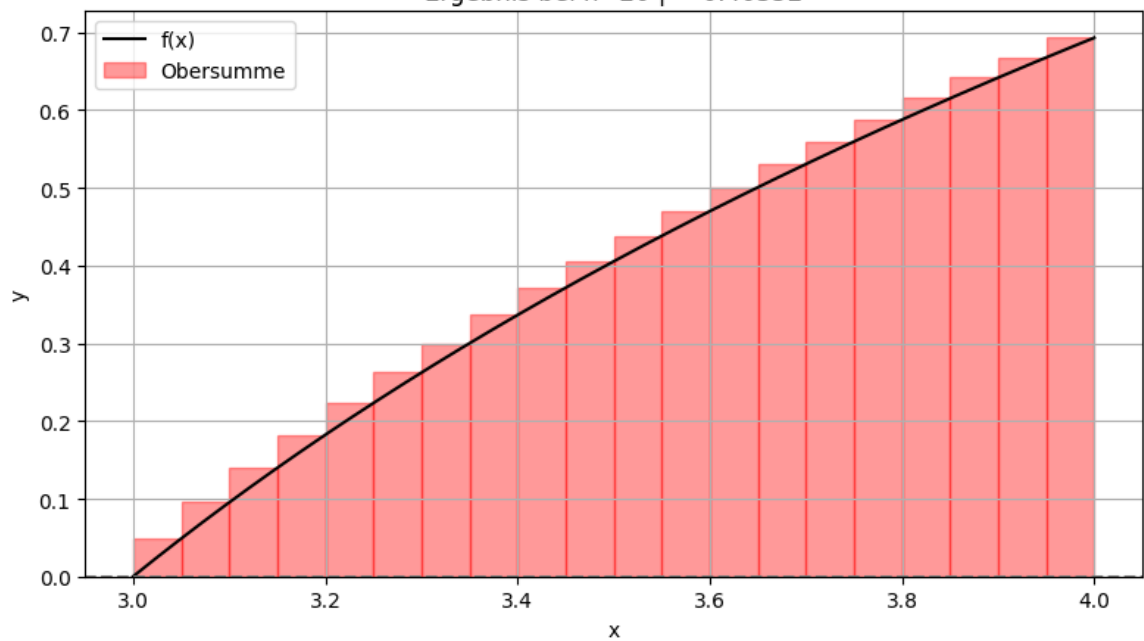


Numerische Integration mittels Monte-Carlo
Ergebnis bei $n=200$ | ≈ 0.39856

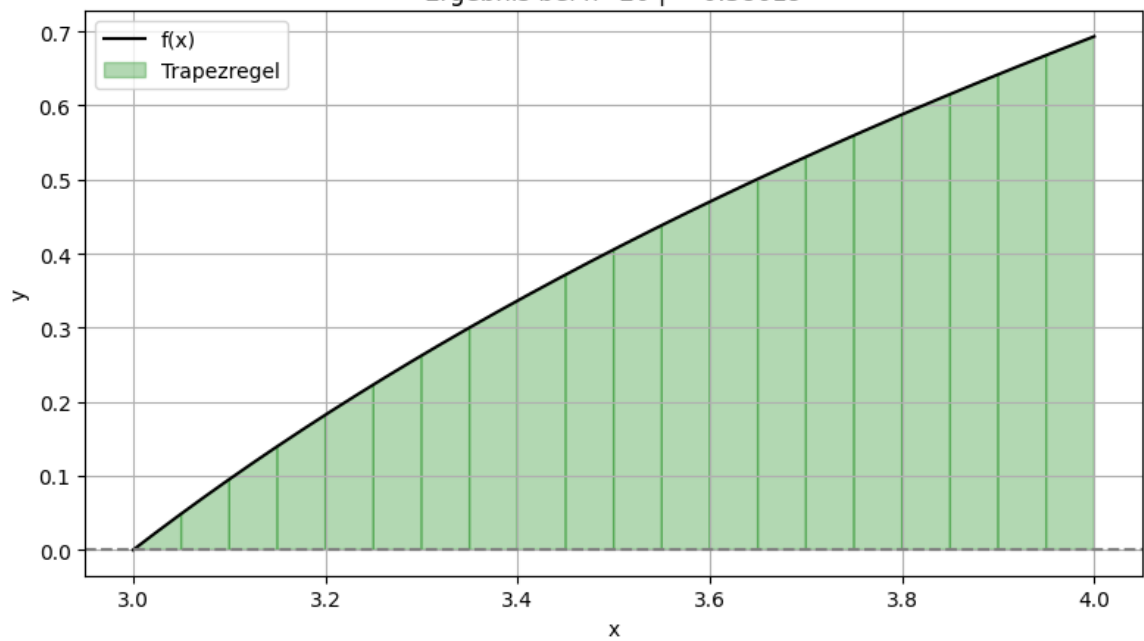




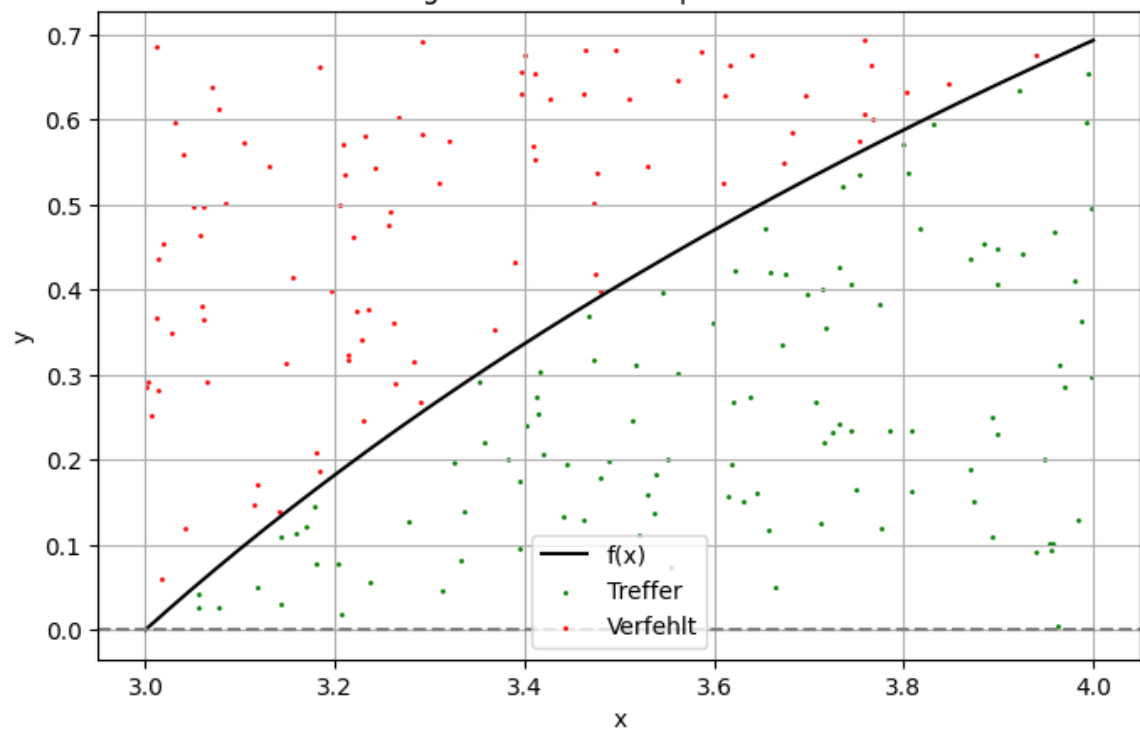
Numerische Integration mittels Obersumme
Ergebnis bei $n=20$ | ≈ 0.40352



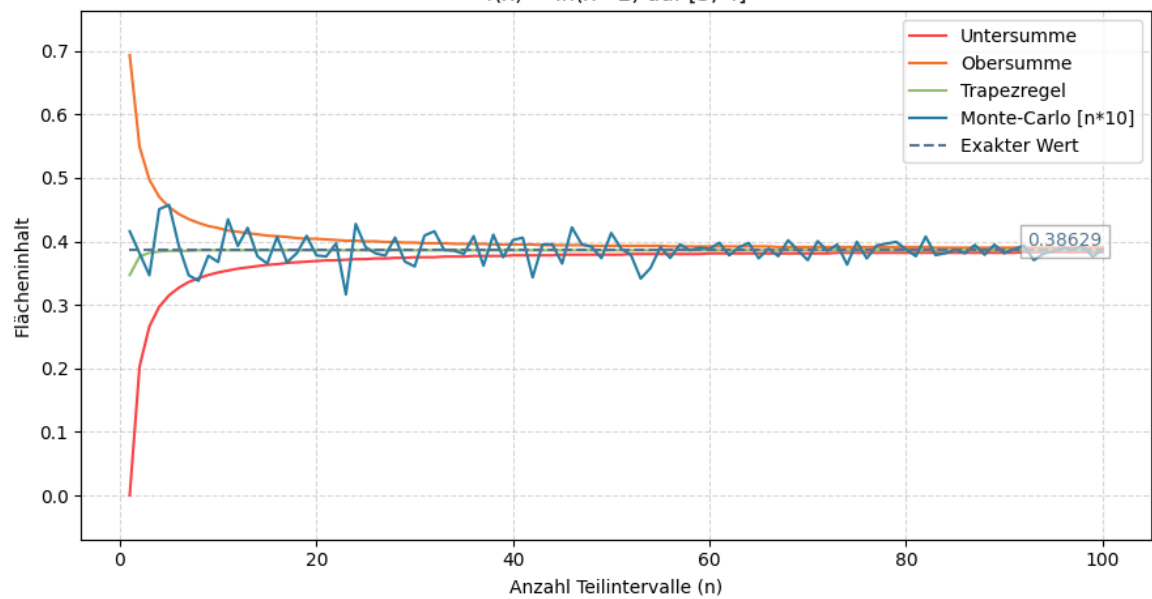
Numerische Integration mittels Trapezregel
Ergebnis bei $n=20$ | ≈ 0.38619

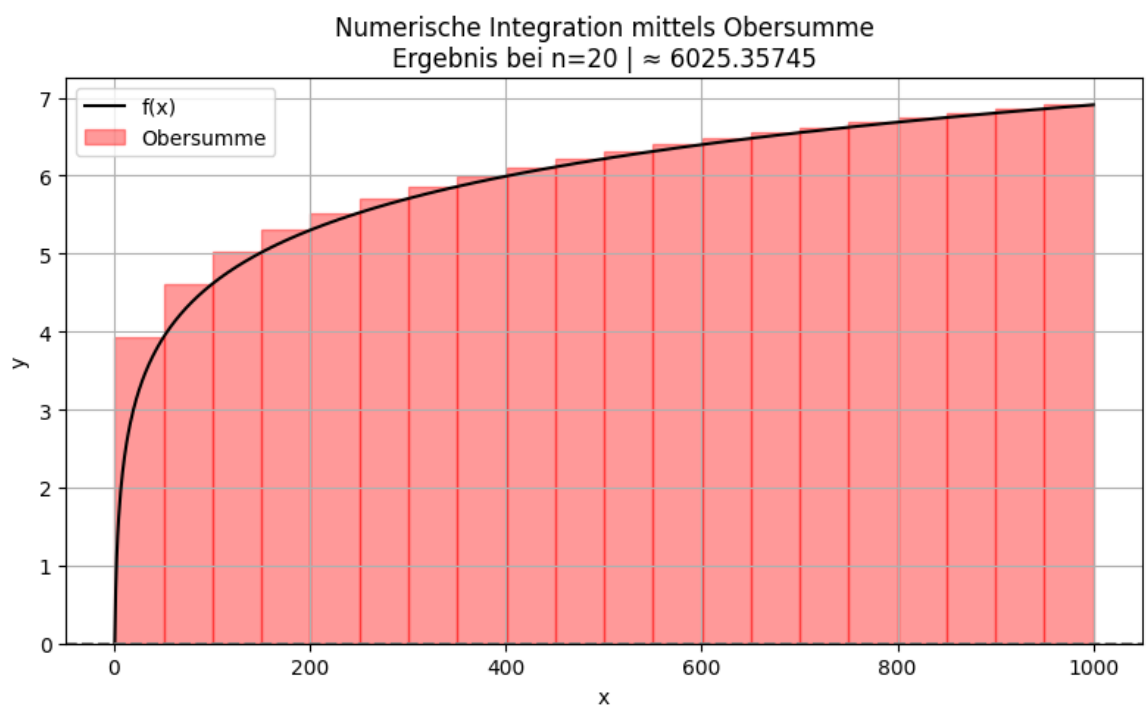
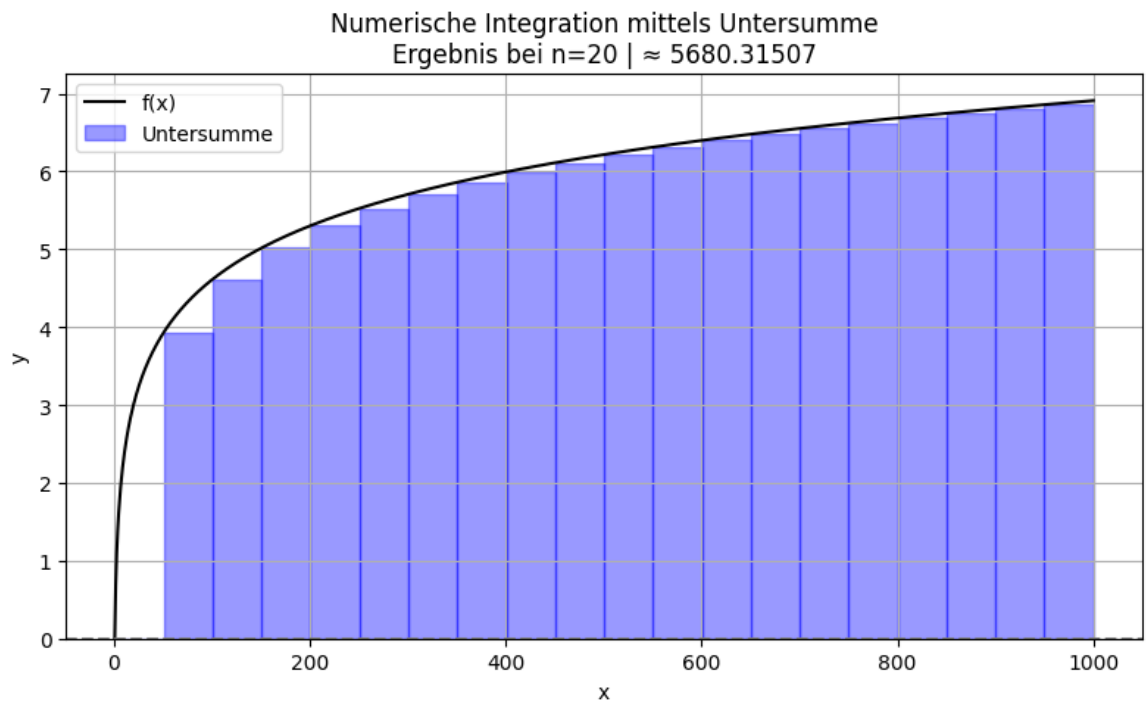


Numerische Integration mittels Monte-Carlo Ergebnis bei $n=200$ | ≈ 0.37777

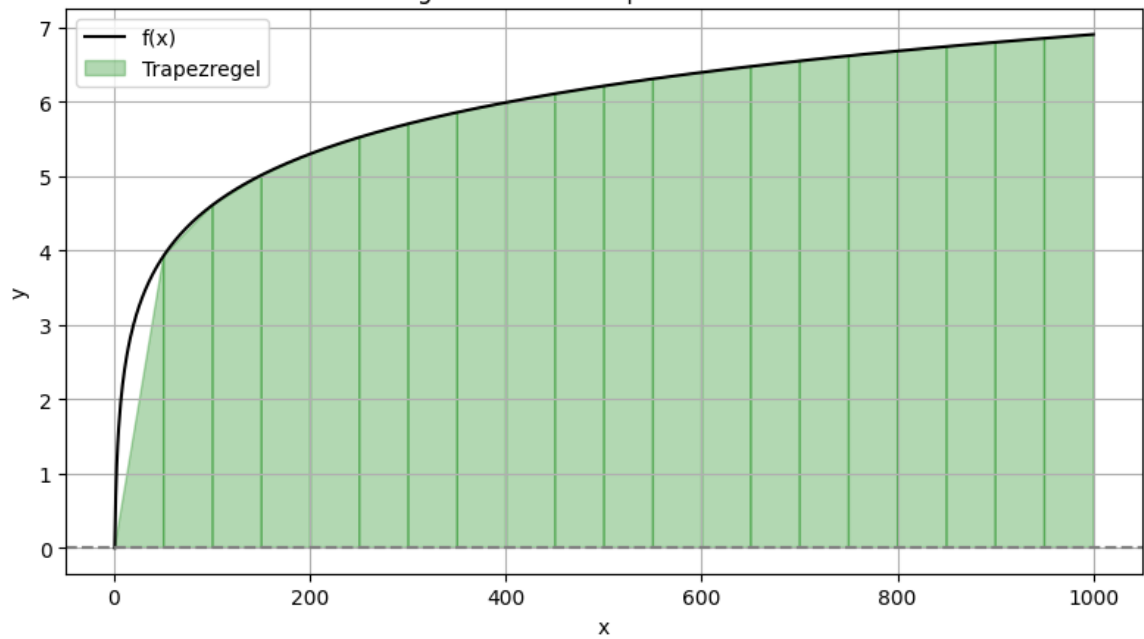


$f(x) = \ln(x - 2)$ auf $[3, 4]$

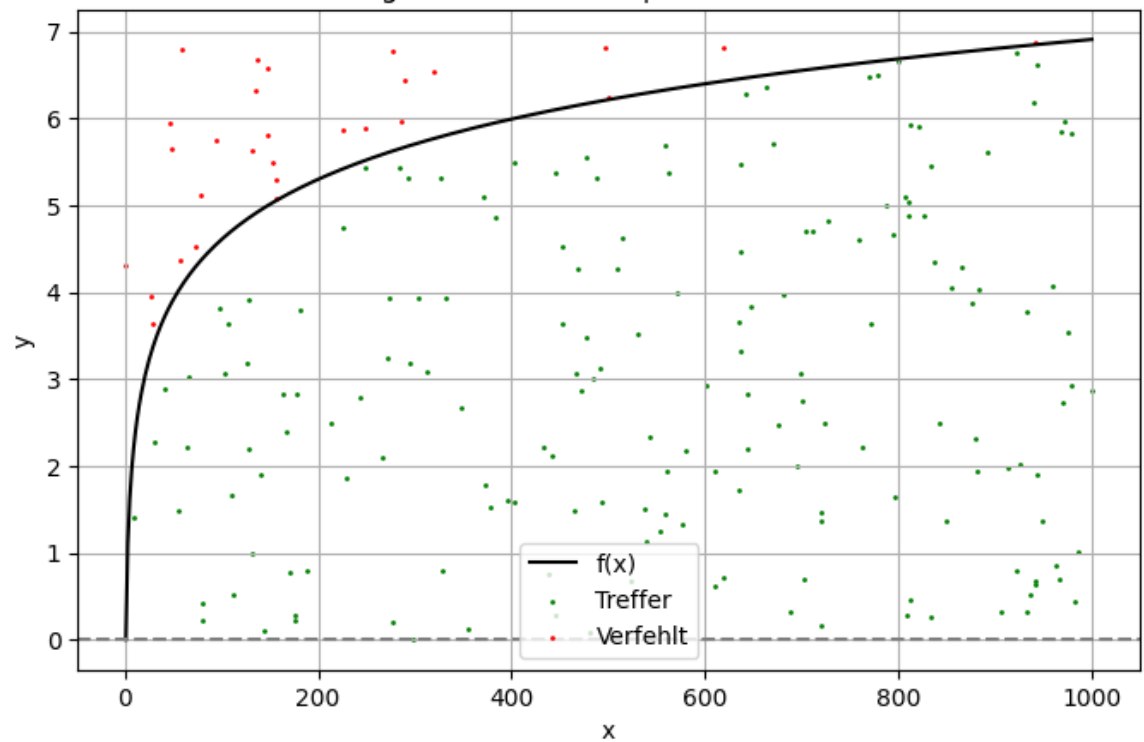


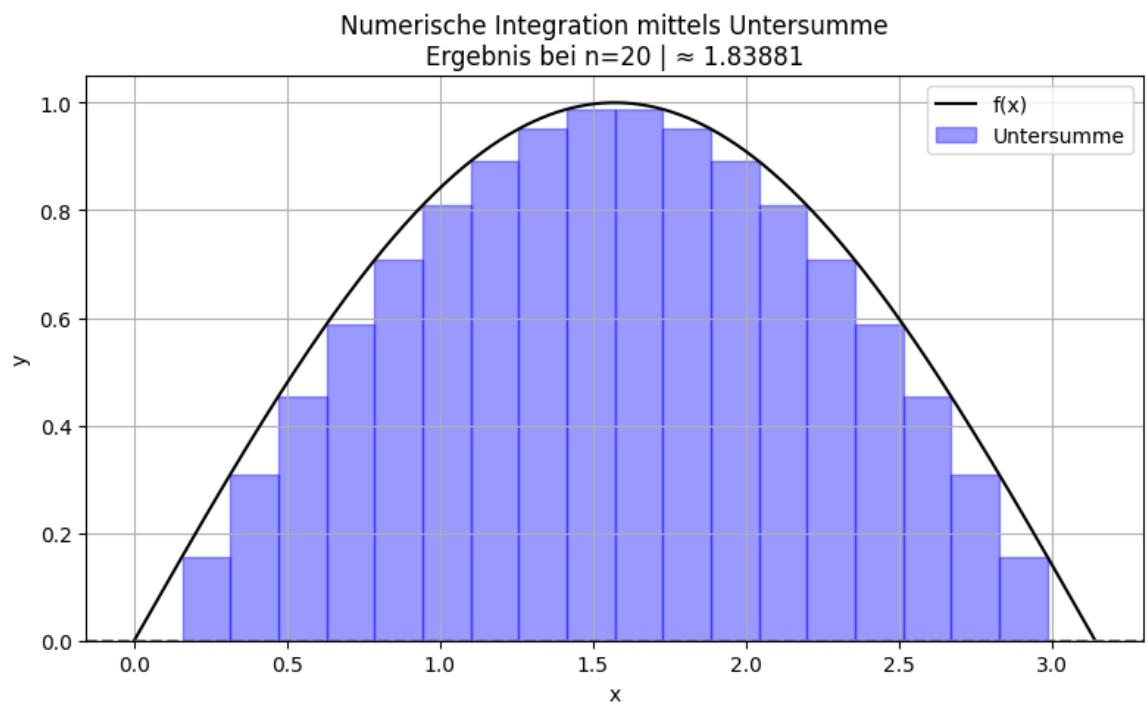
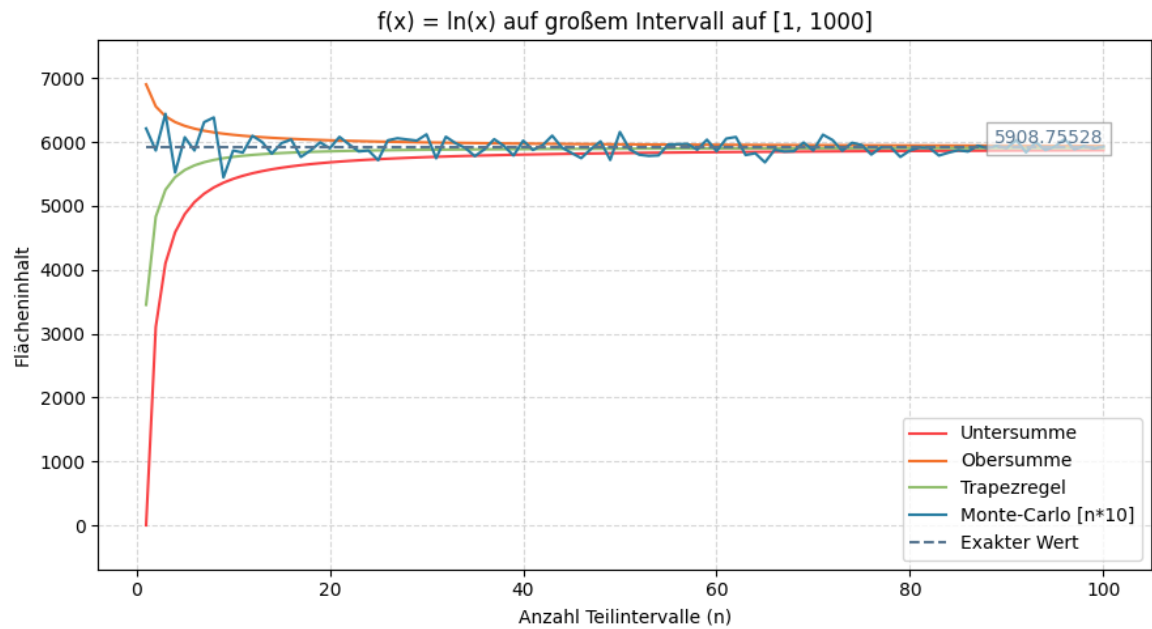


Numerische Integration mittels Trapezregel
Ergebnis bei $n=20$ | ≈ 5852.83626

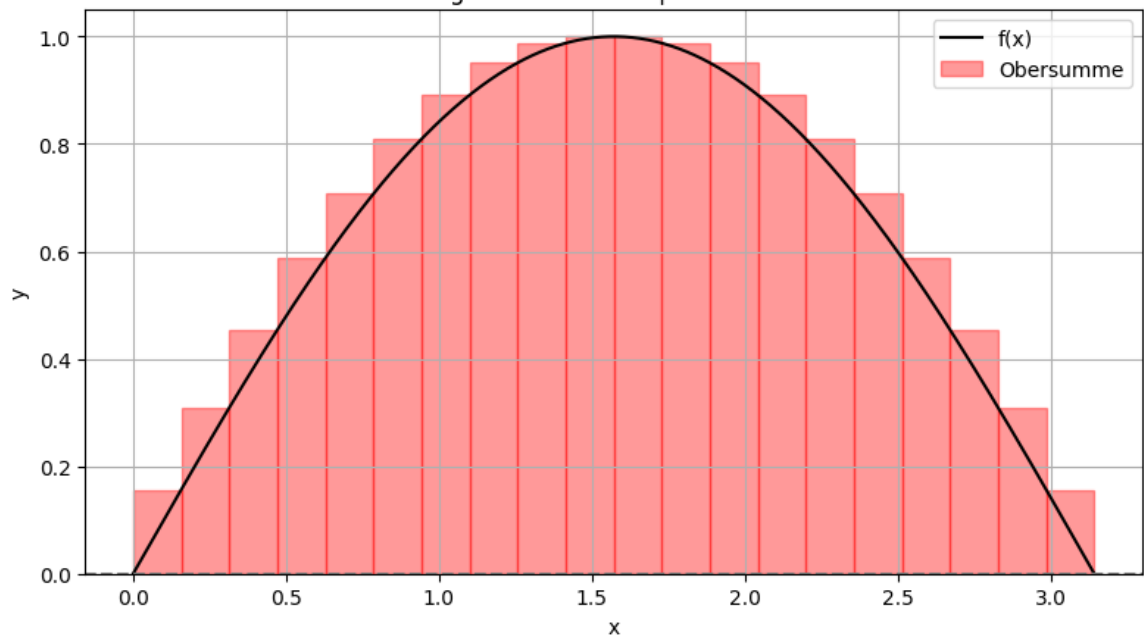


Numerische Integration mittels Monte-Carlo
Ergebnis bei $n=200$ | ≈ 5900.22463

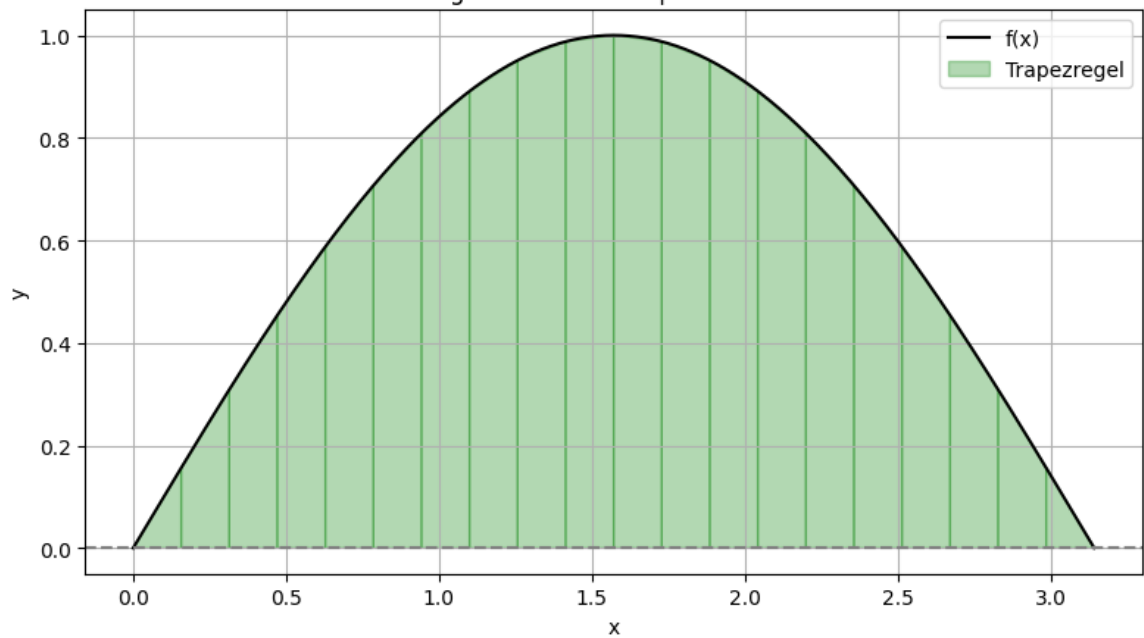




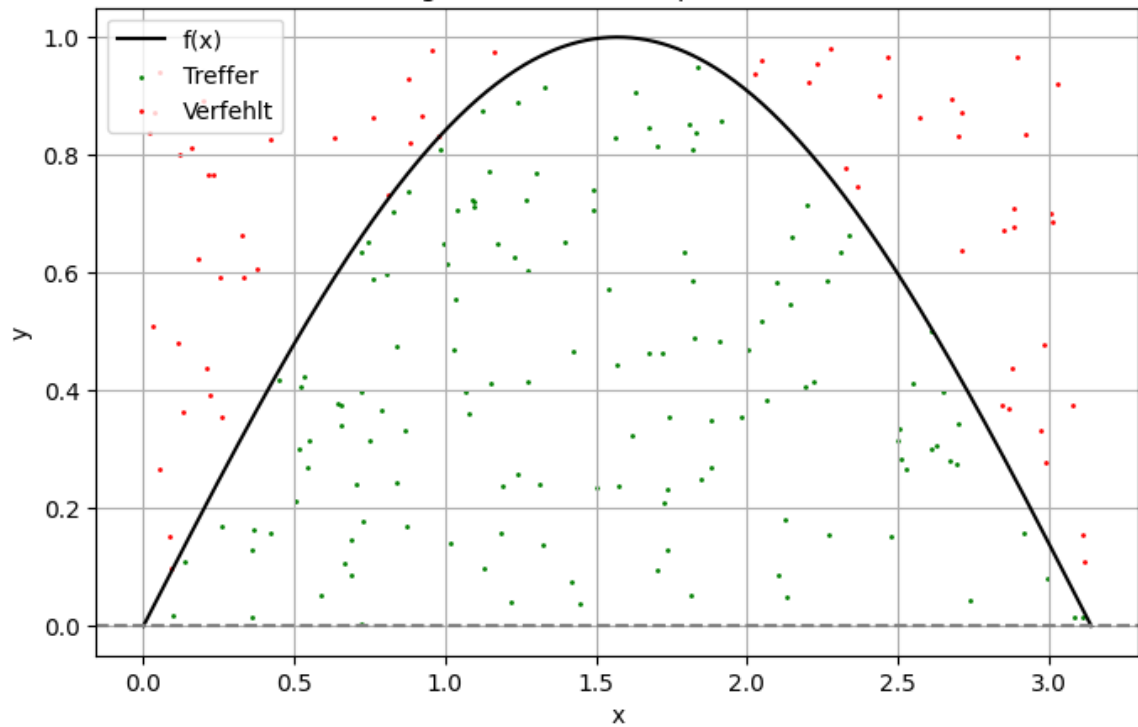
Numerische Integration mittels Obersumme
Ergebnis bei $n=20$ | ≈ 2.15297



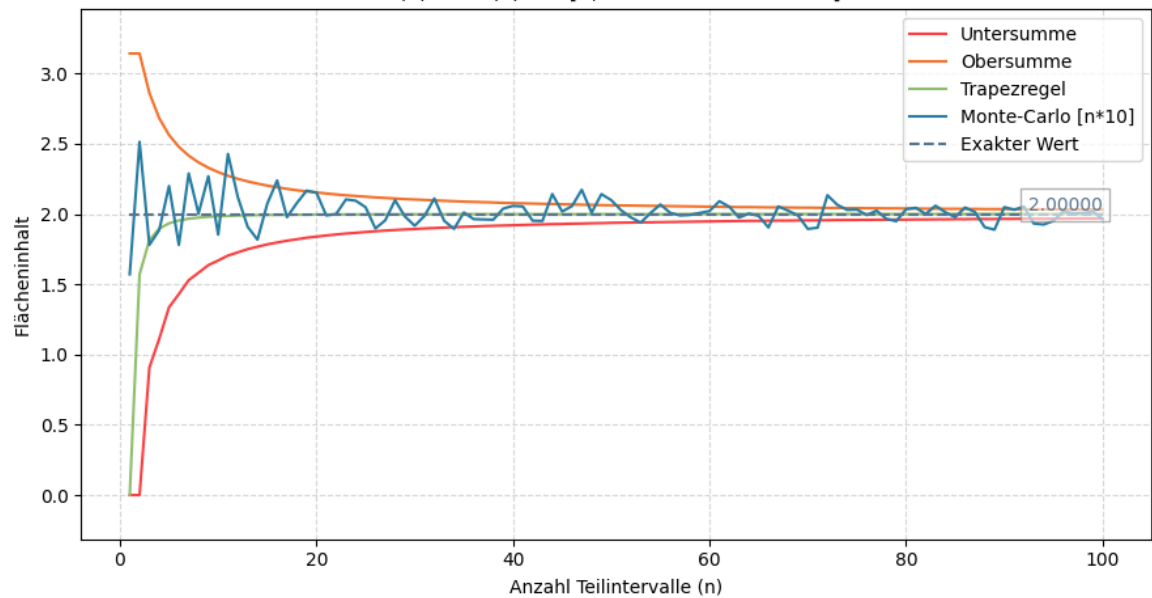
Numerische Integration mittels Trapezregel
Ergebnis bei $n=20$ | ≈ 1.99589

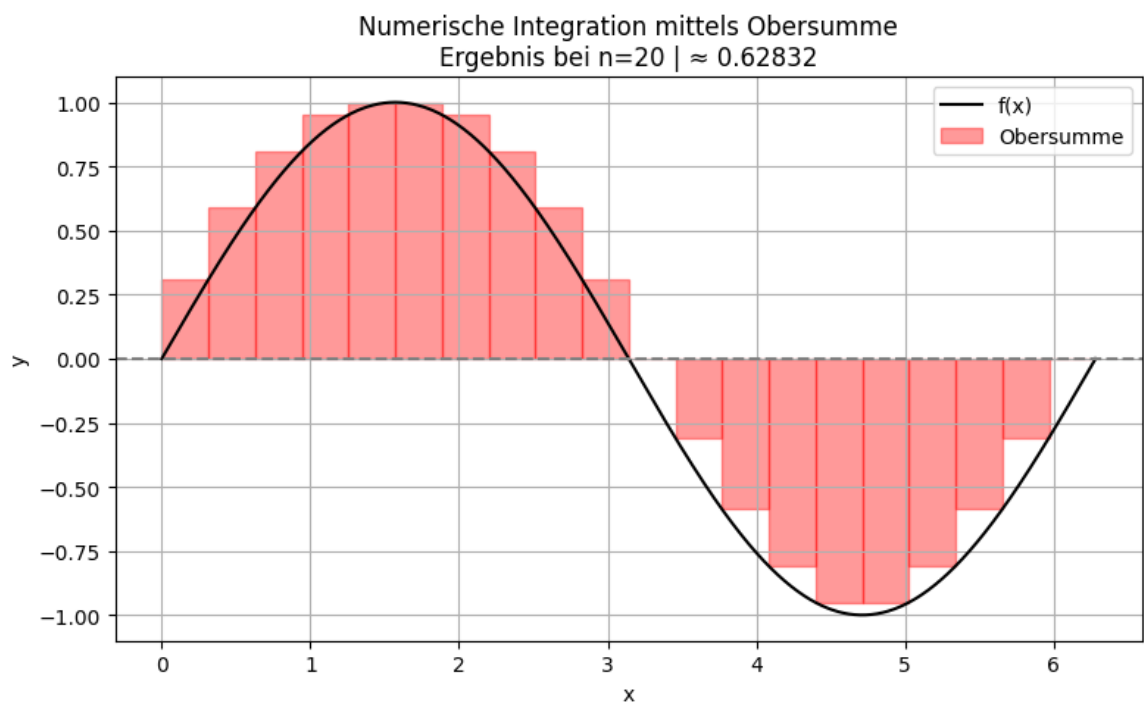
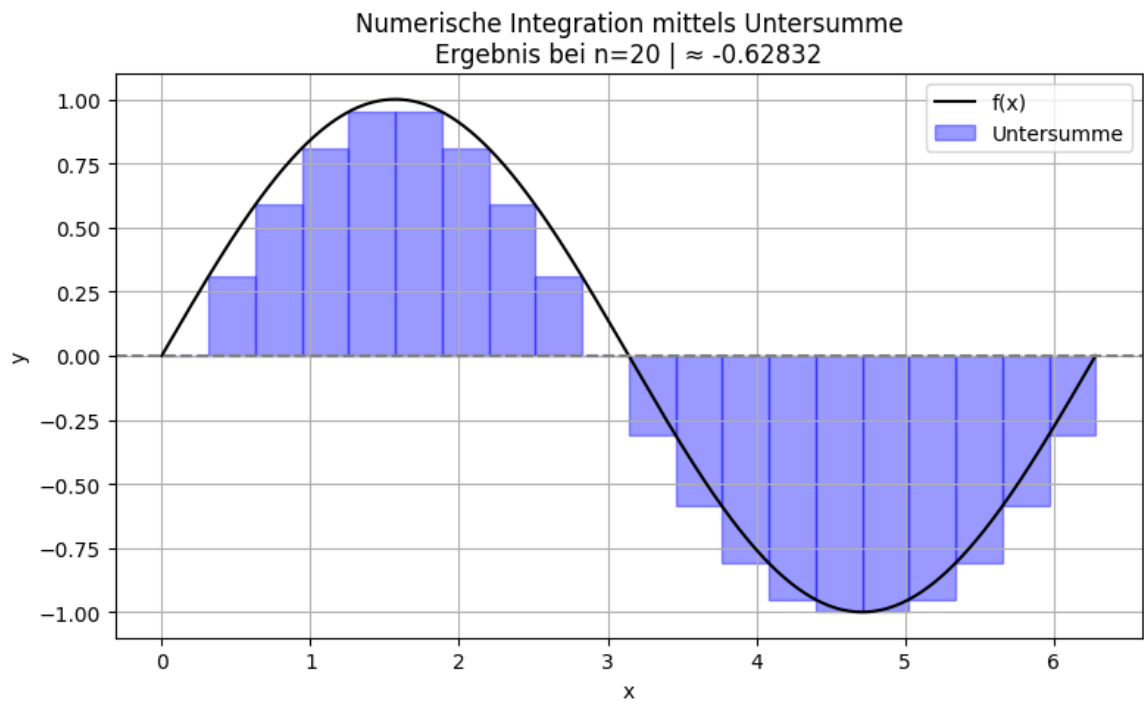


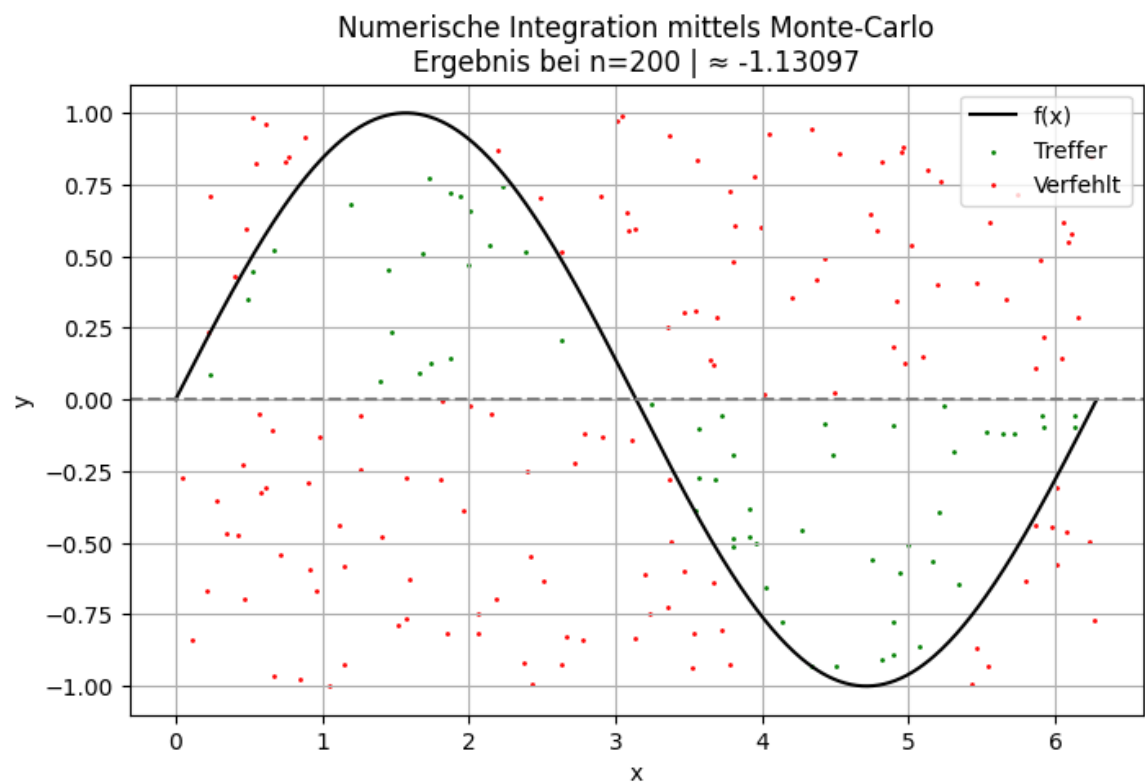
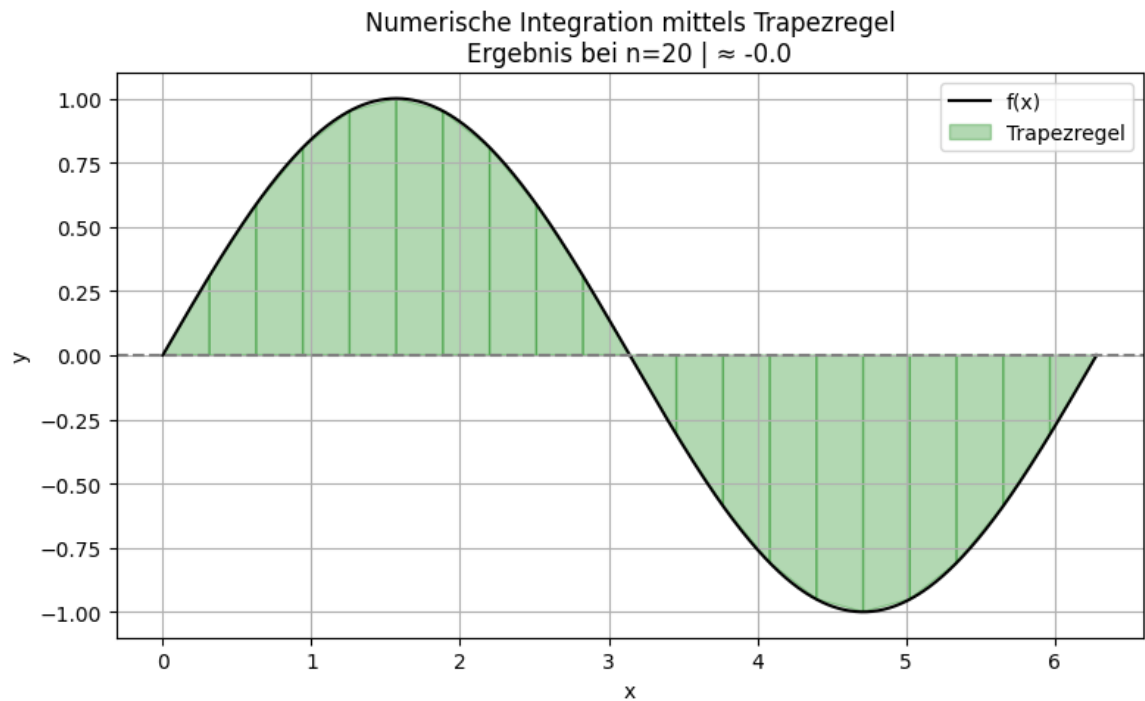
Numerische Integration mittels Monte-Carlo
Ergebnis bei $n=200$ | ≈ 2.15199

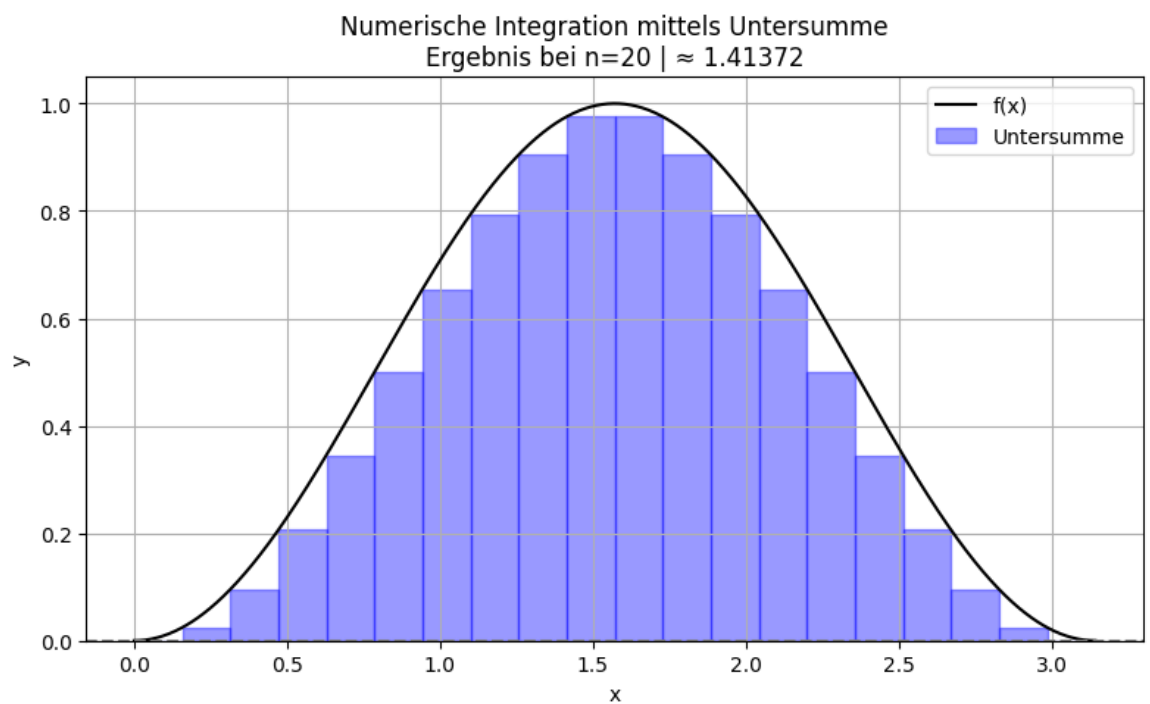
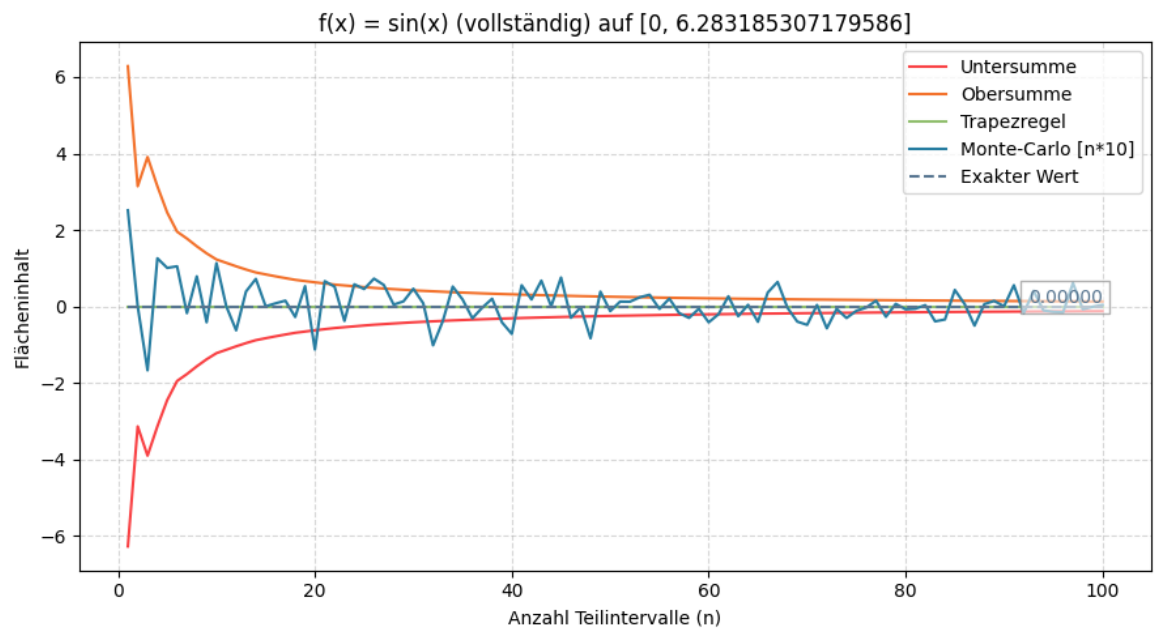


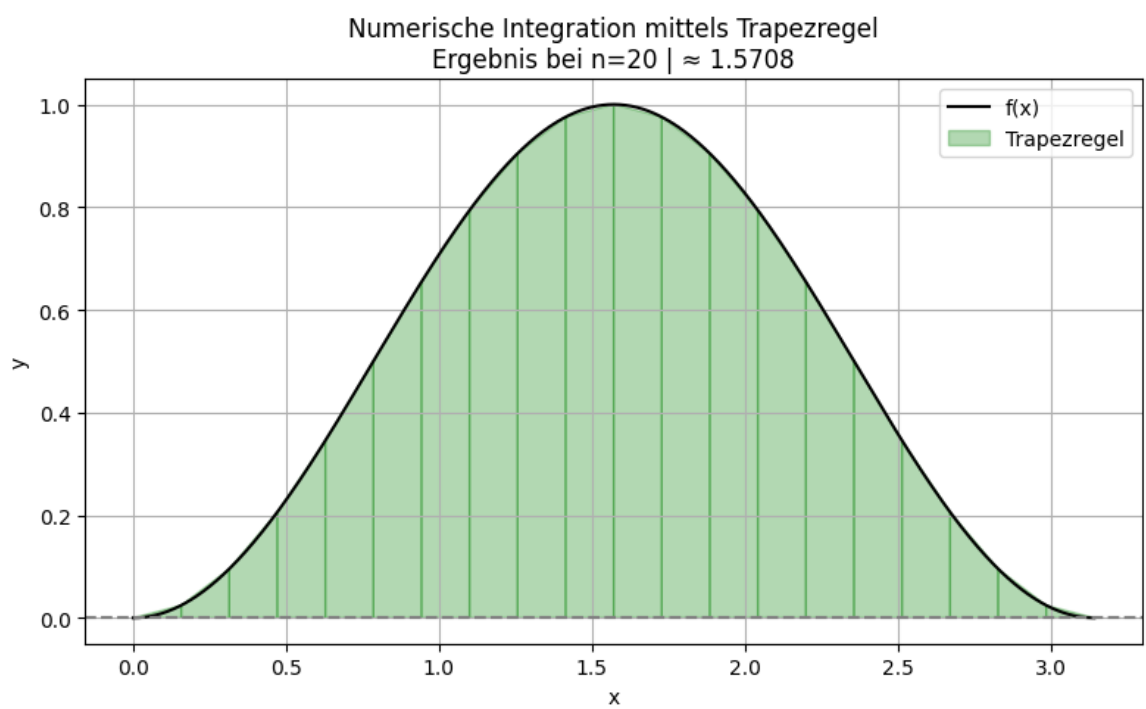
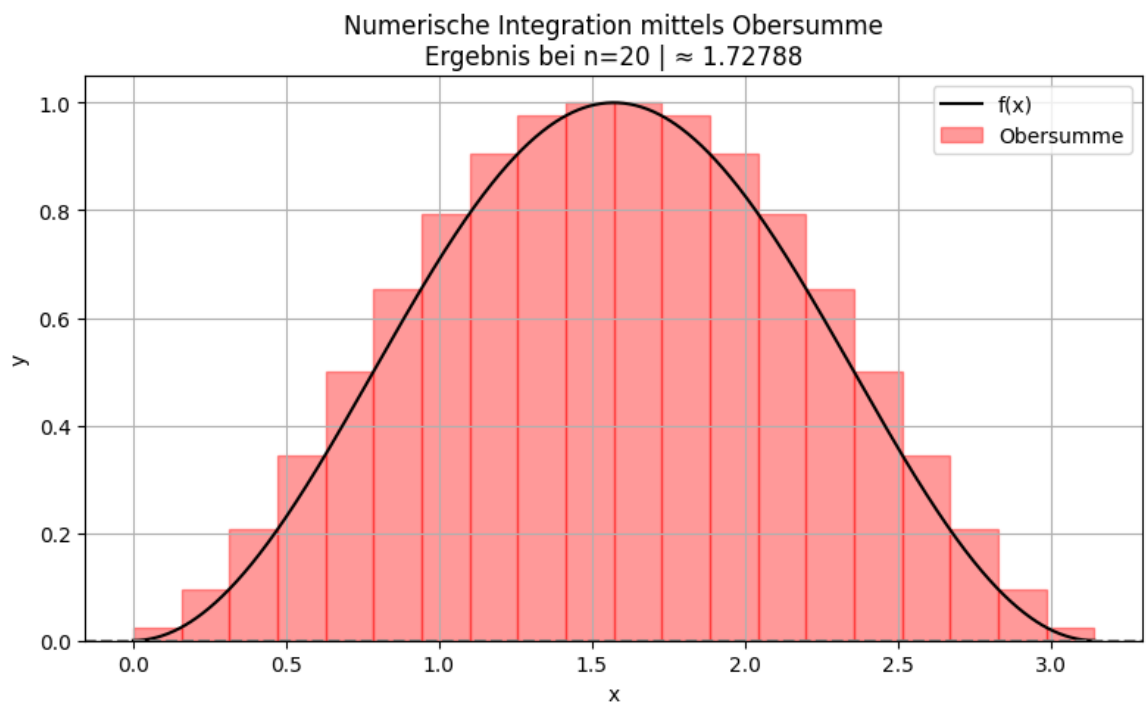
$f(x) = \sin(x)$ auf $[0, 3.141592653589793]$

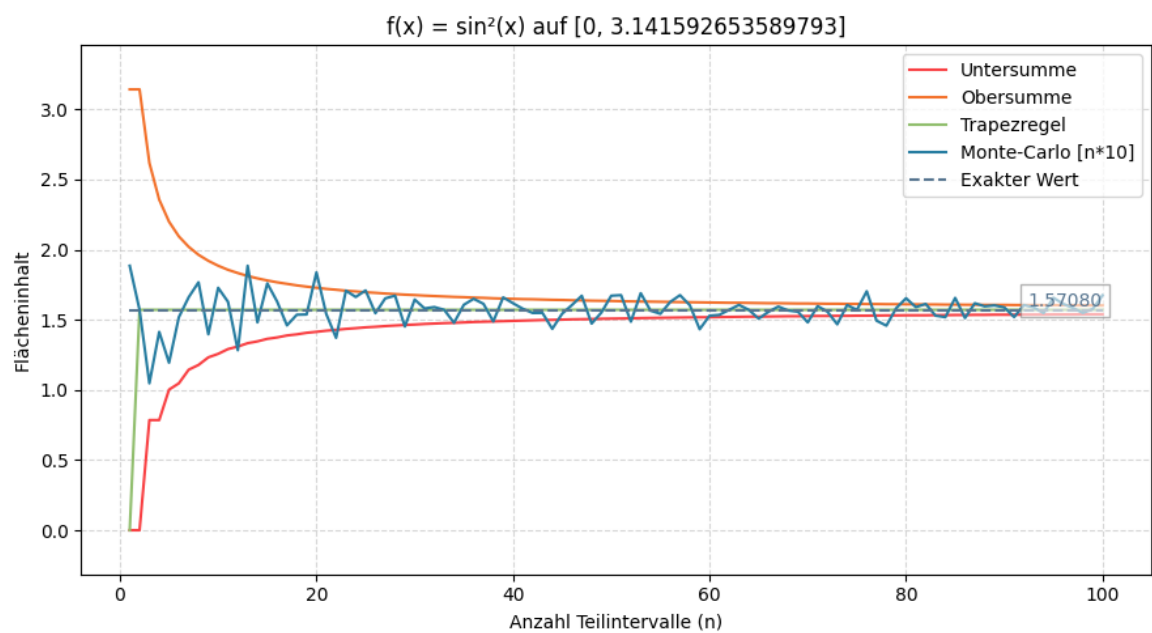
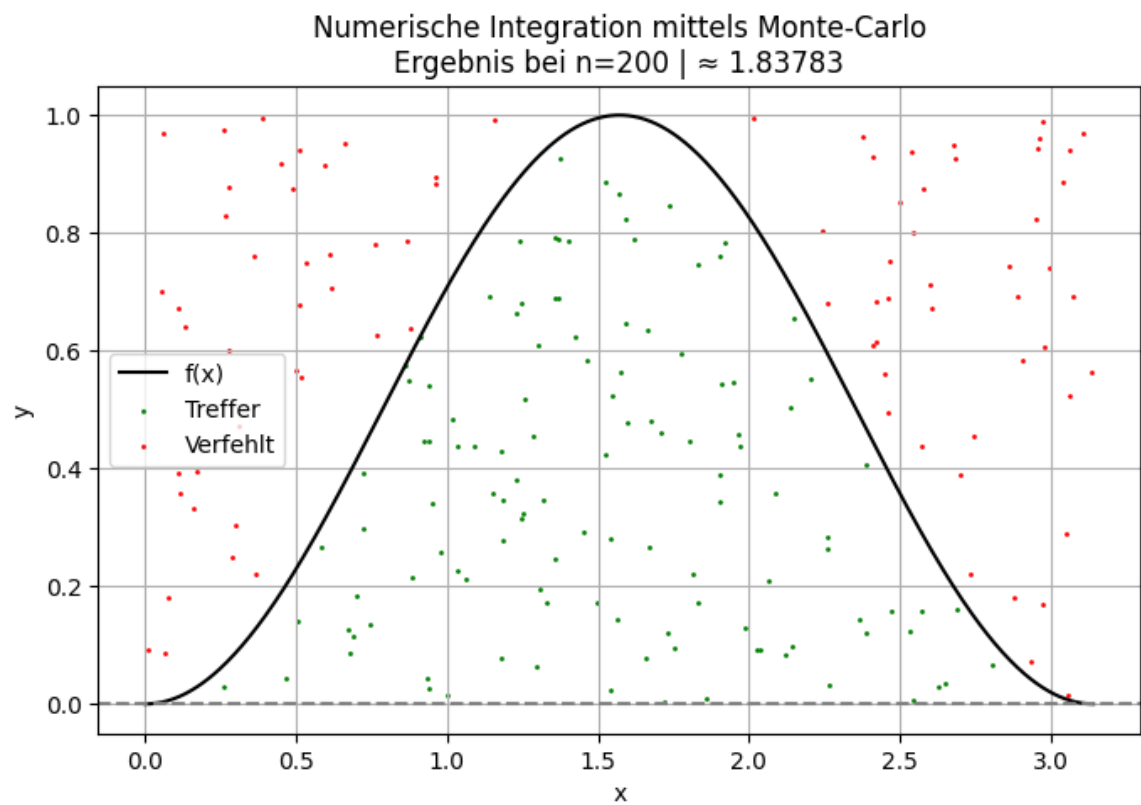


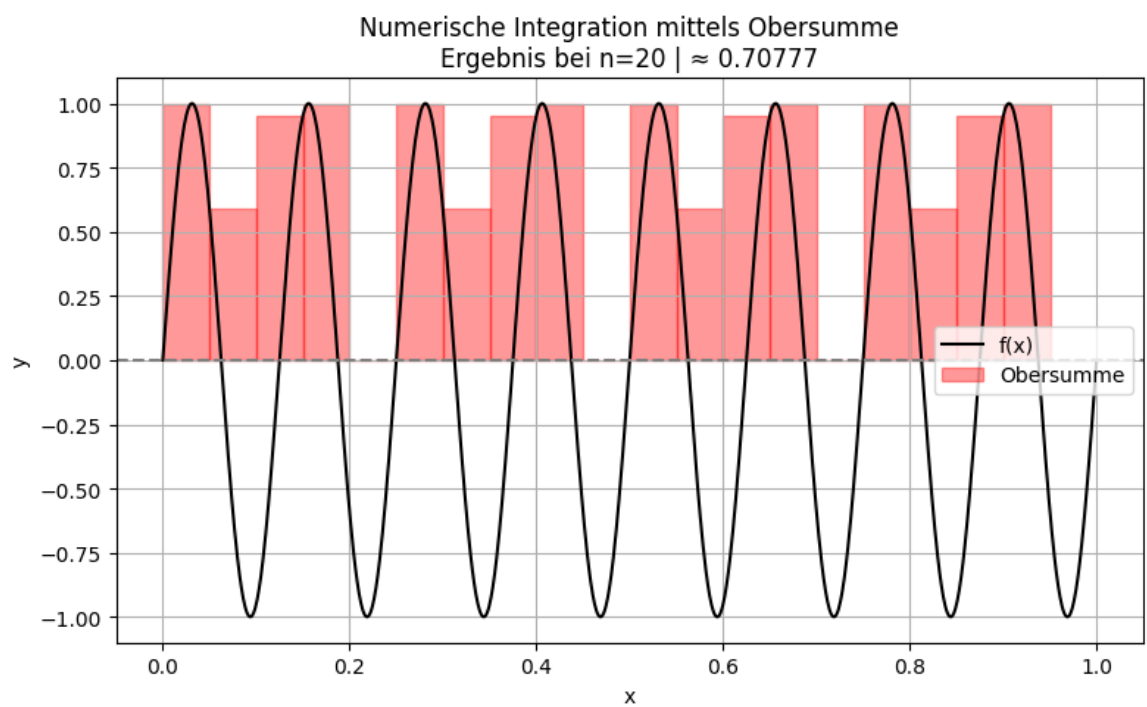
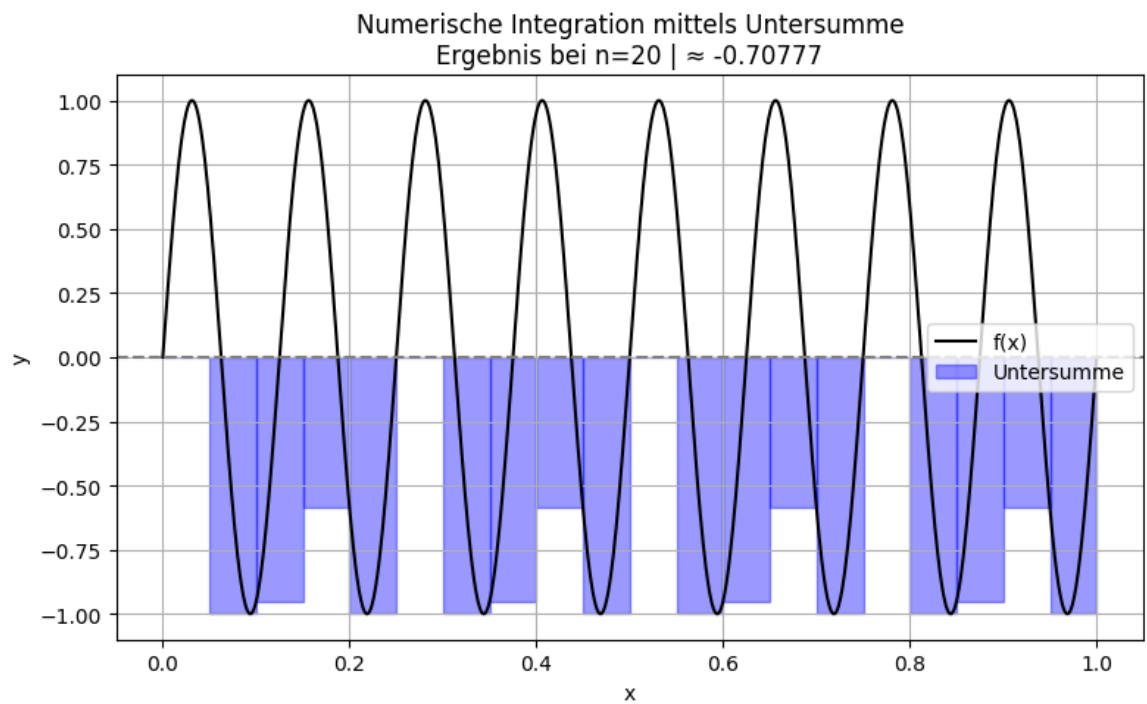


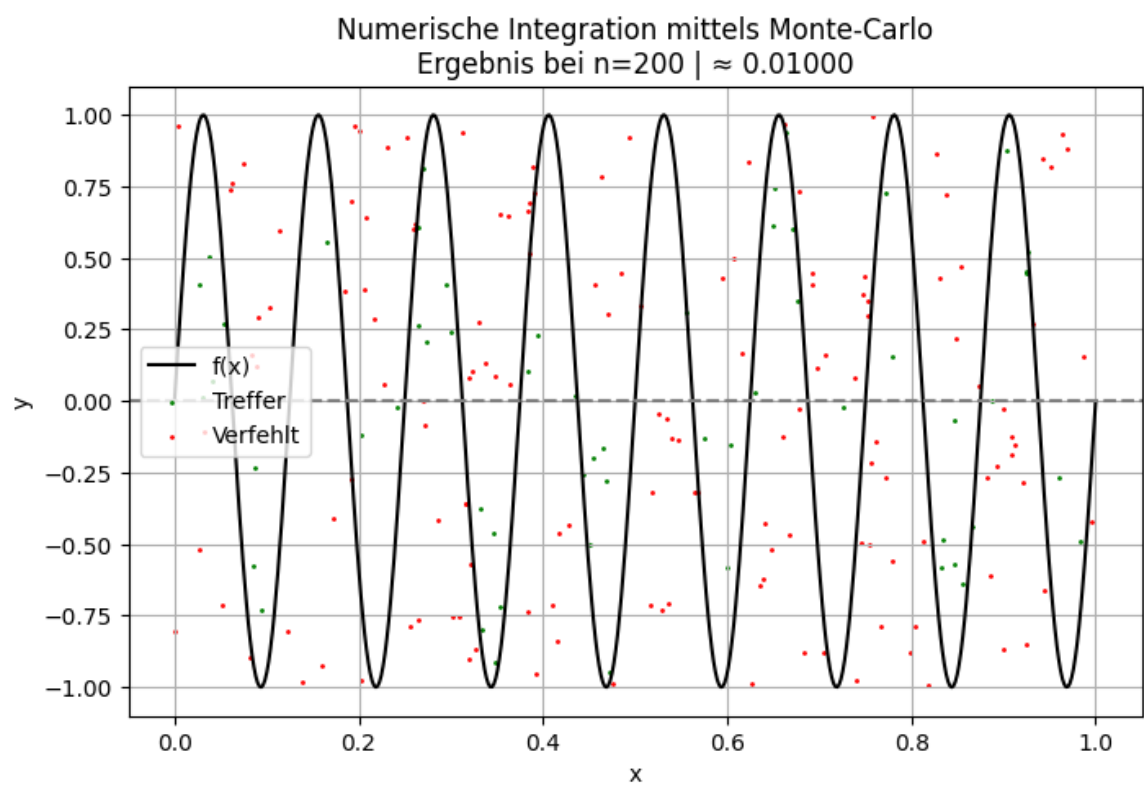
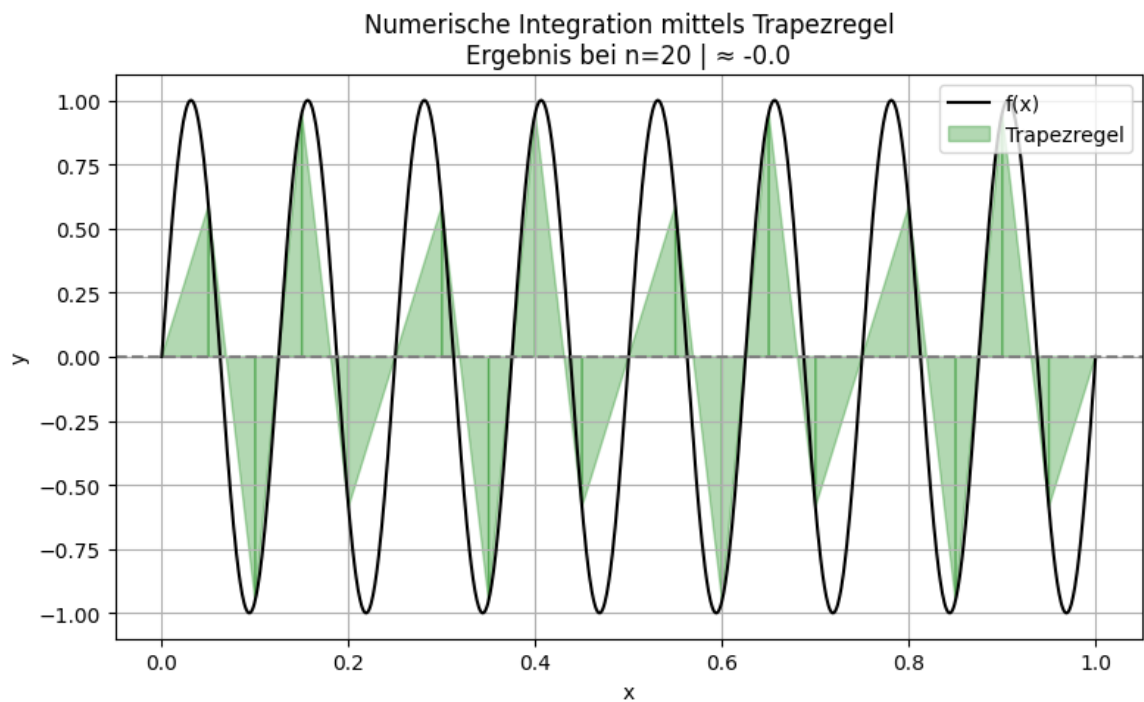


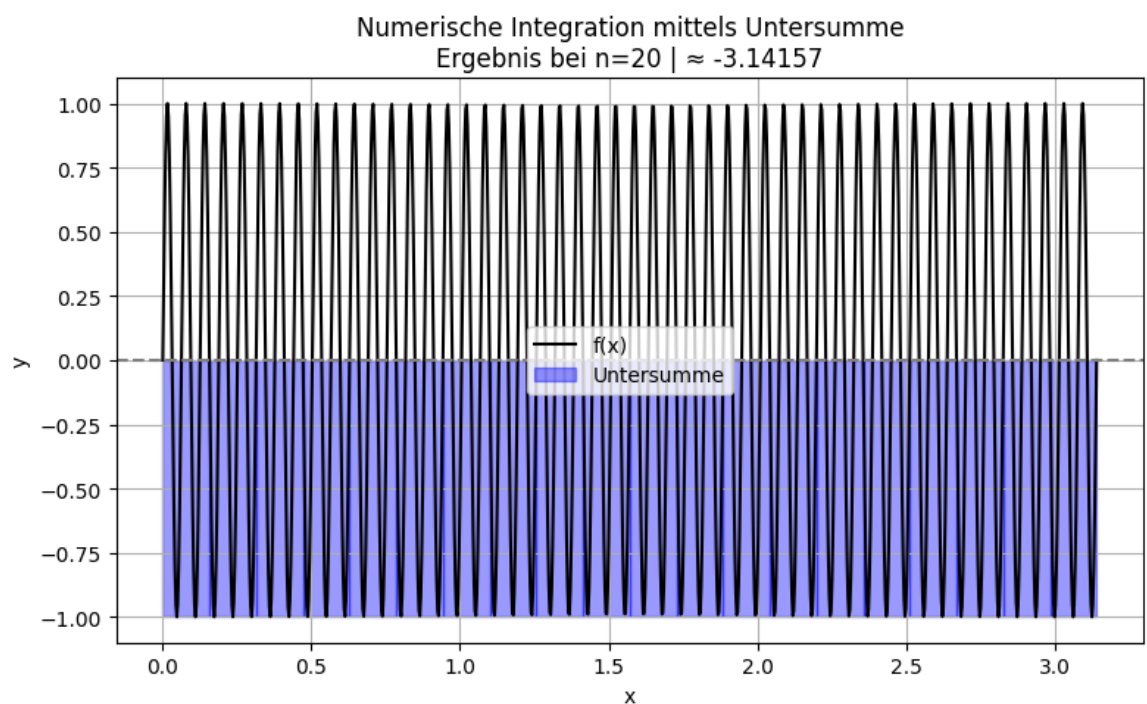
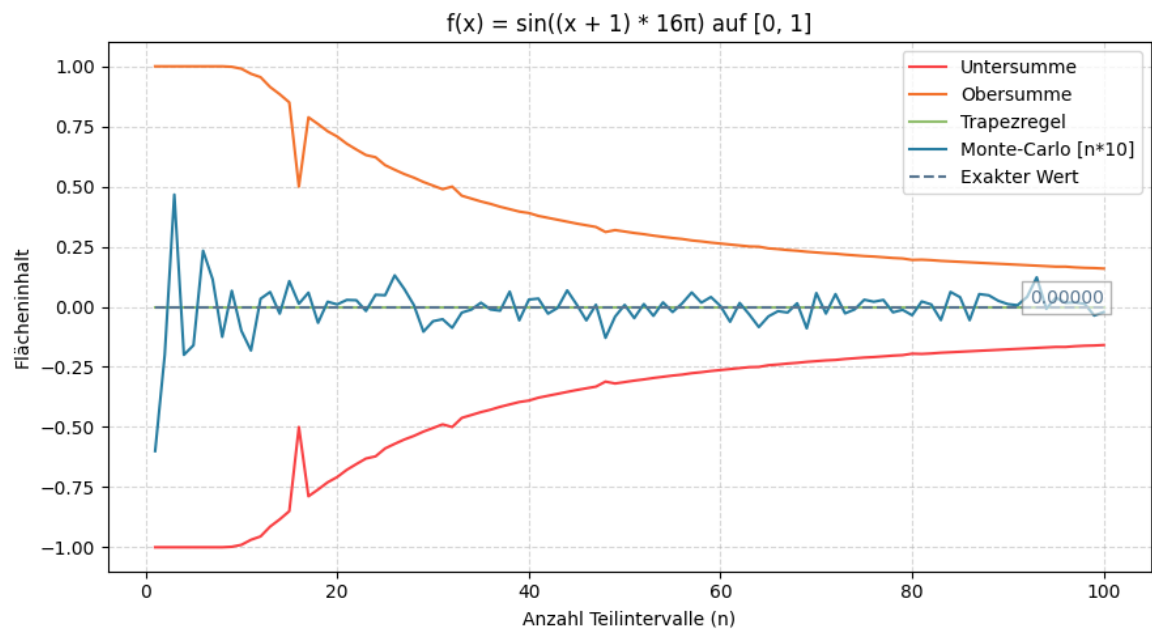


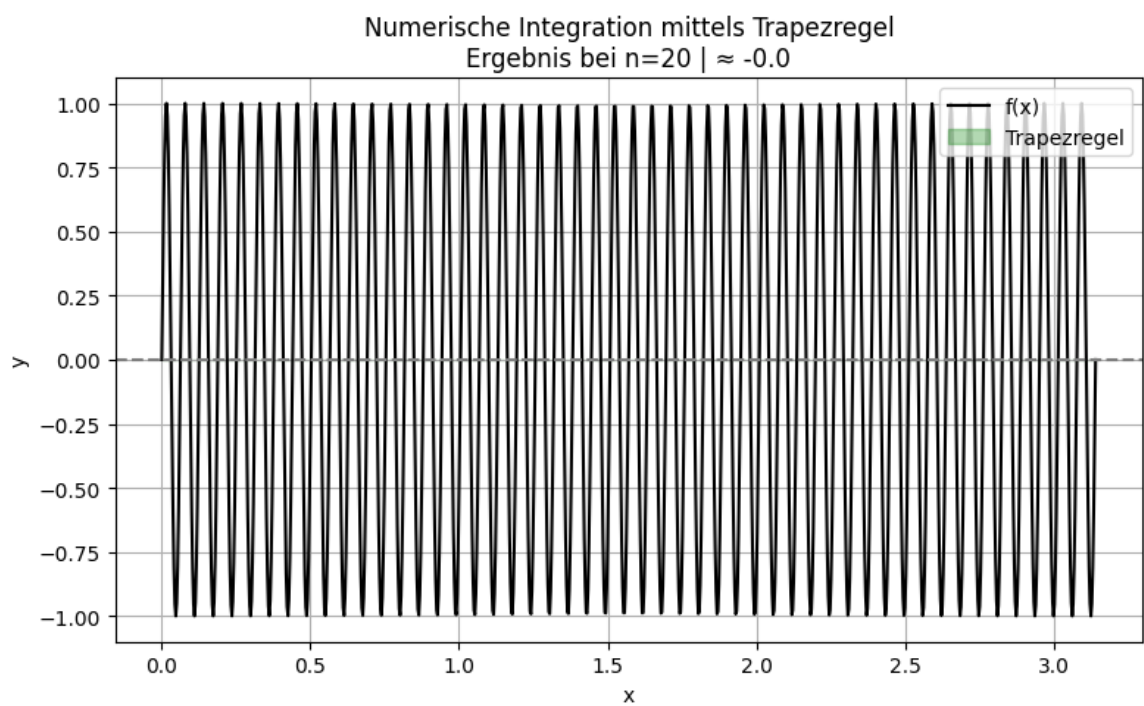
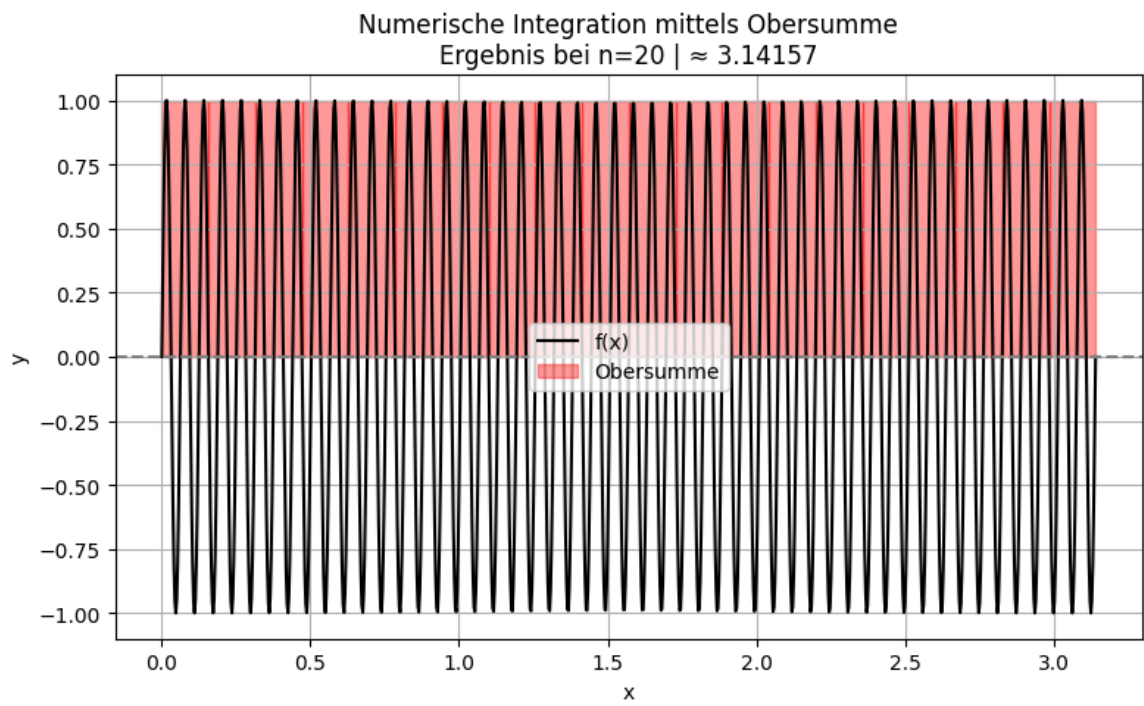


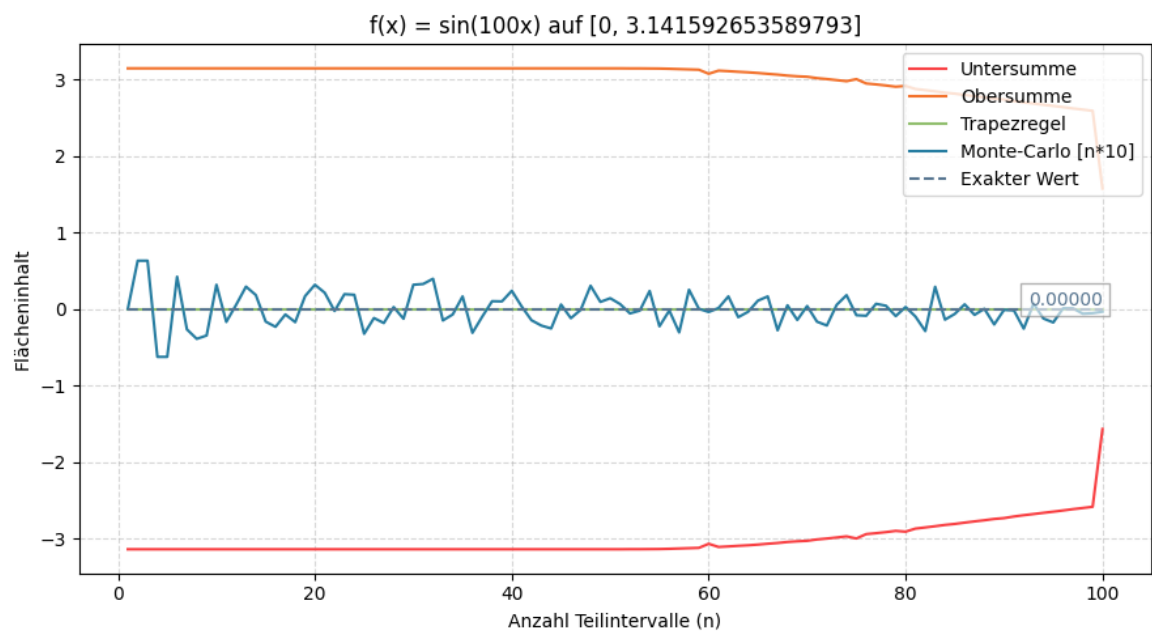
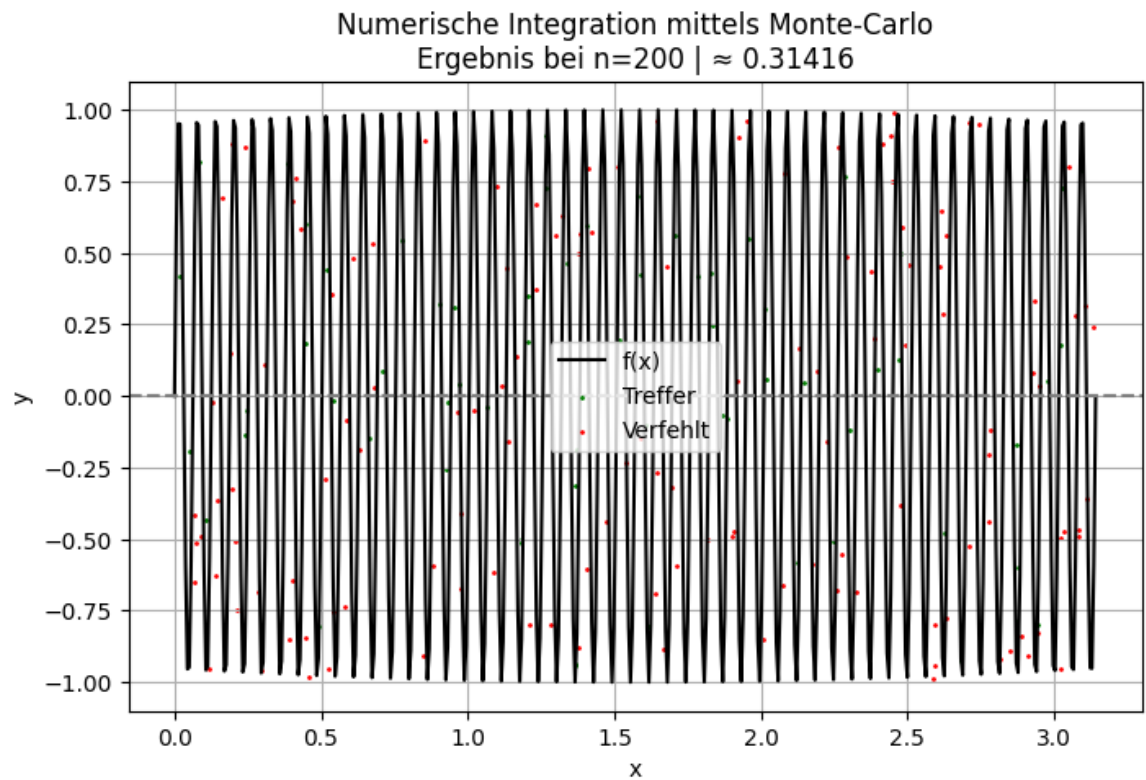




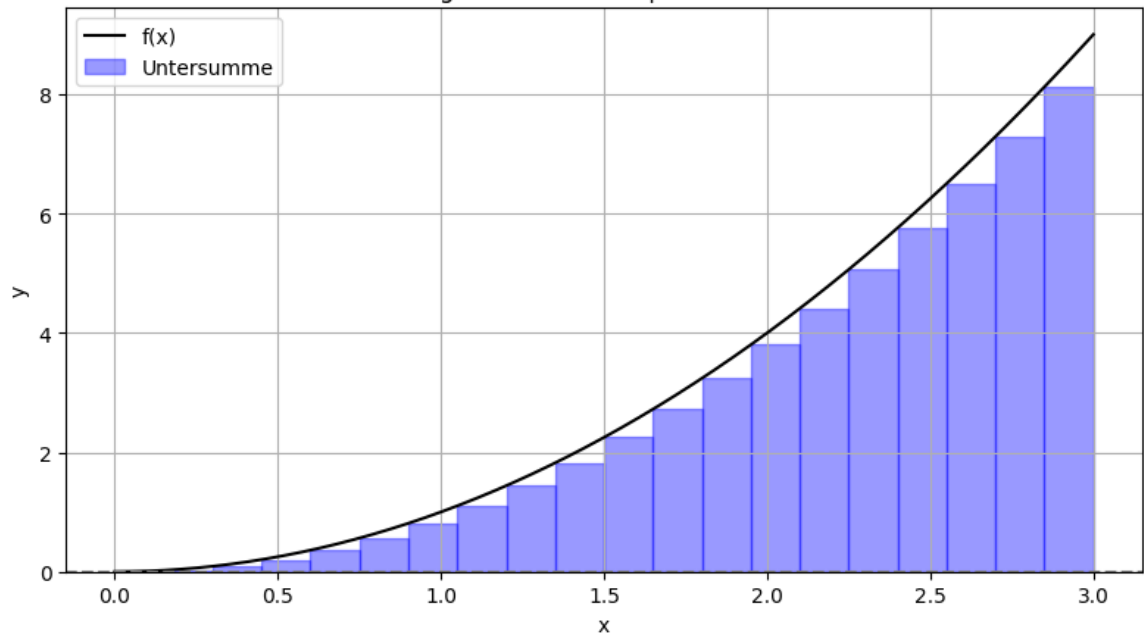




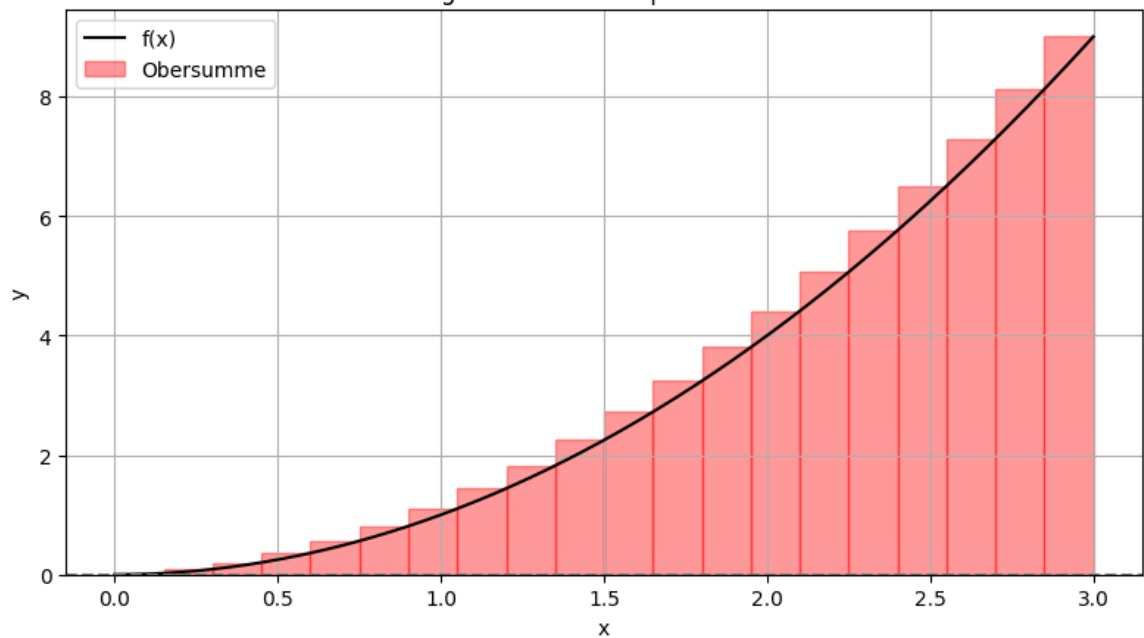




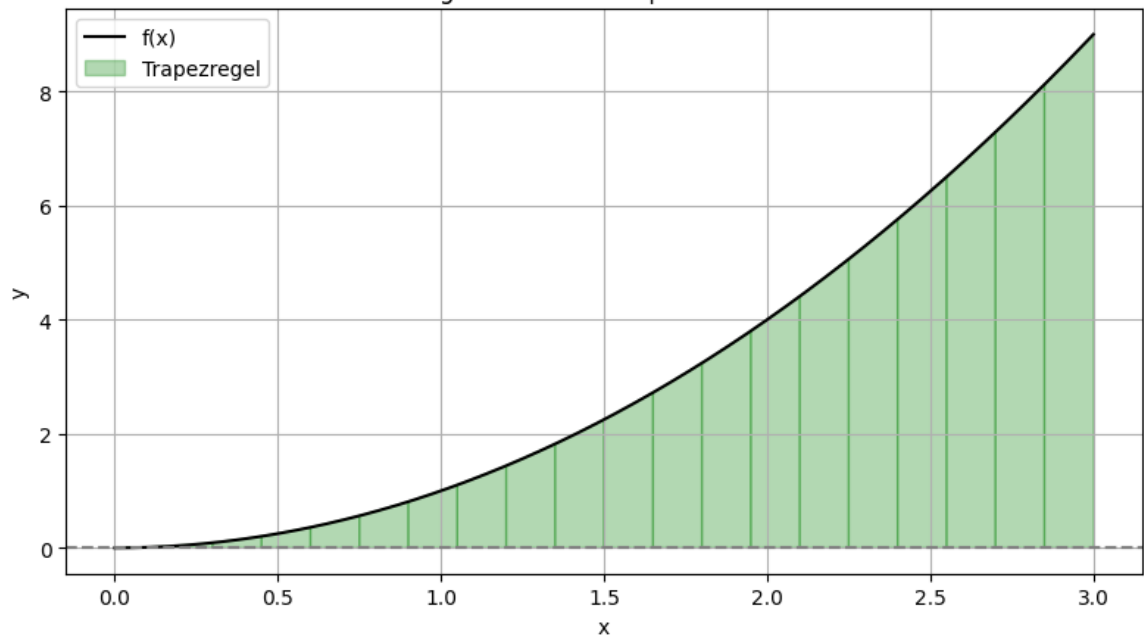
Numerische Integration mittels Untersumme
Ergebnis bei $n=20$ | ≈ 8.33625



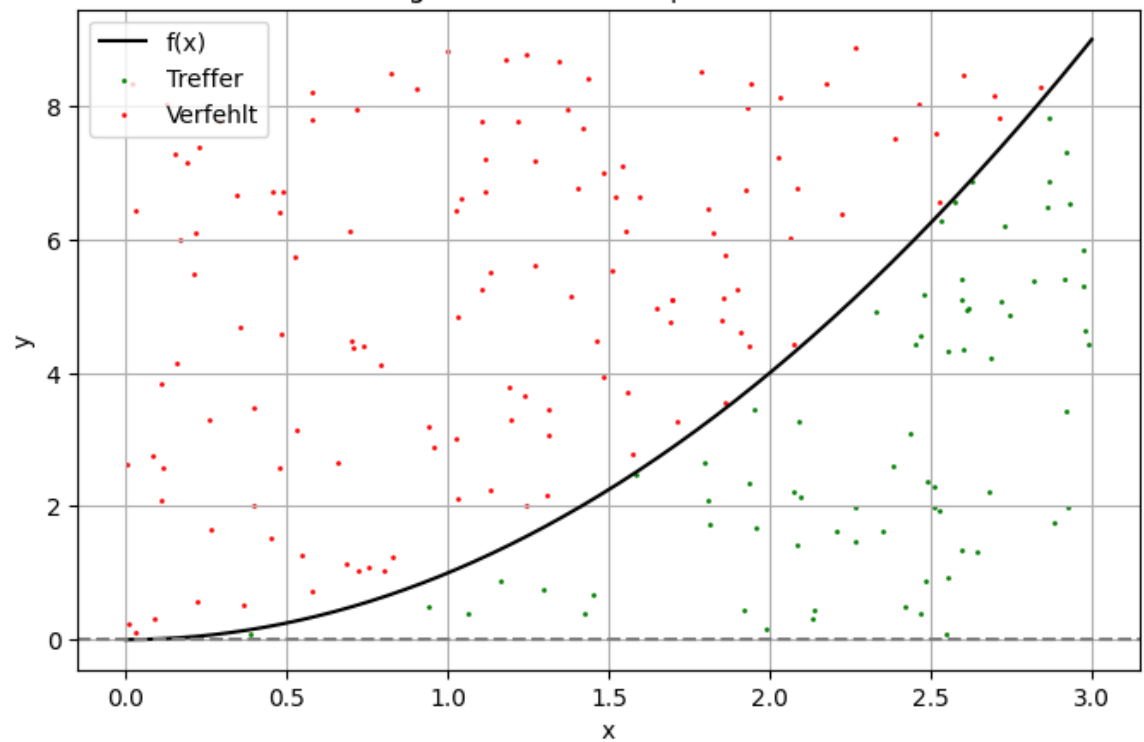
Numerische Integration mittels Obersumme
Ergebnis bei $n=20$ | ≈ 9.68625

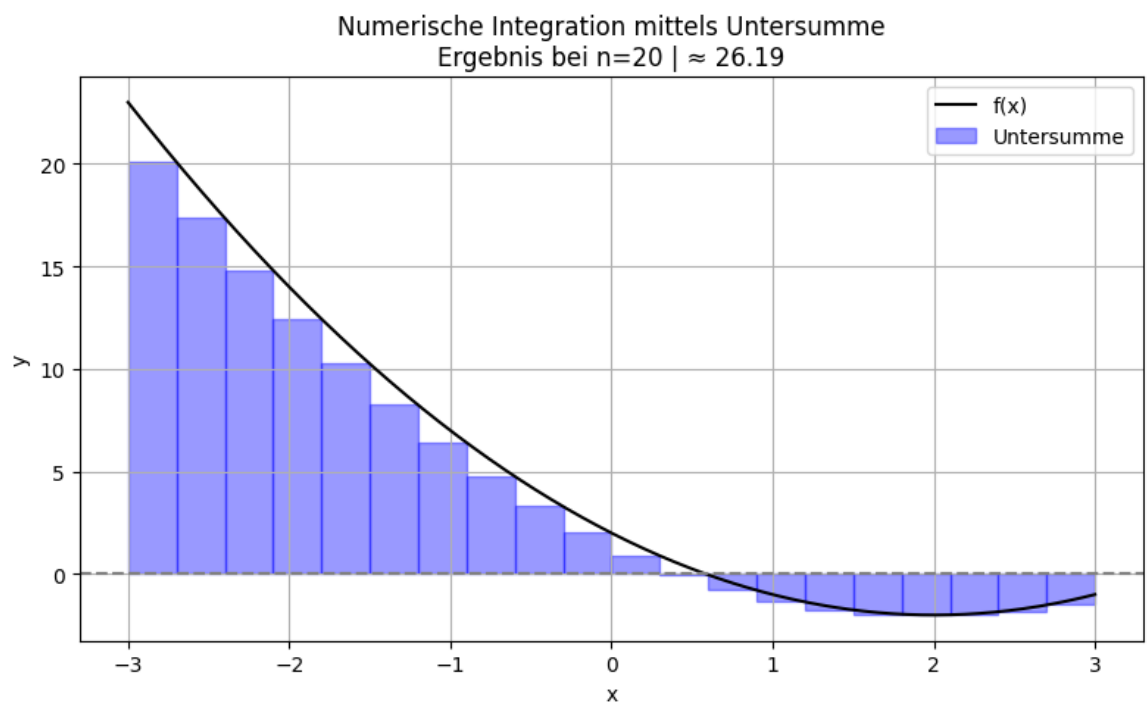
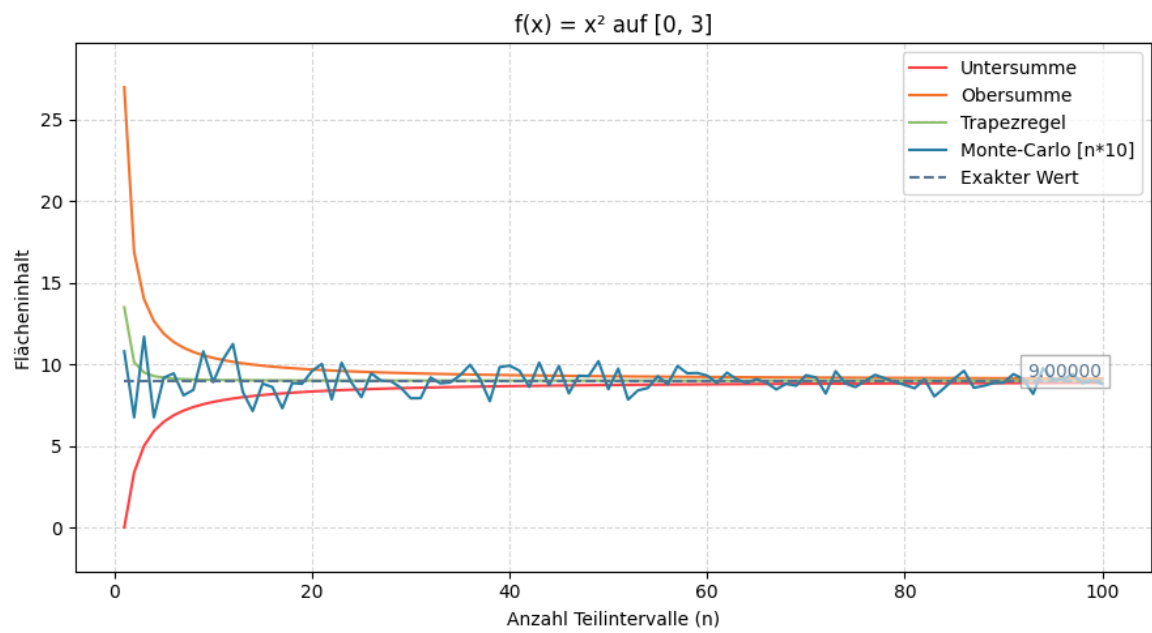


Numerische Integration mittels Trapezregel
Ergebnis bei $n=20$ | ≈ 9.01125

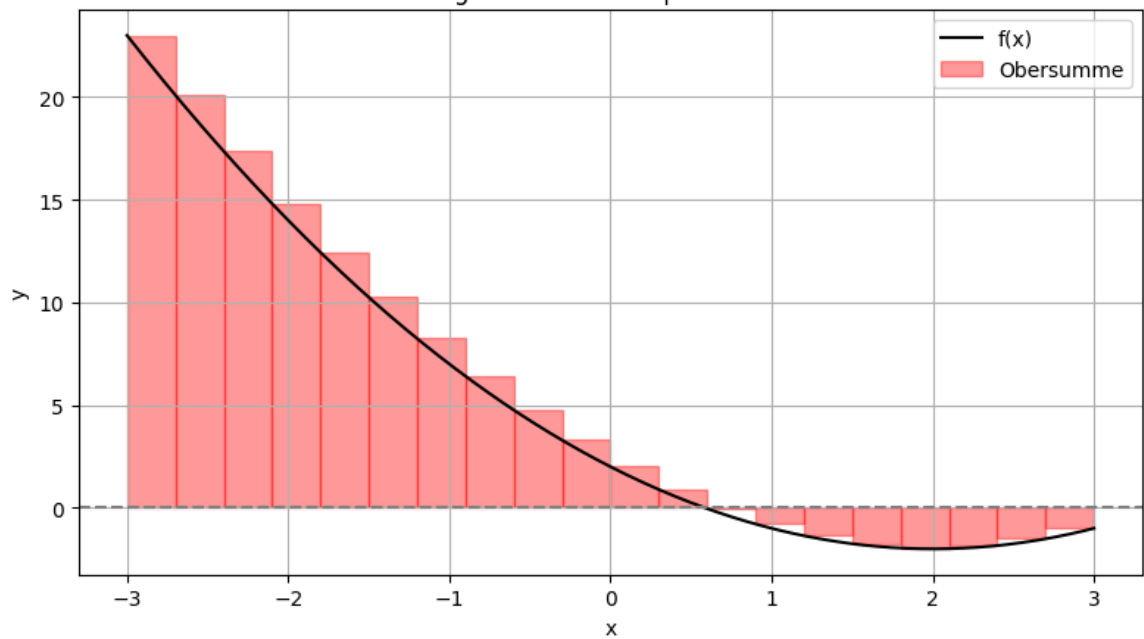


Numerische Integration mittels Monte-Carlo
Ergebnis bei $n=200$ | ≈ 9.58500

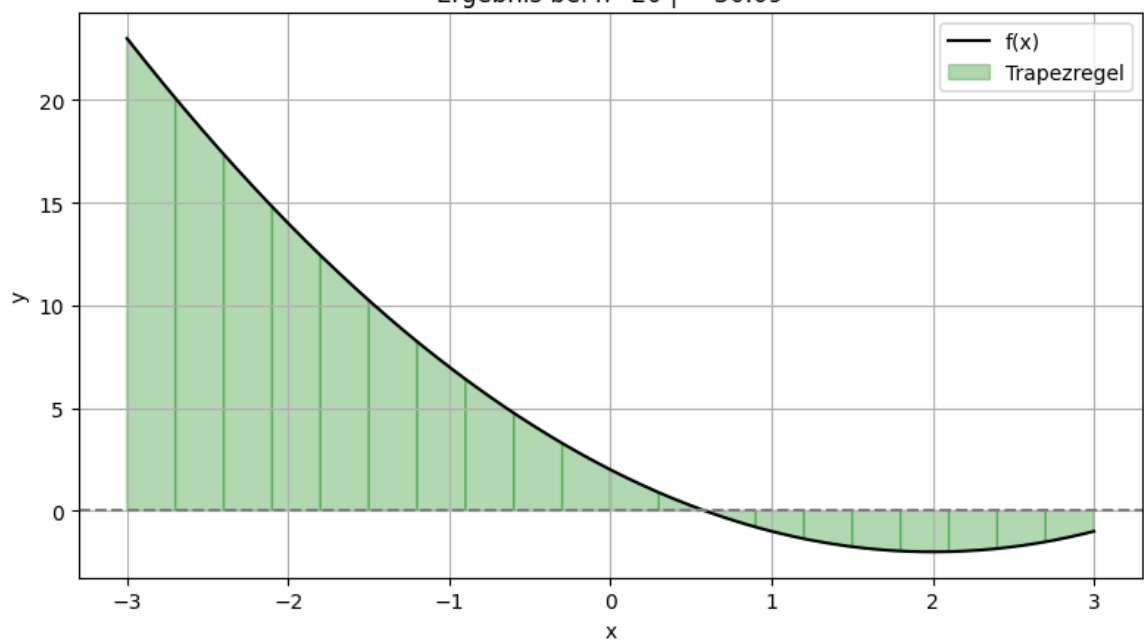




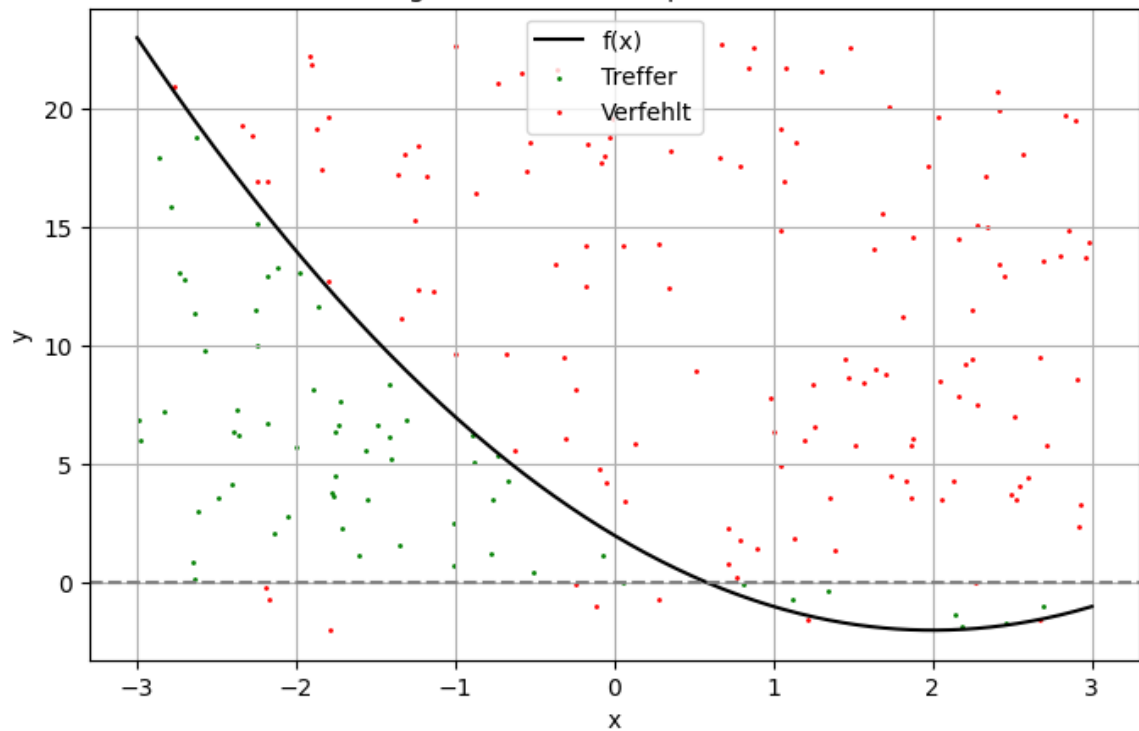
Numerische Integration mittels Obersumme
Ergebnis bei $n=20$ | ≈ 33.987



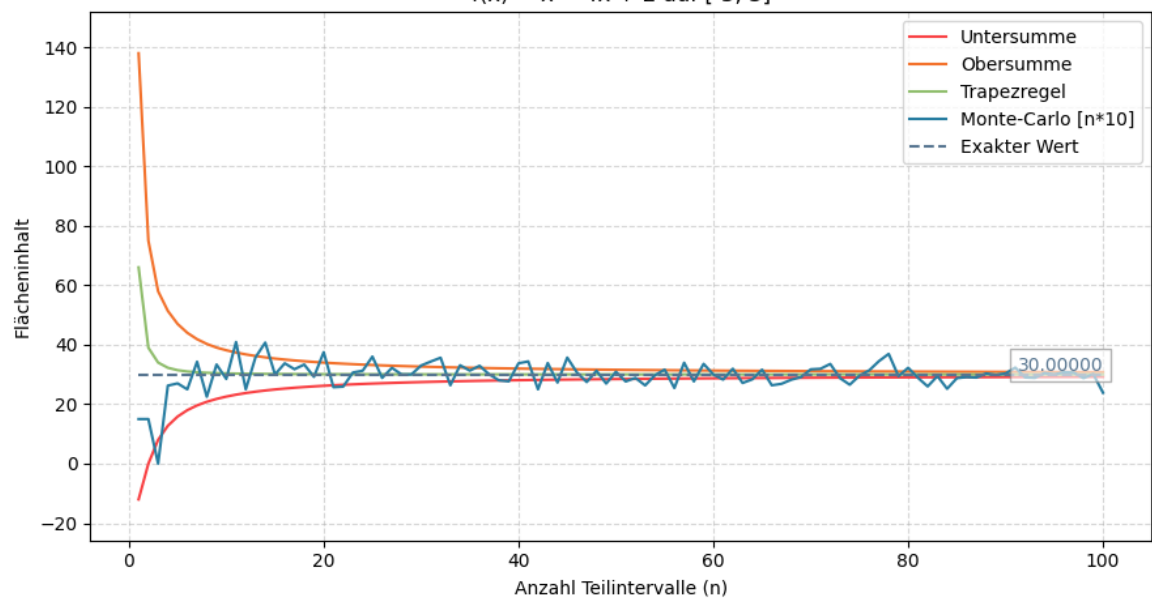
Numerische Integration mittels Trapezregel
Ergebnis bei $n=20$ | ≈ 30.09

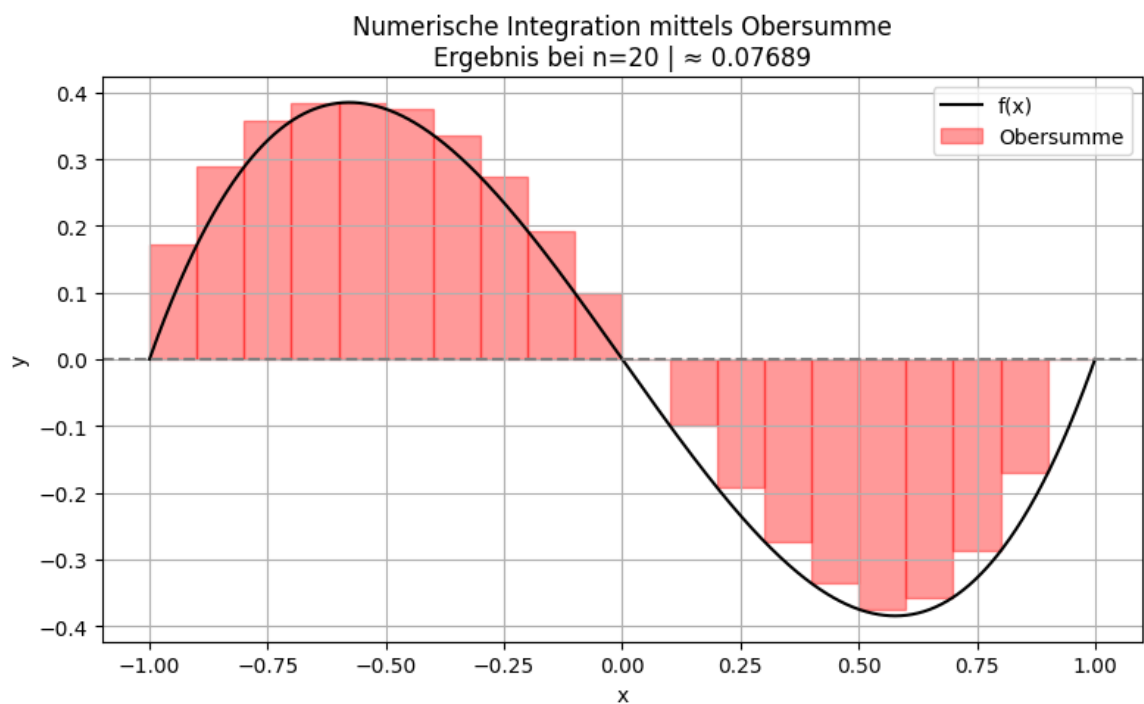
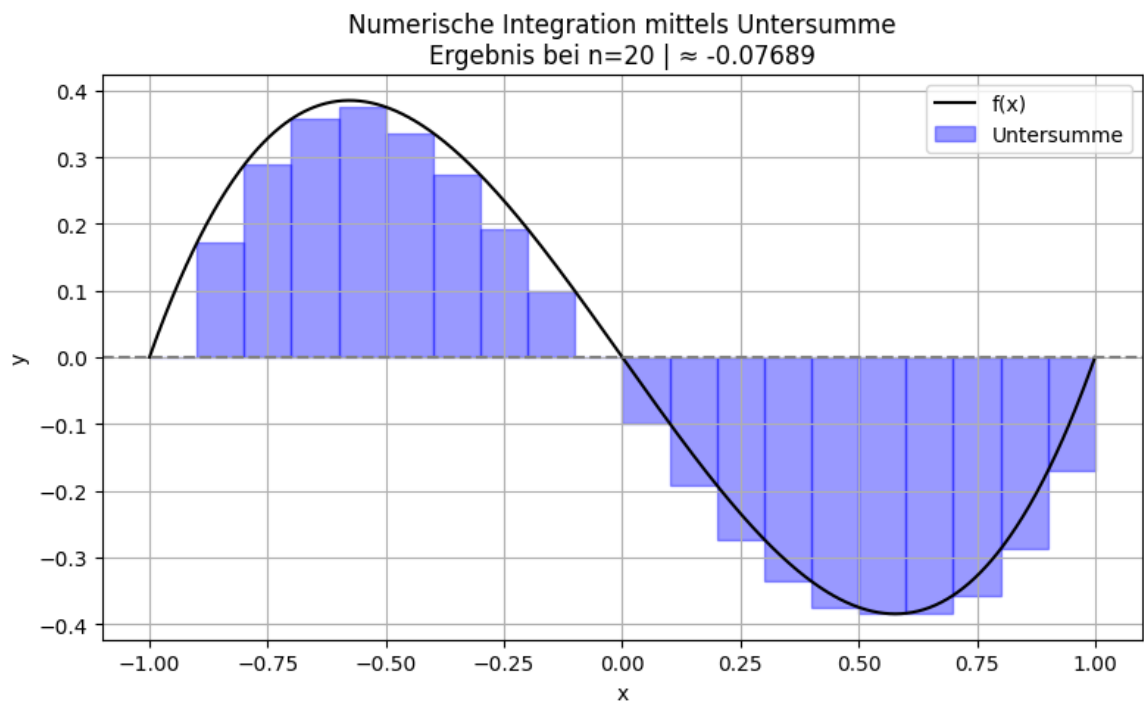


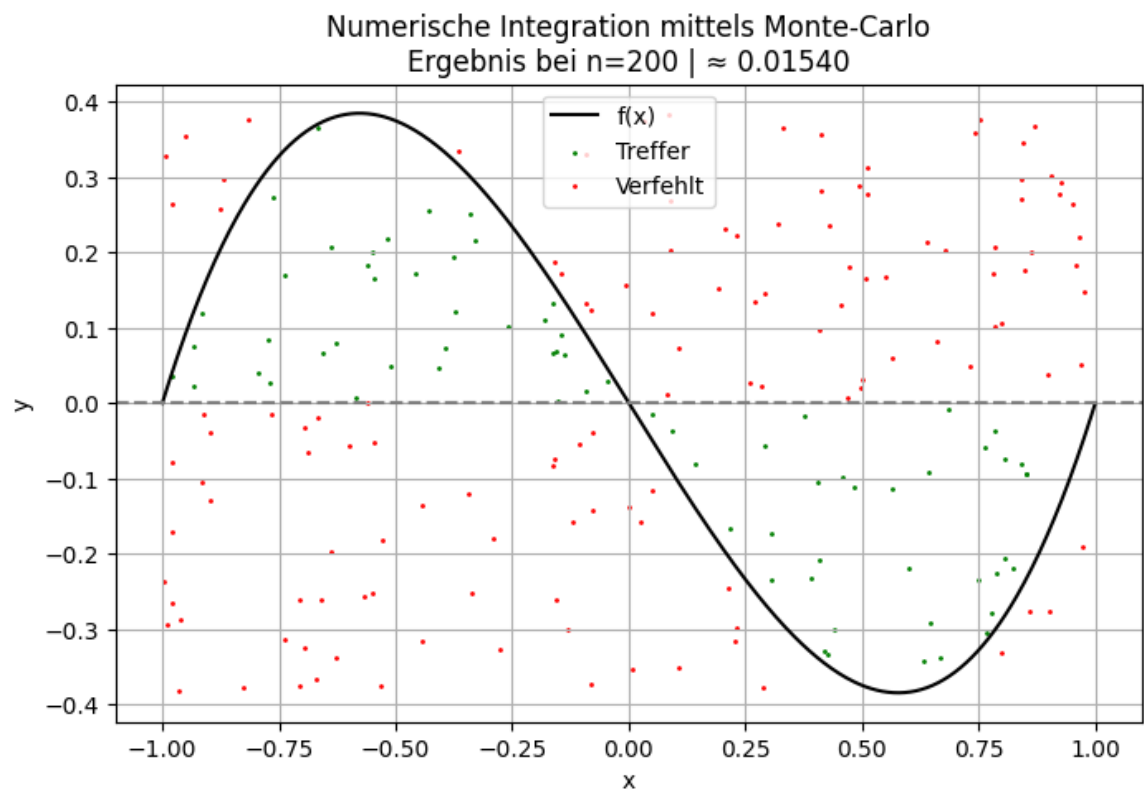
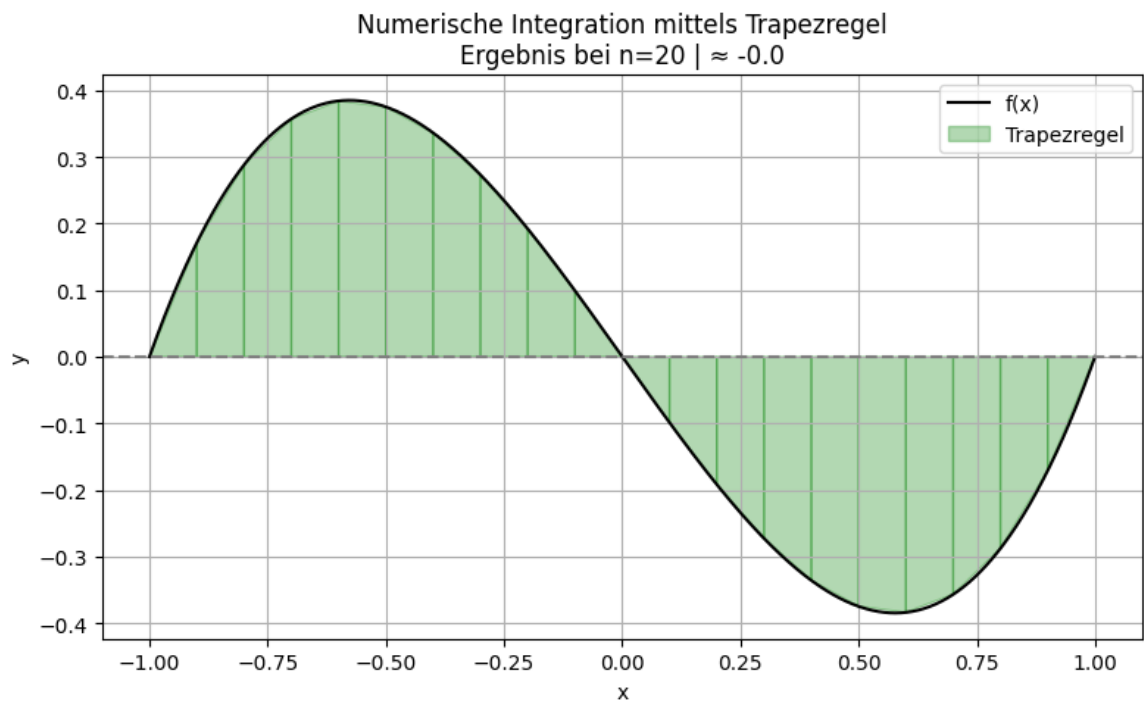
Numerische Integration mittels Monte-Carlo Ergebnis bei $n=200 \mid \approx 37.49999$

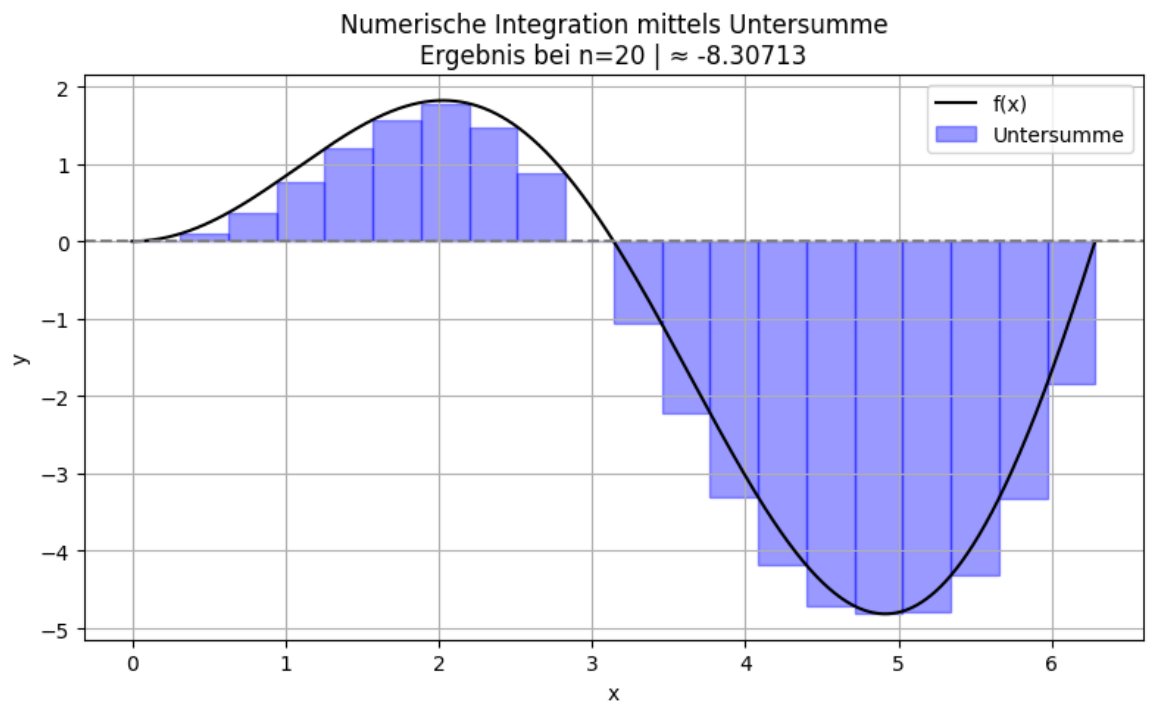
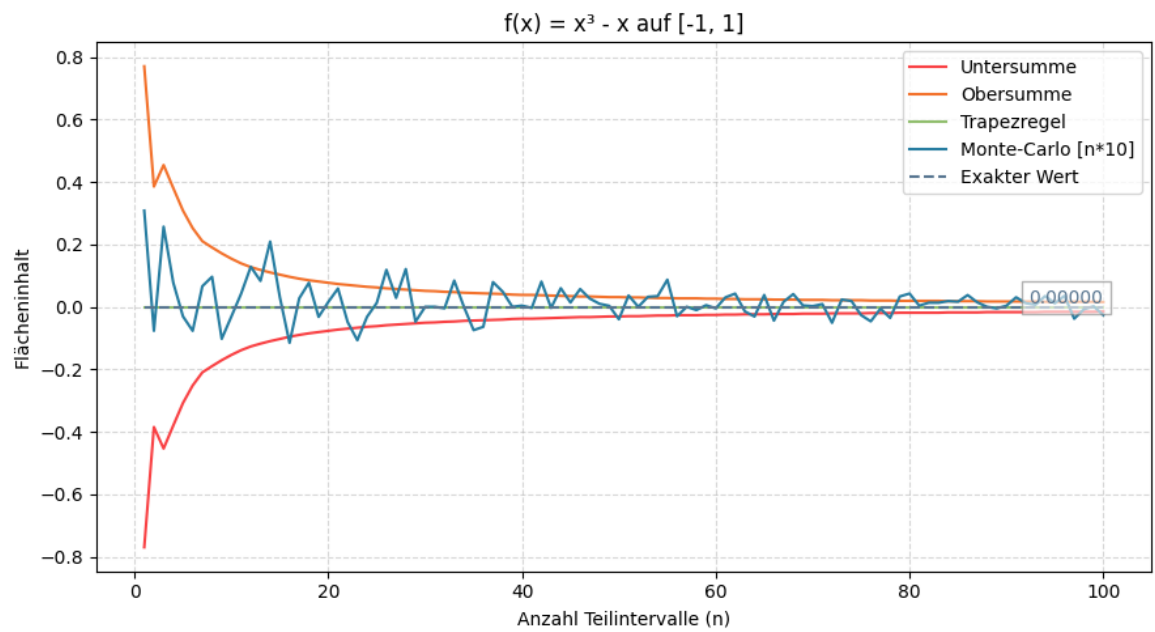


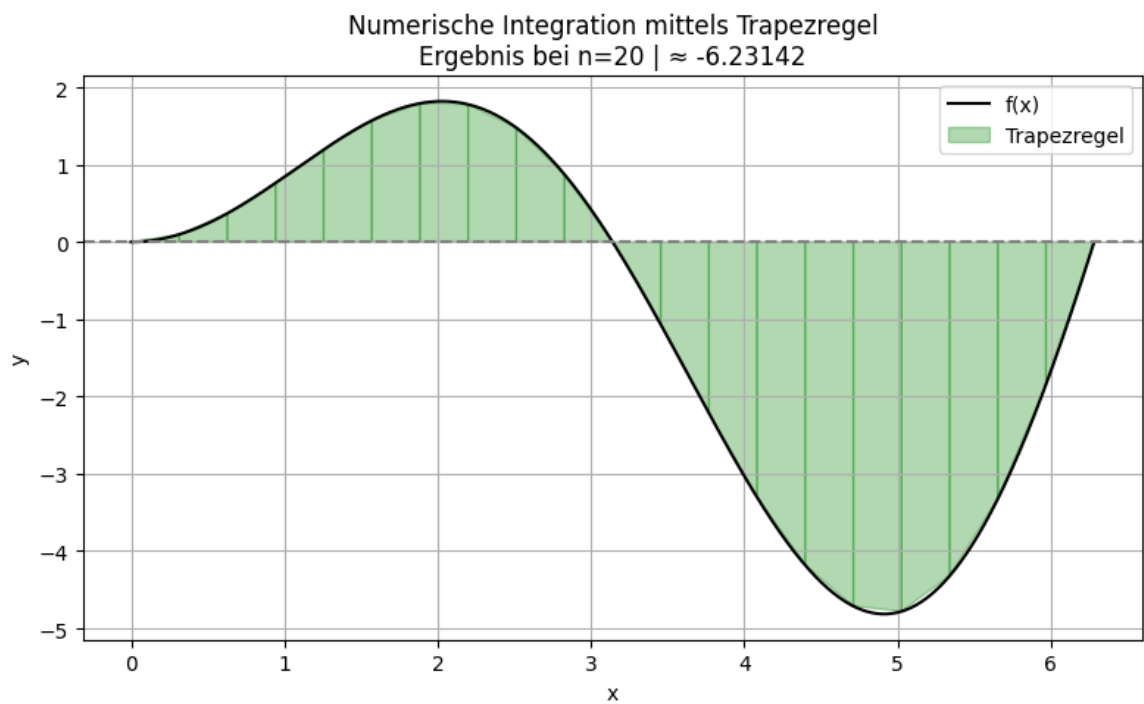
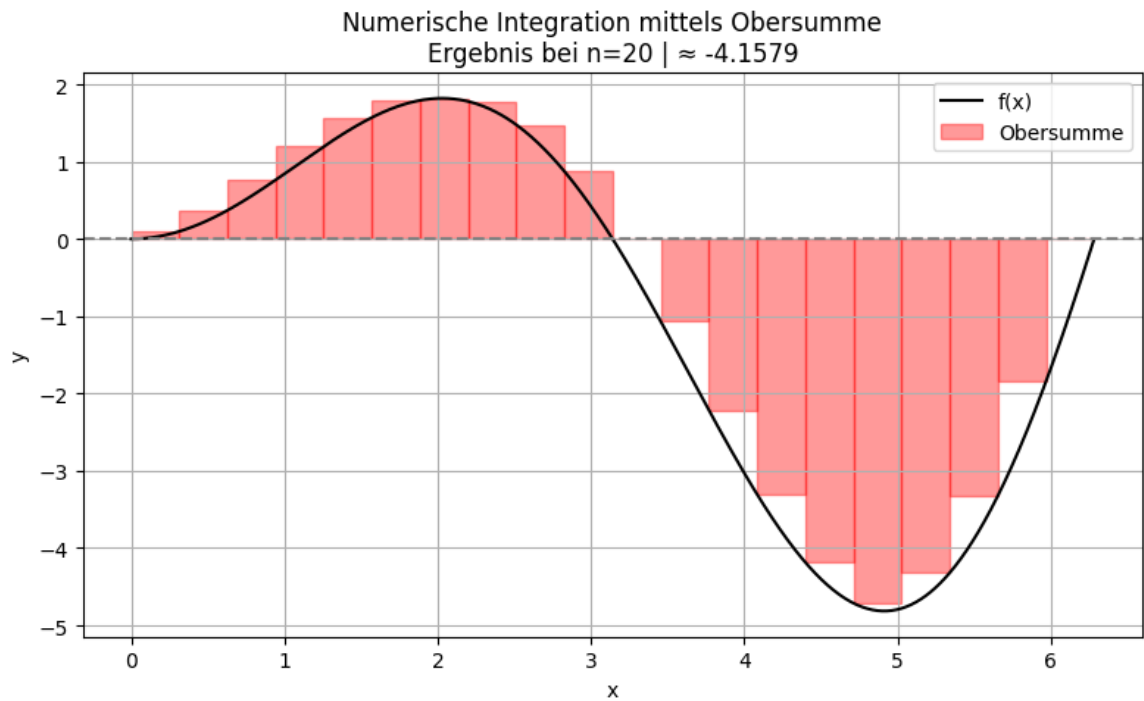
$$f(x) = x^2 - 4x + 2 \text{ auf } [-3, 3]$$

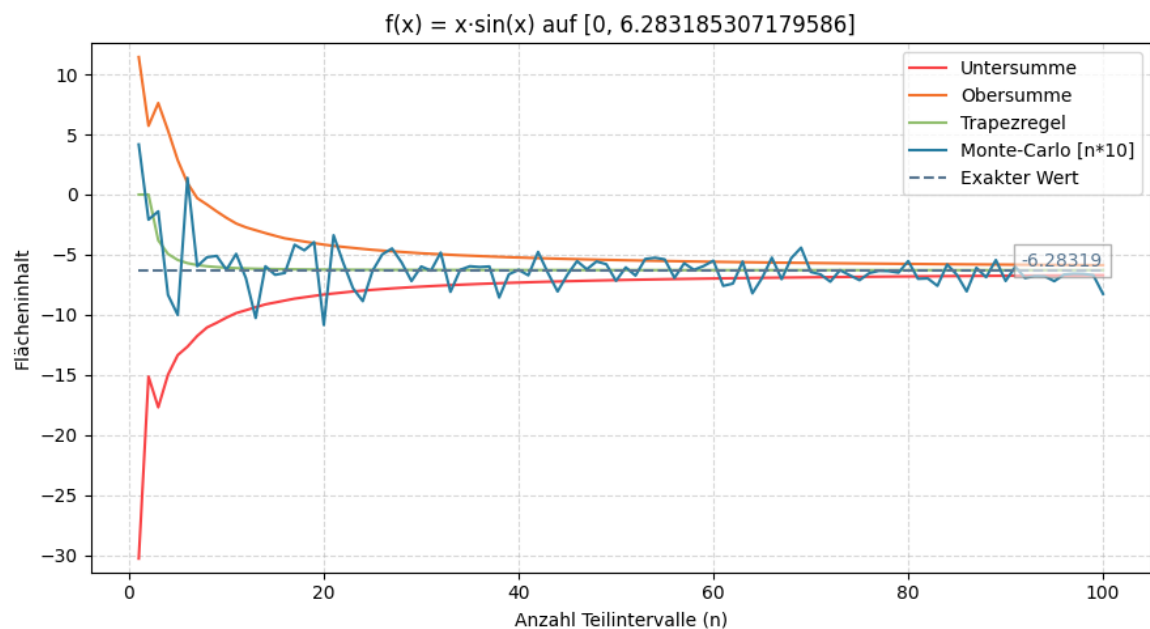
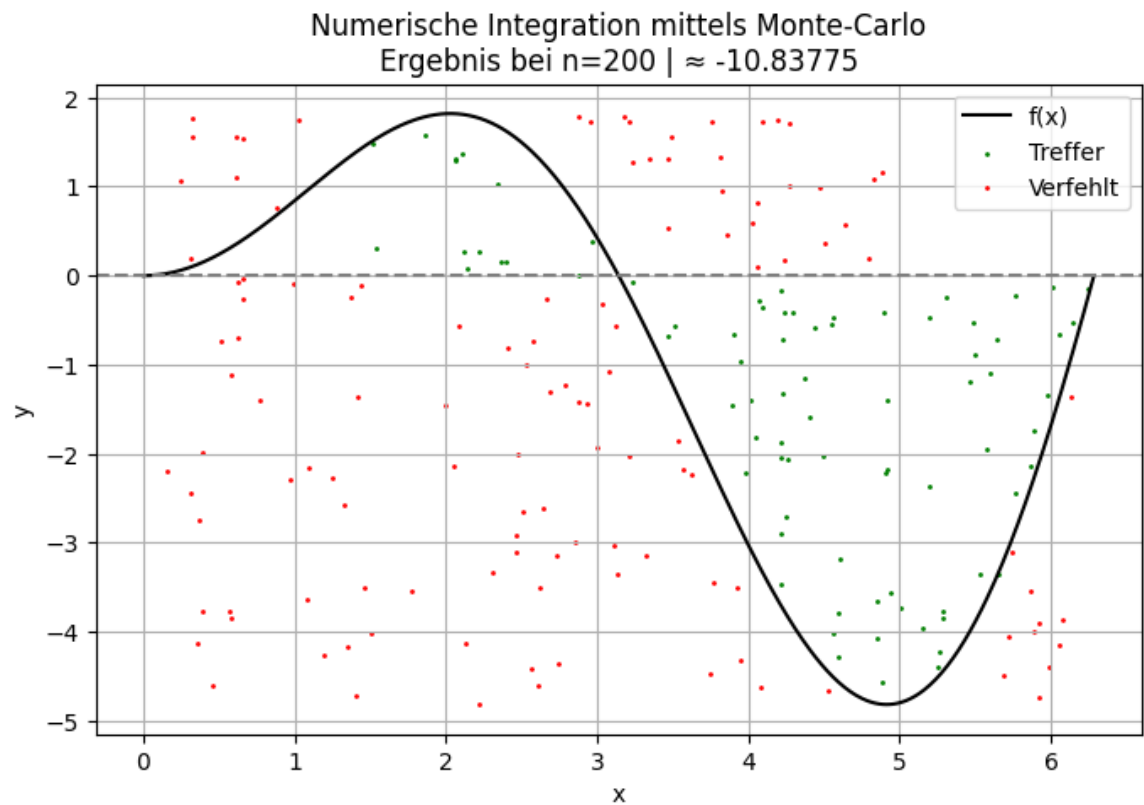




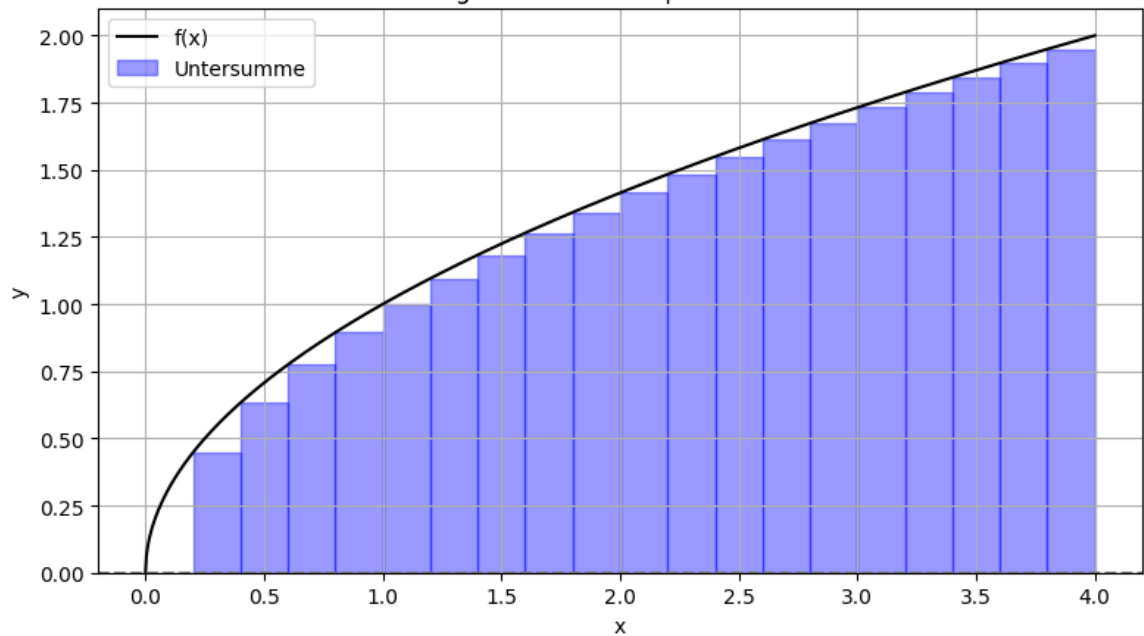




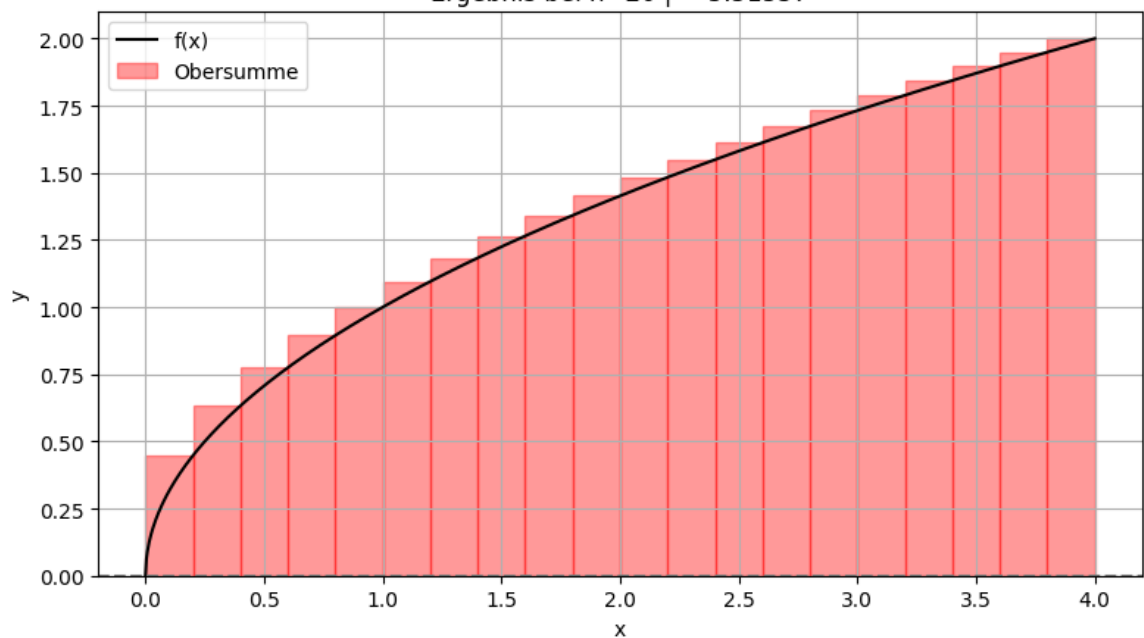




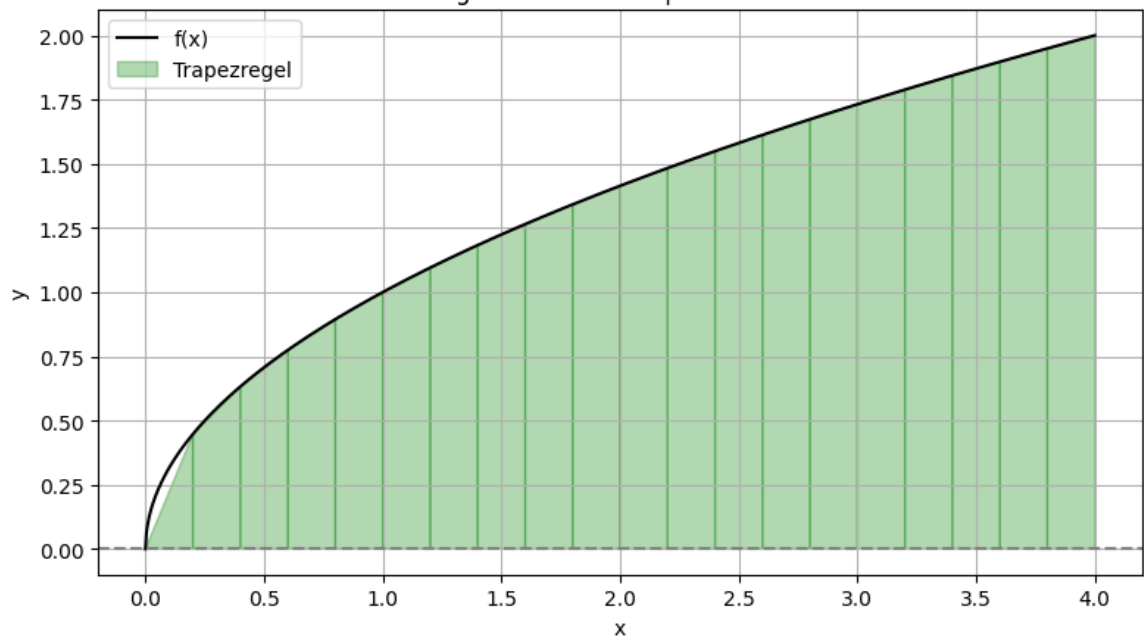
Numerische Integration mittels Untersumme
Ergebnis bei $n=20$ | ≈ 5.11557



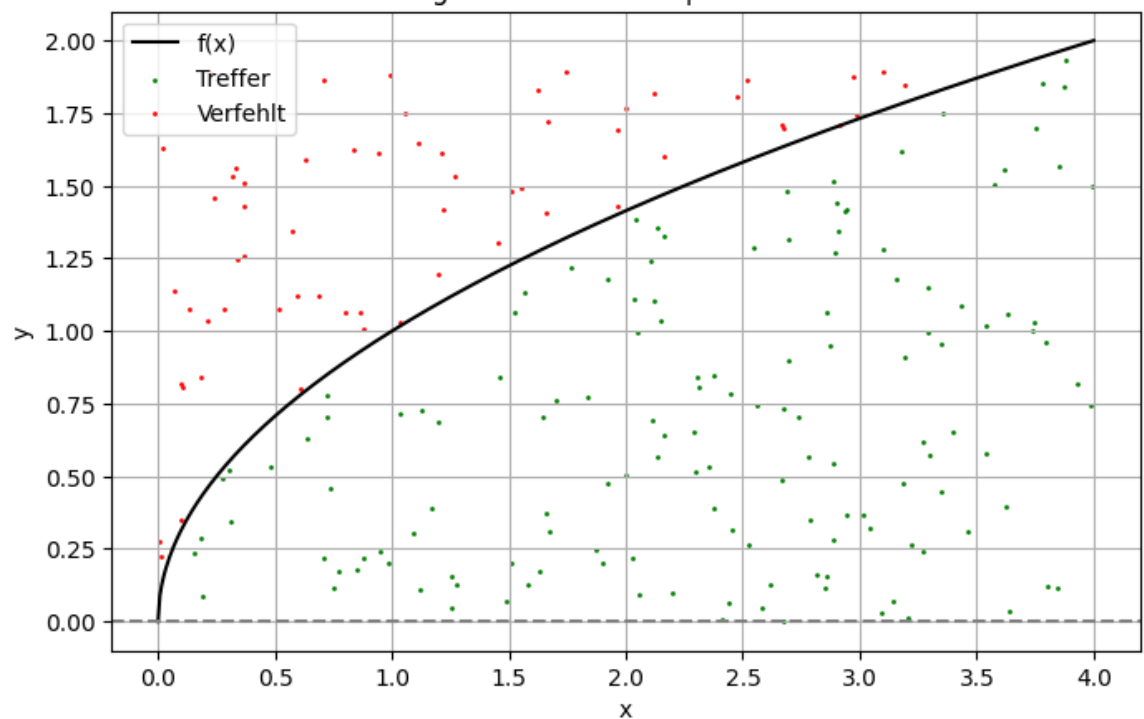
Numerische Integration mittels Obersumme
Ergebnis bei $n=20$ | ≈ 5.51557

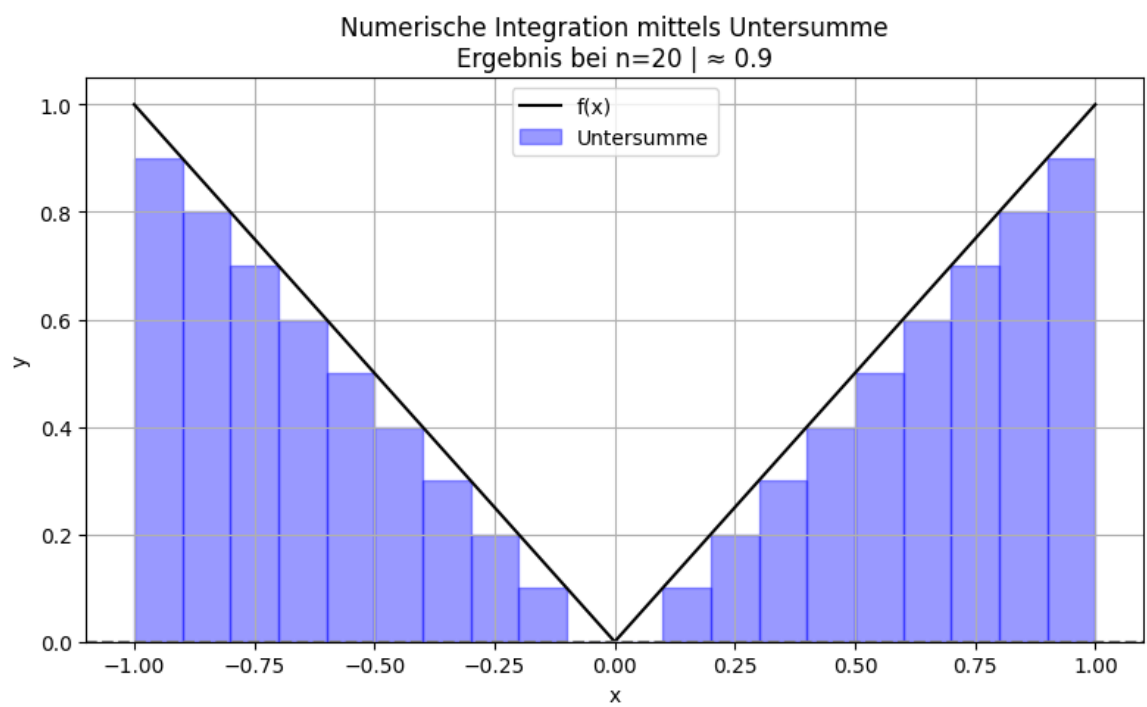
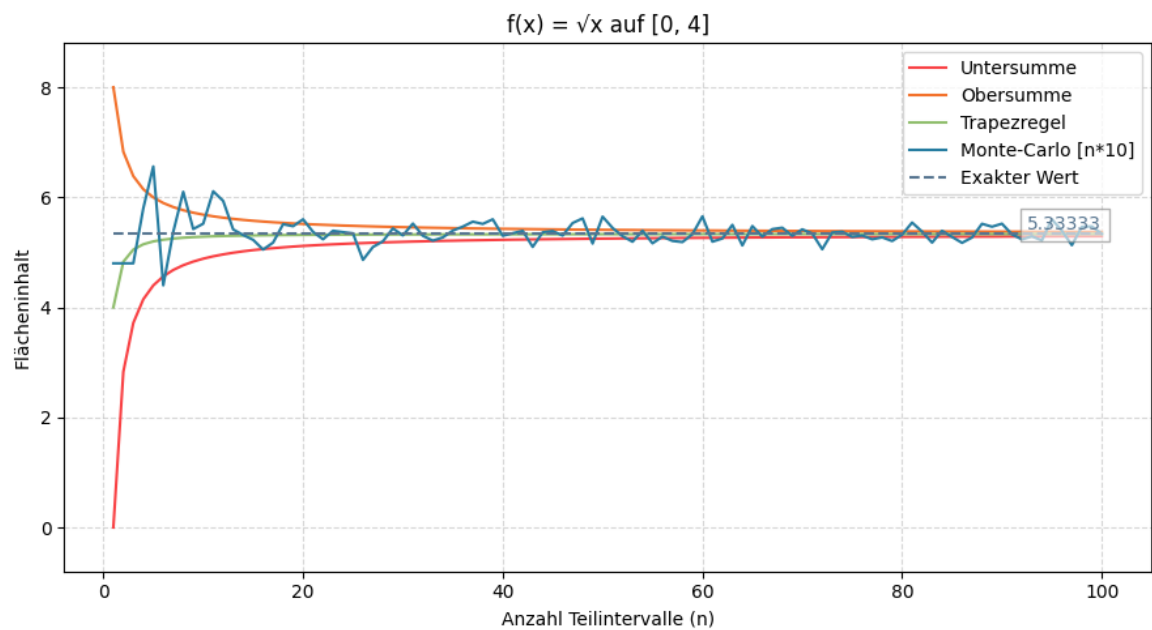


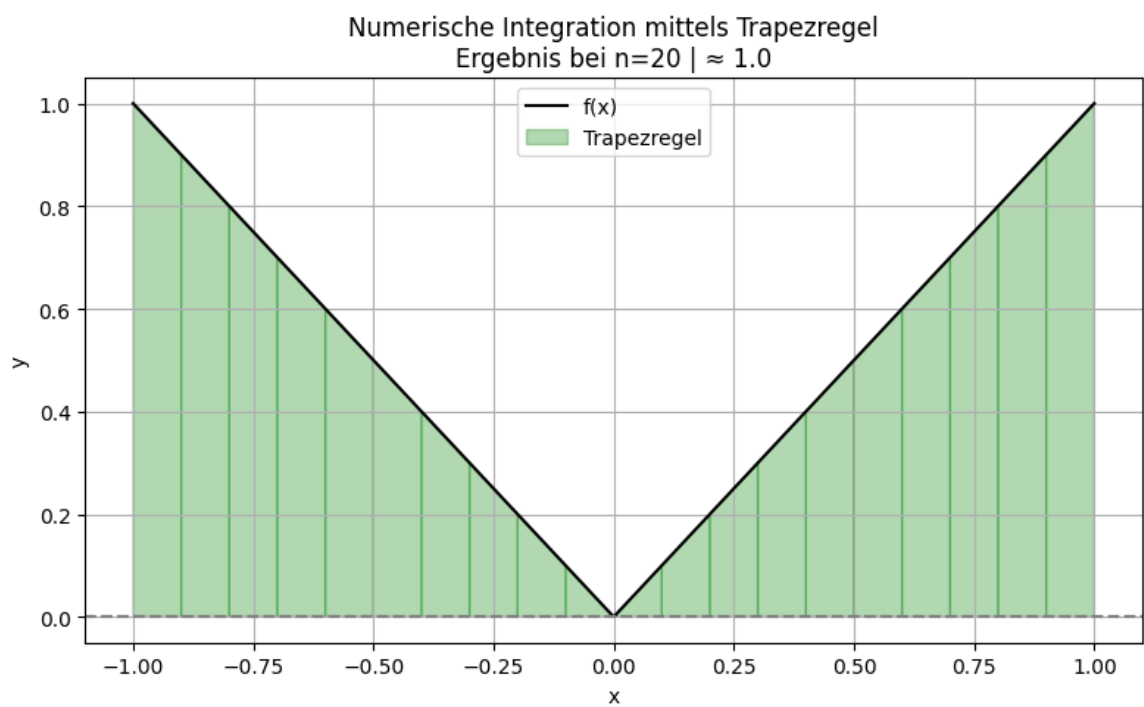
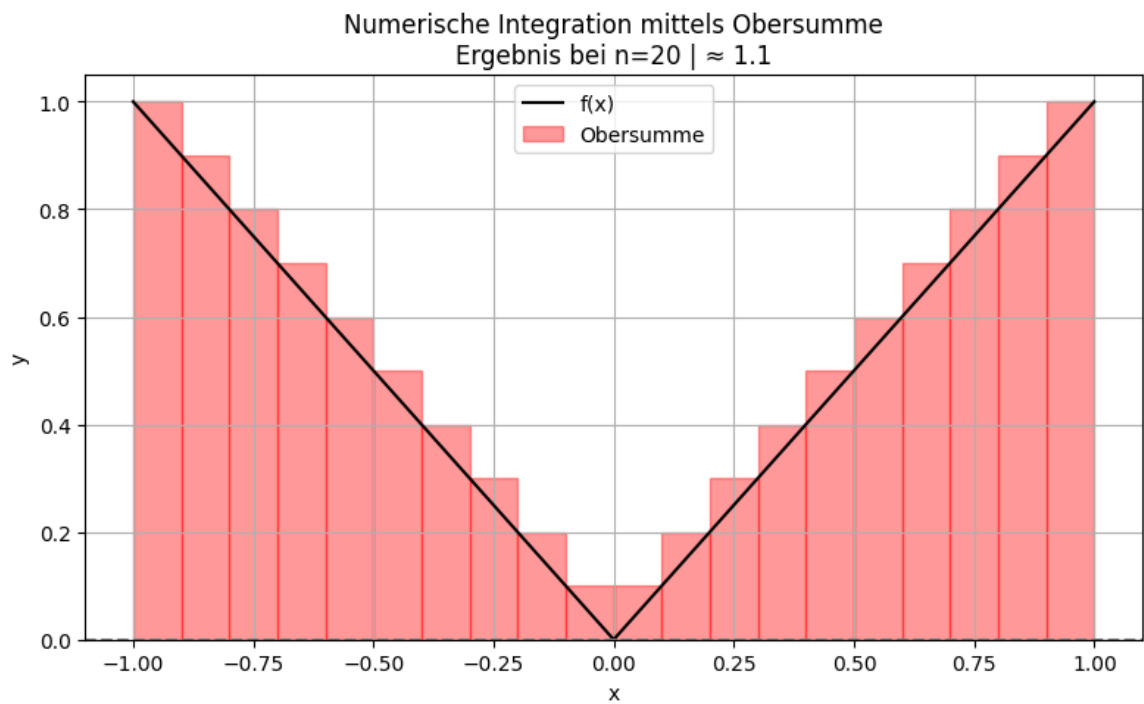
Numerische Integration mittels Trapezregel
Ergebnis bei $n=20$ | ≈ 5.31557

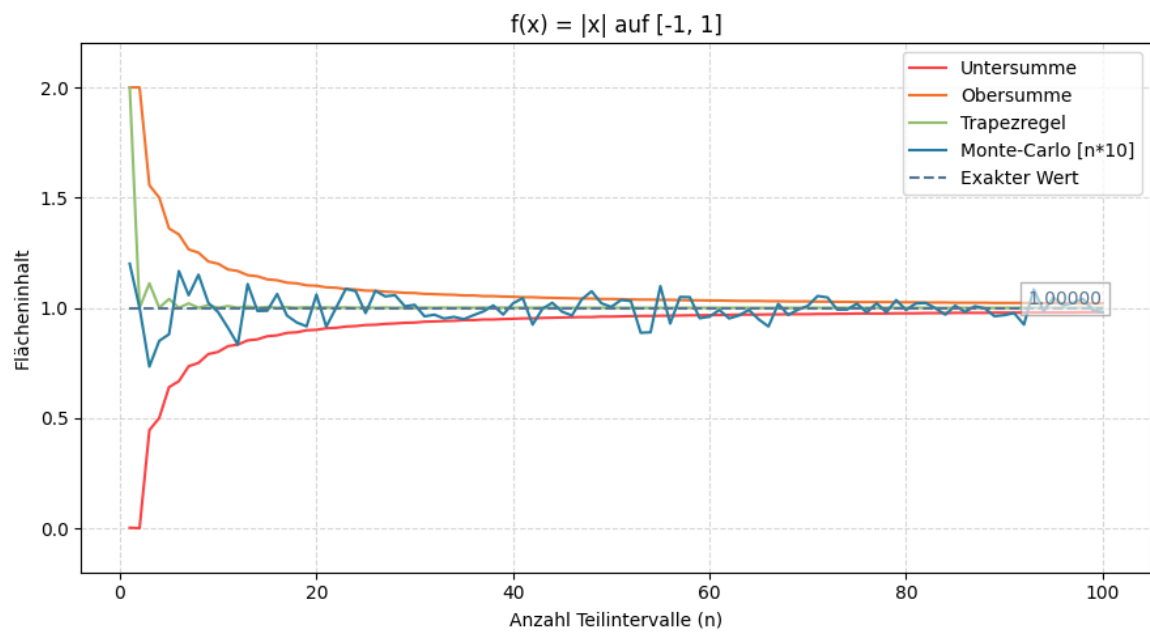
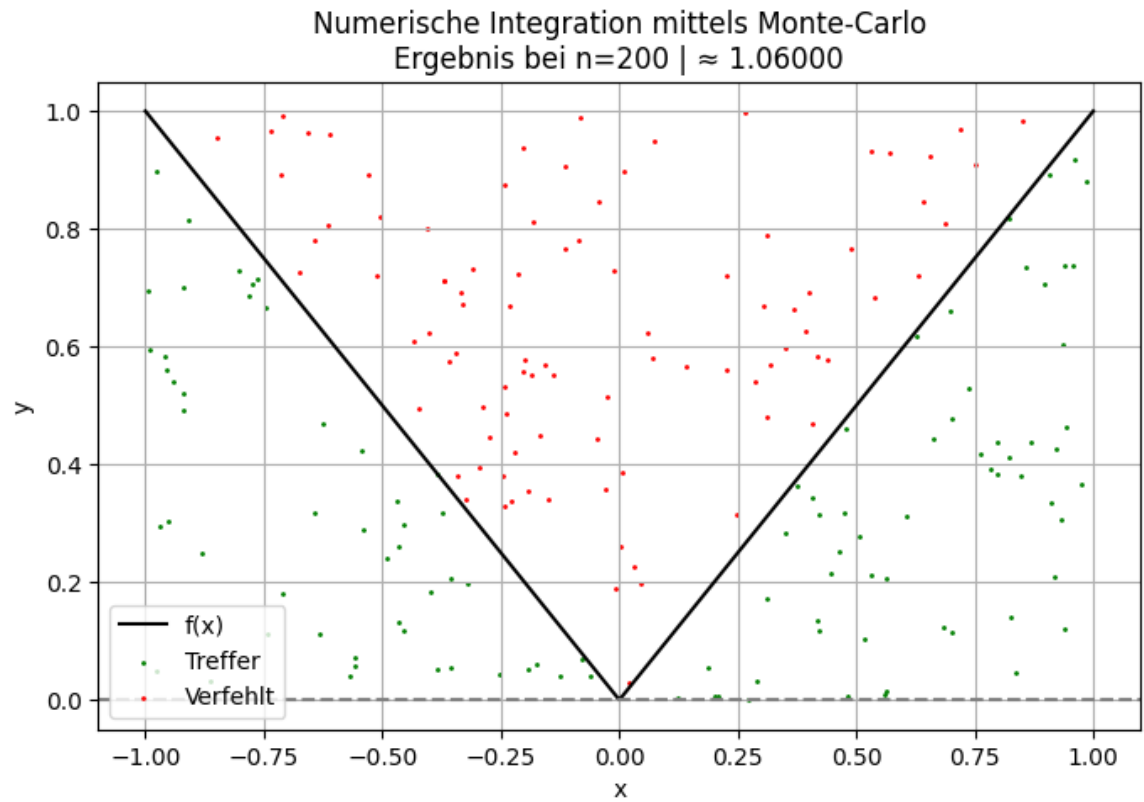


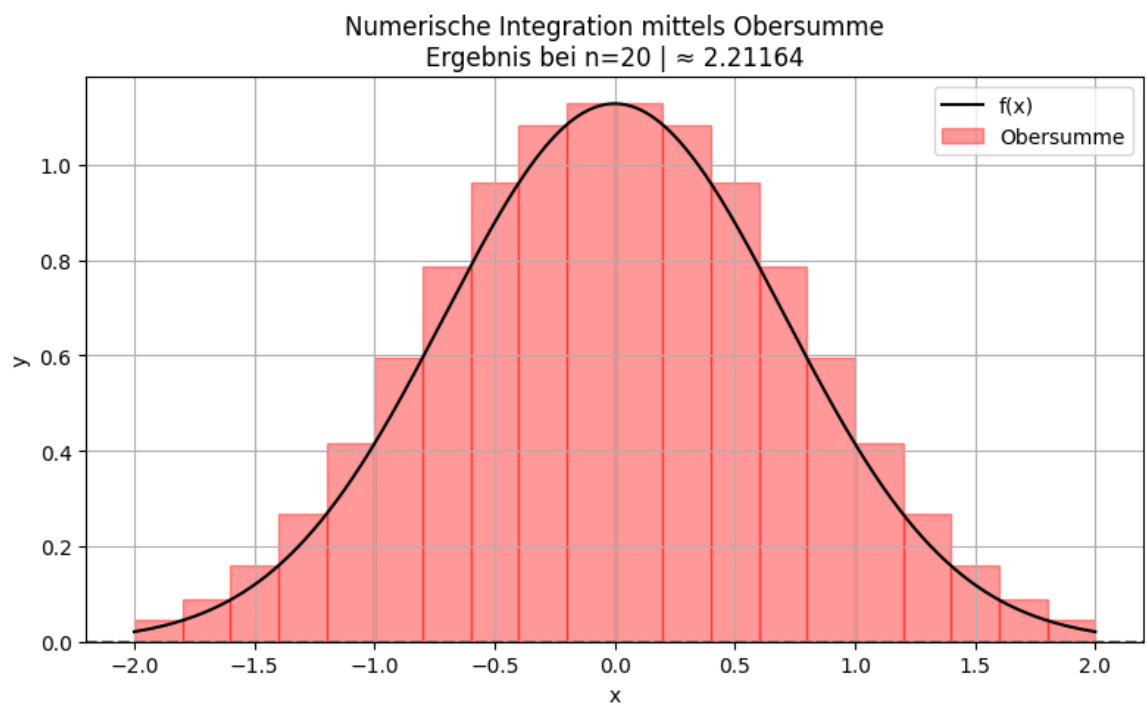
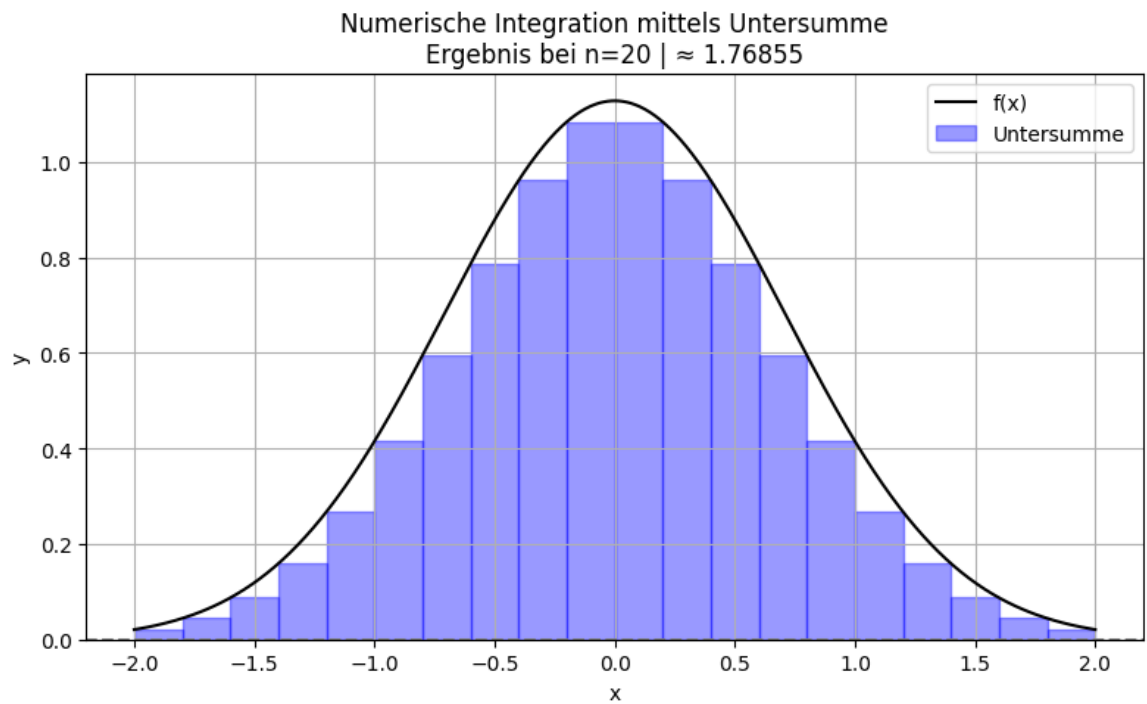
Numerische Integration mittels Monte-Carlo
Ergebnis bei $n=200$ | ≈ 5.60000

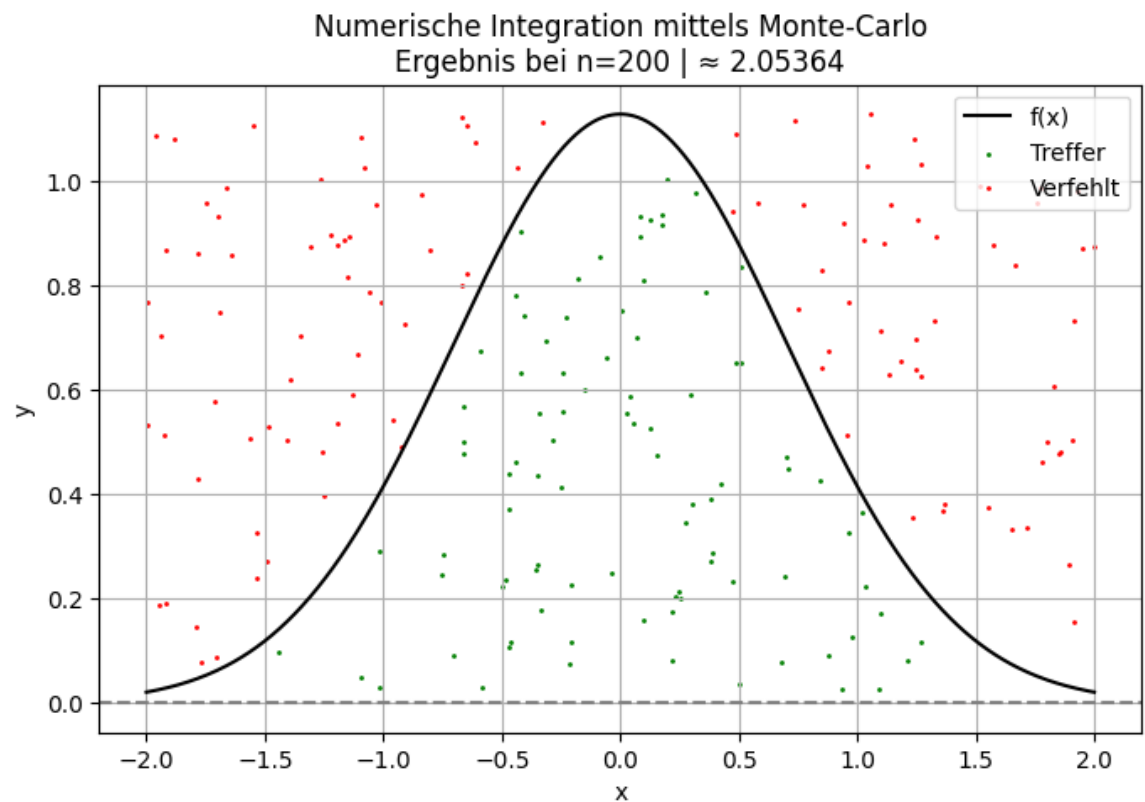
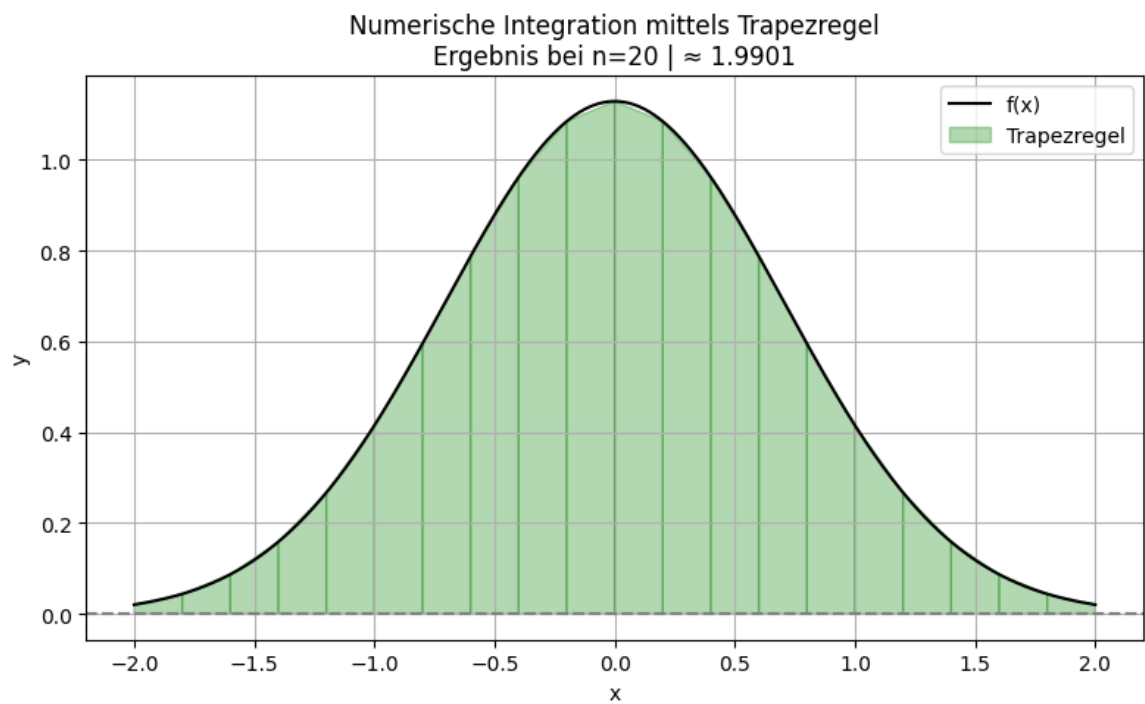


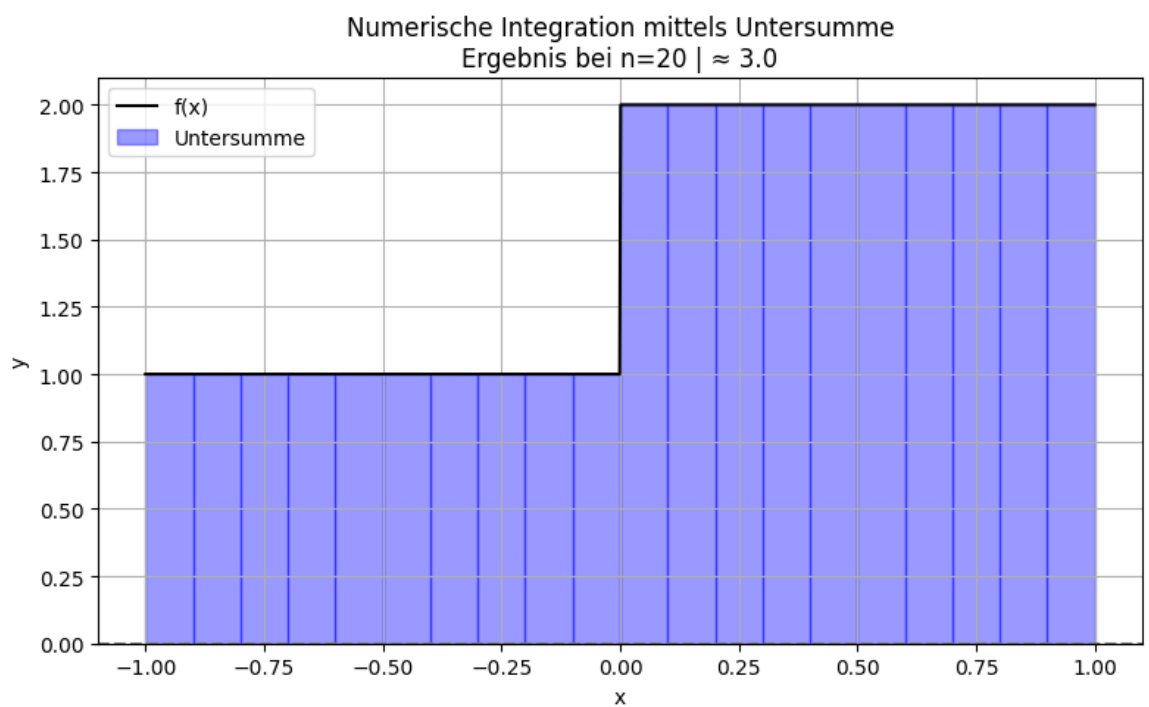
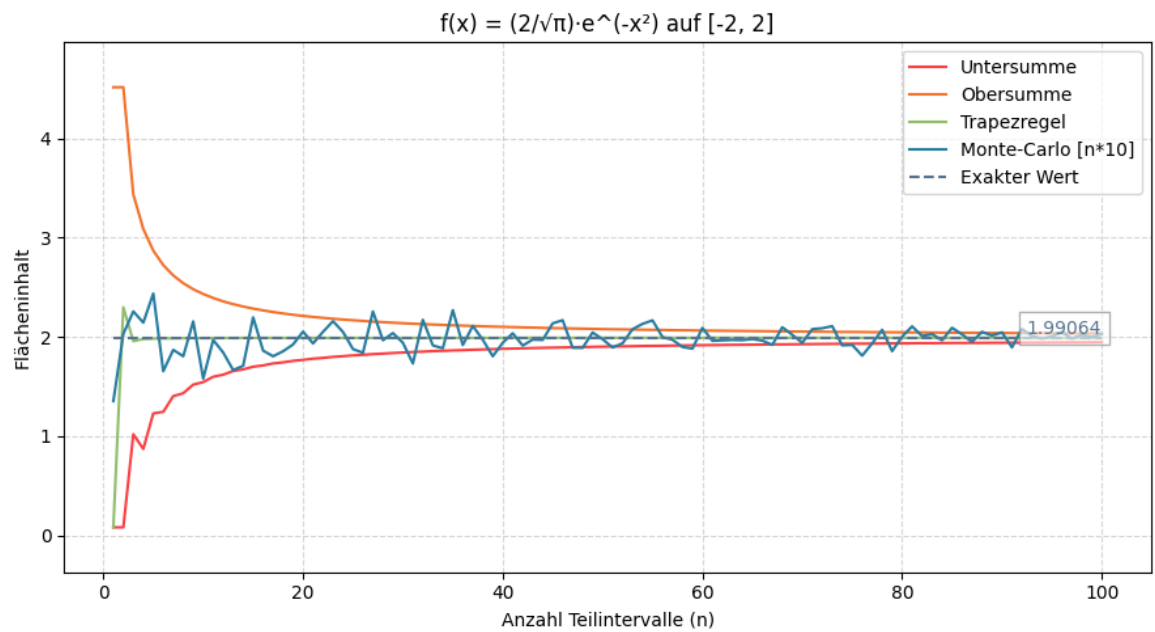




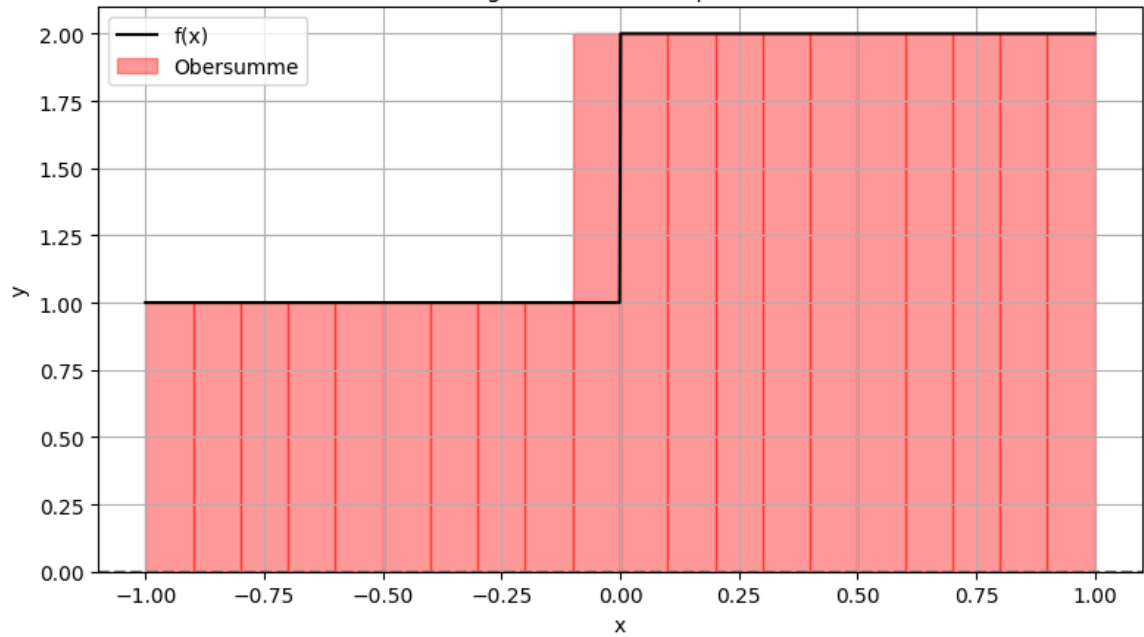




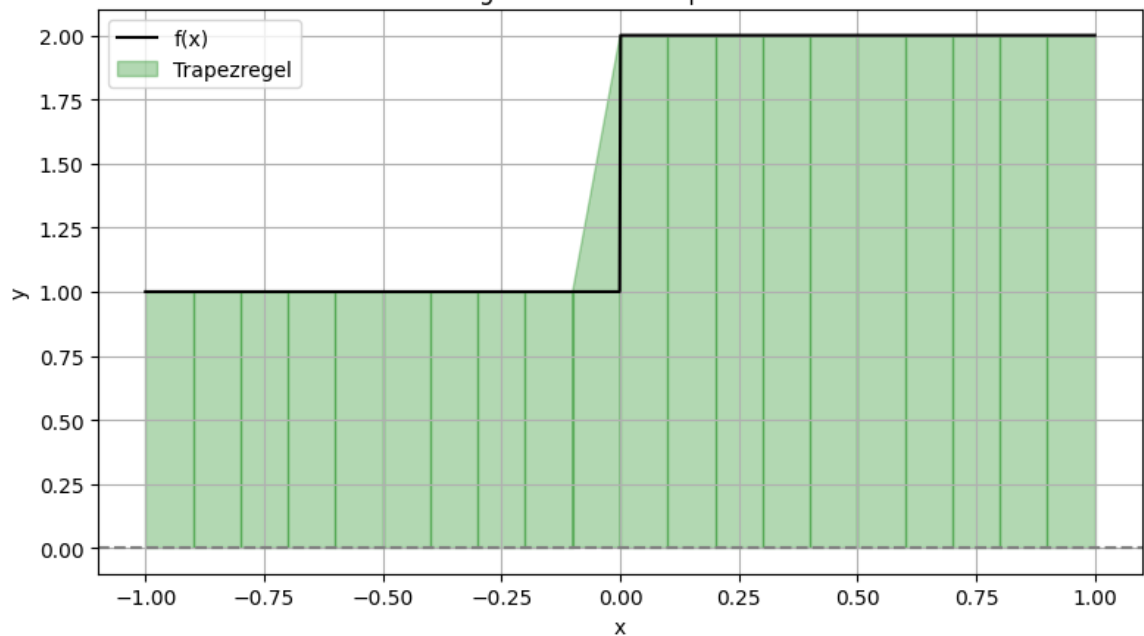


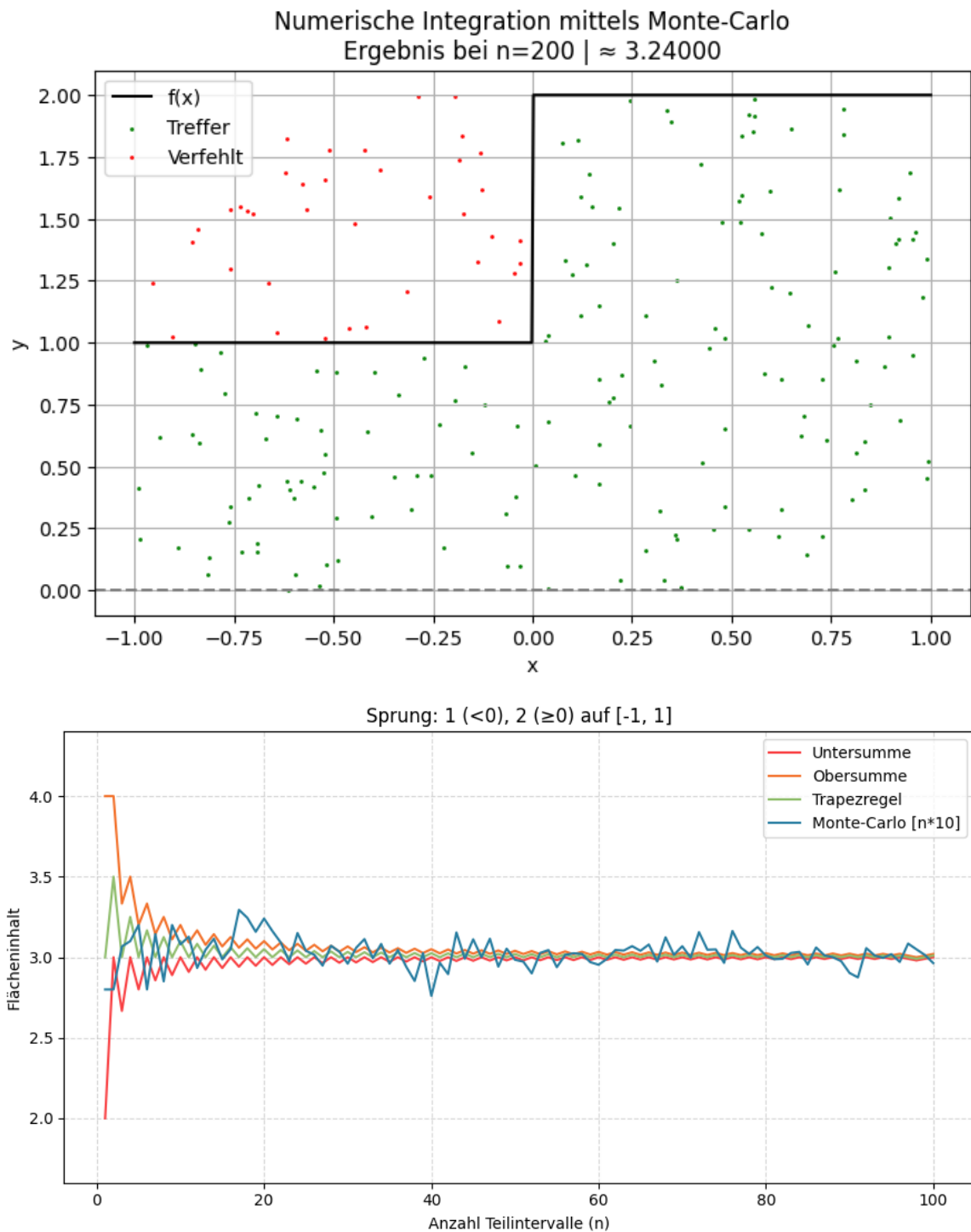


Numerische Integration mittels Obersumme
Ergebnis bei $n=20$ | ≈ 3.1



Numerische Integration mittels Trapezregel
Ergebnis bei $n=20$ | ≈ 3.05





Alle Funktionen erfolgreich geplottet.

Unit Tests

```
In [37]: import unittest
import math
```

```

# -----
# Liste stabiler Testfunktionen für Integrationstests:
# Jede Funktion ist gegeben durch:
# - Beschreibung (str)
# - Funktionsdefinition (lambda x)
# - Intervallgrenzen [a, b]
# - Analytische Stammfunktion F(x) (lambda x)
# -----
testfunktionen_stabil = [
    ("f(x) = -x2", lambda x: -x**2, -2, 2, lambda x: -(1/3) * x**3),
    ("f(x) = 2x + 1", lambda x: 2 * x + 1, 0, 1, lambda x: x**2 + x),
    ("f(x) = 5", lambda x: 5, 2, 6, lambda x: 5 * x),
    ("f(x) = cos(x)", math.cos, 0, math.pi, lambda x: math.sin(x)),
    ("f(x) = sin(x)", math.sin, 0, math.pi, lambda x: -math.cos(x)),
    ("f(x) = x2", lambda x: x**2, 0, 3, lambda x: (1/3) * x**3),
    ("f(x) = x2 - 4x + 2", lambda x: x**2 - 4*x + 2, -3, 3,
     lambda x: (1/3)*x**3 - 2*x**2 + 2*x),
    ("f(x) = x·sin(x)", lambda x: x * math.sin(x), 0, math.pi,
     lambda x: -x*math.cos(x)+math.sin(x)),
]

# -----
# Testklasse für numerische Integrationsmethoden
# -----
class TestIntegration(unittest.TestCase):

    # Hilfsfunktion zur exakten Berechnung des Integrals mittels F(x)
    def berechne_exakt(self, f_stamm, a, b):
        return f_stamm(b) - f_stamm(a)

    # Test: Untersumme
    def test_untersumme(self):
        for name, f, a, b, f_stamm in testfunktionen_stabil:
            with self.subTest(f"Untersumme: {name}"):
                exakt = self.berechne_exakt(f_stamm, a, b)
                # numerische Integration mit Untersumme (n=500 Unterintervalle)
                approx = numerische_integration(f, a, b, n=500, method="left")
                self.assertAlmostEqual(approx, exakt, delta=0.4)

    # Test: Obersumme
    def test_obersumme(self):
        for name, f, a, b, f_stamm in testfunktionen_stabil:
            with self.subTest(f"Obersumme: {name}"):
                exakt = self.berechne_exakt(f_stamm, a, b)
                approx = numerische_integration(f, a, b, n=500, method="right")
                self.assertAlmostEqual(approx, exakt, delta=0.4)

    # Test: Trapezregel
    def test_trapezregel(self):
        for name, f, a, b, f_stamm in testfunktionen_stabil:
            with self.subTest(f"Trapezregel: {name}"):
                exakt = self.berechne_exakt(f_stamm, a, b)
                approx = numerische_integration(f, a, b, n=500, method="trapez")
                self.assertAlmostEqual(approx, exakt, delta=0.2) #

    # Test: Monte-Carlo-Verfahren

```

```

def test_montecarlo(self):
    for name, f, a, b, f_stamm in testfunktionen_stabil:
        with self.subTest(f"Monte Carlo: {name}"):
            exakt = self.berechne_exakt(f_stamm, a, b)
            approx = monte_carlo_integrations(f, a, b, n_punkte=10000)
            self.assertAlmostEqual(approx, exakt, delta=0.2) #

if __name__ == "__main__":
    unittest.main(argv=['first-arg-is-ignored'], exit=False)

```

```

....
-----
--
Ran 4 tests in 3.381s
OK

```

Aufbau des Codes

1. Testfunktionen

Es wird eine Liste stabiler Funktionen `testfunktionen_stabil` definiert. Jede Funktion besteht aus:

- einem **Namen** (z.B. `"f(x) = sin(x)"`),
- einer **Funktionsvorschrift** (z.B. `lambda x: math.sin(x)`),
- dem **Integrationsintervall** `[a, b]`,
- der **analytischen Stammfunktion** `F(x)` zur exakten Berechnung.

Diese Funktionen sind bewusst **glatt, stetig und ohne Singularitäten**, damit die Tests zuverlässig und schnell durchlaufen.

2. Testklasse `TestIntegration`

Diese Klasse enthält vier Methoden, die jeweils eine **numerische Integrationsmethode** testen:

Methode	Beschreibung	Toleranz
Untersumme	Nimmt das Minimum im Intervall (Unterschätzung)	±0.4
Obersumme	Nimmt das Maximum im Intervall (Überschätzung)	±0.4
Trapezregel	Mittelwert der Randwerte – sehr präzise	±0.2
Monte-Carlo	Zufallsbasierte Flächenschätzung (statistisch)	±0.2

Für jede Methode wird:

- das **exakte Integral** mit der Stammfunktion berechnet,

- die **numerische Approximation** ermittelt,
- die Differenz mit `assertAlmostEqual` verglichen.

3. Ausführung in Jupyter Notebook

Der letzte Block:

```
if __name__ == "__main__":
    unittest.main(argv=['first-arg-is-ignored'], exit=False)
```

ist speziell angepasst für Jupyter:

- `argv=['first-arg-is-ignored']` : verhindert Fehler durch Jupyter-Argumente
- `exit=False` : verhindert das Beenden des Kernels nach den Tests

Ergebnis

Wenn alle Tests erfolgreich sind, zeigt das:

- Die **numerischen Methoden** funktionieren korrekt.
- Die **Implementierung der numerische_integration und monte_carlo_integration** liefert realistische Ergebnisse.
- Die Wahl von Parametern (`n` , `n_punkte` , `delta`) ist geeignet für typische, stabile Funktionen.

Kapitel 4: Grenzen numerischer Integration

1. Theoretischer Hintergrund

Beschreibung: $F(x)$ ist eine Stammfunktion der hochfrequent oszillierenden Funktion $\sin((x+1) \cdot 16\pi)$.

Eigenschaften:

- Periodisch mit Periode $\frac{1}{8}$ (aufgrund des Faktors 16π).
- Zeigt schnelle Vorzeichenwechsel durch hohe Frequenz.
- Die Stammfunktion $F(x)$ ist glatt, da sie die Fläche unter $\sin((x+1) \cdot 16\pi)$ aufsummiert.
- Allgemeine Form: $F(x) = -\frac{1}{16\pi} \cos((x+1) \cdot 16\pi) + C$
- Ideal zur Analyse und zum Vergleich numerischer Integrationsverfahren

geeignet.

Exakter Wert des Integrals

Die Funktion kann analytisch integriert werden:

$$F(x) = \int \sin((x+1) \cdot 16\pi) \, dx$$

Setze

$$u = (x+1) \cdot 16\pi \rightarrow du = 16\pi \, dx \rightarrow dx = \frac{1}{16\pi} du$$

$$\rightarrow F(x) = \frac{1}{16\pi} \int \sin(u) \, du = -\frac{1}{16\pi} \cos((x+1) \cdot 16\pi) + C$$

Das bestimmte Integral auf $[0,1]$:

$$\int_0^1 \sin((x+1) \cdot 16\pi) \, dx = -\frac{1}{16\pi} \left[\cos(2 \cdot 16\pi) - \cos(1 \cdot 16\pi) \right]$$

Da $\cos(2 \cdot 16\pi) = \cos(1 \cdot 16\pi) = 1$, ergibt sich:

$$\boxed{I = 0}$$

```
In [38]: # -----
# Testfunktion: Hochfrequente Sinusfunktion mit bekannter Stammfunktion
# -----
hoch0szillierendeFunktionTest = [
    (
        "f(x) = sin((x + 1) * 16π)",          # Funktion

        # Funktionsdefinition:
        # Sinusfunktion mit 16 vollen Perioden auf dem Intervall [0, 1]
        # => hohe Oszillation (32 Nullstellen auf kurzer Strecke)
        lambda x: math.sin((x + 1) * 16 * math.pi),

        0, # Untere Integrationsgrenze
        1, # Obere Integrationsgrenze

        # Stammfunktion F(x):
        # ∫ sin((x+1) * 16π) dx = -1 / (16π) * cos((x+1) * 16π)
        lambda x: -1 / (16 * math.pi) * math.cos((x + 1) * 16 * math.pi)
    ),
]
```

```
In [39]: # Aufruf der Auswertungsfunktion für hochfrequente Sinusfunktion
auswertung_durchfuehren(
    nutze_feste_grenzen=False,          # Verwende die Integrationsgrenzen
    feste_a=-5,                         # Wird ignoriert, da nutze_feste_grenzen=False
    feste_b=5,                         # Wird ignoriert, da nutze_feste_grenzen=False
)
```

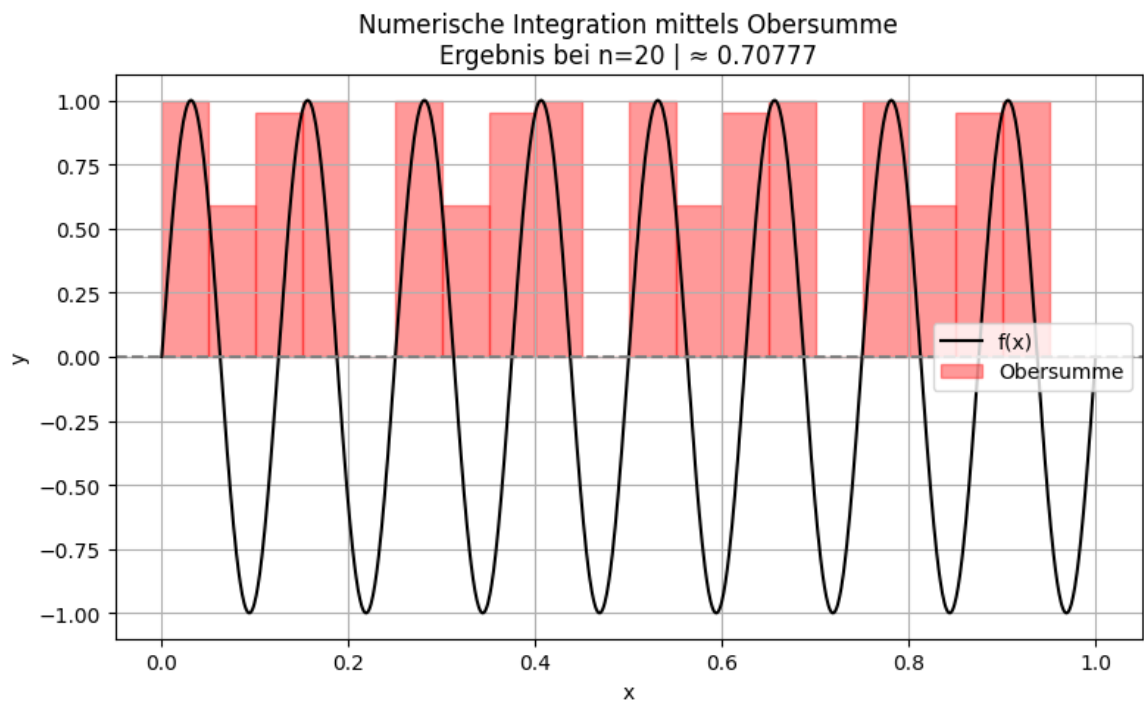
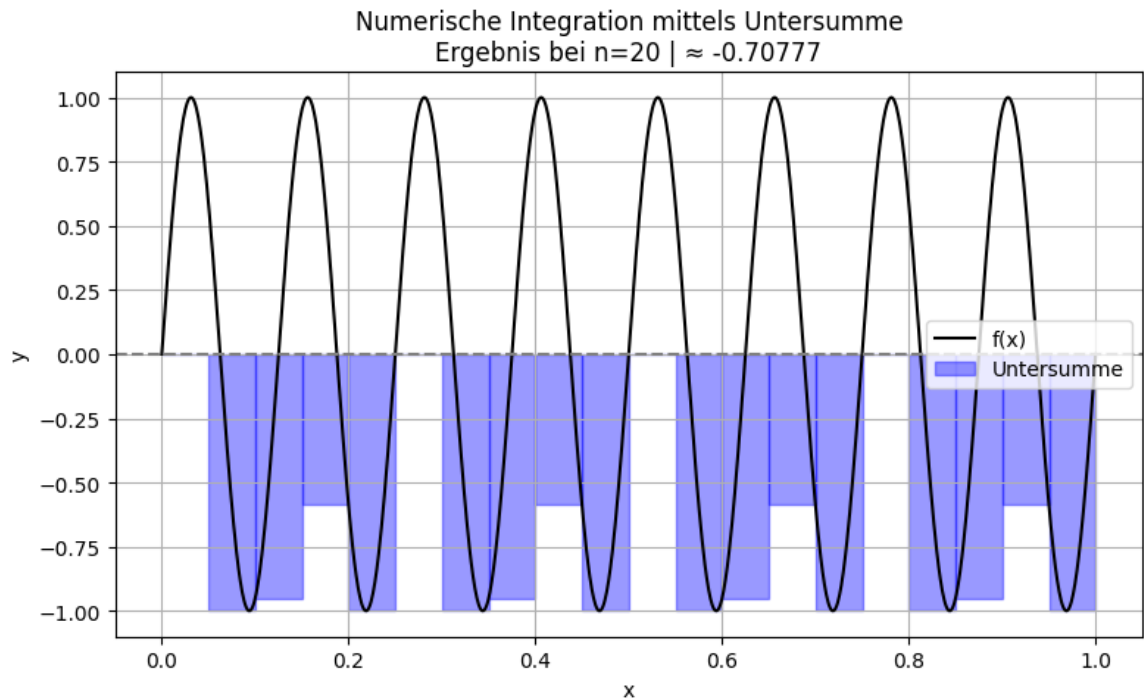
```

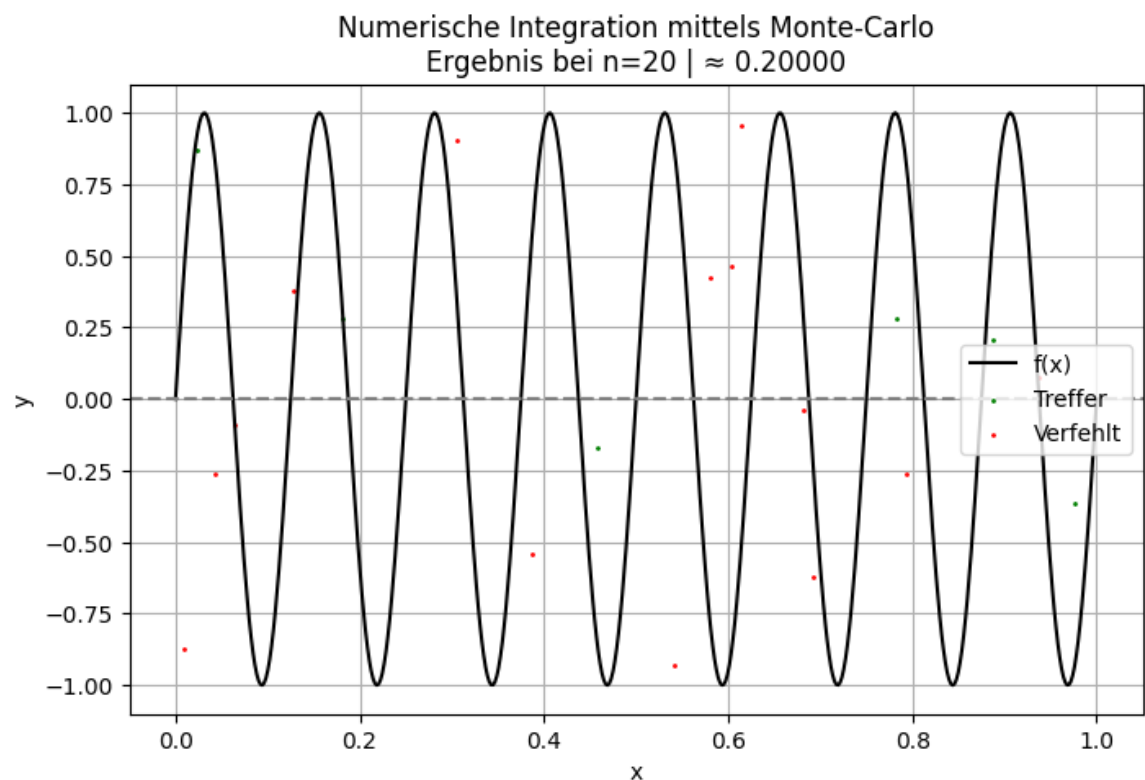
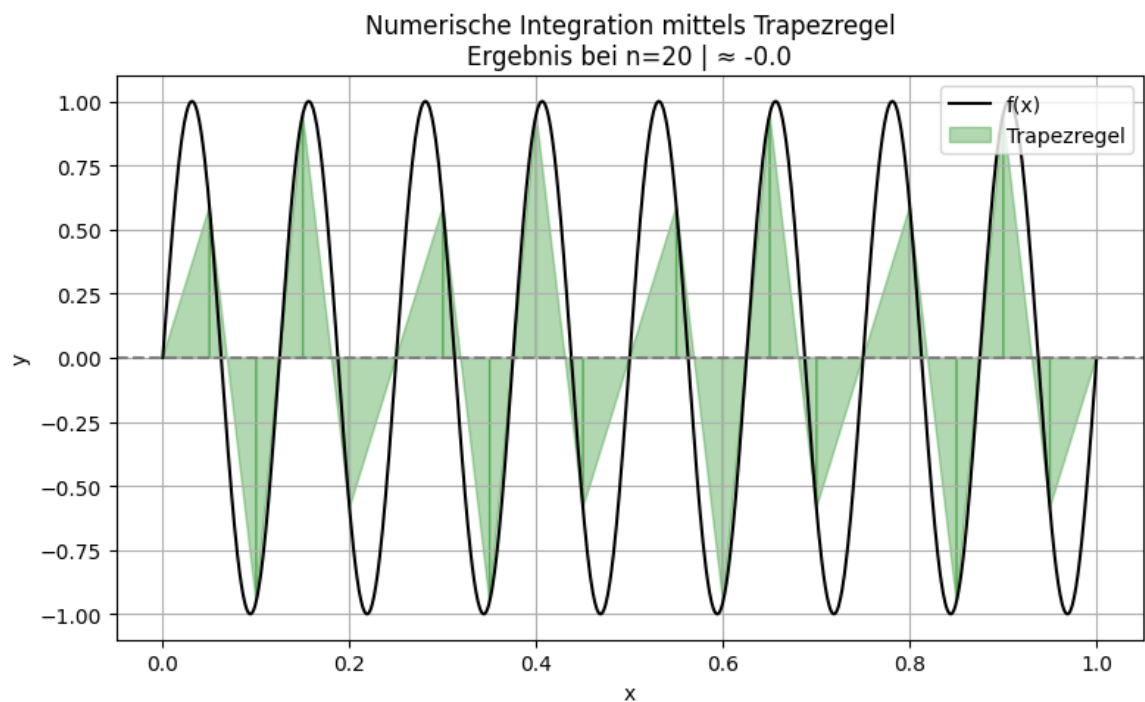
moFaktor=1,                                     # Für Monte-Carlo: Verwende n *
                                                # Hinweis: Geringe Stichprobendi

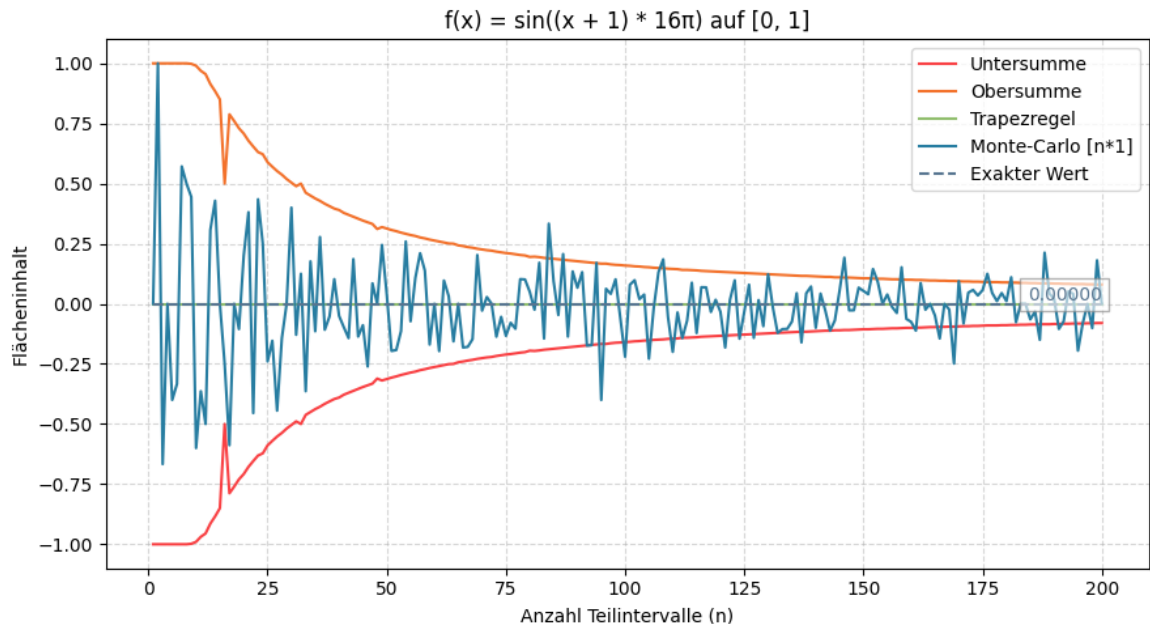
max_n=200,                                     # Teste Teilintervall-Anzahlen v

testfunktionen=hoch0sifizierendeFunktionTest
# Verwende nur eine hochfrequente Funktion:
#  $f(x) = \sin((x+1) \cdot 16\pi)$  auf dem Intervall  $[0, 1]$  mit Stammfunkt.
)

```







Alle Funktionen erfolgreich geplottet.

2. Ergebnisse bei diskreter Aufteilung

Wir betrachten die Werte bei $n = 2, 4, 8, 16$.

n	Untersumme	Obersumme	Trapezregel
2	≈ -0.6374	≈ 0.6374	≈ 0.0
4	≈ -0.3826	≈ 0.3826	≈ 0.0
8	≈ -0.1951	≈ 0.1951	≈ 0.0
16	≈ -0.0981	≈ 0.0981	≈ 0.0

=> Die **Trapezregel** liefert bereits bei wenigen Intervallen einen sehr guten Wert. Die Ober- und Untersummen konvergieren langsamer und liefern einseitige Näherungen.

Monte-Carlo-Integration ist ein stochastisches Verfahren: Die Ergebnisse sind nicht deterministisch und können bei jedem Durchlauf leicht variieren.

Dennoch nähert sich der Mittelwert der berechneten Flächen mit zunehmender Punktzahl schnell dem tatsächlichen Wert an. Bereits bei kleinen n liefert die Methode häufig eine brauchbare Approximation, insbesondere bei stark oszillierenden oder komplexen Funktionen, für die klassische Verfahren versagen können.

3. Erklärung der Ergebnisse

Die Funktion

$f(x) = \sin((x+1) \cdot 16\pi)$ ist **hochfrequent oszillierend**: Sie führt auf dem Intervall $[0,1]$ zu 16 vollständigen Sinusperioden.

Probleme bei Rechteckverfahren (Ober-/Untersumme):

- Diese Verfahren nehmen den **minimalen oder maximalen Funktionswert** im Intervall – sie können die Oszillation innerhalb des Intervalls nicht erfassen, wenn n zu klein ist.
- Bei Funktionen mit stark wechselnden Vorzeichen entstehen **große positive und negative Beiträge**, die sich **bei exakter Integration auslöschen**, aber bei groben Intervallen nicht erkannt werden.

Vorteil der Trapezregel:

- Sie approximiert die Fläche mit **linearen Segmenten** zwischen Funktionswerten und kann so **positive und negative Beiträge besser kompensieren**.
- Bereits bei $n=2$ ergibt sich hier nahezu der exakte Wert.

Vorteile des Monte-Carlo-Verfahrens

- Da das Monte-Carlo-Verfahren auf einer **zufälligen** Verteilung der Punkte basiert, konvergiert das Verfahren bei steigendem n nur sehr langsam gegen den tatsächlichen Wert $I=0$.
- Der Vorteil liegt aber in der schnellen ungefähren Abschätzung, wie im Graph bereits zu sehen liefert Monte-Carlo zwar sehr stark schwankende Ergebnisse, jedoch deutlich bessere als das Verfahren der Ober-/Untersumme.
- Die Varianz ist deutlich durch die starken Ausschläge sichtbar, diese sind bei mehreren Durchläufen auch nicht konstant sondern schwanken in einem gewissen Bereich, je nach Verteilung der Punkte.

4. Kriterien für gute numerische Integration

1. Glattheit und Stetigkeit der Funktion

Numerische Integration funktioniert besser bei stetigen, glatten Funktionen. Diskontinuitäten oder Unstetigkeiten (z. B. Sprungstellen) führen zu hohen Fehlern.

2. Oszillationsverhalten

Bei **schnell oszillierenden Funktionen** muss die Schrittweite klein genug sein, um mindestens 1–2 Stützstellen pro Periode zu setzen:

$$\Delta x \ll \frac{1}{\text{Frequenz}} = \frac{1}{16} \Rightarrow n \gg 16$$

3. Symmetrie ausnutzen

Funktionen mit periodischer Symmetrie (z. B. sinusförmig) können bei geeigneter Intervallwahl exakt integriert werden, da sich die positiven und negativen Flächen gegenseitig aufheben.

4. Wahl der Methode

- **Untersumme/Obersumme:** einseitig, daher nicht für oszillierende Funktionen geeignet.
- **Trapezregel:** geeignet bei glatten, periodischen Funktionen.
- **Monte Carlo:** brauchbar bei hohen Dimensionen, relativ schnell gute Näherung, aber aufgrund der zufälligen Verteilung kein Eindeutiges Ergebnis. Keine Eindeutige Konvergenz, -> Varianz wird einfach nur kleiner

5. Wahl von n

- Generell kann gesagt werden, dass je höher n gewählt wird, desto genau wird der Wert der numerischen Integration
- Dies sieht man auch deutlich am Vergleichsgraphen der verschiedenen Verfahren

Konvergenz aller Verfahren

Trotz unterschiedlicher Geschwindigkeit und Effizienz gilt für alle vorgestellten Verfahren:

$$\lim_{n \rightarrow \infty} I_n^{\text{numerisch}} = I^{\text{exakt}}$$

Das bedeutet: Bei ausreichend feiner Unterteilung ($n \rightarrow \infty$) liefern alle Verfahren – Obersumme, Untersumme, Trapezregel und Monte Carlo – das korrekte Integral, sofern die Funktion integrierbar ist.

Kapitel 5: Fakultativ: Gaußsche Fehlerfunktion

Numerische Integration nicht klassisch

integrierbarer Funktionen – Beispiel: Gaußsche Fehlerfunktion

In der Mathematik gibt es Funktionen, die **nicht in geschlossener Form** integrierbar sind – das heißt, ihre Stammfunktion lässt sich **nicht durch elementare Funktionen** ausdrücken. Dennoch können solche Funktionen mithilfe numerischer Methoden zuverlässig integriert werden.

Die Gaußsche Glockenkurve

Ein klassisches Beispiel ist die **Gaußsche Normalverteilung**:

$$f(x) = e^{-x^2}$$

Diese Funktion tritt in vielen Bereichen der Statistik, Physik und Technik auf. Das bestimmte Integral dieser Funktion über das ganze Intervall $(-\infty, \infty)$ ergibt die Wurzel von π :

$$\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi}$$

Doch diese Aussage basiert auf einem **Spezialbeweis** (z. B. über Polarkoordinaten), **nicht** auf einer elementaren Stammfunktion – denn eine solche existiert nicht.

Die Fehlerfunktion

Die sogenannte **Fehlerfunktion** wird definiert als:

$$\operatorname{erf}(x) := \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt$$

Dies ist eine **Spezialfunktion**, die eigens eingeführt wurde, um das Integral von e^{-x^2} handhabbar zu machen. Sie ist in vielen mathematischen Bibliotheken verfügbar, aber **kein Resultat elementarer Funktionen**.

Numerische Integrierbarkeit

Obwohl $f(x) = e^{-x^2}$ nicht klassisch integrierbar ist, **lässt sich ihr Integral über ein beschränktes Intervall sehr gut numerisch berechnen**. Besonders die Trapezregel oder adaptive Methoden liefern schnell gute Näherungen:

$$\int_{-2}^2 e^{-x^2} dx \approx 1.7642$$

Dieser Wert lässt sich mit gängigen Verfahren (z. B. Trapezregel, Simpson-Regel, Gauß-Quadratur) auf 4 oder mehr Nachkommastellen korrekt

berechnen – bereits bei mäßiger Schrittweite.

```
In [40]: # -----
# Testfunktion: Gaußsche Glockenkurve (Standard-Gauss-Funktion)
# -----
GausscheGlockenKurve = [
    (
        # Funktionsname (wird in Plots und Legenden verwendet)
        "f(x) = e^{-x^2}",

        # Funktionsdefinition: f(x) = e^{-x^2}
        # Symmetrische, glockenförmige Funktion mit Maximum bei x = 0
        # Wird häufig in Statistik (Normalverteilung), Physik und S.
        lambda x: math.exp(-x**2),

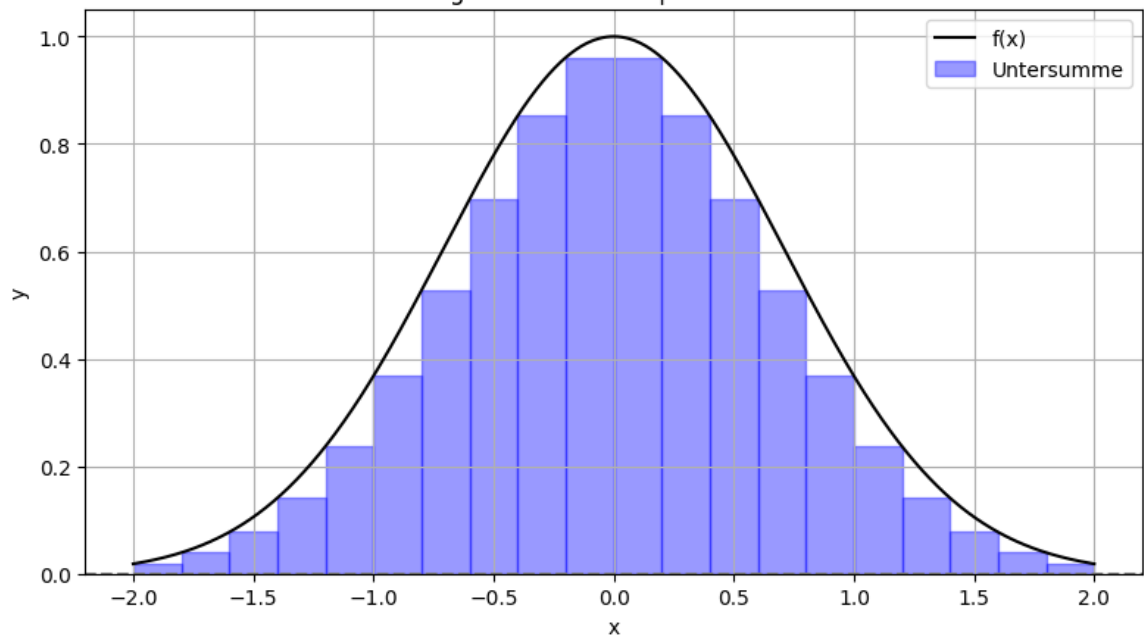
        # Untere und obere Integrationsgrenze
        -2, # Links von 0
        2,  # Rechts von 0

        # Stammfunktion:
        #  $\int e^{-x^2} dx = (\sqrt{\pi} / 2) * \text{erf}(x)$ 
        # (Die Fehlerfunktion erf(x) ist in math.erf verfügbar)
        lambda x: (math.sqrt(math.pi) / 2) * math.erf(x)
    ),
]
```

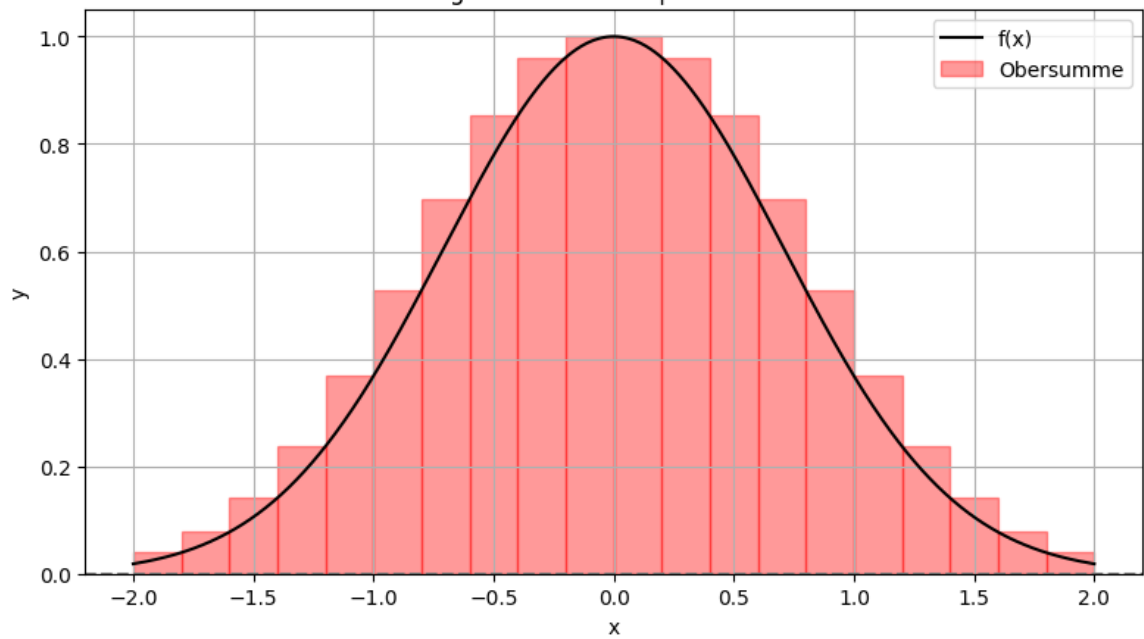
```
In [41]: # Aufruf der Hauptauswertungsfunktion zur numerischen Integration
# für die Testfunktion f(x) = e^{-x^2} auf dem Intervall [-2, 2]

auswertung_durchfuehren(
    nutze_feste_grenzen=False,      # False → verwende die Integrationsgrenzen
    feste_a=-5,                    # Wird ignoriert, da nutze_feste_grenzen=False
    feste_b=5,                     # Wird ignoriert, da nutze_feste_grenzen=False
    moFaktor=1,                    # Monte-Carlo verwendet n * 1 Stichprobe
                                    # → Für glatte Funktionen wie e^{-x^2}
    max_n=200,                     # Teste Teilintervall-Anzahlen von 1 bis max_n
    testfunktionen=GausscheGlockenKurve # Liste mit genau einer Funktion
)
```

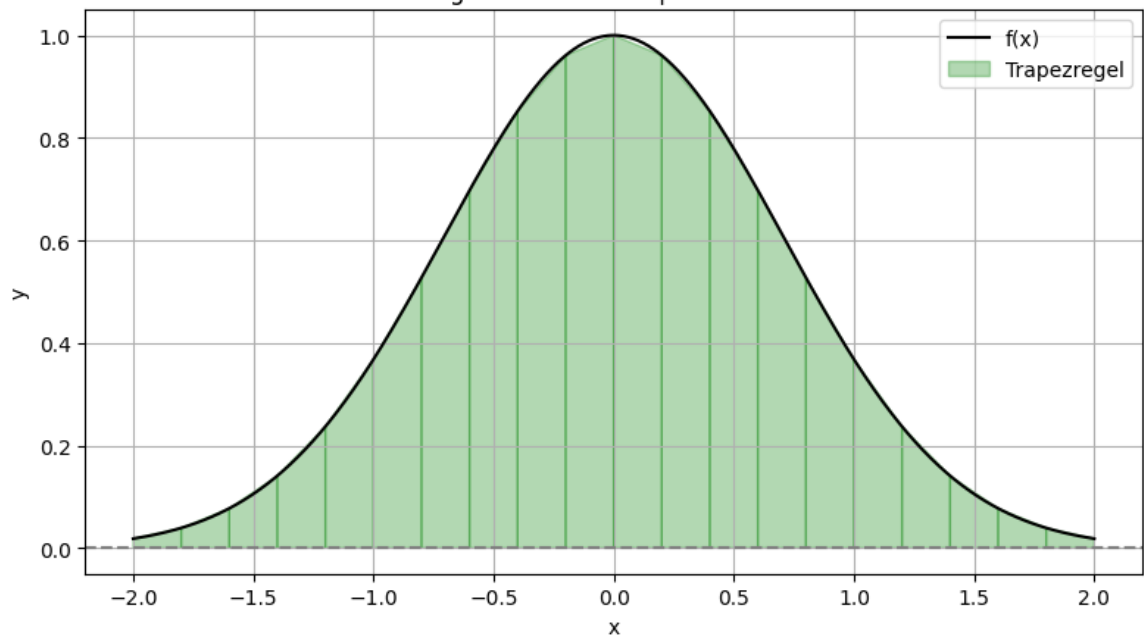
Numerische Integration mittels Untersumme
Ergebnis bei $n=20$ | ≈ 1.56734



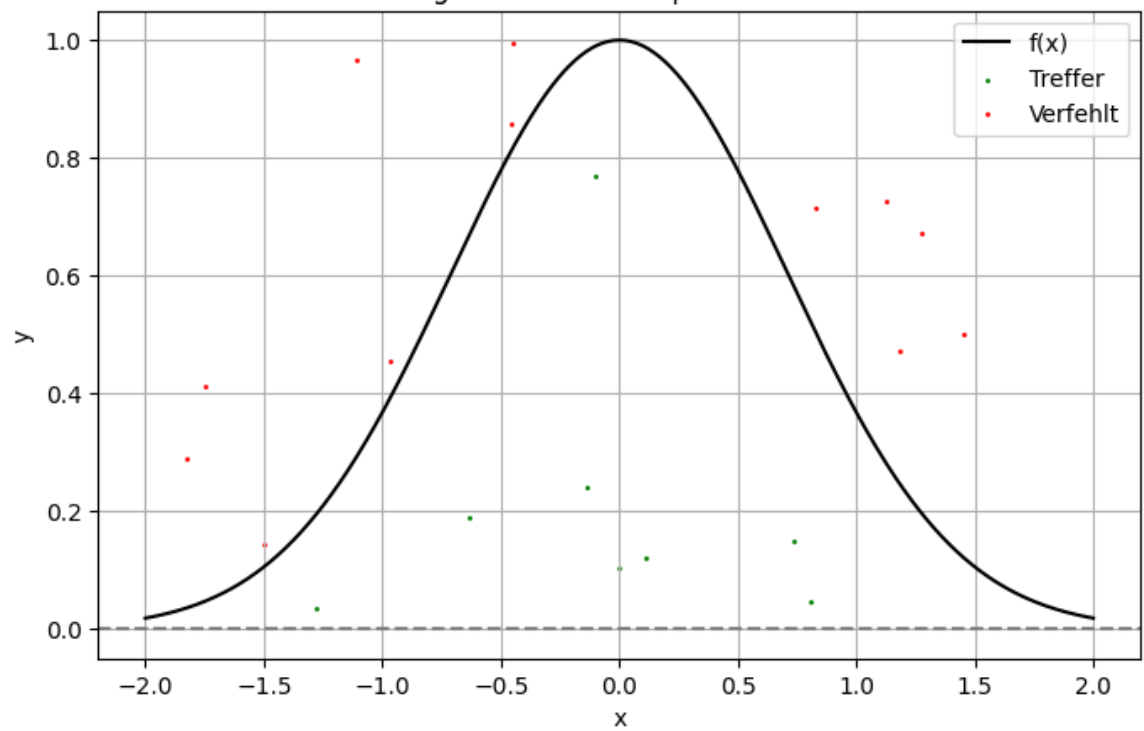
Numerische Integration mittels Obersumme
Ergebnis bei $n=20$ | ≈ 1.96001

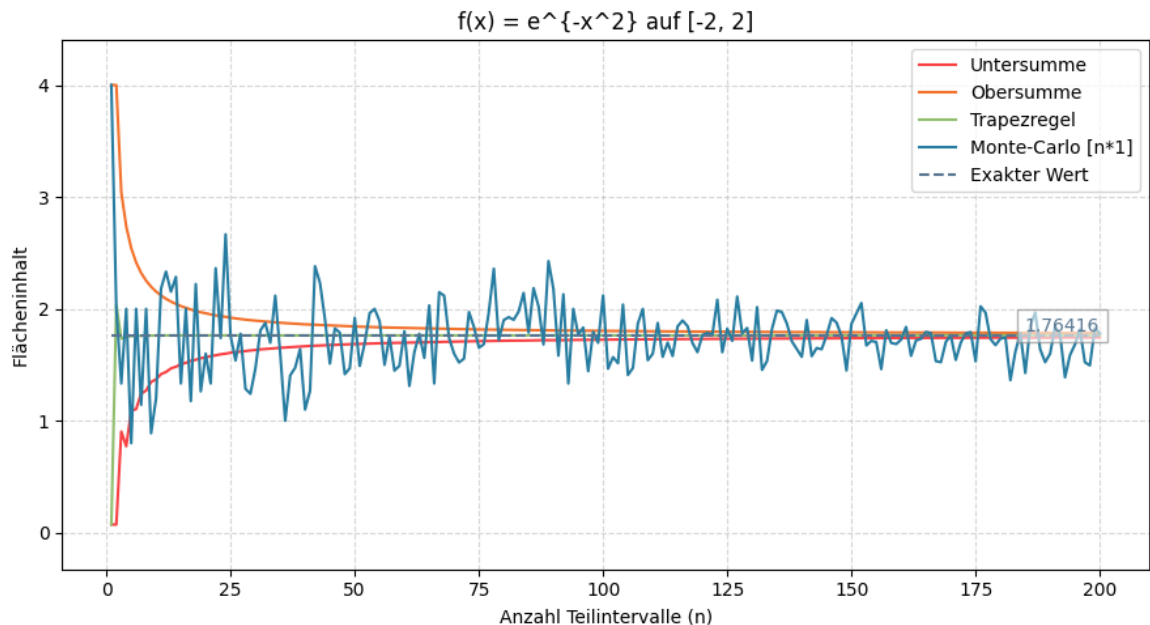


Numerische Integration mittels Trapezregel
Ergebnis bei $n=20$ | ≈ 1.76368



Numerische Integration mittels Monte-Carlo
Ergebnis bei $n=20$ | ≈ 1.59999





Alle Funktionen erfolgreich geplottet.

Fazit

Es gibt Funktionen wie $f(x) = e^{-x^2}$,

- die **keine elementare Stammfunktion** besitzen,
- sich aber dennoch **sehr gut numerisch integrieren lassen**.

Das zeigt:

Die numerische Integrationsfähigkeit hängt **nicht** primär davon ab, ob eine Funktion "klassisch integrierbar" ist – sondern von ihrer **Stetigkeit, Glattheit**, und dem **Verhalten auf dem Integrationsintervall**.

Kapitel 6 - Interaktive Darstellung

Ober- /Untersumme und Trapezregel

```
In [42]: import ipywidgets as widgets

funktions_dict = {
    "x^2": lambda x: x**2,
    "sin(x)": np.sin,
    "e^x": np.exp,
    "ln(x+1)": lambda x: np.log(x+1),
    "1/(1+x^2)": lambda x: 1/(1 + x**2),
}

def numerische_integrations_widget(a, b, n, methode, func_name):
    # Funktion auswählen
    f = funktionen_dict[func_name]
    # Integration & Plot (Plot wird durch numerische_integrations_widget erzeugt)
    ergebnis = numerische_integrations(f, a, b, n, methode, zeige_plot=True)
```

```

print(f"{methode} von f(x)={func_name} im Intervall [{a:.2f}, {b:.2f}]")
print(f"Integral ≈ {ergebnis:.5f}")

widgets.interactive(
    numerische_integration_widget,
    a=widgets.FloatSlider(min=0, max=20, step=0.1, value=0, description='a (linker Rand)'),
    b=widgets.FloatSlider(min=0.5, max=20, step=0.1, value=2, description='b (rechter Rand)'),
    n=widgets.IntSlider(min=2, max=50, step=1, value=10, description='Anzahl der Punkte'),
    methode=widgets.RadioButtons(options=["Untersumme", "Obersumme", "Trapezsumme"], value="Untersumme"),
    func_name=widgets.Dropdown(options=list(funktionen_dict.keys()), value="x^2"),
)

```

Out[42]: interactive(children=(FloatSlider(value=0.0, description='a (linker Rand)', max=20.0), FloatSlider(value=2.0, description='b (rechter Rand)', max=20.0), IntSlider(value=10, description='Anzahl der Punkte', max=50), RadioButtons(options=['Untersumme', 'Obersumme', 'Trapezsumme'], value='Untersumme'), Dropdown(options=['x^2', 'sin(x)', 'e^x', 'ln(x+1)', '1/(1+x^2)'], value='x^2')))

Monte-Carlo-Integration

```

In [ ]: import numpy as np
import matplotlib.pyplot as plt
import ipywidgets as widgets

# Deine Monte-Carlo-Integrationsfunktion und Hilfsfunktion müssen hier definiert werden
# -> monte_carlo_integration und find_extrema

# Funktionsauswahl
funktionen_dict = {
    "x^2": lambda x: x**2,
    "sin(x)": np.sin,
    "e^x": np.exp,
    "ln(x+1)": lambda x: np.log(x+1),
    "1/(1+x^2)": lambda x: 1/(1 + x**2),
}

def monte_carlo_interactive(a, b, n_points, func_name):
    # Funktion auswählen
    f = funktionen_dict[func_name]

    # Berechnung und Plot mit der echten Methode (zeige_plot steuern)
    integral = monte_carlo_integration(f, a, b, n_punkte=n_points, zeige_plot=True)

    # Ergebnis als Text anzeigen
    print(f"Geschätztes Integral von f(x)={func_name} im Intervall [{a:.2f}, {b:.2f}]")
    print(f"≈ {integral:.6f}")

# Widget erstellen
widgets.interactive(
    monte_carlo_interactive,
    a=widgets.FloatSlider(min=0, max=20, step=0.1, value=0, description='a (linker Rand)'),
    b=widgets.FloatSlider(min=0.5, max=20, step=0.1, value=2, description='b (rechter Rand)'),
    n_points=widgets.IntSlider(min=100, max=5000, step=100, value=1000, description='Anzahl der Punkte'),
    func_name=widgets.Dropdown(options=list(funktionen_dict.keys()), value="x^2"),
)

```

Out[]: interactive(children=(FloatSlider(value=0.0, description='a (linker Rand)', max=20.0), FloatSlider(value=2.0, description='b (rechter Rand)', max=20.0), IntSlider(value=1000, description='Anzahl der Punkte', max=5000), Dropdown(options=['x^2', 'sin(x)', 'e^x', 'ln(x+1)', '1/(1+x^2)'], value='x^2')))

Anhang

Gruppe: Pyoneers

Mitglieder: Simon Riedinger, Damian Schmidt, Laurin Ruoff, Colin Vierheller

Quellen

1. **Burden, R. L., & Faires, J. D. (2010).** *Numerical Analysis* (9th ed.). Cengage Learning.
– Enthält fundierte Erklärungen zur Trapezregel, Fehlerabschätzungen und numerischen Integration allgemein.
 2. **Wolfram MathWorld – Riemann Sum**
– Definitionen und Visualisierungen zur Untersumme, Obersumme und Riemann-Integration.
 3. **Wolfram MathWorld – Trapezoidal Rule**
– Übersicht zur Trapezregel mit Formeln und Beispielabbildungen.
 4. **Wolfram MathWorld – Monte Carlo Integration**
– Theorie und Anwendung der Monte-Carlo-Methode zur Integrationsschätzung.
 5. **Wikipedia – Fehlerfunktion (erf)**
– Mathematische Definition und Anwendungen der gaußschen Fehlerfunktion.
 6. **Wikipedia – Numerische Integration hochoszillierender Funktionen**
– Herausforderungen bei der Integration von Funktionen mit starker Oszillation.
- **Monte Carlo Integration (Scratchapixel)**
– Praktische Anwendung der Monte-Carlo-Integration in der Computergrafik.
 - **Monte-Carlo-Verfahren – Uni Münster PDF**
– Physikalisch fundierte Einführung mit Beispielen.
 - **StudySmarter – Riemann'sches Integral**
– Kompakte Einführung für Studierende mit anschaulichen Beispielen.
 - **TU Graz – Skriptum zur Numerischen Physik (Kapitel 6)**
– Kapitel zur numerischen Integration und deren Herausforderungen.